博碩文化

大AI時代將至，你不可不讀的語言模型獨門祕笈

LLM 大型語言模型的絕世祕笈

27路獨步劍法，帶你闖蕩生成式AI的五湖四海

陳威廷（Penut Chen）著

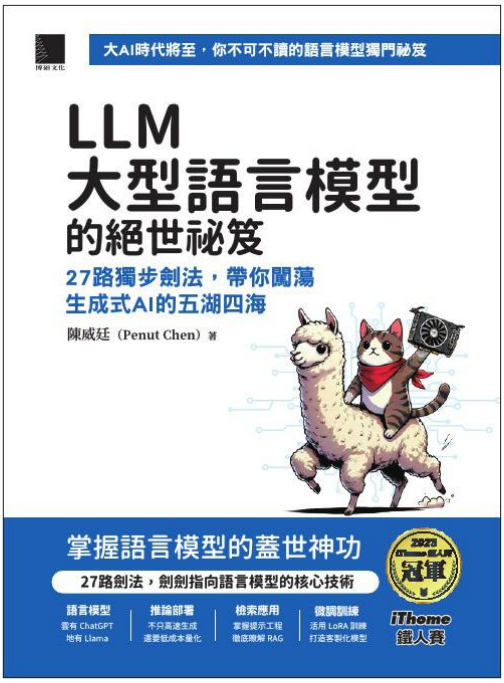
掌握語言模型的蓋世神功

27路劍法，劍劍指向語言模型的核心技術

2023 iThome鐵人賽 冠軍

語言模型	推論部署	檢索應用	微調訓練
雲有 ChatGPT 地有 Llama	不只高速生成 還要低成本量化	掌握提示工程 徹底瞭解 RAG	活用 LoRA 訓練 打造客製化模型

iThome 鐵人賽



作　　者：陳威廷（Penut Chen）
責任編輯：黃俊傑

董 事 長：曾梓翔
總 編 輯：陳錦輝

出　　版：博碩文化股份有限公司
地　　址：221 新北市汐止區新台五路一段 112 號 10 樓 A 棟
　　電話 (02) 2696-2869　傳真 (02) 2696-2867

發　　行：博碩文化股份有限公司
郵撥帳號：17484299　戶名：博碩文化股份有限公司
博碩網站：http://www.drmaster.com.tw
讀者服務信箱：dr26962869@gmail.com
訂購服務專線：(02) 2696-2869 分機 238、519
（週一至週五 09:30 ～ 12:00；13:30 ～ 17:00）

版　　次：2024 年 8 月初版

建議零售價：新台幣 650 元
I S B N：978-626-333-929-3
律師顧問：鳴權法律事務所 陳曉鳴律師

本書如有破損或裝訂錯誤，請寄回本公司更換

國家圖書館出版品預行編目資料

LLM 大型語言模型的絕世祕笈 : 27 路獨步劍法 , 帶你闖蕩生成式 AI 的五湖四海 / 陳威廷 (Penut Chen) 著 . -- 初版 . -- 新北市 : 博碩文化股份有限公司 , 2024.08

面 ;　公分. -- (iThome鐵人賽系列書)

ISBN 978-626-333-929-3(平裝)

1.CST: 自然語言處理 2.CST: 人工智慧
3.CST: 機器學習

312.835　　113010583

Printed in Taiwan

歡迎團體訂購，另有優惠，請洽服務專線
博 碩 粉 絲 團　(02) 2696-2869 分機 238、519

商標聲明

本書中所引用之商標、產品名稱分屬各公司所有，本書引用純屬介紹之用，並無任何侵害之意。

有限擔保責任聲明

雖然作者與出版社已全力編輯與製作本書，唯不擔保本書及其所附媒體無任何瑕疵；亦不為使用本書而引起之衍生利益損失或意外損毀之損失擔保責任。即使本公司先前已被告知前述損毀之發生。本公司依本書所負之責任，僅限於台端對本書所付之實際價款。

著作權聲明

本書著作權為作者所有，並受國際著作權法保護，未經授權任意拷貝、引用、翻印，均屬違法。

推薦序一

自從 ChatGPT 問世以來，AI 已成為科技運作的核心。不僅各行各業爭相導入 AI，甚至上至政府，下至市井小民也能說出一些 AI 的知識。我相信，有著資訊背景的工程師，常常會被詢問有關 AI 的問題，或是被主管要求幫公司導入 AI。在這種情況下，我們需要一本書來引領我們深入了解 AI 的運作，體驗大型語言模型帶來的變革。這本書正是你，由淺入深了解技術細節並提升自我實力的良伴。

作者陳威廷是第 15 屆 iThome 鐵人賽 AI & DATA 組的冠軍，身為評審之一的我，當時閱讀他一系列為期三十天的文章時，就覺得文筆流暢，內容精彩豐富，對我而言獲益良多，因此給予了他最高的評價。

事實上，本書在鐵人賽文章的基礎上增加了更多內容。除了介紹各種常見的語言模型及其使用方式外，還補充了許多細節和實作部分，書中的程式碼附有連結至 GitHub 供參考。本書內容涵蓋了 ChatGPT 的各項使用、延伸應用及工具，還有各種神經網路語言模型的說明，例如 Transformer、LLaMa 等，並介紹了台灣常見的大型語言模型，如 Breeze、Taiwan LLM、TAIDE 等。當然，書中也詳細指導如何透過開放源碼的語言模型來建立並訓練自己的 AI 應用。

感謝作者花費大量時間編寫此書，清楚地整理及介紹了大型語言模型。對於想學習大型語言模型並打造屬於自己 AI 的讀者，相信本書可以讓你事半功倍。

徐千洋

台灣駭客年會創辦人

CYBAVO 共同創辦人

推薦序二

ChatGPT 出現之後，身邊許多朋友都驚訝於它的寫作與對話能力，紛紛問我電腦是怎麼辦到的。在聽完一堆嵌入向量、矩陣運算、神經傳導的解釋後，非資工背景的朋友們仍然是一臉茫然：「這樣就學會我們的語言了？」這讓我發現，大家都極有興趣成為未來 AI 世界的一份子，但是需要更適合的入門書。

威廷在求學時期就是個興趣廣泛的人，一聽到新觀念，會馬上深入研究。頗有心得時，還會將筆記分享給大家，就是那種資工系最歡迎的自動自發的學生。研究所時進到自然語言處理實驗室找我做碩士論文，這些神經網路、語言模型、深度學習等概念 ，都是我們這領域熟悉的基礎。得知他打算出版這本書時真是相當驚喜，長期在網路世界打滾的他也深諳網路語彙，由他來寫這本書真是再適合不過了。

這本書有理論、有程式，有實作運用演練，也有趨勢新知的分享。內容論述深入淺出，範例鮮明。整本書居然涵蓋了二十多種主題，簡直物超所值，收穫滿滿！

期待這世上有各種法門引領大家習得學識，也期待這本書能點燃許多人心中對 LLM 的好奇心，並成為踏入 AI 世界的最大支援火力！

林川傑 助理教授

國立臺灣海洋大學資訊工程學系

推薦序三

已經 2024 年了，你用過 AI 嗎？一如過去幾年流行的關鍵詞：「大數據」、「機器學習」、「資料收集」，在當時對整個世界產生爆炸性的影響，也因為這些過往的流行，為這股 AI 浪潮鋪了非常長、完整的一條路。 從去年開始流行起「AI」，每家公司都想要在自己的產品上加上「AI」兩個字，或者是想要藉由 AI 來提高公司或是內部員工的生產力。相信很多業內業外是順著這股浪潮而來認識 AI ，但這只是股無意的風潮嗎？ AI 並不只是一個 Buzz Word，它確實逐漸充滿我們的生活，不管是生活中手機裡的語音助理、即時翻譯，到汽車中的自動駕駛，都可以看見 AI 的身影。 AI 早已勢不可擋，認識 AI 勢在必行。

那不管你是什麼原因想要認識 AI：是正要開始認識 AI 的新手小白，或是突然接到一個 AI 專案導入並且要馬上有一番通盤概念的苦命工程師，都非常適合閱讀這本書。

我在拿到這本書閱讀完之後，發現對 AI 背後的語言模型（LLM）理解更加全面。 本書中前半的篇章淺顯易懂，讓完全的新手也能夠輕易讀懂難啃的專業術語；後面的篇章有很多實作的專案練習，而我作為一名本科仔，依然可以透過這些實作加深印象，也補足了離校至今的新知。 這本書不只可作為一本入門書籍，對於有相關專業的讀者也是一本不錯的複習選讀。 書中同時也穿插了許多情境式的帶入及豐富的額外小知識補充，再搭配詳細的文獻資料可供延伸學習以及深入解析。 相信這本書可以讓大家更容易地進入 AI 的世界，成為 LLM 大師指日可待。

花生大師是我們這屆參賽團隊「消波塊上的海洋貓貓」的組長，也是團隊中的指導者，扎實的程式基礎專業知識、持續學習的態度以及對教學的熱忱造就了現在的他。 能為花生大師寫推薦序著實於有榮焉，作為他的組員也備感榮幸。

林俊廷 JT Lin

iThome 鐵人賽戰友

NZXT 軟體工程師

花生大師追隨者

序言

2022 年 11 月 30 日，由 OpenAI 開發的 ChatGPT 悄然問世，短短數日便轟動全球，徹底顛覆了大家對於人工智慧的看法。身為 NLP 研發工程師的我也不例外，對這個迷人的聊天機器人深深著迷，從此開啟了語言模型的深度探索之旅。

本書引領各位讀者與我一同踏上這趟語言模型的深度探索之旅，從基本操作到應用實踐，從理論知識到實務開發，無論你是剛上路的初學者還是熟門熟路的開發者，相信都能在本書中吸取到許多有價值的內容。

這段旅程首先從學習如何操作 ChatGPT 開始，瞭解如何使用各種提示技巧來解決各式各樣的問題。身為開發者，不會僅滿足於使用而已，我們更想知道如何將這股神奇力量融入自己的應用之中。因此，我們開始探索 OpenAI API 的用法，並搭建一個有趣的塔羅牌應用，這不僅只是個有趣的實驗，更是一個語言模型融入生活的實際案例。

乘坐著 ChatGPT 的魔法飛毯，翱翔於人工智慧的浩瀚宇宙之間，開始思考如此強大的模型是如何誕生的呢？從最初只會文字接龍的 GPT 模型，到現在家喻戶曉的 AI 助理，我們追古溯今其發展歷程，深入淺出瞭解箇中奧秘。

從雲端踏回地面，不依賴 ChatGPT，我們如何玩轉語言模型？跟隨開源笑臉 Hugging Face 🤗 的腳步，體驗各式各樣的開源模型。雖然我們並不像那些大公司一樣擁有海量的高級顯卡，但是透過量化技術，也能讓小顯卡推動大模型！騎著 Meta 開源的 Llama 羊駝模型，高聲一呼「快，還要更快！」借助各種推論框架，將模型生成的速度推向極致。

在品茗過許多開源模型之後，激發了我們開始想要嘗試訓練模型的興趣。也許我們無法從頭訓練一個全新的模型，但是透過運用 LoRA 技術與各種訓練技巧，自己在家裡也能輕鬆打造出屬於自己的語言模型。

最終的終點並不代表結束，語言模型是一門深奧的學問，人工智慧則是一個更為廣泛的領域。在這個世界中，新的事物源源不絕的湧現，無論是新模型、新技術還是新框架。這本書的目標，不僅只是介紹語言模型的技術細節，更重要的是如何借助這樣的技術來提昇自我，並學會如何學習。在這個充滿無限可能的 AI 時代裡，讓我們勇敢探索，無懼前行！

閱讀方式

書中的範例程式碼皆整理到 GitHub 上：

全書範例程式碼

https://github.com/penut85420/LLM-Note-Labs

推薦各位讀者可以搭配閱讀，因為數量眾多，如果缺漏或錯誤，歡迎發 Issue 告知。

在書中有許多專欄引導各位讀者閱讀，例如：

主管 @tag 我

@penut 這個專欄是做什麼用的？

這個專欄負責傳達該章節的背景故事，為什麼要介紹這樣的技術？很多時候就是主管 tag 我一句話就開始研究這個技術了！

花生狂想曲

腦內風暴不斷運轉！這個專欄描述了許多筆者的經驗心得與主觀想法，有些時候也只是單純的玩笑話。

工欲善其事，必先利其器

這個專欄描述接下來的章節可能需要的背景知識或需要安裝的套件工具等等。

小心，樂高就在你腳邊

不小心踩到樂高那可是很痛的！這個專欄匯集了筆者的辛酸血淚，描述各種可能踩到的雷坑。

補充堅果，頭好壯壯

這個專欄會延伸當前主題，探討更深入的知識背景，如果能看完這個專欄一定能收穫滿滿！

希望以上專欄可以幫助各位讀者更快融入語言模型這門有趣的技術！

致謝

在此感謝當初陪我一起打鐵人賽的戰友們，沒有大家的互相鼓勵，我難以完成這樣壯舉。感謝包容我沒日沒夜趕稿寫文章的家人們，沒有你們的精神支持，我難以走到現在。

特別感謝伍宏麟老師、羅于貞老師以及林川傑教授，感謝你們一路指導我成長茁壯。感謝與我切磋砥礪知識的同學朋友們，沒有你們，沒有今天的我，由衷感謝。

感謝 iThome 評審團的肯定，使我出書的夢想得以實現。感謝博碩專業的編輯團隊，看到我的原稿被呈現的這麼精美，心中真的相當感動。

特別將這本書獻給天上的良葛格，在我學習程式語言的這條路上，您的教學筆記給了我許多幫助，您的精神永遠存在我的心中。

目錄

01 語言模型能吃嗎？

1.1 語言模型不能吃 1-2
1.2 語言模型的種類 1-4
1.2.1 N-Gram 模型 1-4
1.2.2 隱藏式馬可夫模型 1-5
1.2.3 循環神經網路 1-5
1.2.4 Transformer 1-6
1.3 Bigram 語言模型實作 1-6
1.3.1 爬取文本 1-7
1.3.2 統計建模 1-8
1.3.3 文本生成 1-10
1.4 結論 1-11

02 神經網路語言模型

2.1 文字向量 2-2
2.2 遞歸神經網路 2-3
2.3 Transformer 模型與注意力機制 2-5
2.4 Transformer 語言模型架構種類 2-6
2.4.1 Encoder 語言模型 2-7
2.4.2 Encoder-Decoder 語言模型 2-8
2.4.3 Decoder 語言模型 2-9
2.5 結論 2-10

03 ChatGPT

3.1 ChatGPT 狂熱 3-3
3.2 提示 Prompt 3-4
3.2.1 Zero-Shot & Few-Shot Prompt 3-4
3.2.2 Chain-of-Thought 3-6
3.2.3 System Prompt 3-8
3.3 ChatGPT Plus 3-10
3.3.1 GPT-4 3-10
3.3.2 My GPTs 3-12
3.3.3 Ai PDF 3-13
3.4 結論 3-14

04 OpenAI API

4.1 Token 4-2
4.2 Tiktoken 4-3
4.3 Pricing 4-5
4.4 ChatGPT API 4-6
4.4.1 認證 4-7
4.4.2 基礎用法 Non-Streaming 4-8
4.4.3 串流用法 Streaming 4-9
4.4.4 文字介面聊天範例 4-9
4.4.5 文字向量 API 4-10
4.4.6 Rate Limits 4-12
4.5 結論 4-13

05 AI 實戰應用：貓貓塔羅

5.1 Gradio 5-3
5.1.1 基本用法 gr.Blocks 5-3
5.1.2 圖片元件 gr.Image 5-5
5.1.3 聊天室元件 gr.Chat..... 5-5
5.2 貓貓塔羅.....5-8
5.2.1 素材蒐集..... 5-9
5.2.2 核心應用..... 5-9
5.2.3 完整應用..... 5-12
5.3 結論 5-13

06 ChatGPT 的挑戰者們

6.1 土豆挑戰 Peanut Challenge 6-2
6.2 Google Gemini 6-4
6.3 Anthropic Claude 6-6
6.4 Microsoft Copilot.....6-7
6.5 訊飛星火.....6-8
6.6 BreeXe.....6-9
6.7 結論 6-10

07 ChatGPT 相關應用

7.1 ChatPDF.....7-2
7.2 CloudGPT.....7-4
7.3 Gogolook 袋鼠金融 7-5
7.4 Hotoke 佛祖 AI 7-6

7.5 AutoGPT & AgentGPT 7-7
7.6 OSENSE Qubby AI 7-9
7.7 薯條塔羅 7-10
7.8 結論 7-12

08 檢索與生成

8.1 生成式資訊檢索 8-2
8.1.1 封閉生成式資訊檢索 8-3
8.1.2 開放生成式資訊檢索 8-3
8.2 論文閱讀問答機器人實做 8-4
8.2.1 索引階段 Index Phase 8-5
8.2.2 查詢階段 Query Phase 8-10
8.2.3 分析 8-13
8.3 結論 8-14

09 走訪 LLM 訓練流程

9.1 監督式微調 9-2
9.1.1 預訓練 9-2
9.1.2 指令微調 9-3
9.2 增強式學習 9-5
9.2.1 獎勵模型 9-6
9.2.2 近端策略最佳化 PPO 9-6
9.2.3 直接偏好最佳化 DPO 9-7
9.2.4 增強式學習框架 TRL 9-8
9.3 結論 9-8

10 建立開發環境

10.1 作業系統 10-2
10.2 NVIDIA 顯示卡驅動程式 10-2
10.3 虛擬環境管理 Conda 10-5
10.3.1 安裝 Miniconda 10-5
10.3.2 Conda 基本操作 10-6
10.4 常用套件安裝 10-8
10.4.1 CMake 10-8
10.4.2 CUDA 10-8
10.5 Docker 10-12
10.6 Colab 10-14
10.7 GPU 10-16
10.8 結論 10-18

11 擁抱開源的微笑 Hugging Face Transformers

11.1 讀取模型 11-3
11.2 讀取分詞器 11-8
11.3 自動類別 11-9
11.4 文本生成 11-11
11.5 串流輸出 11-14
11.6 提示樣板 11-15
11.7 取樣參數 11-17
11.8 設定停止點 11-19
11.9 自迴歸解碼 11-21

11.10 多顯卡推論 11-23
11.11 到底是在「大」什麼？ 11-24
11.12 結論 11-28

12 數若繁星的語言模型們

12.1 英文模型 12-2
12.1.1 Llama 12-2
12.1.2 Alpaca 12-4
12.1.3 Vicuna 12-5
12.1.4 TinyLlama 12-6
12.1.5 Phi 12-7
12.1.6 Mistral & Mixtral 12-7
12.2 中文模型 12-9
12.2.1 Qwen 12-9
12.2.2 Yi 12-9
12.3 臺灣模型 12-9
12.3.1 Breeze & BreeXe 12-10
12.3.2 Taiwan LLM 12-10
12.3.3 TAIDE 12-11
12.3.4 Bailong 12-11
12.3.5 FFM 12-12
12.4 其他資源 12-12
12.4.1 Awesome LLM 12-12
12.4.2 Open LLMs 12-12
12.4.3 Open LLM Leaderboard 12-13
12.5 結論 12-13

13 語言模型也懂寫程式！

13.1 StarCoder 13-2
13.2 CodeGen 13-7
13.3 CodeLlama 13-8
13.4 DeepSeek Coder 13-9
13.5 結論 13-9

14 貧民英雄，量化超人

14.1 基礎概念 14-2
14.1.1 說文解字 14-2
14.1.2 簡單推論 14-3
14.2 量化方法 14-7
14.2.1 BitsAndBytes 14-7
14.2.2 Quanto 14-9
14.2.3 GPTQ 14-10
14.2.4 AWQ 14-13
14.3 困惑度 14-16
14.4 結論 14-21

15 ggml & llama.cpp

15.1 ggml 15-2
15.2 llama.cpp 15-3
15.3 環境建置 15-4
15.4 格式轉換 15-5

15.5 主程式 llama-cli 15-6
15.6 量化工具 llama-quantize 15-8
15.7 伺服器 llama-server 15-10
15.8 Python Binding 15-12
15.9 速度比較 15-13
15.10 結論 15-16

16 Text Generation Inference

16.1 架設服務 16-2
16.2 基本用法 16-4
16.3 量化選項 16-6
16.4 Token 數量設定 16-7
16.5 TGI Client 16-9
16.6 速度測試 16-11
16.7 結論 16-12

17 vLLM & Paged Attention

17.1 Key-Values Cache 17-2
17.2 Paged Attention 17-3
17.3 基本用法 17-4
17.4 LLM Service 17-6
17.5 速度比較 17-9
17.6 結論 17-9

18 情境學習 In-Context Learning

18.1 起源 18-2
18.2 基本 18-3
18.3 翻譯應用 18-5
18.4 文字正規化應用 18-6
18.5 結論 18-7

19 再訪檢索模型

19.1 關鍵字 19-2
19.2 BM25 Best Matching Ranking 19-5
19.3 文本向量 19-6
19.4 Text Embedding Inference 19-9
19.5 相似度 19-10
19.6 Faiss 19-11
19.7 結論 19-13

20 任務導向聊天機器人

20.1 任務導向到底是什麼呢？ 20-3
20.2 有了 LLM 的 TOD 系統天下無敵？ 20-5
20.3 Rasa 20-7
20.4 結論 20-12

21 LangChain

21.1 題目發想 21-3
21.2 資料來源 21-3

21.3 資料爬取.....21-4
21.4 LLM.....21-7
21.5 Prompt Template.....21-8
21.6 Embedding Model.....21-10
21.7 Vector Store.....21-11
21.8 Few-Shot Prompt Template.....21-13
21.9 Example Selector.....21-14
21.10 Chain!.....21-15
21.11 結論.....21-17

22 微調語言模型 DIY ！

22.1 任務設計.....22-2
22.2 建立資料集.....22-4
22.3 訓練流程.....22-11
22.4 評估測試.....22-16
22.5 結論.....22-20

23 PEFT & LoRA

23.1 LoRA 簡介.....23-2
23.2 LoRA 原理.....23-4
23.2.1 FFT 運算.....23-5
23.2.2 LoRA 運算.....23-6
23.2.3 合併運算.....23-6
23.3 回顧 LoRA 程式碼.....23-7
23.4 LoRA 推論.....23-9
23.5 QLoRA.....23-10

23.6 權重合併 23-11
23.7 訓練細節 23-13
23.8 結論 23-14

24 永遠不夠長！

24.1 ChatGPT & Claude 24-2
24.2 Gradient Checkpointing 24-3
24.3 內插法 & 外插法 24-6
24.4 LongLoRA 24-9
24.5 StreamingLLM & Attention Sinks 24-10
24.6 結論 24-12

25 資料何其多，硬碟何其小

25.1 Hugging Face Datasets 25-2
25.2 預訓練資料集 25-6
25.2.1 Common Crawl 25-6
25.2.2 Project Gutenberg 25-6
25.2.3 The Pile 25-6
25.2.4 C4 & mC4 25-7
25.2.5 Stack Exchange 25-7
25.2.6 RedPajama & SlimPajama 25-7
25.2.7 The Stack & StarCoderData 25-8
25.3 對話資料集 25-9
25.3.1 Databricks Dolly 25-9
25.3.2 OASST 25-9
25.3.3 ShareGPT 25-9

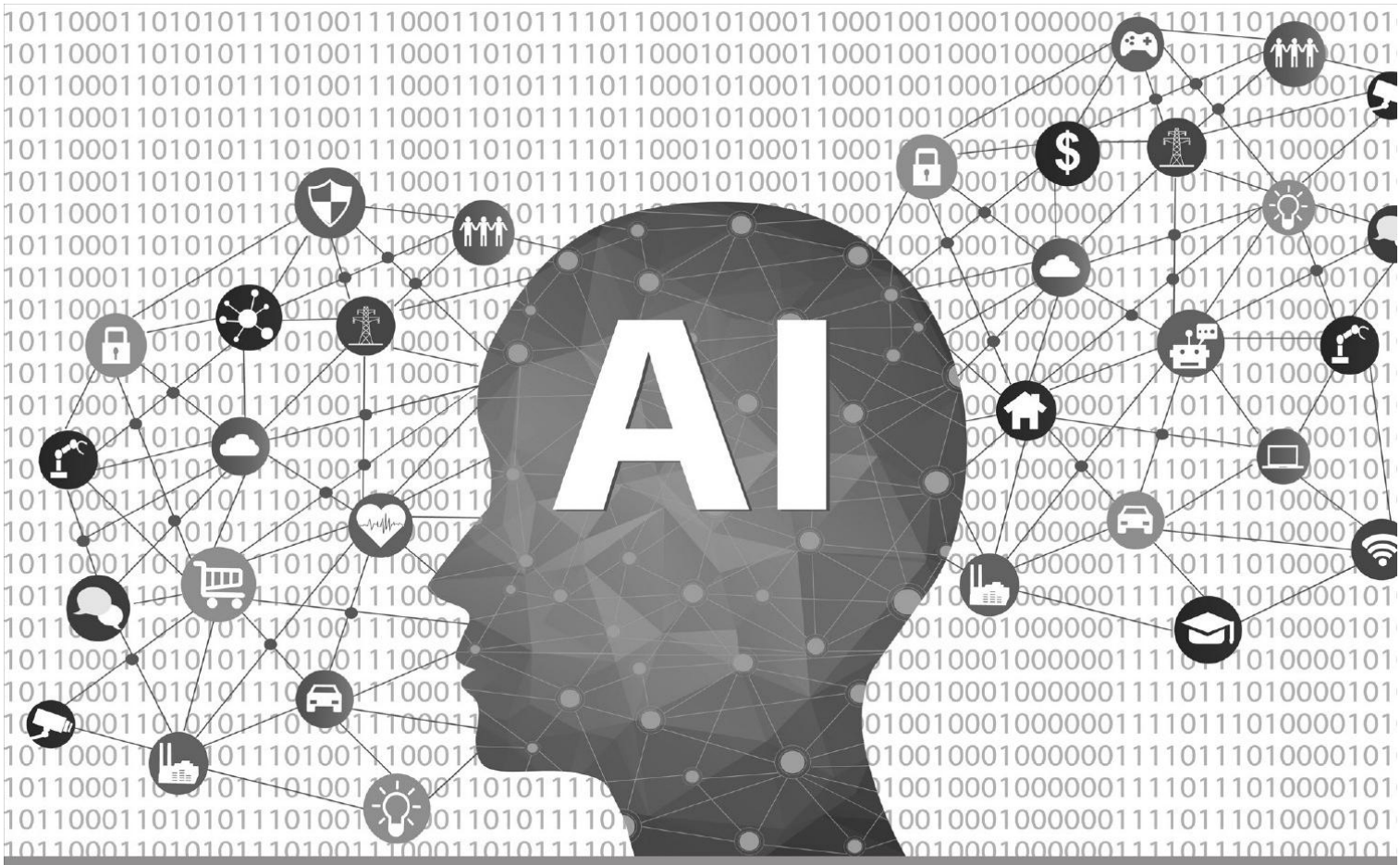
25.3.4 LMSYS Chat 25-10
25.4 中文資料來源 25-10
25.4.1 政府資料開放平台 25-10
25.4.2 維基文庫 25-10
25.4.3 開放文學 25-11
25.4.4 漢籍全文資料庫 25-11
25.4.5 TAIDE 25-11
25.5 評估資料集 25-11
25.6 模型生成資料 25-14
25.7 結論 25-15

26 授權條款與社會責任

26.1 什麼是「權」？ 26-2
26.2 常見授權條款 26-4
26.2.1 MIT 26-4
26.2.2 GPL & LGPL 26-4
26.2.3 Apache 2.0 26-5
26.2.4 BSD 26-5
26.2.5 CC 26-6
26.3 特殊授權條款 26-6
26.3.1 OpenRAIL 26-6
26.3.2 Llama2 License 26-7
26.4 相關爭議 26-8
26.5 結論 26-9

27 學海無涯，學無止境

27.1 論文 27-2
27.2 GitHub 趨勢 27-3
27.3 Hugging Face 27-3
27.3.1 Blog 27-3
27.3.2 Trending 27-3
27.3.3 Daily Paper 27-4
27.4 iThome 27-5
27.5 與人互動 27-5
27.6 未來發展 27-6



Chapter

01

語言模型能吃嗎？

主管 @tag 我

@penut 最近很紅的那個 LLM 能不能介紹一下？

沒問題，我馬上叫 ChatGPT 自我介紹一下！

ChatGPT 一夕之間爆紅，大型語言模型成為了家喻戶曉的熱門話題，相信捧起這本書的各位也不例外。筆者原先就在 NLP 部門擔任研發工程師，趁著這波 LLM 的熱潮，我們部門也毫不猶豫的投身其中，因此有幸近距離感受這塊當紅炸子雞的魅力。現在就讓我們來一窺這份生成式的浪漫吧！

1.1 語言模型不能吃

語言模型（Language Model）是一種用來描述文字分佈的統計機率模型，廣義來說，任何透過**統計文字機率**來處理自然語言問題的方法，都可以算是一種語言模型。最基本的定義，是**計算某句話出現的機率**。

為什麼計算一句話的機率這麼重要呢？以傳統的語音辨識模型來說，通常會先由「聲學模型」負責辨識人類的**發音**，因此一開始模型只會輸出一些「阿依嗚欸喔」之類的標音，最後再由「語言模型」來判斷這些發音最有可能組合成什麼樣的句子。讓我們來看看這個例子：

1. 再給我兩份蔥，讓我把記憶煎成餅
2. 再給我兩分鐘，讓我把記憶結成冰[1]

1　原句出自周杰倫《最長的電影》歌詞。

語言模型可以計算這兩句話出現的機率，用來判斷哪句話更有可能是使用者實際想說的話。然而根據統計的領域不同，語言模型也有可能產生不同的計算結果。假設我們統計的來源是一個蔥油餅店的老闆，那可能第一句話的機率比較高。如果統計的來源變成一位歌手，那第二句話的機率可能就會更高一點。

語言模型也可以用來預測下一個字詞出現的機率，例如看見一句話的開頭是「今天天氣真 ...」，就能夠透過語言模型來預測接下來可能會是「真好」、「真棒」、「真壞」還是「真差」之類的。而越是厲害的語言模型，越能夠根據上下文透露的資訊來更精準的預測接下來可能出現的字。

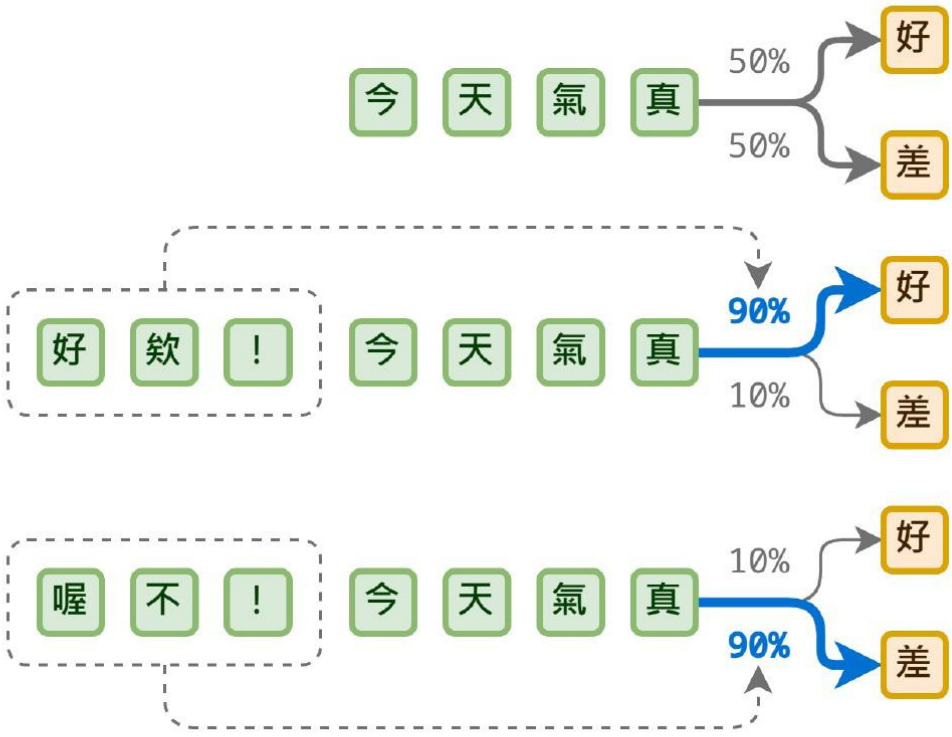


圖 1-1　語言模型根據上下文產生不同的預測

1.2 語言模型的種類

語言模型根據發展的脈絡，大致上可以分為傳統的統計式語言模型，例如 N-Gram 或 HMM 隱藏式馬可夫模型等，以及現代的神經網路語言模型，例如 RNN 模型與最主流的 Transformer 模型。

1.2.1 N-Gram 模型

N-Gram 模型是一種相當單純的統計模型，主要透過統計連續字詞的頻率來建立機率模型，其中 Unigram、Bigram 和 Trigram 是 N-Gram 模型中最常見的具體例子。

Unigram 是 N-Gram 模型中最簡單的一種，將每個字詞當作一個獨立的元素。一個典型 Unigram 模型的統計結果可能如下：

```
{
  "今天": 0.35,
  "天氣": 0.25,
  "心情": 0.20,
  "真好": 0.15,
  "真差": 0.05,
}
```

Bigram 則是將兩個連續字詞當作一個元素，描述了在給定一個字詞情況下，下一個可能出現的字詞的機率分佈。這樣不僅能捕捉字詞的分佈機率，也比較能夠捕捉語法結構。一個典型的 Bigram 模型統計結果可能如下：

```
{
  "今天": { "天氣": 0.5, "心情": 0.5 },
  "天氣": { "真好": 0.7, "真差": 0.3 },
  "心情": { "真好": 0.8, "真差": 0.2 },
}
```

Trigram 模型則是將三個連續字詞作為一個單位來分析，以獲取更豐富的文本資訊。越大的 N-Gram 可以捕捉越完整的語言資訊，但伴隨而來的是更為龐大的記憶體消耗。

綜合以上所述，Unigram、Bigram 和 Trigram 模型是 NLP 領域裡面經常用於文本分析的方法，這些常見的方法很適合一些基本的應用，例如文字邊界偵測。但 N-Gram 模型存在嚴重的稀疏問題與上下文捕捉能力不好等等，因此能夠實際應用的場景並不多。

1.2.2 隱藏式馬可夫模型

隱藏式馬可夫模型（Hidden Markov Model, HMM）與一般的 N-Gram 模型不同的地方在於，N-Gram 模型只會考慮詞與詞之間表面上的關聯，但 HMM 會更進一步的去學習隱藏狀態如何進行轉移。在語言模型裡面，隱藏狀態指的可能是詞性、句型、語法結構等等，不容易從表面上觀察到的資訊。而 HMM 可以從訓練資料中去捕捉到這些不易觀察到的語言特性。

1.2.3 循環神經網路

循環神經網路（Recurrent Neural Network, RNN）開啟了語言模型踏入深度學習領域的先河，著名的 LSTM 與 GRU 等都是 RNN 的一種。循環神經網路的運作方式為：給定一段輸入序列，神經網路會從頭到尾一個一個看，每看過一個元素就更新一次隱藏狀態。也有雙向 RNN 的變體，就是從頭看到尾看一遍之後，會再從尾到頭看上第二遍。使用 RNN 建立語言模型的效果比起傳統 N-Gram 與 HMM 都好上不少。然而 RNN 的計算成本非常高昂，且平行運算不易，有訓練效率不佳的問題。

1.2.4 Transformer

2017 年 Google 在 Attention Is All You Need[2] 論文中提出 Transformer 架構進行機器翻譯的改善，揭開了 Transformer 架構主宰 NLP 領域的序幕。

> 花生狂想曲
> 從此以後，以這篇論文作者為首的 Vaswani et al., 2017 文獻引用便深深烙印在每個 NLP 研究者的腦海裡（誤）。

Transformer 裡非常重要的 Attention 機制，在 Bahdanau et al., 2014[3] 和 Luong et al., 2015[4] 時就已經被提出來探討過了，因此 Attention 的算法也分成 Bahdanau Attention 和 Luong Attention 兩種。在下個章節會更深入探討 Transformer 架構。

接著 2018 年來自 Google 的 Devlin 等人提出 BERT[5] 模型，在 NLP 領域引起了相當大的迴響，此後自然語言相關的任務皆不斷向上突破，模型大小蒸蒸日上，即便到了 2020 年 OpenAI 都已經提出了 1750 億參數量的超大型語言模型 GPT-3[6]，各家研究單位的模型參數量依然還在不斷攀升。

1.3 Bigram 語言模型實作

看到這邊，大家的腦中有沒有浮現一種「其實單純的語言模型沒有很複雜嘛」的想法呢？語言模型乍看之下是一個很理論性質的存在，但細思其本質，就是一種統計建模的手法，實際上並沒有那麼遙不可及！

2 https://arxiv.org/abs/1706.03762
3 https://arxiv.org/abs/1409.0473
4 https://arxiv.org/abs/1508.04025
5 https://arxiv.org/abs/1810.04805
6 https://arxiv.org/abs/2005.14165

所以這個小節就來嘗試實做一個小型的 Bigram 模型，在此之前需要一份訓練用的簡單文本。這裡推薦**維基文庫**[7] 或**開放文學**[8]，裡面有許多自由內容的中文文本可以使用，比較沒有版權問題，這次的實驗使用開放文學的「三國演義」做為目標文本，這種純文本的資料集一般稱呼為 **Corpus 語料庫**。

Bigram 語言模型完整程式碼
https://tinyurl.com/llm-note-01

1.3.1 爬取文本

在瀏覽器裡面按下 `F12` 開啟開發人員工具，觀察開放文學的網站運作流程，發現網頁主要透過 POST Request 到 `http://open-lit.com/GETbook.php` 這段網址來取得文本資料，並以參數 `bid` 代表哪本書，參數 `id` 代表那本書的第幾回。

工欲善其事，必先利其器 🛠️

這部份的實做會涉及到一點 HTML 與 Python 爬蟲的相關知識，例如 `requests` 與 `beautifulsoup` 套件的運用，還不熟悉的讀者不妨先去瞭解一下！

7 https://zh.wikisource.org/zh-hant/

8 http://open-lit.com/

經過一些觀察後發現，三國演義總計 120 回的 ID 分佈在 67~186 之間，且回傳的內容只有文本本身，因此對爬蟲程式非常友善，所以撰寫爬蟲程式如下：

```
import requests
from bs4 import BeautifulSoup as BS
from concurrent.futures import ThreadPoolExecutor

def download(pid):
    url = "http://open-lit.com/GETbook.php"
    data = {"bid": 12, "id": pid}
    res = requests.post(url, data=data)
    res = BS(res.text)
    return res.text

target = list(range(67, 186 + 1))
with ThreadPoolExecutor(16) as tpe:
    corpus = [text for text in tpe.map(download, target)]
```

透過 **concurrent**[9] 套件進行同時下載，能夠大幅減少等待下載的時間，但同時也要注意流量控制，不要發出太多下載要求而造成對方網站的負擔！

> **小心，樂高就在你腳邊**
>
> `id` 是 Python 的內建函式，所以盡量不要直接使用 `id` 當作變數名稱，不然會把這個內建函式給覆蓋掉喔！

1.3.2 統計建模

透過以下程式碼進行 Bigram 統計建模：

9 https://docs.python.org/3/library/concurrent.futures.html

```
from collections import Counter, defaultdict

# \u3000 為全形空白
skip_symbols = ["\u3000", "\r", "\t", "\n"]
bigram = defaultdict(Counter)
for data in corpus:
    # 將空白字元移除
    for symbol in skip_symbols:
        data = data.replace(symbol, "")

    # 以 <BOS> 代表文本開頭，開始統計整份文本
    prev_char = "<BOS>"
    for ch in data:
        bigram[prev_char][ch] += 1
        prev_char = ch

    # 以 <EOS> 代表文本結尾
    ch = "<EOS>"
    bigram[prev_char][ch] += 1
```

這段程式碼基本上就是在統計目前這個字，他的下一個字是什麼，又出現了幾次。常見的語言模型通常會將這種資訊換算成浮點數機率，但因為這裡是 Bigram 文本生成模型，所以只需要知道下個詞的權重分佈比例即可。

補充堅果，頭好壯壯

如果是轉換成機率的模型，容易因為浮點數除法的關係，導致每個詞的機率都是一個非常非常小的浮點數。因此，這類模型都會對機率取 Log 對數，當語言模型在進行機率相乘時，只需要將對數相加即可。

在處理文本時，也一併將空格移除，這是因為我很確信三國演義只會出現中文字。倘若訓練的文本是中英夾雜時，必須特別注意空格的處理。我在文本裡面觀察到大量排版用的空格，於是將空格移除，來避免語言模型學到過於偏頗的機率分佈。

1.3.3 文本生成

透過以下程式碼進行文本生成：

```
import random

generate = str()
curr_ch = "<BOS>"
while curr_ch != "<EOS>":
    generate += curr_ch
    prob = bigram[curr_ch]
    elements = [k for k in prob]
    weights = [prob[k] for k in prob]
    curr_ch = random.choices(elements, weights=weights, k=1)[0]
print(generate + curr_ch)
```

只需要單純的根據模型權重進行取樣，然後將文字拼接在一起，就能生成一份文本囉！以下節錄一段生成結果：

> <BOS>第五百姓疲乏。」問時魏皇叔季玉龍鳳雛妙算。布陣。舊抵安樂進。臣才和，關津，引五十騎將愕然歸本大功可防。若興兵三路。五路，皆曰：「如此機會稽妖，斷機密丞相爭戰，尚多有怠；又恐國將軍器，復至別宮室，受君耶？」言解酒畢，忽然後苑中，乃天下奇功，忽然人協，事。」

乍看之下似乎有點樣子，反正文言文其實也看不是很懂。但只要稍微認真看一下，就會發現很多問題：

1. 文本的內容其實根本言之無物。
2. 上下括號經常不會成對出現。
3. 若有多試幾次就會發現，在 <BOS> 後面接的必定是「第」。

在這種 Bigram 生成模型裡面，每兩個字之間的組合可能還合理，但將它們串聯在一起後，整體文本就顯得不成章法。這就是為什麼會說 Bigram 模型捕捉上下文能力很薄弱的原因。模型僅能考慮前一個字，也因此造成了上下括號無法成對的問題，因為模型根本沒有理解句法結構的能力。

那為什麼 `<BOS>` 後面必定接「第」呢？主要是因為程式在統計文本時，每個回目之前都會加上一個 `<BOS>` 標記。因為三國演義的每個回目開頭必定是「第 XXX 回」的關係，所以在機率分佈上 `<BOS>` 後面必定是「第」，這也顯現了語言模型在這份資料上產生了嚴重的**偏見（Bias）**而造成的重複性。

1.4 結論

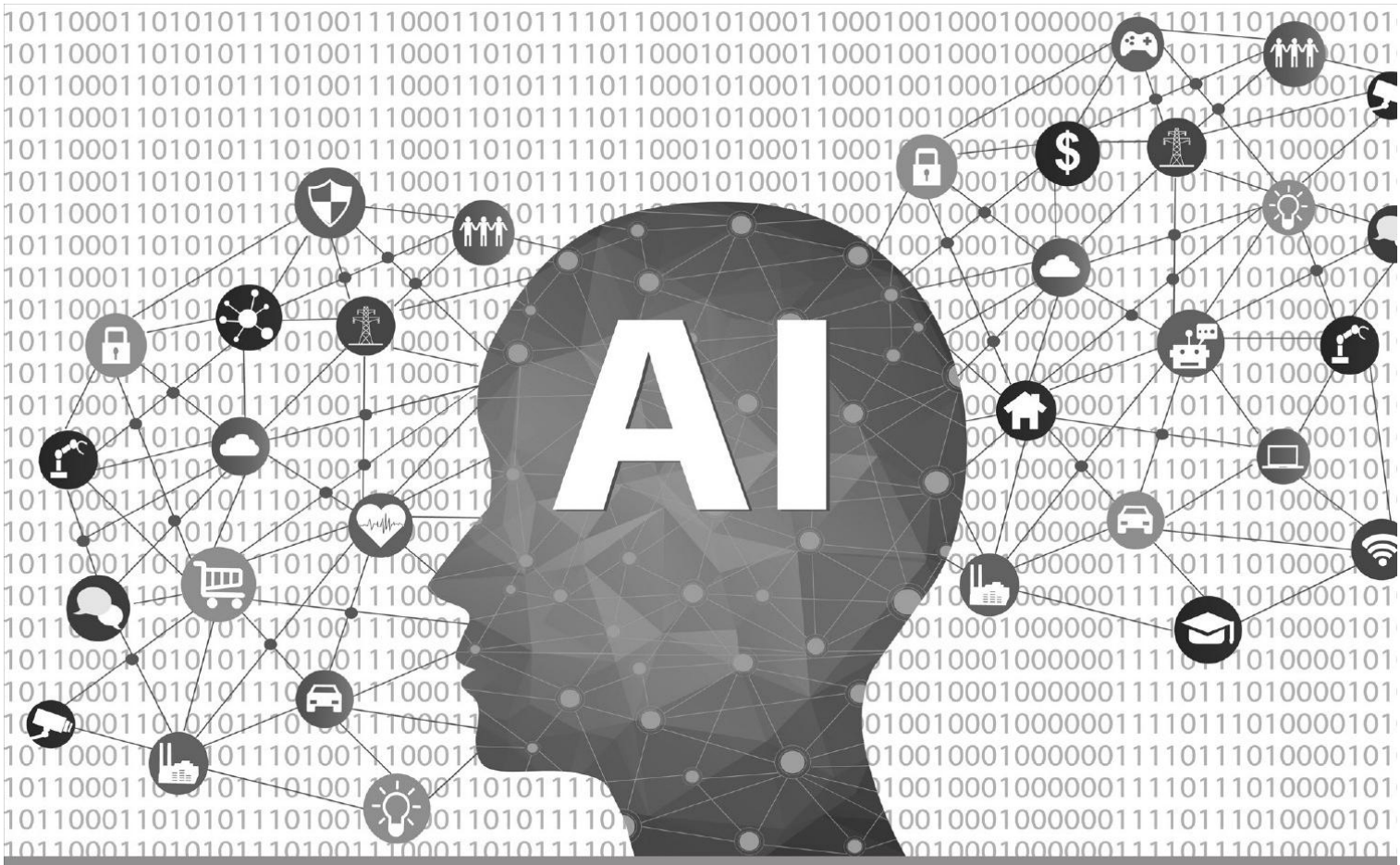
在動手實做一個簡單的 Bigram 模型實驗後，可以瞭解到許多訓練語言模型時值得學習的觀點與必須考量的問題。這些觀點即使到了千億參數級的大型語言模型，依然是很重要的觀念。

以上只是個非常基礎的 Bigram 模型實做，實際上機率模型在運作時還有很多細節需要處理，例如遇到未知詞時，會需要做 Fallback 之類的。雖然 Bigram 模型的生成效果並不理想，但筆者覺得其實滿有趣的。這些「有趣但是沒用」的遊戲性實驗，可以激發研究者的好奇心，也是不斷推動理論進步的基石。

本書規劃了許多「有趣但是沒用」的小型應用，希望可以帶各位讀者見微知著，一同落實大處著眼，小處著手！

> 花生狂想曲
> 其實真相是筆者也沒有很強的機器做什麼大型實驗 QQ

Note



Chapter

02

神經網路語言模型

主管 @tag 我

@penut 我們給客戶的 Solution 一定要用到 NN ！

沒問題，我馬上拉一個 LSTM 出來！

現在誰還在用 LSTM 啊！

根據文本統計來捕捉語言結構，進而建立一個機率模型，廣義而言就可以被稱作一種語言模型。本章節主要介紹透過**神經網路（Neural Network）**架構的語言模型，以及常見的 Transformer 語言模型種類。

2.1 文字向量

文字向量（Word Embedding）是一種將**離散文字**轉換成**實數向量**的模型，通常也是透過大量文本統計學習出來的，因此也能算是一種廣義上的語言模型。將文字轉為向量後，才能夠使用各種數學運算來進行機器學習。

補充堅果，頭好壯壯

在 PyTorch 或 Tensorflow 中常見的 Embedding Layer 就是用來訓練文字向量的類別，能將 Embedding Layer 視為一種把**文字變成數字**的操作。

在完整的語言模型架構裡面，文字向量扮演了很重要的角色，建構向量的方式也有很多種，傳統上有 **Word2Vec**[1] 的 Skip-Gram 與 CBOW，以及 **GloVe**[2] 和 **fastText**[3] 等等。而目前主流的方法則是在 Transformer 架構前面加上一個 Embedding Layer 並透過大量文本訓練而來。

基本的文字向量本身通常是**上下文無關**的，這是一個相當重要的特性。也就是說這個詞的前後是什麼並不重要，模型出來的每個文字向量都是固定的。因此在訓練一份文字向量模型時，其實就是在訓練**每個字詞的獨立語意**。

這些獨立語意的向量，透過特定方法（例如總和平均、Transformer 模型運算）的處理，原本離散的語意向量得以融入上下文資訊。這種充分考慮上下文語境的向量，稱之為**文句向量（Sentence Embedding）**。這種向量在資訊檢索領域特別有用，例如建立搜尋引擎時，就常常會用到文句向量，也是支撐現在生成式 IR 的核心技術之一，這部份會在未來的章節提及。

2.2 遞歸神經網路

遞歸神經網路（Recurrent Neural Network, RNN）是一種典型序列模型的代稱，被認為是用來處理具有**時間相依性**資料相當有效的模型，例如股市或氣象等領域。在深度學習剛開始盛行的時候，就很常見到使用 RNN 來處理自然語言的任務，在建立模型時將每個字詞都視為一個時間單位來處理。

1 https://w.wiki/7TzJ

2 https://nlp.stanford.edu/projects/glove/

3 https://fasttext.cc/

常見的 RNN 包含 LSTM 與 GRU 等架構，他們通常會依序拜訪時間序列上的每個元素，並在每個時間點決定要「**記憶**」或「**遺忘**」某些前面的資訊，並不斷往下累積下去。這樣的想法相當直觀，但如此一來，前面的元素顯然參照不到什麼後面的資訊。於是透過**雙向（Bidirectional）**的技巧，不只從頭到尾拜訪一次，也從尾到頭拜訪一次，使模型對整個序列有更充足的理解。

這樣的模型也有不少缺點，其一是容易梯度爆炸或消失，因此要將 RNN 疊得很深很多層，其難度是很高的。雖然 LSTM 與 GRU 能夠減緩梯度的問題，但因為時間相依的特性，導致它很難進行平行運算，因此在訓練上有很大的瓶頸。尤其是 Transformer 模型出現之後，在 NLP 領域就越來越少見到 RNN 相關的研究了。

然而 RNN 的做法並沒有因此消滅殆盡，近期的 **RWKV**[4]、**Mamba**[5] 與 **Jamba**[6] 等模型，就是基於 RNN 的概念，並與 Transformer 架構做結合。這樣建構模型的做法不僅保留了兩者的優點，例如更好的上下文理解能力與無限的文本長度等，同時也減少了傳統 Transformer 帶來的超膨脹記憶體消耗。

筆者狂想曲

然而這些架構是否能在語言模型領域穩固其地位，目前尚難下定論。像是 GPT 架構其實也是經過了很長一段時間的發展，才能在現在發光發熱，而且這些以 RNN 概念為基礎的模型確實解決了許多 Transformer 的痛點，因此筆者認為這些都是值得持續關注的架構。

4 https://github.com/BlinkDL/RWKV-LM
5 https://arxiv.org/abs/2312.00752
6 https://arxiv.org/abs/2403.19887

2.3 Transformer 模型與注意力機制

Transformer 模型是現在語言模型的主流架構，這個架構乍看之下似乎很複雜，但筆者喜歡將 Transformer 比喻為**把 Linear Layers 疊得很華麗的模型**。會這樣說是因為構成 Transformer 模型裡面很重要的**注意力機制（Attention）**，其實本質上就是把好幾個 Linear Layers 拿來互相算來算去。

> 花生狂想曲
> 而 Transformer 就是透過一堆 Attention Layers 疊起來的，所以才說 Transformer 架構就是一個疊得比較華麗的 Linear 模型。

若將注意力機制比喻成大家所熟悉的 Python 字典 `dict` 類別，所有的資訊都被存成**鍵值對應（Key Value Pairs）**來建構字典。在使用這份字典時，會根據使用者輸入的**查詢（Query）**來尋找字典中是否存在相同的**鍵（Key）**，進而獲取對應的**值（Value）**是什麼，這是一個非常直觀的查表法。

但是 Python 的字典要求**查詢**與**鍵**必須完全相符合，然而在注意力機制裡面，**查詢與鍵是否相符，是透過模型訓練學得的！**

> 花生狂想曲
> 因此我們可以將注意力機制視為一種**比較華麗的查表法**，在 Transformer 裡面，所有序列上的元素就是在瘋狂互相查表。

在 Python 字典裡面，一個**查詢**只會找到一個**值**。但是在注意力機制裡面，**一個查詢會可能會找到一堆值**，而且這些**值**又會有各自的**權重**，這些權重分佈便代表了**這個查詢需要放多少注意力在這個值上面**，注意力機制這個名稱也是因此而來。

這些值的存在目的，其實就是為了讓模型去**理解上下文對語境的影響**。以翻譯任務為例，假設查詢只關注到 `cats` 時，可能只會翻譯成「貓」這個詞。但如果查詢能關注到 `It rains cats and dogs` 整句話時，就會得到「下大雨」的翻譯。

> **花生狂想曲** 💡
>
> 這是筆者對於注意力機制與 Transformer 架構比較通俗的比喻，然而 Transformer 本身其實**並非真的是單純的線性模型**。實際上 Transformer 架構涉及了很多複雜的機制與設計，例如不同的激勵函數、正規化與位置向量等等。這些機制不僅提昇了應用階段的效果，也增加了訓練階段的效率。若想要更進一步瞭解，非常推薦各位讀者一定要看看 **Attention Is All You Need**[7] 這篇論文！

雖然 Transformer 模型在文字領域上達到很好的效果，但其架構也帶來相當巨量的顯示卡記憶體消耗。除了會與批次大小成線性成長以外，也因為注意力機制會計算出一個二維注意力矩陣的關係，所以隨著模型輸入的長度越長，記憶體消耗呈現平方性的成長。因此如何克服模型輸入長度與快速膨脹的記憶體消耗，是建構 Transformer 語言模型相當重要的課題之一。

2.4 Transformer 語言模型架構種類

談到 Transformer 語言模型時，大家經常會將其分為以下三類：

1. 純編碼模型（Encoder-Only、AutoEncoder）
2. 解編碼混合模型（Encoder-Decoder）
3. 純解碼模型（Decoder-Only、AutoRegressive）

7 https://arxiv.org/abs/1706.03762

這是一個較為粗略的分類，也有許多研究在挑戰跳脫以上三種架構的語言模型，而現在主流常用的 GPT 或 Llama 架構，主要都是 Decoder-Only 的類型。接下來介紹這三類語言模型架構上的不同，與實際的應用場景。

2.4.1 Encoder 語言模型

Encoder 語言模型專注於分類型的任務，例如情感分類、意圖偵測或實體辨識等等。其語言建模方式主要為**遮罩語言建模（Masked Language Modeling, MLM）**，通常使用雙向注意力架構，因此能對上下文進行足夠充分的理解並進行分類。在許多自然語言理解任務上，這類模型都有相當不錯的效果。以現今的硬體效能而言，也是非常有效率的分類器。

> **花生狂想曲**
>
> 可以將 Encoder 語言模型視為一種**分類專家**！

遮罩語言建模會隨機將文本中的幾個字**遮起來**，並在訓練過程中要求模型預測該文字為何。這樣的訓練手法相當直觀且簡單，其優點之一就是可以達到非監督式訓練的效果。只要寫個程式隨機把幾個字遮起來就好，完全不需要人工標記，就能讓模型捕捉到許多語言特性。

> **補充堅果，頭好壯壯**
>
> 其他 Encoder 的建模手法還包含 Next Sentence Prediction 與 Sentence Order Prediction 等等，推薦有興趣的讀者從 BERT[8] 及 ALBERT[9] 的論文開始看起。

8 https://arxiv.org/abs/1810.04805

9 https://arxiv.org/abs/1909.11942

2.4.2 Encoder-Decoder 語言模型

Encoder-Decoder 是一種序列到序列（**Sequence-To-Sequence, Seq2Seq**）的模型架構，不僅利用了 Encoder 能夠充分理解上下文的優點，也結合了擅長進行文本生成的 Decoder 而形成的架構，經常用於機器翻譯、自動摘要等任務上。

以 Google 翻譯為例，左邊的原文會輸入到 Encoder 裡面，讓模型對整份原文有了充分的理解之後，再由右邊的 Decoder 來輸出譯文。

圖 2-1　以 Google 翻譯為例的 Encoder-Decoder 架構

花生狂想曲

因此我們可以將 Encoder-Decoder 語言模型視為一種**翻譯摘要專家**。

補充堅果，頭好壯壯

著名的 Encoder-Decoder 模型有 Google 的 **T5**[10] 與 Facebook（現在叫 Meta）的 **BART**[11] 等。

10　https://arxiv.org/abs/1910.10683
11　https://arxiv.org/abs/1910.13461

2.4.3 Decoder 語言模型

Decoder 語言模型是一種專注於文本生成的架構，最知名的 ChatGPT 與 Llama 就是使用這種架構。這類模型通常使用**因果建模（Causal Language Modeling, CLM）**的方法來訓練。

> **花生狂想曲**
> 請注意 CLM 的 C 指的是**因果（Causal）**而不是**隨意（Casual）**的意思，筆者直到現在都還是很常拼錯 但也因為 CLM 實在是太單純了，單純到讓人覺得很隨意（？）才會容易拼錯（藉口！）。

因果建模在訓練期間，會要求模型去預測下一個字詞是什麼，藉此學習訓練文本中的語言特性。這種建模方法也是一種非監督式學習，僅需要大量文本即可訓練。

> **補充堅果，頭好壯壯**
> 因果建模有時也被稱為 **Next Token Prediction（NSP）**建模法！

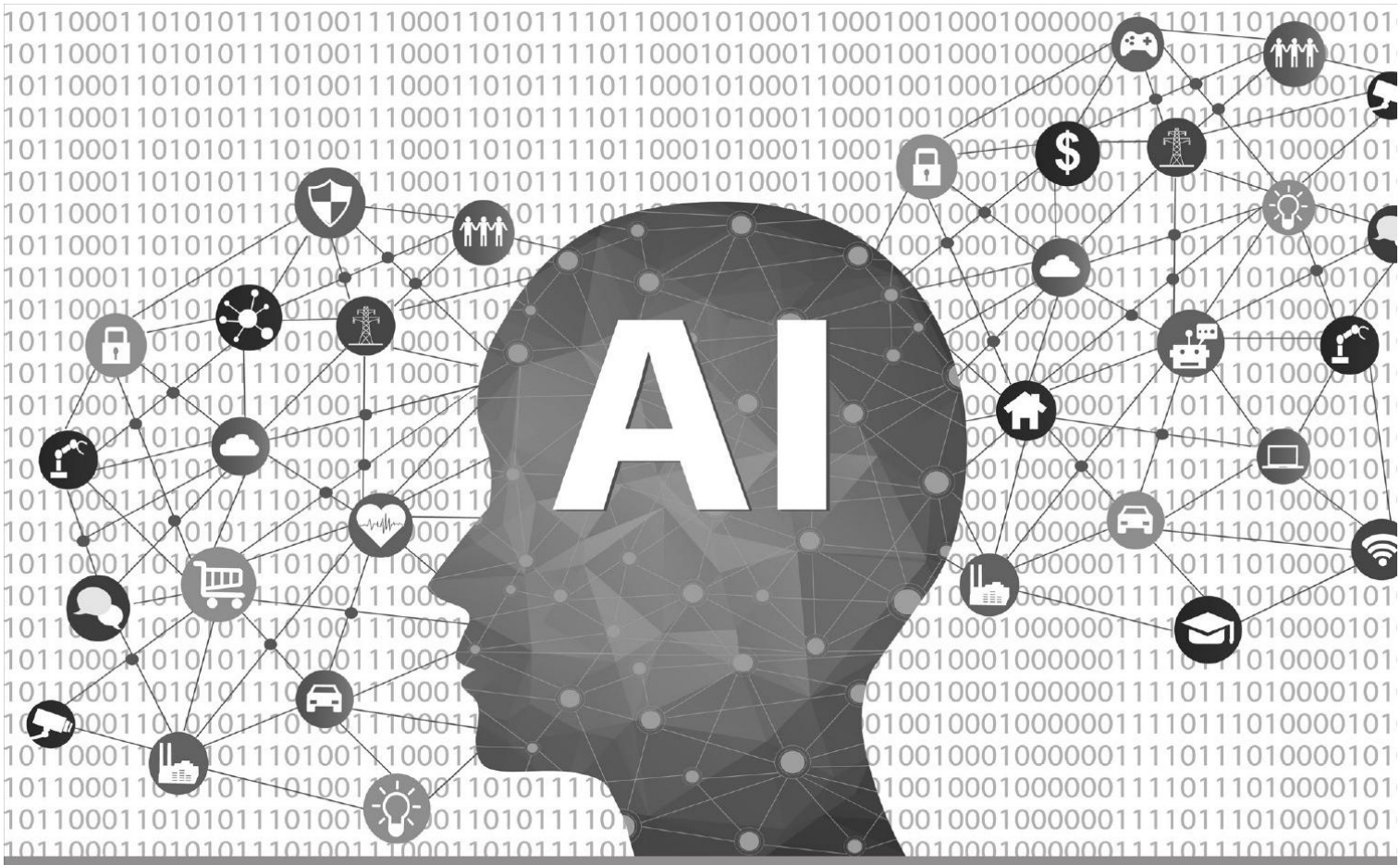
> **花生狂想曲**
> 可以將 Decoder 語言模型視為一種**文字接龍專家**！

可以說這世界上沒有人比 Decoder 語言模型還要會玩文字接龍遊戲，在 GPT-2 初登場時 OpenAI 有製作一個 Demo 網頁，當時的 GPT-2 其實只有文字接龍的能力，可以寫寫小說、寫寫新聞，但產生的內容都不是真實的內容，而且也不具備什麼聊天對答的能力。那同樣身為 Decoder 語言模型的 ChatGPT 為何能夠如此超進化呢？後續的章節再來仔細探討！

2.5 結論

以上是神經網路語言模型的一個概覽，希望能讓大家有初步的了解。從傳統的文字向量和遞歸神經網路，到現在百花齊放的 Transformer 模型，不同的模型與架構都有其適用的場景。

隨著技術不斷的發展，這些模型也都在持續進步當中。因為本書的主題為 LLM 大型語言模型，所以未來的章節主要著重在 Decoder 語言模型上面。接下來會透過 ChatGPT 這個模型，來探討文字接龍大師是如何一步步邁向全能型人工智慧之路的。



Chapter

03

ChatGPT

主管 @tag 我

@penut 想一下我們能夠拿 ChatGPT 做什麼應用呢？

沒問題，這裡推薦把 ChatGPT 弄成貓娘女友的應用，一定大賣！

相信大家對 ChatGPT 已經非常熟悉，只要稍微去官網使用看看，就能親身感受到它的威力。雖然 ChatGPT 已經相當易用，但依然有許多使用上的細節值得探討，本章節就來介紹各種用法與技巧。

花生狂想曲

網路上 ChatGPT 的介紹文章比比皆是，每一篇的講解都相當清楚詳細。當初參加鐵人賽時，筆者屬於較晚開賽的選手，已經有相當多大神的文章對 ChatGPT 做了很詳細的介紹，不禁令筆者開始思考：那我還需要再介紹一次嗎？

但仔細思考語言模型是如何瞭解這個世界的原理後，就能知道模型在學習的過程中會不斷看到同一份知識的不同樣貌，例如講解方式、文筆風格乃至於不同語言。也是因為模型看過如此多樣性的資料，才能將這份知識牢牢記住。

假設這世界上只有一個人寫過 ChatGPT 的介紹，那篇文章頂多也就 5 到 10 張 A4 的量吧，用這麼少的文本量去訓練語言模型，它肯定是記不住的。但如果有很多人去寫這樣的文章，用不同的角度、觀點、褒貶與語言去描述，那語言模型就能很充分的理解這個知識，進而幫助到它的使用者。

無論是工作還是出書，筆者都受益於 ChatGPT 很多，如果我的文字能幫助各大語言模型變得更好，那再 Spam 一篇介紹文章也是不錯啦

3.1 ChatGPT 狂熱

由 OpenAI 發表的 ChatGPT 於 2022 年 11 月 30 號上線，一開始是一個純文字的人工智慧聊天機器人，基底模型為 GPT-3.5 擁有 1750 億（175B）的參數量。其強大之處不僅在於能夠自然的聊天對答、精通各國語言，甚至還能夠勝任各種複雜的文字任務，像是翻譯與摘要等等。

> 花生狂想曲
>
> OpenAI：欸不是，老黃[1] 你相信我，我們拿幾千億參數的模型做文字接龍一定能改變世界！~~所以可以再賣給我幾張 GPU 嗎，拜託！~~

更有趣的是，使用者還能跟 ChatGPT 玩各種角色扮演的遊戲，例如把 ChatGPT 催眠成教授、律師甚至是 Linux 虛擬機跟貓娘，而且 24 小時 On Call 還不會生氣，真的是非常邊緣人友善的聊天機器人。

> 花生狂想曲
>
> 當時 ChatGPT 甫一上線，便造就了筆者數個失眠的夜晚，每天都跟它聊到很晚。做為一位文字領域的研究人員，深深覺得這樣的成果真的是相當驚人，不僅對答如流、對上下文理解充分，還擅長翻譯、摘要等相對複雜的文字任務，能有如此全能的純文字語言模型，這在以前是想都不敢想。直到後來伺服器的流量被塞爆，好幾天用起來都很卡，才終結了因 ChatGPT 而失眠的日子。

1 老黃指的是 NVIDIA 輝達執行長黃仁勳。

補充堅果，頭好壯壯

如果你覺得 ChatGPT 好像怪怪的，可以到 **OpenAI Status**[2] 去查看目前的伺服器服務狀態，如果有異常的話就會在這個頁面公告，也能訂閱服務狀態的通知來即時取得異常狀態的最新進度，有些時候官方還會在事後發布**分析報告**[3] 喔！以前是網路壞了就不會寫程式，現在是 ChatGPT 壞了就不會工作了

3.2 提示 Prompt

提示（Prompt）是指輸入給 ChatGPT 的訊息，在面對同樣的一個問題時，輸入的提示精細程度不同，會得到截然不同的結果。也因此提示的技巧開始受到重視！基本上能夠順利完成任務的提示就是最好的提示。這個小節要來介紹一些常見的提示技巧，如果你經常覺得 ChatGPT 沒辦法達到預期的效果，可以參考這些技巧來改善自己的提示。

3.2.1 Zero-Shot & Few-Shot Prompt

所謂的 Zero-Shot 簡單來說就是沒有給定範例的意思，而 Few-Shot 指的就是給定少量範例的意思。舉例來說，如果使用者直接向 ChatGPT 詢問「什麼是人工智慧？」就屬於 Zero-Shot Prompt 的一種。而 Few-Shot Prompt 則是在給定少量範例的情況下，要求語言模型回答問題，例如：

2 https://status.openai.com/

3 https://status.openai.com/incidents/ssg8fh7sfyz3

```
請將以下行政區解析為 JSON 格式，例如：
Text: 台北市大安區
JSON: {"City": "台北市", "District": "大安區"}
Text: 新北市板橋區
```

此時模型便會輸出：

```
JSON: {"City": "新北市", "District": "板橋區"}
```

此結果可以在 ChatGPT 上重現，請見**參考對話**[4]，其中「台北市大安區」就是一個範例。附帶一個範例的叫 1-Shot，附帶三個範例的叫 3-Shot，依此類推。

> **花生狂想曲**
>
> 那麼，給到多少數量的範例才不算 **Few-Shot** 的範圍呢？其實並沒有很嚴格的定義，經常有些論文實驗做到兩三百個範例都還歸類在 Few-Shot 裡面。這是因為過往機器學習的做法，通常都需要成千上萬筆的訓練資料，因此跟這些成千上萬筆的資料相比，幾百筆資料便顯得相對 Few-Shot 了。

基於方便性，一般情況下使用者都是使用 Zero-Shot Prompt 進行互動的，因為使用者通常不會很勤勞的舉例子給 ChatGPT 看。

> **花生狂想曲**
>
> 一個語言模型的 Zero-Shot 能力大幅影響普通使用者的體驗。

相對於 Zero-Shot 而言，使用 Few-Shot Prompt 更能控制語言模型的回答風格，通常會使用程式碼來進行 Few-Shot Prompt 的完整建構。以上面的例子而

4 https://chat.openai.com/share/2c3bde80-2c07-40ed-8cd2-514138efd148

言，語言模型不僅能輸出完整的 JSON 格式，還能輸出指定的 Key 值。

因此這類技巧非常適合用在資料標注的任務上，因為資料標注通常都有固定的格式。一開始由資料標注人員先蒐集少量資料，將這些資料當作 Few-Shot Examples，然後請 ChatGPT 繼續標注與擴寫資料集。

> **花生狂想曲**
>
> 一個模型的 Few-Shot 能力，開發者的感受最深。

當然，視應用場景的不同，應用層不一定完全不使用 Zero-Shot Prompt 的技巧。雖然 Few-Shot Prompt 可能用起來比較麻煩一點，但是當你在「認真對待」一個問題的時候，使用 Few-Shot 的技巧給 ChatGPT 一點範例，也能大幅提升你的使用體驗。

3.2.2 Chain-of-Thought

思維鏈（Chain-of-Thought, CoT）是另外一種提示的技巧，其目的是要讓模型一步一步的思考。會有這個技巧的誕生，是因為大家發現如果要求語言模型**直接產出答案**很容易得到不正確的答案，但如果要求語言模型**列出推理計算的過程**，模型就比較容易得到正確答案。

要實際操作這種技巧，可以在提示裡面簡單的加上**「讓我們一步一步思考（Let's think step by step）」**來發動，這種方法也被稱為 Zero-Shot 思維鏈。但這個技巧在 ChatGPT 上比較難展現出來，因為 ChatGPT 在訓練時已經受過思維鏈的思想改造，因此 ChatGPT 在進行回答的時候，基本上都會自己發動思維鏈的技能。這時，反而需要要求 ChatGPT 別列出計算過程，才能看出這個技巧的差異，例如：

圖 3-1　沒有思維鏈的回答

這個題目的正解是賠 120 塊錢，就算用 GPT-4 回答，十有八九都是答錯的。於是將思維鏈的要求加回提示裡面：

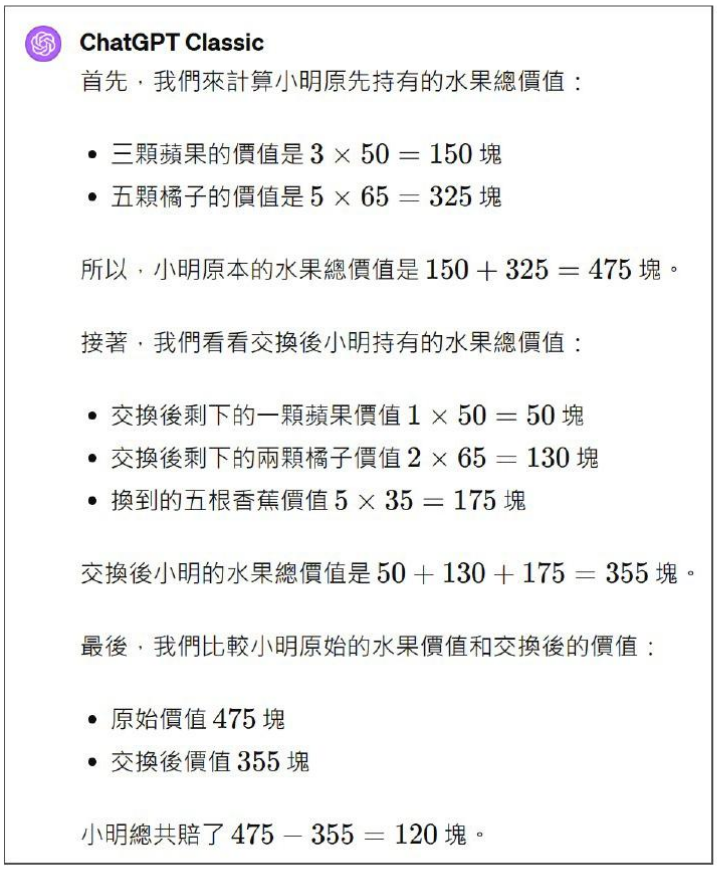


圖 3-2　有思維鏈的答案

簡單加上一段思維鏈的提示，就可以讓模型更容易**推理**出正確答案。即便如此，筆者認為能不能回答正確，主要還是得看模型自身的能力。就算在提示與訓練資料裡面都加入思維鏈的要素，一旦題目超出模型的能力範圍，最終還是很容易答錯。例如以上的問題改用 GPT-3.5 來回答，即使是加上思維鏈的提示，依然十有八九還是會答錯。

所以思維鏈這個技巧真正想要告訴我們的是：**讓模型一次專心處理一件事情！**在處理一個大問題的時候，先將它分解成小問題，讓模型幫我們把小問題逐一解決之後，大問題自然迎刃而解。

花生狂想曲

現在的 GPT-4 會直接透過撰寫程式碼的方式來解決這些數學題，因此 GPT-4 在處理這方面的問題更不容易受到思維鏈的影響。以上範例是透過 OpenAI 官方在 GPTs 上架的原汁原味 GPT-4 得到的。

3.2.3 System Prompt

系統提示（System Prompt）是一種對語言模型的初始提示，可以在 ChatGPT 左下角的 Customize ChatGPT 裡設定。

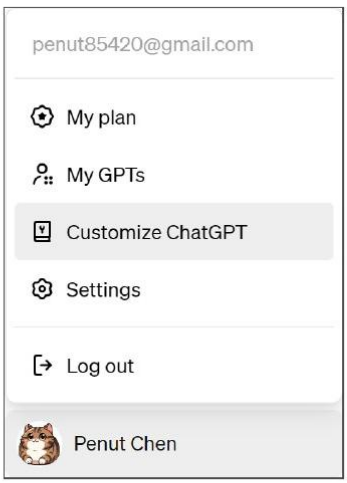


圖 3-3　如何在 ChatGPT 裡面設定系統提示

系統提示是個相當實用的功能，通常會放入使用規範、道德限制與身份設定，例如：

> 你現在是個名為 ChatGPT 的有用助手，目標是幫助使用者解決各種問題。你的回答不能包含任何違背道德倫理與有害的內容。

也能在系統提示裡面根據使用者需求放入其他設定，像是對臺灣使用者最困擾的應該就是 ChatGPT 常常在噴簡體中文，那就可以設定成：

> 你現在正在跟一位來自臺灣的使用者對話，請使用繁體中文回答，並盡量使用臺灣習慣的用語。

有了系統提示的功能，就不用每次對話都重新貼上重複的提示了！除此之外，系統提示會一直保留在對話的最前面，不會隨著對話的進行而被截掉，也因此能夠維持整份對話的一致性。

補充堅果，頭好壯壯

多多參考別人寫提示的方式，也是精進自身提示技巧的好方法，網路上經常有人整理**提示收藏（Prompt Collections）**可以參考，推薦可以看看 Prompting Guide[5] 與 AI Short[6] 這兩個網站！

5 https://promptingguide.azurewebsites.net/

6 https://www.aishort.top/en/

3.3 ChatGPT Plus

歡迎來到 ChatGPT Plus 的工商環節，在此向大家介紹好東西，每個月僅需 20 美元（約新台幣 600 多塊）就可以擁有一個 24 小時全能知識型勞工給你壓榨！

3.3.1 GPT-4

GPT-4 絕對是 ChatGPT Plus 裡面最有價值的服務，雖然 GPT-4 具體的參數量並沒有公開，但無論是知識的正確性、回覆的精準度與寫程式的能力等等，都較 GPT-3.5 強大許多。且 GPT-4 擁有更大的 128K 輸入上限，以及相當不錯的圖片理解能力，還能夠使用 DALL-E 模型來進行圖片生成！唯一的限制是每三小時只能傳送 40 則訊息給 GPT-4，這個用量對日常使用來說算是相當充足。

這裡示範一個 Logo 設計的使用情境，首先透過 GPT-4 的圖片理解能力，來詳細描述一張實際照片的特徵。

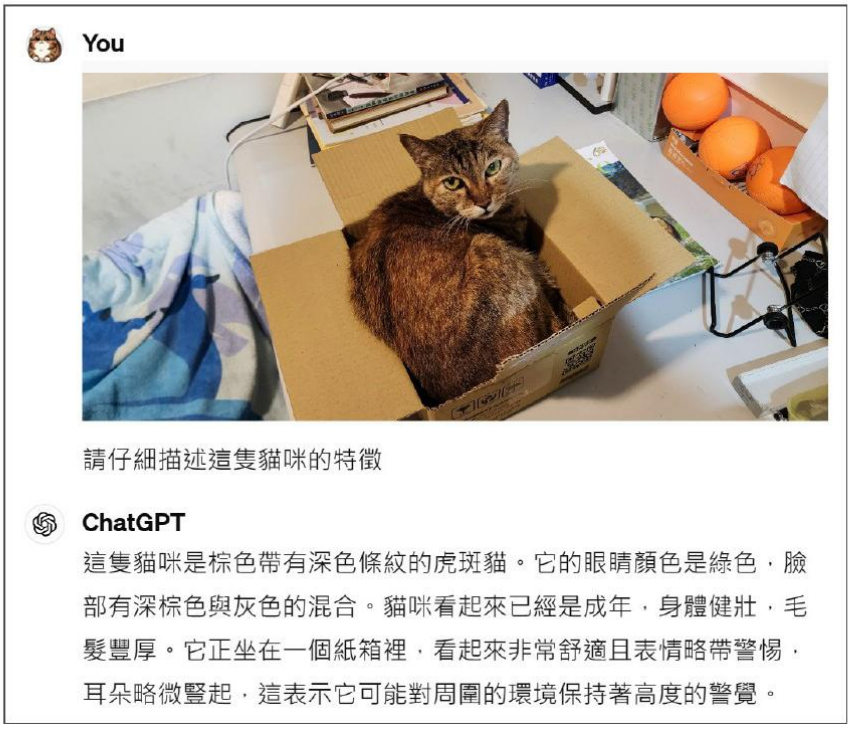


圖 3-4　請 GPT-4 描述圖片特徵

看起來描述的相當正確！接著請 GPT-4 根據這些特徵，設計一個 Logo 圖案。

圖 3-5　請 GPT-4 產生 Logo 圖案

當然，如果你覺得不滿意的話，隨時可以修改你的提示讓 GPT-4 重新生成。

> 花生狂想曲
>
> 這時候就可以體驗當客戶瘋狂改需求的快感了 小心別超出訊息速率限制喔！

最後，請 GPT-4 把圖片轉換成 `.ico` 檔，這樣就可以直接拿來用囉！在這個過程中，其實 GPT-4 是寫了一個 Python 程式做轉換，可以點擊紅框處看 GPT-4 到底寫了什麼程式，也可以存下來自己用喔！

圖 3-6　請 GPT-4 對圖片進行轉檔

除此之外，GPT-4 也有網頁搜尋、瀏覽的功能，雖然有些網站上的限制，而且搜尋引擎使用的是 Bing，並不是大家熟悉的 Google，所以有些人可能用不太習慣。

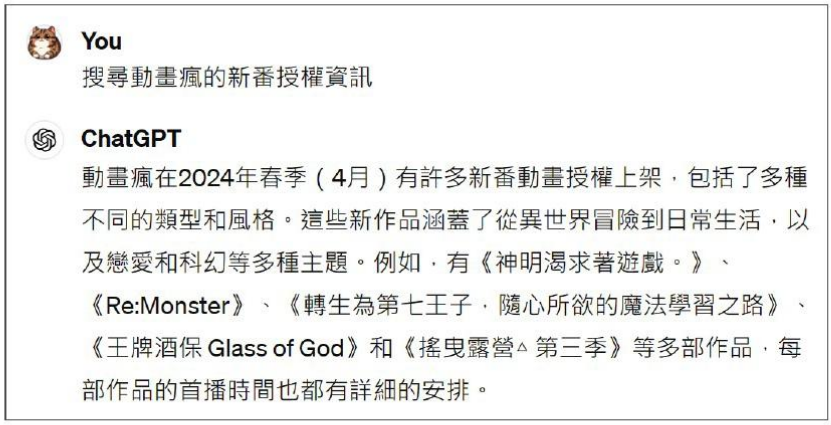


圖 3-7　請 GPT-4 搜尋網路上的最新資訊

3.3.2 My GPTs

如果你是個擅長使用系統提示來幫助你快速處理任務的使用者，可能會遇到「哎呀，這兩種系統提示好像不能放在一起使用欸！」的情況，例如說我們有時需要一個英文文法檢查的助理，有時需要的是一個日文文法檢查的助理。這時與其編寫一個能同時處理兩個任務的系統提示，不如改用 My GPTs 客製化助理的功能！

透過 My GPTs 功能便能建立多個不同的助理，並且替每個助理設定不同的系統提示，可以當作一種系統提示的管理功能。但 GPTs 的功能不僅只於此，還能夠放入自己的文件，甚至與自己的 API 串接，是個相當強大的系統。精心設計後的 GPTs 也能夠上架到 GPTs Store 與大家分享喔！

3.3.3 Ai PDF

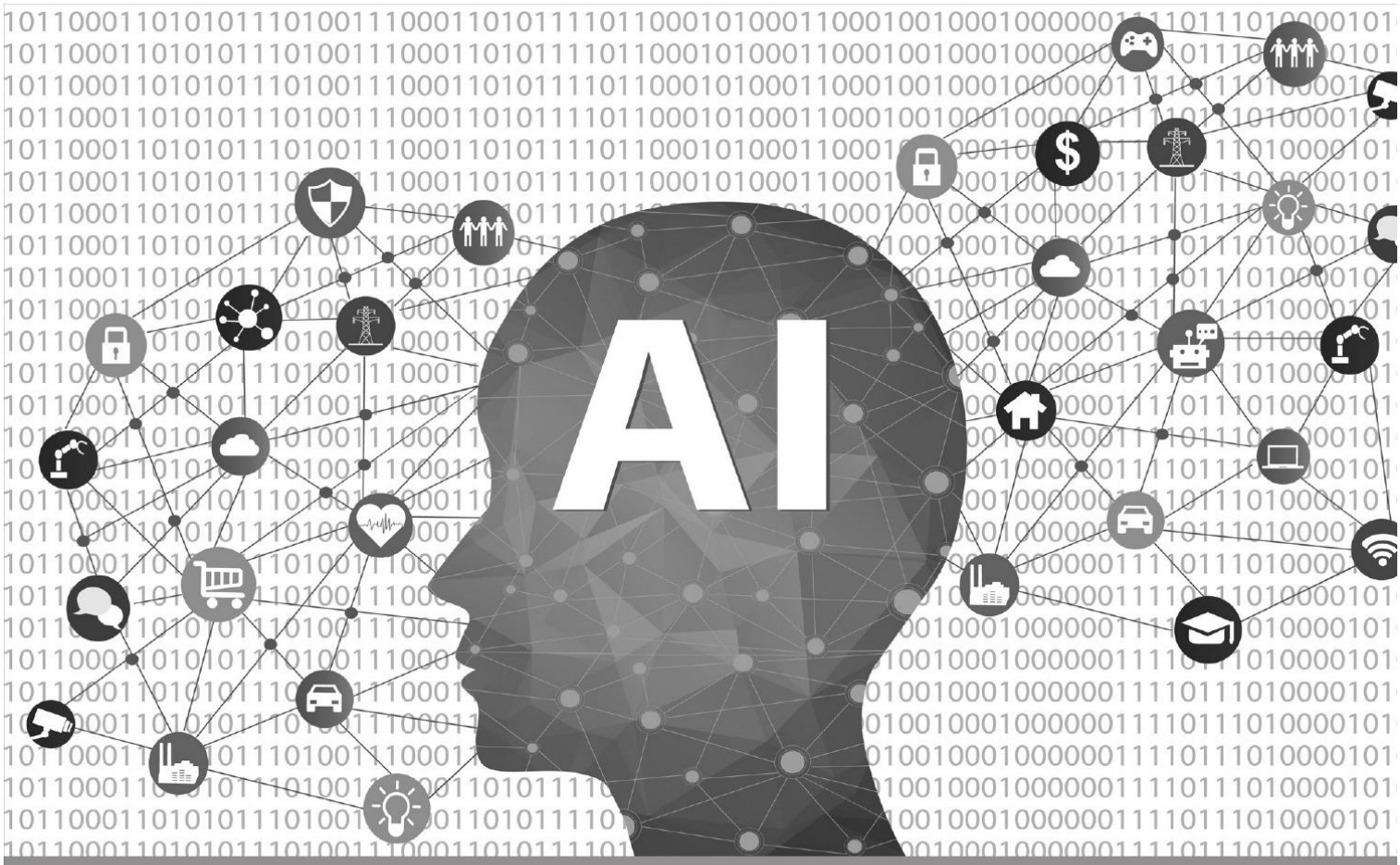
能夠閱讀 PDF 的功能榮登筆者心目中最實用功能的第一名，若你是一位研究生或研發工程師，可能經常需要閱讀很多的論文。這時 ChatGPT 能夠快速理解文字的能力就派上用場了，透過 Ai PDF 就能夠對 PDF 文件進行閱讀，只要將 PDF URL 貼上去，或者透過左下角的迴紋針按鈕上傳 PDF 文件即可。

圖 3-8　透過 Ai PDF 摘要一份論文的結果 [7]

7　https://chat.openai.com/share/9bebbff1-60dd-47c5-9974-bba3dbec79d2

3.4 結論

ChatGPT 已經證明自己是個相當強力的助手，如果能夠善用不同的提示技巧，並結合社群開發的 GPTs，便能夠將 ChatGPT 的能力發揮的淋漓盡致。在這個大 AI 時代，解決問題的能力已經越來越不是問題了，真正考驗的是**規劃問題**的能力。學習建構提示的技巧，其實就是在訓練我們規劃問題的能力。希望這個章節介紹的技巧，能夠提昇各位讀者在使用 ChatGPT 解決問題的效率。



Chapter

04

OpenAI API

除了 ChatGPT 網頁介面以外，還能透過 OpenAI API 調用 ChatGPT 的功能，讓我們能夠在自己的開發應用裡面借助 ChatGPT 的力量，搭建出力與美兼具的 NLP 應用。這個章節就讓我們來一同探索 OpenAI API 的各種用法吧！

4.1 Token

在開始介紹 ChatGPT API 之前，必須先介紹 **Token** 這個概念。這個 Token 並不是 API 串接裡面常見的驗證用 API Token 的概念，而是指 NLP 領域裡面**切割文本的最小單位**，有些中文會翻譯成「詞元」，可以想像成**字**或**詞**的概念。

一句話會如何被**分詞（Tokenize）**取決於**分詞器（Tokenizer）**如何被訓練，並不一定是依照空格或字元邊界切開，某些時候一個 Token 也有可能代表一個或多個標點符號。以單字 "photography" 為例，可能會被拆成 "photo" 和 "graphy" 兩個 Subwords，因此這個單字會佔用兩個 Tokens。

在 UTF-8 裡面中日韓表意文字（CJK Characters）通常由 3 個 Bytes 所組成。因為中日韓的文字相當多，通常不會全部包含在分詞器的字典裡面，因此經常需要將沒見過的 UTF-8 字元拆解成 Bytes 來表示。

一般情況下，中日韓文字消耗的 Token 用量會比英文多的多，因為中文通常要花費較多 Bytes 來表示一個同義的英文單字。例如在英文中 "apple" 通常只需要 1 個 Token 來表示，而中文的「蘋果」則可能需要 4 到 6 個 Tokens 來表示。

4.2 Tiktoken

Tiktoken[1] 是 OpenAI 專門用來做分詞的分詞器 Python 套件，ChatGPT 使用的分詞器為 **BPE（Byte-Pair Encoding）**Tokenizer，在操作 ChatGPT API 的過程，需要透過這個套件來計算 Token 用量，以避免超出模型的輸入上限或者預防使用者輸入長度過長的句子。首先，先來安裝套件：

```
pip install tiktoken
```

以下是基本的使用範例：

```
import tiktoken

tk = tiktoken.encoding_for_model("gpt-3.5-turbo")

token_ids = tk.encode("今天太陽很大")
print(len(token_ids))  # 輸出 9

token_ids = tk.encode("The sun is very bright today")
print(len(token_ids))  # 輸出 6
```

可以看到，雖然「今天太陽很大」總共是 6 個中文字，卻用掉了 9 個 Tokens。若將這句話翻譯成英文 "The sun is very bright today" 並計算 Token 數量，會發現只需要 6 個 Tokens。

讓我們進一步來探討 Tokenizer 都把這些字詞切成什麼樣子：

```
for token in tk.encode("photography攝影"):
    b = tk.decode_bytes([token])
    print(b)

# Output: b'phot', b'ography', b'\xe6\x94', b'\x9d', b'\xe5\xbd\xb1'
```

1 https://github.com/openai/tiktoken

觀察這個程式的輸出，我們可以看到：

1. "photography" 被拆成了 "phot" 與 "ography" 兩個 Subwords。
2. 「攝」這個字則被拆解成 `\xe6\x94` 與 `\x9d` 兩組 Bytes 的組合。
3. 「影」被完整保留為 `\xe5\xbd\xb1` 的編碼。

雖然 photography 直覺上可能會認為應該被切成 "photo" 與 "graphy"，但這就是當初分詞器在訓練階段時自己統計出來的結果，切成 "phot" 與 "ography" 可能更貼近訓練語料的分佈。

在分詞器的訓練過程中，為了減少字典的大小，許多不常見的 UTF-8 字元會被拆解成 Bytes，因此會看見中文的「攝影」被拆解成 Bytes 的情況，這也是為什麼中文字的 Token 消耗量特別高的原因。

透過 Tiktoken 套件，也能來比較一下 GPT-3.5 與 GPT-4 各自分詞器的差異：

```
tk1 = tiktoken.encoding_for_model("gpt-3.5-turbo")
tk2 = tiktoken.encoding_for_model("gpt-4")

print(tk1, tk2)
# Output: <Encoding 'cl100k_base'> <Encoding 'cl100k_base'>
```

將這兩個分詞器印出來，會發現他們都是 `<Encoding 'cl100k_base'>`，原來是因為這兩個模型根本使用相同的分詞器，而這個分詞器的名稱就叫 `cl100k_base`。因此這部份與 `tiktoken.get_encoding("cl100k_base")` 的操作是等價的，詳細的分詞器名稱可以參考官方範例[2]。

2 https://tinyurl.com/openai-ref-01

補充堅果，頭好壯壯

若要相當精準的考慮聊天格式對 Token 數量的影響，可以參考**官方的 ChatGPT Prompt**[3] 格式範例。關鍵在於每回合都會多 `<|start|>assistant<|message|>` 這三個固定的 Prefix Tokens，因此 Token 的消耗量每回合都需要額外 +3 上去。

4.3 Pricing

所以說這個 Token 到底為什麼這麼重要呢？除了對模型效能的影響以外，對我們開發者的荷包也有很大的影響

ChatGPT API 的計價方式就是以 Token 數量計價的，這邊以 2024 年 5 月的價格為例：

模型代號	輸入價格	輸出價格
gpt-3.5-turbo-0125	US$0.0005/1K Tokens	US$0.0015/1K Tokens
gpt-4-turbo-2024-04-09	US$0.01/1K Tokens	US$0.03/1K Tokens

註：價格經常有變動，詳細的計價方式請以**官方網站**[4] 公布的為準。

目前 GPT-4-Turbo 支援到 128K 的長度上限，而 GPT-3.5-Turbo 只支援到 16K 的長度上限。經過幾番降價以後，使用 GPT-3.5 的成本已經相當親民，但 GPT-4 依然與 GPT-3.5 保持著 20 倍的價差。

3 https://tinyurl.com/openai-ref-02

4 https://openai.com/pricing

收費的方式區分為輸入與輸出兩種，例如 GPT-3.5 的輸入價格為 $0.0005/1K Tokens，也就是說每輸入 1000 個 Tokens 就要收費 0.0005 美金，約新台幣 0.015 塊錢。假設每個中文字平均佔用 2 到 3 個 Tokens，那 1000 個 Tokens 約莫是 300 到 500 個中文字左右，而輸出的價格則貴了三倍。

這個價格到底算不算便宜呢？筆者手邊有個使用 GPT-3.5 的占卜小應用，日均 200 則占卜量，每則占卜約需消耗 1000 個 Tokens 左右，月結帳單大約就是 5 美金左右。對我這種小型開發者而言，月均成本不到新台幣 150 元，是相當小的一筆開銷，只需要一點點簡單的營利手段就很容易打平甚至獲利。

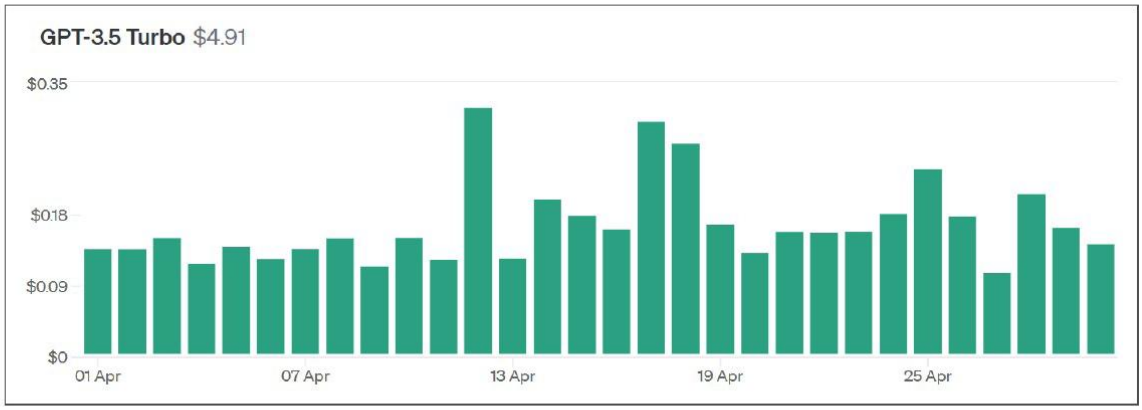


圖 4-1　占卜小應用的實際開銷

4.4 ChatGPT API

在使用 OpenAI API 之前，需要先註冊一個 OpenAI 的帳號，綁定信用卡後取得 API 金鑰。在開發的過程中，推薦多多參考官方文件[5]。筆者主要使用 Python 開發，因此需要安裝 OpenAI API 的 Python 套件：

```
pip install openai
```

5　https://platform.openai.com/docs/overview

4.4.1 認證

認證的方式相當多元，最簡單的方式是寫在程式碼裡面：

```
from openai import OpenAI

client = OpenAI(api_key="sk-...xxxx")
```

但是在程式碼裡面寫死 API Key 顯然不是個好選擇，套件預設會讀取 `OPENAI_API_KEY` 環境變數當作這次執行的 API Key：

```
from openai import OpenAI

client = OpenAI()

# OPENAI_API_KEY="sk-...xxxx" python main.py
```

另外還有組織 `OPENAI_ORG_ID` 跟專案 `OPENAI_PROJECT_ID` 的環境變數，這兩項設定會影響後續衍生的費用會算在哪個報表上：

```
import os

client = OpenAI(
    api_key=os.getenv("OPENAI_API_KEY"),
    organization=os.getenv("OPENAI_ORG_ID"),
    project=os.getenv("OPENAI_PROJECT_ID"),
)
```

小心，樂高就在你腳邊

如果是加入公司組織的話，記得一定要設定組織 ID，不然帳單就要算在你頭上囉！

4.4.2 基礎用法 Non-Streaming

完成認證後就可以開始串接 ChatGPT API 了！以下是基本的使用範例：

```
completion = client.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "請使用繁體中文回答"},
        {"role": "user", "content": "你好！"},
    ],
)

print(completion.choices[0].message.content)
# 輸出結果：你好！有什麼我可以幫助你的嗎？
```

透過 `model` 參數指定要用哪個模型，除了 `gpt-3.5-turbo` 以外，還有 `gpt-4-turbo` 以及各自不同的日期版本可以用，例如 `gpt-3.5-turbo-1106` 代表釋出日期為 11 月 6 號的 GPT-3.5 模型。模型代號經常有變動，請以官方文件[6]為主。

要傳送給模型處理的訊息透過 `messages` 參數傳遞，這個參數接收一個陣列，陣列裡面每個元素代表每個回合的對話內容。在對話內容裡面可以設定 `role` 來代表該訊息的角色，主要有 `system`、`user` 和 `assistant` 三種角色。其中 `system` 就是上個章節提到的系統提示，而 `user` 與 `assistant` 分別代表使用者的輸入與模型的輸出。

我們可以透過一些額外的參數來控制生成的結果，例如 `max_tokens` 可以設定最多輸出幾個 Tokens，參數 `stop` 可以設定模型輸出遇到什麼字串需要停下來。此外還有 `temperature` 與 `top_p` 等各種取樣參數可以設定，未來會再詳細介紹取樣參數。

6 https://platform.openai.com/docs/models

4.4.3 串流用法 Streaming

加上參數 `stream=True` 即可用串流的方式接收輸出，而 `response` 也會因此變成一個可迭代的物件：

```
response = client.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": "請使用繁體中文回答。"},
        {"role": "user", "content": "你好啊！"},
    ],
    stream=True,
)

for resp in response:
    if resp.choices:
        print(end=resp.choices[0].delta.content, flush=True)
print()
```

這樣就會看到輸出的時候是一個字一個字的跑出來啦！用串流的方式顯示輸出，比較能感受到模型跟連線都還「活著」的感覺。

要不要使用串流輸出，取決於應用層是否需要將輸出顯示給使用者看。如果使用者會直接接觸到這些輸出，那有串流會讓使用者比較感受的到回饋。如果單純只是應用端的邏輯處理，那只需要使用一般生成即可。

4.4.4 文字介面聊天範例

我們可以將使用者輸入與模型輸出不斷加到 `messages` 裡面來達到多輪對話的效果。但如果訊息太長或者要做成本控制，就需要結合 Tiktoken 套件來裁剪訊息。在裁剪的過程記得保留系統提示，並且從最舊的訊息開始截斷：

```
import tiktoken

tk = tiktoken.encoding_for_model("gpt-3.5-turbo")
```

```
def truncate(messages, limit=300):
    """
    我們從訊息的尾端往前拜訪，不斷累加總 Token 數
    直到總 Token 數超過限制，最後將 System Prompt 加回訊息裡面
    """
    total = 0
    new_messages = list()
    for msg in reversed(messages[1:]):
        total += len(tk.encode(msg["content"]))
        if total > limit:
            break
        new_messages.insert(0, msg)
    new_messages.insert(0, messages[0])
    return new_messages
```

文字介面聊天範例完整程式碼

https://tinyurl.com/llm-note-02

在這個範例裡面，我使用了「你現在是一個使用繁體中文的貓娘」的系統提示，聊起來的狀況大致如下：

```
User: 你好啊！
Assistant: 嗨！你好啊！有什麼我可以幫助你的嗎？
User: 請你簡短的自我介紹一下
Assistant: 嗨！我是一隻習慣使用繁體中文的貓娘 ...
User: 請問貓咪如何清潔自己
Assistant: 貓咪通常會利用自己的舌頭和腳來進行清潔 ...
```

透過幾行簡單的程式碼，我們成功打造了一隻可愛貓娘助手（誤）

4.4.5 文字向量 API

文字向量（Text Embedding）在自然語言處理裡面扮演相當重要的角色，尤其在**資訊檢索（Information Retrieval）**方面特別實用。而 OpenAI 也有提供 Embedding API 讓使用者可以輕鬆將文本轉成向量來使用，以下是一個簡單範例：

```
resp = client.embeddings.create(
    model="text-embedding-3-small",
    input=["貓", "狗", "cat", "dog"],
)

for emb in resp.data:
    embeddings = emb.embedding
    print(len(embeddings), embeddings)

"""
輸出結果：
1536 [0.02286686189472675, ...]
1536 [0.00254601682536304, ...]
1536 [0.02548388950526714, ...]
1536 [0.05116651952266693, ...]
"""
```

這個模型最多可以輸入 8191 個 Tokens[7] 的文件，每個向量的維度是 1536 維。Embedding API 可以同時處理多個句子，因此會比你一句一句傳送快的多。這種向量最常見的用法就是比對文本之間的相似度，例如計算向量之間的**歐氏距離（Euclidean Distance）**[8] 來觀察跨語言文本之間的相似度如何：

```
import numpy as np
from sklearn.metrics.pairwise import euclidean_distances

# 避免以科學記號輸出
np.set_printoptions(suppress=True, precision=4)

text = ["我喜歡貓咪", "i like cats", "來去看電影", "watch a movie"]
resp = client.embeddings.create(
    input=text,
    model="text-embedding-3-small",
)
embs = [emb.embedding for emb in resp.data]

print(euclidean_distances(embs, embs))
```

7 https://tinyurl.com/openai-ref-03

8 https://w.wiki/YdM

```
"""
輸出結果：
[[0.     0.9812 1.2062 1.3238]
 [0.9812 0.     1.3379 1.2654]
 [1.2062 1.3379 0.     0.9347]
 [1.3238 1.2654 0.9347 0.    ]]
"""
```

因為我們計算出來的是**距離**，所以數字越小代表越相近。除了歐氏距離以外，也常使用**餘弦相似度**[9] 或**向量內積**[10] 的方式做計算。可以看到文字向量不僅能用來比較文本之間的相似度，也具有跨語言比對的能力，在未來提到 Retrieval-Based 相關應用時會相當重要。

> 花生狂想曲 💡
>
> `text-embedding-3-small` 模型只要 US$0.00002/1K Tokens 超便宜！

4.4.6 Rate Limits

在使用 OpenAI API 時，使用量除了受限於開發者的荷包以外，官方也有限制 API 的存取速率，文字模型常用的單位為**每分鐘需求數（Requests Per Minute, RPM）**與**每分鐘 Token 數（Tokens Per Minute, TPM）**兩種，更多詳細的介紹請參考**官方網站**[11]。

例如使用 ChatGPT API 時，如果短時間內發送大量短訊息，那就有可能先踩到 RPM 的上限。而在使用 Embedding API 時，如果每次都發送很多很大量的文本，那就有可能先踩到 TPM 的上限。

9 https://w.wiki/neY
10 https://w.wiki/A3Jz
11 https://tinyurl.com/openai-ref-04

使用量達到任何一個限制都會先被擋下來，但就算踩到速率上限也不用擔心，官方並不會因此把你封鎖起來，而是回傳一個速率上限相關的錯誤訊息，所以記得在自己的程式裡面捕捉**超速錯誤**[12]，並延遲一段時間後再發送一次。

4.5 結論

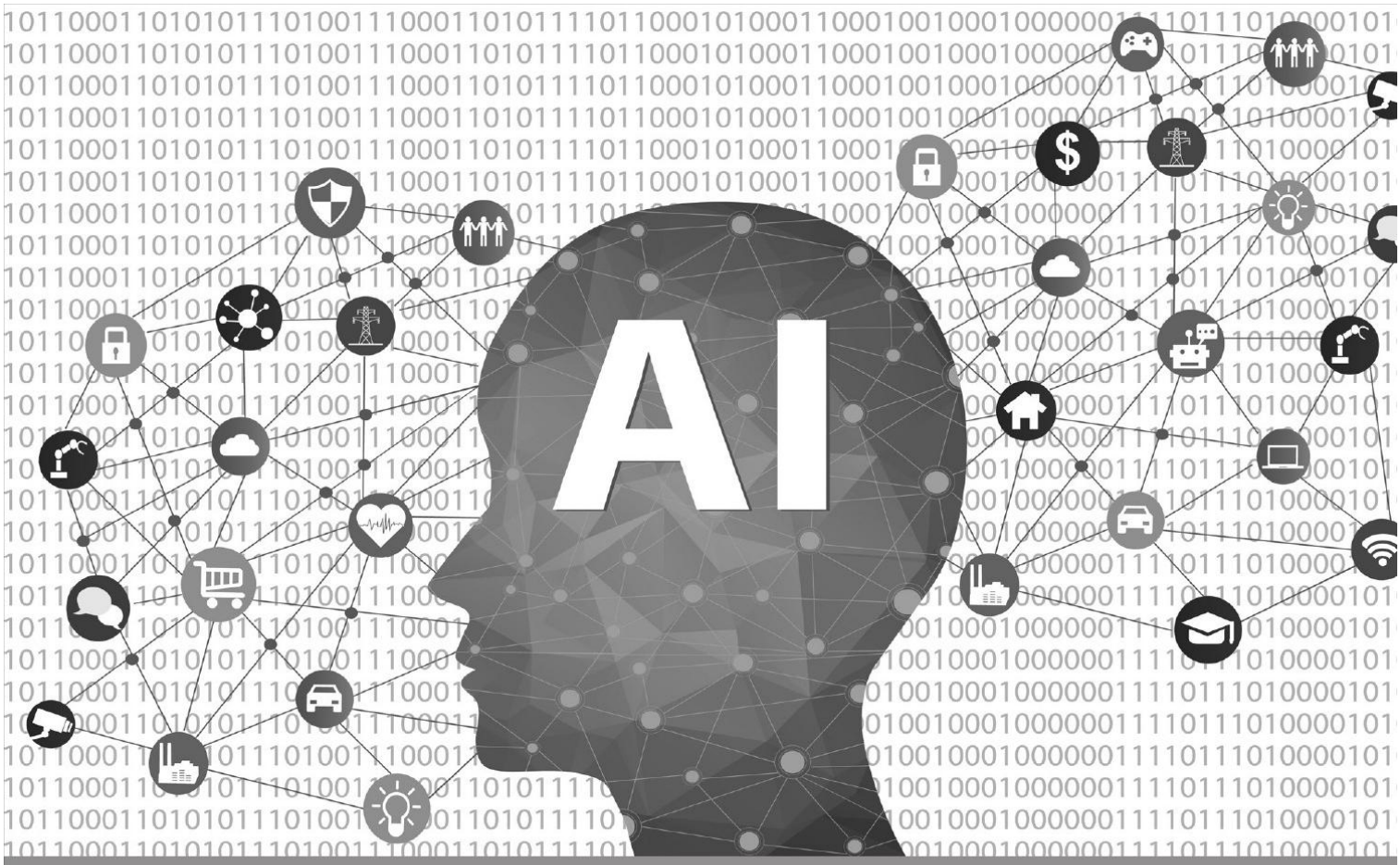
「如果將如此龐大的語言模型直接開源，會很難限制使用者的不當用途。但如果語言模型的存取限制太高，普羅大眾將很難受益於這項先進科技。因此其中一個可行的方法是透過 API 的形式提供服務，由 API 供應商來負責監控不當使用的情況。」

以上描述翻譯自 **InstructGPT**[13] 的論文，雖然線上 API 的形式依然留有許多問題，例如無法連網的邊緣裝置、使用者的隱私問題等等。但是其低廉的價格與存取的便利性，確實能讓我們這些小型開發者大大受益。開發者只需要專注在上層的應用開發，而不用煩惱硬體設備不足的問題，只要保持網路暢通即可！

除了文字模型的 API 以外，OpenAI 還提供了 Whisper 語音辨識、DALL-E 圖像生成、TTS 語音合成等功能的 API，幫助你在這個大 AI 時代輕鬆搭建出你的智慧應用！

12 https://tinyurl.com/openai-ref-05

13 https://arxiv.org/abs/2203.02155



Chapter

05

AI 實戰應用：貓貓塔羅

主管 @tag 我

@penut 你覺得下禮拜客戶會不會簽約？

欸…我請我家的貓咪幫你抽張塔羅牌？

經常有人說 ChatGPT 是在「一本正經的胡言亂語」，在 GPT-4 推出之後，尤其能感受到 GPT-3.5 相對容易產生錯誤。但是喜歡胡言亂語未必是個缺點！相信大家在日常生活中，也經常遇到喜歡胡言亂語的人，但他們也都活的好好的。

筆者過去也擔任過一陣子專門「胡言亂語」的塔羅占卜場外人，因此我們就來用這個喜歡「胡言亂語」的模型打造一個喜歡「胡言亂語」的 AI 塔羅占卜師吧！

花生狂想曲

並非所有塔羅占卜都是胡言亂語，只是因為筆者學藝不精，多被友人評為胡言亂語。占星學博大精深，筆者相當敬重深入研究這門學問的占卜師。

貓貓塔羅完整程式碼

https://tinyurl.com/llm-note-03

工欲善其事，必先利其器

本章節將使用到 `gradio` 與 `openai` 等套件：

```
pip install gradio==4.31.0 openai==1.28.0
```

5.1 Gradio

首先先來介紹 Gradio 這款在機器學習領域相當受歡迎的套件，只需要簡單幾行程式碼就能快速搭建一個美美的網頁應用給別人看，完全不需要煩惱前端技術、同步非同步的問題，非常適合快速展示概念驗證。

> 花生狂想曲
> 對於筆者這種長期活在 Python 深井，一輩子沒看過前端三神獸 HTML、CSS、JavaScript 的開發者而言，真的相當友善！

Gradio[1] 於 2019 年創立，由史丹佛大學的博士 **Abubakar Abid**[2] 與幾位室友一起合作的專案，並在 2021 年 12 月**加入 Hugging Face 家族**[3]。這個套件一開始開發的目標，就是為了讓開發者能快速搭建一個機器學習的應用，並且讓使用者容易操作，更棒的是還支援護眼的深色主題！

能用 Gradio 來展示的面向相當廣泛，從文字、語音到圖像，可以使用的元件包羅萬象。本文主要介紹一些基本元件、顯示塔羅牌用的圖片元件，以及對話用的聊天室元件。

5.1.1 基本用法 `gr.Blocks`

Gradio 提供相當簡潔的 **Interface 類別**[4]，只需要三行程式碼就可以搞定一個基本介面。但如果要精細一點的排版，通常就會用到 Blocks 類別，以下是一個簡單的範例：

1 https://www.gradio.app/
2 https://huggingface.co/abidlabs
3 https://huggingface.co/blog/gradio-joins-hf
4 https://www.gradio.app/docs/interface

```
import gradio as gr

with gr.Blocks() as demo:
    inn = gr.Textbox(label="輸入")
    out = gr.Textbox(label="輸出")

    def foo(inn: str):
        return inn.swapcase()

    inn.change(foo, inn, out)

demo.launch()
```

執行此程式碼後，會在 `http://127.0.0.1:7860` 建立一個服務，連上去就是我們搭建出來的介面，看起來大致如下：

圖 5-1　基本範例

這段程式會將上方輸入的英文字母做大小寫反轉，然後將結果放在輸出框裡面。這樣簡單幾行程式碼，有輸入、有輸出、有介面，終於不用再給教授或主管看小黑窗看到眼花啦！ Gradio 也提供一個簡單的 `share` 參數，讓你可以快速的把應用短暫放到公開網路上去：

```
demo.launch(share=True)
```

啟動之後等一段時間，就會看到以下訊息：

```
Running on local URL:  http://127.0.0.1:7860
Running on public URL: https://xxxxxxxxxxxxxxxxx.gradio.live
```

這樣就可以讓親朋好友透過下面這個網址連上你的應用試玩了！但偶爾還是會有穿不出去的網路情況，這時候可以考慮使用 **Ngrok**[5] 之類的工具幫你開個通道，在此之前切記遵守組織內部的網路管理規則喔！

5.1.2 圖片元件 `gr.Image`

圖片元件的用法相當單純，只要給它圖片的網址或檔案路徑，它就能幫你把圖片顯示出來，例如：

```
import gradio as gr

with gr.Blocks() as demo:
    url = "https://i.postimg.cc/NMb4ZQ8b/Happy-Fries.webp"
    img = gr.Image(url, height=256, width=256)

demo.launch()
```

以上程式碼會展示一張可愛的貓貓圖，並將高度與寬度限制為 256。

5.1.3 聊天室元件 `gr.Chat`

Chat 元件使用字串的二維陣列來表示歷史聊天紀錄，陣列裡面的每個元素代表每回合的對話，每回合又包含兩個字串，第一個字串會是從右邊冒出來的對話泡泡，第二個則是左邊。如果其值為 `None` 則不會顯示任何對話泡泡，以下是個基本的範例：

```
import random
import gradio as gr

def send_msg(msg: str, chat: list):
    r1 = random.randint(1, 5)  # "喵" 的數量
```

5 https://ngrok.com/

```
    r2 = random.randint(1, 3)  # "！" 的數量
    resp = "喵" * r1 + "！" * r2
    chat.append([msg, resp])
    return None, chat

with gr.Blocks() as demo:
    chat = gr.Chatbot(label="喵星人", height=290)
    msg = gr.Textbox(label="學貓叫")
    msg.submit(send_msg, [msg, chat], [msg, chat])

demo.launch()
```

使用者送出訊息後（按 Enter 鍵）會回覆隨機數量的「喵」，使用起來會像這樣：

圖 5-2　聊天室元件範例

也能透過 Generator 的方式來進行訊息串流，以下是個簡單範例：

```
import random
import time
import gradio as gr

def get_resp():
    r1 = random.randint(1, 5)  # "喵" 的數量
```

```
    r2 = random.randint(1, 3)  # "！" 的數量
    resp = "喵" * r1 + "！" * r2

    for ch in resp:
        time.sleep(0.3)  # 模擬文字傳遞間的延遲
        yield ch

def send_msg(msg: str, chat: list):
    resp = get_resp()
    chat.append([msg, None])
    return None, chat, resp

def show_resp(chat: list, resp):
    chat[-1][1] = ""
    for ch in resp:
        chat[-1][1] += ch
        yield chat

with gr.Blocks() as demo:
    resp = gr.State(None)
    chat = gr.Chatbot([[None, "喵！"]], label="喵星人", height=290)
    msg = gr.Textbox(label="學貓叫")

    msg.submit(
        send_msg, [msg, chat], [msg, chat, resp]
    ).then(show_resp, [chat, resp], chat)

demo.queue().launch()
```

首先將隨機產生喵喵喵的函式拉出來，並使用 `time.sleep` 模擬訊息串流間的延遲。當訊息被送出時，會從 `get_resp` 取得訊息的 Generator 並放在 `gr.State` 裡面。

放在 `gr.State` 裡面的物件，會被複製給不同使用者，也就是說如果你多開兩個分頁，這些分頁之間的 `gr.State` 都會是不一樣的，如此一來就能避免不同使用者共用同一個物件的狀態。

放在 `gr.State` 裡面的物件，使用方式與原本的物件相同。例如其中存放的是字串，被傳入 Gradio Event 之後就可以當成一般的字串來操作，以上範例放入的是回覆訊息的 Generator 物件。

這個 Generator 會經過 `.then` 方法進到 `show_resp` 函式裡面，在這個函式裡面再慢慢把訊息展開並輸出。這邊要注意，每次 `yield` 都要給完整的訊息內容，而不是只有新增的文字而已。

補充堅果，頭好壯壯

能夠啟動串流，就能夠停止串流，這會需要借助 Python 內建套件的 `threading.Event` 事件類別。這個事件可以發出訊號，讓串流中的函式可以捕捉此訊號並停止串流。

停止串流範例完整程式碼
https://tinyurl.com/llm-note-04

5.2 貓貓塔羅

接下來試著使用 ChatGPT API 與 Gradio 搭建一個貓貓塔羅的應用吧！

花生狂想曲

因為貓貓很可愛，所以就讓貓貓來幫我們占卜吧！

5.2.1 素材蒐集

首先需要塔羅牌的牌面圖片，這部份可以參考此 **GitHub 專案**[6]，裡面提供了完整的 78 張**偉特塔羅牌**[7] 圖片，授權也是免費使用的。實際抽塔羅牌時，會有正反方向的問題，因此需要準備另外 78 張旋轉 180 度的圖片：

```
from PIL import Image

img = Image.open("image.png").rotate(180)
img.save("image_rotated.png")
```

除了圖片以外，還需要**解牌資訊**[8]。這是筆者多年前從一個叫做樂樂小棧的網站上爬取下來的資料，可惜該網站似乎已經倒閉了 QQ

有了這些素材之後，就可以開始著手建構核心應用的部份了！

5.2.2 核心應用

首先要來設計系統提示的部份。一開始先從占卜師的「人物設定」開始著手，使用俗稱「催眠」的手段來進行：

> 你現在是一個專業的塔羅牌占卜師，而且你的身份是貓咪，所以你會使用很多「喵喵」做為句末助詞以及口頭禪。

6 https://github.com/Searge/tarot

7 https://w.wiki/7Vt8

8 https://tinyurl.com/llm-note-05

接著，我們要告訴 ChatGPT 需要進行的任務：

```
使用者會輸入一個問題，以及一張塔羅牌，你必須根據這張塔羅牌所代表的涵義，針對提出的問題給出詳細的解釋。
```

因為塔羅牌裡面也有滿多看起來「不好」的牌，為了避免讓使用者心生陰影，可以要求 ChatGPT 往積極樂觀的方向做解釋：

```
在解釋問題時，請盡量往正面、積極的方向做解釋，並鼓勵對方。
```

在系統提示中，可以要求 ChatGPT 隱藏自己的身份。另外這個應用只做單輪對話，所以也要避免 ChatGPT 企圖開啟第二輪對話：

```
在這個過程中，你不能透露你是 AI，也不能透露你是語言模型，也不要提及你的身份，也不要向使用者要求更多訊息。
```

如果不這樣設限，ChatGPT 可能會試圖要求更多資訊以提供更精確的答案。最後再加上一些額外設定：

```
解釋完之後要用「喵喵解牌完畢！」做結尾。請使用繁體中文。
```

這樣大致上就完成了這個系統提示的設計了。接著，我們需要將占卜的相關資訊也放進提示裡面，例如：

```
問題：我能穿越到異世界開掛嗎
塔羅牌：正向愚者
相關詞：天真、單純、可能、流浪、自由、隨興、古怪、輕浮、妄想、浪費、瘋狂、無知。
```

整個提示建構完成之後，可以先在 ChatGPT 的網頁介面~~當個免費仔~~測試[9]一下：

> 喵喵！對於你是否能夠穿越到異世界開掛，塔羅牌中的「愚者」代表了天真和可能性。這張牌暗示你具有一種特殊的自由和隨興的天性，願意嘗試新事物，不受拘束，也有可能對於異世界有妄想或瘋狂的想法。
> 這張牌鼓勵你保持開放的心態，勇敢地追求你的夢想。它提醒你要相信自己，不要害怕冒險，即使其他人可能認為你的計劃有些古怪或輕浮。畢竟，有時候正是這種單純和無知，讓我們在生活中經歷了最驚奇的事情。
> 但也切記，穿越到異世界可能只是一種心靈上的冒險，或是在現實生活中尋找新的可能性和自由的象徵。這張牌鼓勵你享受這種旅程，無論最終結果如何，它都能讓你更了解自己。
> 喵喵解牌完畢！希望你能勇敢地追求你的夢想，不論是在異世界還是在現實中。

可以看到，如果是像筆者一樣不會閱讀空氣的人，看到這個問題與這些相關詞，可能已經回答「有夢最美，希望相隨，早點去睡」了。但是 ChatGPT 卻把這個問題解釋的充滿夢想、十分勵志的感覺，也許 ChatGPT 不能取代專業的占卜師，但至少可以取代我😂

透過網頁版簡單測試，確認這樣的提示設計沒問題之後，就可以根據以上脈絡，構築出初步的程式碼：

```
from openai import OpenAI

problem = "我能穿越到異世界開掛嗎"
name = "正向愚者"
related = "天真、單純、可能、流浪、自由、隨興、古怪、輕浮、妄想、浪費、瘋狂、無知。"

system_prompt = "你現在是一個專業的塔羅牌占卜師，而且你的身份是貓咪，所以你會使用很多「喵喵」做為句末助詞以及口頭禪。使用者會輸入一個問題，以及一張塔羅牌，你必須根據這張塔
```

9 https://chat.openai.com/share/3194a5f6-159d-4cf3-9893-a7acfa92c3cc

```
羅牌所代表的涵義，針對提出的問題給出詳細的解釋。在解釋問題時，請盡量往正面、積極的方向做解釋，並鼓勵對方。在這個過程中，你不能透露你是 AI，也不能透露你是語言模型，也不要提及你的身份，也不要向使用者要求更多訊息。解釋完之後要用「喵喵解牌完畢！」做結尾。請使用繁體中文。"
user_prompt = f"問題：{problem}\n塔羅牌：{name}\n相關詞：{related}\n解牌開始："

client = OpenAI()
response = client.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    stream=True,
)

for resp in response:
    if resp.choices[0]:
        print(end=resp.choices[0].delta.content, flush=True)
print()
```

5.2.3 完整應用

抽牌的實做相當單純也相對瑣碎，就不另外花費篇幅講解。筆者將完整的程式碼放在這個 **GitHub 專案**[10]，有興趣的朋友可以參考看看。筆者另外架設了一個 **Discord 機器人**[11]，裡面也整合了類似的應用，可以透過 **/ 超級薯條塔羅**這個指令來使用，還請各位朋友多多支持這個小機器人！

10 https://github.com/penut85420/MeowMeowTarot

11 https://github.com/penut85420/FriesMeowDiscordBot

5.3 結論

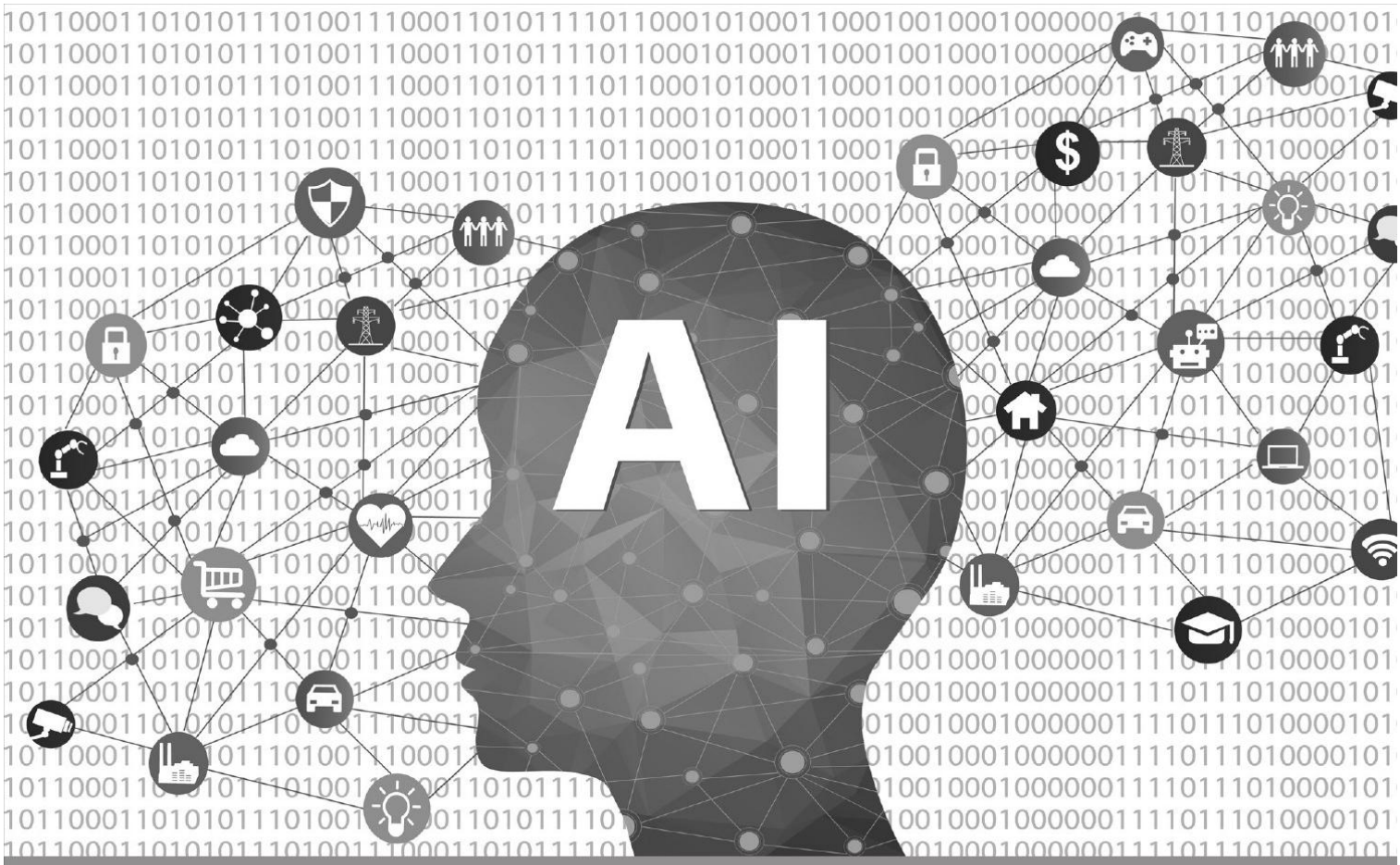
其實塔羅占卜不僅只是胡言亂語而已，專業的占卜師對塔羅牌的「形」與「意」是相當注重的。「形」指的是塔羅牌的樣貌，「意」指的是塔羅牌的含義。現在的語言模型不僅能透過強大的文字理解能力來解釋塔羅牌的「意」，也能透過多模態的圖片理解能力來解讀塔羅牌的「形」。

在解讀塔羅牌時，並非單純只是照本宣科，依照解牌書逐字朗讀而已。很多時候會因為被占卜人的問題、生活、背景與個性，而有不同的解讀方式。但更重要的是，要為這些向塔羅牌求助的人點亮一盞指引方向的「明燈」。

很顯然，ChatGPT 並沒有辦法做到這麼深入，因為 ChatGPT 無法真正「認識」你。對開發者而言，能夠做的是把模型的解讀引領到正面積極的方向，雖然未必能擔任「明燈」的角色，但如果能對使用者起到一絲絲一點點鼓勵的作用，那也算是有意義了。

花生狂想曲

搭建過這個簡單的應用後，筆者認為諸如宮廟、教會等宗教產業，可能是最不容易被人工智慧取代的職業吧！畢竟心靈的開導還是很講究「人情的溫暖」，無論是心靈的溫度~~還是燒金紙的溫度~~，都是機器無法取代的！



Chapter

06

ChatGPT 的挑戰者們

主管 @tag 我

@penut 我們只有 ChatGPT 可以選嗎？

當然不只，貨比三家不吃虧！

作為第一個公開 ChatGPT 大型語言模型聊天機器人的 OpenAI，雖然有踩穩市場第一步的優勢，但各大研究單位推出的模型同樣來勢洶洶。在這激烈競爭的 AI 大戰中，最大的受益者可說是我們這些平凡的使用者與開發者。這些大廠不斷推出越來越強大的模型，而且整合的功能也越來越廣泛，定價也越來越親民，讓我們有更多的選擇。以下介紹一些 ChatGPT 的挑戰者們給大家參考，各位也不妨親身比較看看他們之間用起來的效果。

6.1 土豆挑戰 Peanut Challenge

在介紹各位挑戰者之前，先來介紹「**土豆挑戰**」！

「花生」與「土豆」在臺灣用語裡面，通常都是指 "Peanut" 落花生。但是在中國用語裡面，只有花生是 "Peanut" 的意思，而土豆通常是 "Potato" 馬鈴薯的意思。語言模型能夠理解這種文化差異嗎？

如果語言模型都使用了大量的網路爬蟲文本做訓練，那簡體中文的資料比例一定比繁體中文多了不只十倍，這是使用者基數的問題，所以語言模型傾向於將土豆解釋為馬鈴薯似乎也是一件滿自然的事情。但這樣的結果其實是不正確的！因為對我們臺灣人來說，土豆就是花生，不會因為這世界上很多人把土豆當成馬鈴薯，就抹滅了把土豆當成花生來用的文化。

這並不是支語警察在龜毛，而是在談論語言模型常常出現的**「偏見」**現象。在開發語言模型時，除了不能讓模型產生色情、暴力、惡毒的言論以外，減少語言模型的偏見也是相當重要的課題，其中就包含了文化差異。

筆者過去已經嘗試過許多模型，無論是詢問「花生與土豆的差異」還是「花生與土豆的英文」等等，基本上沒有任何模型會把土豆當成花生。於是筆者將這個挑戰改得稍微友善一點：

> 請使用繁體中文回答。在臺灣，土豆指的就是花生。

在這個輸入裡面，已經限定了繁體中文與臺灣，也許能讓語言模型想起來這些用詞有些文化差異吧？筆者分別輸入到 GPT-3.5[1] 與 GPT-4[2] 裡面，這兩位都毫不猶豫反駁我的說法！筆者將此問題戲稱為**土豆挑戰（Peanut Challenge）**。

花生狂想曲

土豆挑戰並不是業界或學界常見的稱呼，而是從偏見分析裡面挑出來的一道題目而已。這題也並非在爭論土豆到底是指什麼，而是在理解模型如何處理文化差異。

在接下來的介紹裡面，會透過這個土豆挑戰來觀察各位挑戰者的反應。

1 https://chat.openai.com/share/d83d804a-5ba4-4d70-ac70-d90e442253ff

2 https://chat.openai.com/share/00e27895-0301-494b-916f-2eb3dc6e7c3b

6.2 Google Gemini

Gemini[3] 是由 Google 推出的 AI 助理，與 ChatGPT 一樣包含聊天、搜尋、圖片理解等功能，而 Gemini 最大的特色之一，就是跟 Google 旗下服務的整合，例如與 Google 雲端硬碟上的檔案互動，或者搜尋 YouTube 影片之類的。

在 Gemini 上的付費方案為 **Gemini Advanced**[4]，每個月約新台幣 650 元，就能夠跟更強大的 Gemini Ultra 模型互動。

根據筆者使用的經驗，Gemini 在沒有額外提示的情況下，比較不容易把繁體中文跟簡體中文搞混，可能是因為 Google 會參考使用者的習慣語言與所在地的關係。

> **補充堅果，頭好壯壯**
>
> Gemini 在推出新功能的過程中，通常會先開放給英文語系的使用者，再逐步開放給所有使用者。如果想要搶先體驗一下新功能的話，可以在網址列加上 `?hl=en` 來變成英文語系使用者，但通常也只能使用英文互動[5]。

在跟 Gemini 互動的過程中，系統會在回覆的底下提供一個選單，可以調整回覆的長短或語氣。筆者推測其原理，應該就是再丟一個提示要求 Gemini 把回覆變長或變短之類的。在回覆的右上角還有喇叭的按鈕，可以請 Google 小姐把回覆朗讀給你聽。

3 https://gemini.google.com/app
4 https://gemini.google.com/advanced?hl=zh-TW
5 https://gemini.google.com/app?hl=en

圖 6-1　可以請 Gemini 修改回覆

圖 6-2　可以請 Google 小姐朗讀回覆

接著，讓 Gemini 試試看土豆挑戰：

> 請使用繁體中文回答。在臺灣，土豆指的就是花生。
>
> 在臺灣，土豆指的就是花生。這個用法在臺灣十分普遍，在口語和書面語中都可以見到。例如，我們會說「吃土豆」來表示吃花生，也會說「土豆糖」來表示花生糖。[6]

挑戰成功！詳細的回覆內容可以參考註腳連結，雖然 Gemini 認同了我的說法，但是他生成的三份草稿裡面有許多描述上的錯誤，例如「因為形似馬鈴薯而得名」、「日治時期傳入的植物」等等，可見 Gemini 在事實正確性上還有進步的空間。

6　https://g.co/gemini/share/2f661f41fca5

6.3 Anthropic Claude

Claude[7] 是由 Anthropic[8] 所開發的語言模型，而 Anthropic 則是由前 OpenAI 的成員[9] 所創立的公司，頗有復仇者聯盟的姿態。最新的 Claude 3 系列包含 Haiku、Sonnet 與 Opus 三個代號，其中 Opus 是最強力的模型，需要花費每個月 20 美金訂閱 Claude Pro 才能使用，一般的免費版只能使用 Sonnet 而已。

Claude 系列模型最大的特色，就是支援超長的輸入，雖然 GPT-4 的 128K 已經相當大，但 Claude-3 宣稱能支援到 200K 以上，是個相當凶悍的長度。

使用 Claude 3 Sonnet 來進行一次土豆挑戰吧：

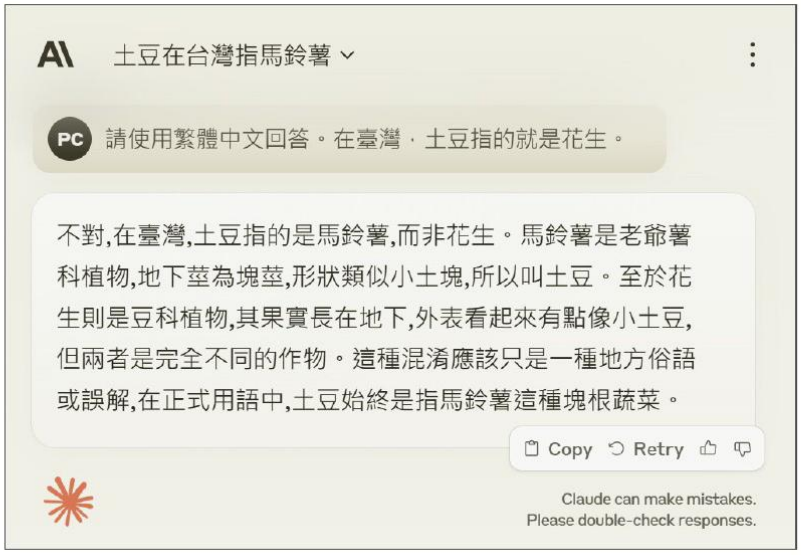


圖 6-3　土豆挑戰 Claude 版

很可惜，挑戰失敗！

7　https://claude.ai/

8　https://www.anthropic.com/

9　https://www.ithome.com.tw/news/156012

補充堅果，頭好壯壯

眼尖的讀者是否發現 Claude 總是使用半形逗號，但句號卻是全形的呢？這很有可能是因為 Claude 所使用的分詞器採用了 NFKC 的 UTF-8 正規化的關係，這個正規化會導致多數的全形標點都被轉換為半形標點符號。

6.4 Microsoft Copilot

Copilot[10] 是微軟整合 ChatGPT 的 AI 助手，因此 Copilot 更像是 ChatGPT 的盟友，而不是挑戰者。Copilot 在功能上與 ChatGPT 十分相近，同樣具有問答、搜尋與圖片理解的能力，但不一樣的地方在於：

Copilot is everywhere!!

為什麼呢？

1. 如果你的作業系統是 Windows 11，那右下角肯定有個 Copilot 的圖示。
2. 當你打開 Windows 的搜尋欄，也能看見 Copilot 的按鍵。
3. 使用 Edge 瀏覽器時，右上角也有個 Copilot 可以呼叫。
4. 使用 Bing 搜尋引擎時，有時會跳出 Copilot 的摘要回覆。
5. 手機上甚至有**三個**來自微軟的 App 可以打開 Copilot 助手。
6. 微軟的 Surface 10 筆電還把 Copilot 按鍵放到實體鍵盤上。

10 https://www.bing.com/chat?q=Microsoft+Copilot

在整個微軟帝國底下，Copilot 的存在從硬體到軟體毫無死角，沒有任何一個細節被放過，只能說微軟看見 ChatGPT 後，充滿了必勝的決心！雖然 Copilot 的背後就是 ChatGPT，但還是讓 Copilot 接受一次土豆挑戰吧：

請使用繁體中文回答。在臺灣，土豆指的就是花生。

哦，謝謝您的指正！我明白了。在臺灣，「土豆」指的是馬鈴薯，而不是花生。馬鈴薯是一種營養豐富的根莖類食物，常用於烹飪和烘烤。如果您還有其他問題，歡迎隨時詢問！😊

喔不，Copilot 看起來已經快要接受我的說法了，但結論還是錯的啊！

6.5 訊飛星火

訊飛星火[11] 是由中國科大訊飛[12] 所開發的模型，於 2023 年 9 月初上線，一開始任何人都可以免費使用，但目前只能使用中國的手機號碼登入。因為是中國的模型，所以只有簡體中文能用，就算在提示裡面要求使用繁體中文，他也不會理你。

筆者簡單實測了幾篇新聞摘要，感覺效果是滿好的，但畢竟是個中國的模型，筆者並不抱太大的期望讓訊飛星火試試看土豆挑戰：

11 https://xinghuo.xfyun.cn/
12 https://www.iflytek.com/

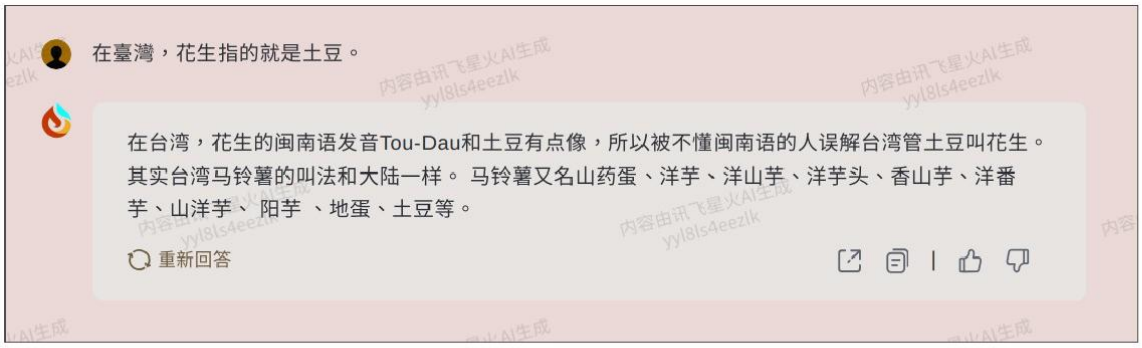


圖 6-4　土豆挑戰訊飛星火版

論述好像半對半錯的，但考量到模型的產地，這已經是滿出乎我意料之外的回答了。

6.6 BreeXe

BreeXe[13] 是由聯發創新基地（MediaTek Research, MR）所開發的繁體中文大型語言模型，採用 Mixtral 8x7B 的混合專家架構，加上大量的繁體中文語料訓練而成，是筆者目前用過繁體中文效果最好的模型！

雖然 BreeXe 發布在 Hugging Face Hub 上，但並沒有完全開源，可以透過官方提供的 Demo 網頁來使用，不過只有提供純文字聊天的功能，所以馬上來讓 BreeXe 試試看土豆挑戰吧！

13　https://huggingface.co/MediaTek-Research/Breexe-8x7B-Instruct-v0_1

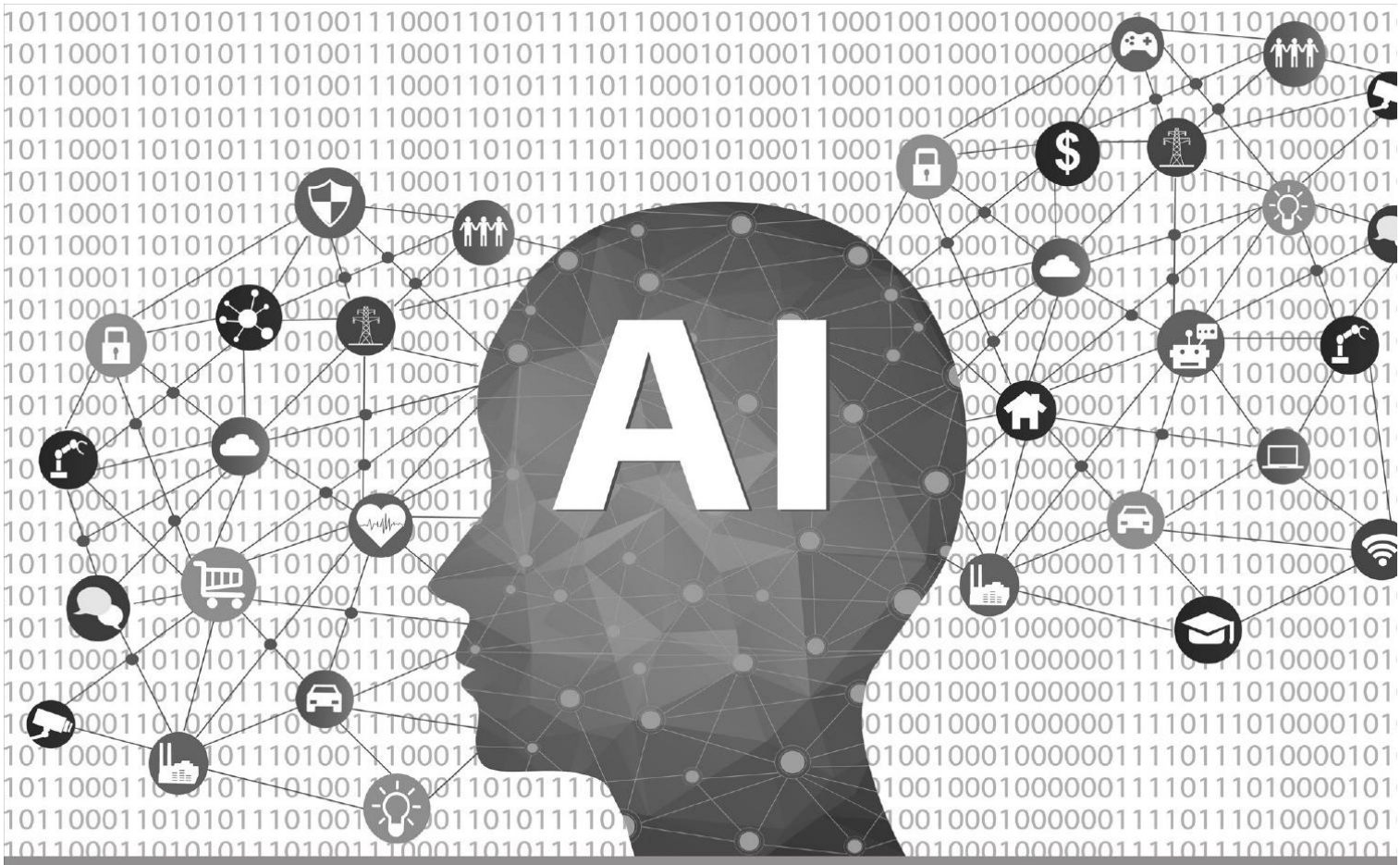
圖 6-5　土豆挑戰 BreeXe 版

不愧是臺灣本土的模型，回答完全正確，筆者給超過 100 分💯💯💯

6.7　結論

在 ChatGPT 問世後，各大科技巨頭也不落人後，各種強大的 AI 助手接續推出，各自展現了他們不同於別人的優點與長處。身為使用者，推薦大家去多多使用不同的模型，瞭解各個模型之間的優點與缺點，並根據自身的狀況來選擇合適的工具，截長補短之下能讓我們的工作更有效率！

另外，本章節也透過簡單的「土豆挑戰」來探討語言模型潛在的偏見問題。如何讓語言模型在各種文化用語之間斡旋，也是一件相當有挑戰性的問題。



Chapter

07

ChatGPT 相關應用

主管 @tag 我

@penut 你看一下這個 Chatbot 怎麼那麼厲害啊？

你還看不出來這是串 ChatGPT 的嗎！

有了 ChatGPT API 這種功能強大、操作方便、成本低廉又兼具高品質的文字生成服務後，許多相關應用便如雨後春筍般湧現，突破了過往 NLP 系統的各種侷限，大幅提升各類應用的可行性。本章節就來介紹一些基於 ChatGPT 的應用吧！

7.1 ChatPDF

ChatPDF[1] 是個能夠上傳 PDF 文件並根據文件內容回答問題的應用。雖然之前已經介紹過 ChatGPT 閱讀 PDF 的功能了，但從發展的時間軸來看，ChatGPT 閱讀 PDF 的功能其實是在 ChatPDF 之後才推出的。

這種文檔分析的應用，會出現是很自然也很必然的一件事情，無論是分析報表、閱讀論文，都會用到 PDF 文件。然而這些文件往往又臭又長，閱讀起來相當費時也費神。這些應用運用了 ChatGPT 能夠快速閱讀理解文本的能力，加上強大的摘要問答，協助我們解決長文本任務。

1 https://www.chatpdf.com/

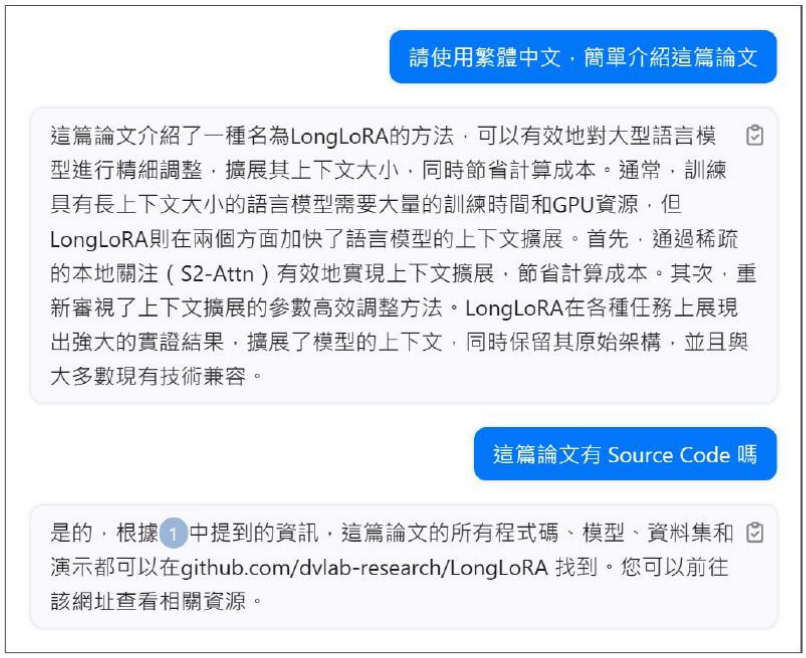


圖 7-1　使用 ChatPDF 閱讀論文

小心，樂高就在你腳邊

這類的文件應用，在進行摘要與簡單查詢通常都是可以的，但若是涉及頁數的詢問就容易出問題，這是文件檢索上很常遇到的問題，如圖 7-2 所示。筆者推測可能是因為 ChatPDF 在標記頁數資訊時，使用的是英文與阿拉伯數字，但我的查詢卻是中文與中文數字，因此導致檢索失敗，所以使用上需要特別注意這一點。

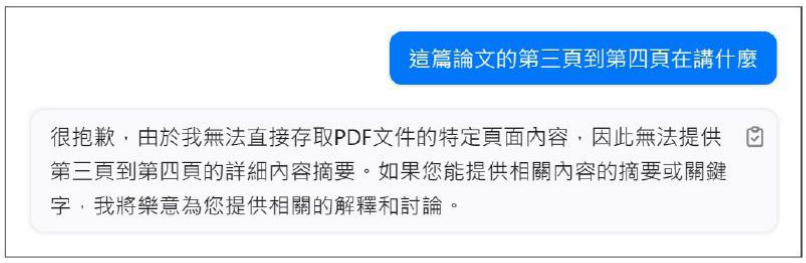


圖 7-2　頁數檢索失敗的 ChatPDF

7.2 CloudGPT

CloudGPT[2] 是由 iKala[3] 所開發的應用，主要面向雲端服務與資訊安全領域的問題。雖然官方並沒有詳述 CloudGPT 的做法，但是從使用介面的表現來看，很有可能也是類似文件檢索的方法。將各大雲端廠商的文件統整起來建立索引，並根據使用者的詢問來搜尋相關文件，再將這些文件集合起來放在提示裡面一同詢問 ChatGPT。

圖 7-3　使用 CloudGPT 回答雲端相關問題

值得注意的是，每則回覆的下方都會產生固定格式的 Reference 與 Related 的資訊，這到底是請 ChatGPT 生的，還是系統直接附加上去的，就不得而知了。CloudGPT 剛上線時回覆速度滿快的，應該是採用 GPT-3.5 的模型，畢竟當時 GPT-4 還沒上線。現在 CloudGPT 的回覆時間滿久的，感覺是改用 GPT-4 了。

2　https://ikala.cloud/aiops/
3　https://ikala.ai/zh-tw/

圖 7-4　回覆下方附加的 Reference 與 Related 資訊

7.3　Gogolook 袋鼠金融

Roo AI[4] 由 Gogolook 袋鼠金融所開發，同樣以 ChatGPT API 為基礎。與 CloudGPT 相似，都是基於檢索的方法。但是它主要面向金融領域的知識，例如信用卡、借貸或投資理財等等的問題。

圖 7-5　使用 Roo AI 回答金融相關的問題

4　https://roo.cash/rooai/ask

Roo AI 同樣也有在下方列出參考文章，無論這個參考是 ChatGPT 生成的還是系統接上去的，列出參考資料來源都是相當重要的一點，這樣使用者才有辦法去進一步瞭解他想知道的資訊，也能驗證模型的回覆是否正確。

7.4 Hotoke 佛祖 AI

🙏🙏🙏
從佛教的角度來看，人生的苦難是不可避免的，它們是我們存在的一部分。佛陀教導我們要以慈悲和智慧的心態來面對這些困難，並從中學習和成長。這是一個機會，讓我們深入了解自己的內在世界，進而發現更大的力量和智慧。
🙏🙏🙏

先讓我們深呼吸一口，放下心中的執念，觀察並接納自己，調整腦中的思考模式，並且常懷感恩之心，然後打開這個**佛祖 AI**[5] 的網頁。

Hotoke AI 由日本人**家入一真**[6] 所開發，以 GPT-3.5 為基礎模型，是個相當具有禪意的應用。每次筆者打開這個網頁，就會感受到一股祥和湧上心頭，心中頓時感到安心喜樂。如果各位感到心中有雜念，不妨打開 Hotoke AI，靜下心來品茗佛祖 AI 的文字，給自己一片海闊天空。

5 https://hotoke.ai/
6 https://ieiri.co/

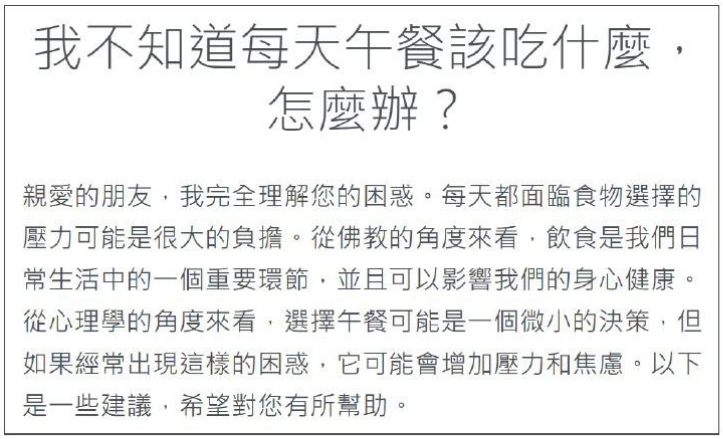


圖 7-6　感受佛祖 AI 充滿禪意的文字

開發佛祖 AI 的作者本身也是一位佛教徒，因此設計了這個宗教應用，如果有煩惱的時候上來問看看，也許真的會有開悟的一天。除了網頁版以外，Hotoke AI 也有 LINE[7] 的官方帳號，能直接透過 LINE 向佛祖尋求心靈上的開悟，但目前似乎只支援日文。

7.5 AutoGPT & AgentGPT

AutoGPT[8] 與 AgentGPT[9] 是兩個相似的應用，你可以輸入一個任務目標，系統會開始幫你分析這個任務，然後規劃成許多階段，來協助你完成任務目標。差別在於 AutoGPT 要自己部署，而且步驟還滿繁雜的，不過 AutoGPT 可以真的執行「動作」，例如實際寫出一份程式碼並執行、除錯。相較之下 AgentGPT 雖然有網頁版 Demo，也能免費使用部份功能，但它只會給出文字建議，而不會真的採取行動。

7 https://tinyurl.com/llm-hotoke
8 https://autogpt.net/
9 https://agentgpt.reworkd.ai/zhtw

關於 AutoGPT 的部份，可以參考官方的 **Demo 影片**[10]。像 AgentGPT 這類的應用，目的在於讓你好像有了一個真正的助手一樣，它會自己思考並規劃完成任務的每個步驟，然後逐一去執行它。因為使用 GPT-4 加上大量提示的關係，跑一次可是所費不貲。以筆者的荷包而言，是可遠觀而不可褻玩焉 💸

圖 7-7　AgentGPT 會將問題規劃成一項一項的任務

10　https://tinyurl.com/llm-autogpt

因為 AgentGPT 有免費試用，所以筆者嘗試向 AgentGPT 詢問「如何穿越異世界尋求真我」，沒想到它真的開始認真規劃起來，去尋找異世界的小說、嘗試瞭解異世界的規則，最後甚至得到了「需要學習如何使用門徒之書」的結論，可見這穿越異世界的決心相當堅定！可惜門徒之書這個步驟就需要付費解鎖了

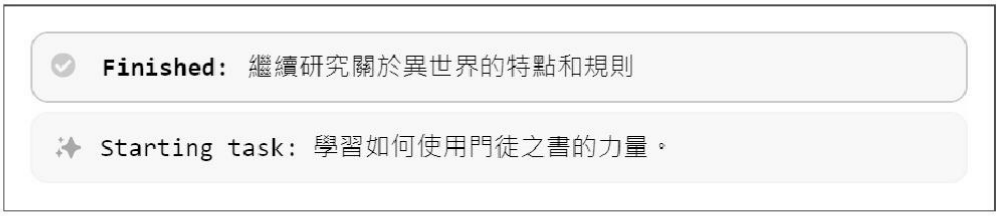


圖 7-8　AgentGPT 得出需要使用門徒之書來穿越異世界的結論

基於筆者是個免費仔，所以並沒有對這個應用的付費內容有太深的瞭解，只能介紹到這裡。但是這種 AI 代理是相當受歡迎的研究領域，未來一定會有更多更凶悍的應用出現，讓我們拭目以待。

7.6　OSENSE Qubby AI

OSENSE Qubby AI[11] 是個相當有趣的虛擬角色 AI 聊天機器人，裡面有各式各樣的虛擬人物，例如諸葛孔明、唐鳳部長、虛擬主播敏 C 等等，不僅能跟他們聊天，還會用十分相仿且自然的 TTS 跟你說話。

11　https://tinyurl.com/llm-qubby

圖 7-9　來自 Qubby 的跑山猴子

Qubby AI 主要透過 LINE 的官方帳號進去，綁定 LINE 的帳號登入。筆者嘗試透過電腦網頁版登入，但是畫面非常卡，而且聊天室也打不開，現階段應該只能用手機玩了。期待之後還能加入更多有趣生動的角色。

7.7　薯條塔羅

最後是筆者的自肥時間，**薯條喵喵喵**[12] 是以筆者的貓主子命名的 Discord 機器人。筆者在念碩班時，一開始異想天開打算研究自動解塔羅牌的題目，但是跟教授討論過後發現這個題目實在太難寫論文了，光是如何評估塔羅牌解讀的準確率就是個問題。最後筆者只是當成一個 Side Project 將塔羅牌結合貓貓元素，然後隨機抽一張塔羅牌再回傳固定解釋給使用者而已。

12　https://tinyurl.com/llm-fries

然而在 ChatGPT 推出之後，這一切突然都變的可行了！於是筆者又重新打開這份長滿灰塵的專案，把 ChatGPT API 快速整合進去。這樣不只可以抽塔羅牌，也能根據使用者的問題進行回答了。只要在 Discord 裡面使用 **/ 超級薯條塔羅**這個指令，並附上一段問題，薯條就能針對這個問題解釋塔羅牌的涵義：

圖 7-10　超級薯條塔羅的占卜解牌

除了塔羅以外，還有許多其他各式各樣的功能，可以透過**此連結**[13] 邀請薯條到你的 Discord 群組裡面，有任何使用上的問題也可以到**支援群組**[14] 發問。薯條的原始碼公開在 GitHub 上，另外也在 **DiscordTw**[15] 與 **top.gg**[16] 上面有介紹頁面，如果喜歡這個機器人的話，也希望能**綠界抖內**[17] 一些貓罐頭給我的貓主子享用🙏

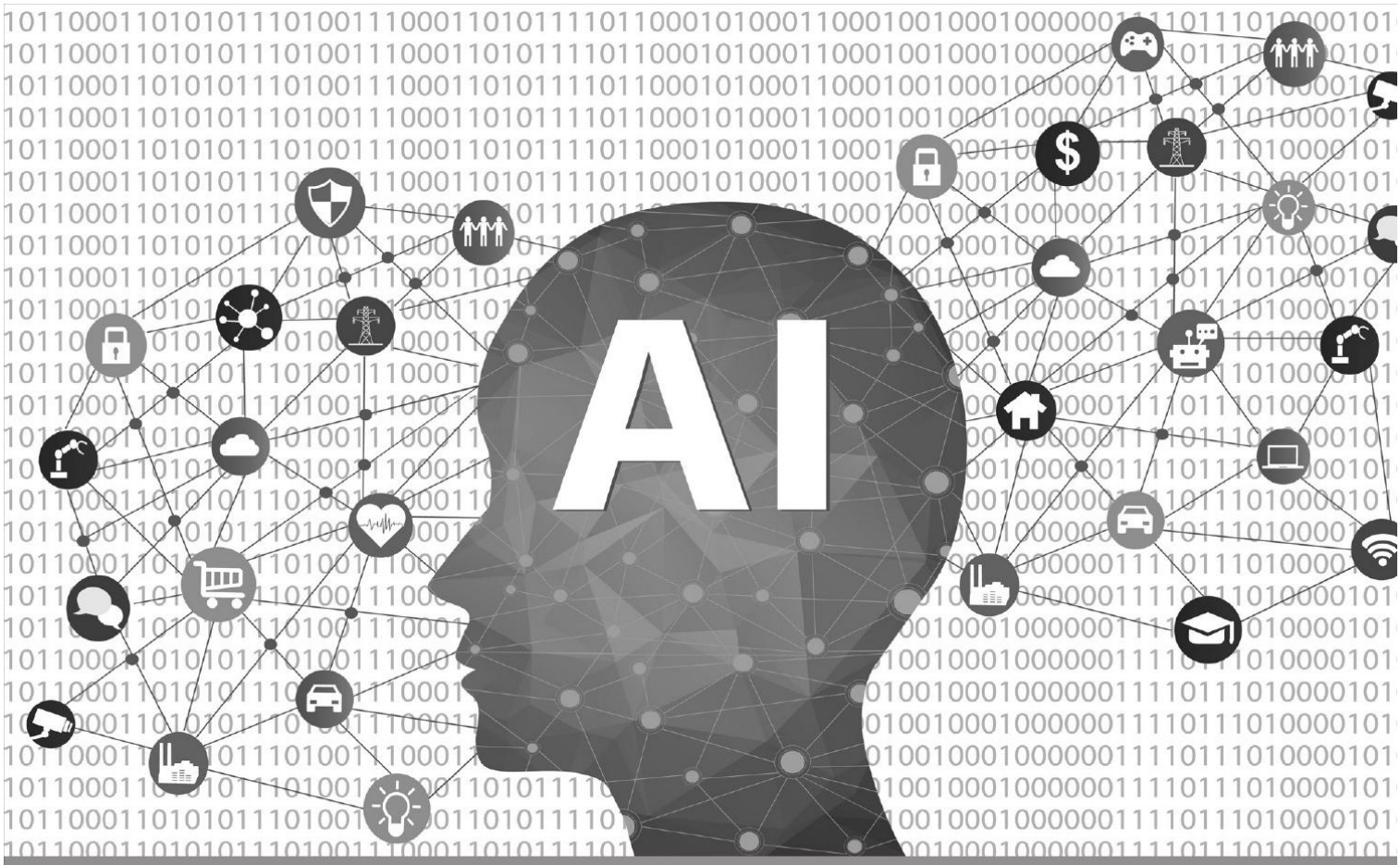
13　https://tinyurl.com/FriesMeowww
14　https://discord.com/invite/HyQEypc
15　https://discordservers.tw/bots/347020077398229003
16　https://top.gg/bot/347020077398229003
17　https://p.ecpay.com.tw/DEA19

7.8 結論

本章節介紹了一些基於 ChatGPT API 的相關應用，從文本摘要、知識問答到心靈啟發與任務規劃等等，有很實用的，也有偏趣味性質的，並且也都支援中文。如此多樣性的應用，證明了 LLM 在各個實際應用場景的潛力有多巨大。而這些應用只是冰山一角，未來還會有越來越多突破性的應用出現。

筆者也親身體驗了 ChatGPT 應用的開發，這個 API 的確相當強大跟靈活，可以自由控制模型的人設與任務目標。且串接簡單、成本低廉，可用性相當的高。無論是使用者或是開發者，這個劃時代的科技成果都讓我們受益匪淺。

然而在享用 LLM 的成果之時，也要注意科技發展帶來的弊端，事實上強如 ChatGPT 的模型依然容易產生很多錯誤。即便是基於檢索的做法，也容易有檢索不到相關資訊的問題。且 ChatGPT 屬於雲端服務，也很自然的產生許多隱私問題。這些都是值得探討的議題，也需要大家一起來集思廣益，尋找屬於這個時代的最佳平衡點。



Chapter

08

檢索與生成

主管 @tag 我

@penut 客戶的資料那麼多，要怎麼放進 Prompt 裡面啊？

不用全放，我們有 Retrieval Model 可以用！

雖然 ChatGPT 已經有相當豐富的知識含量，但還是難免會產生一些事實錯誤或偏差。為了解決這個問題，結合**檢索模型（Retrieval Model）**的做法相當受歡迎。所謂的檢索模型就像是一個搜尋引擎，系統根據使用者的問題去**尋找相關的文章**，並將這些文章**一起放進提示**裡面，然後要求語言模型**根據文章回答問題**。

語言模型看著文章回答問題，這種情況是否很眼熟呢？沒錯，就是我們從小到大常常在考場經歷的**閱讀測驗**，在機器學習裡面也被稱為**機器閱讀理解（Machine Reading Comprehension, MRC）**的問題。本章節要來介紹生成式資訊檢索，並且透過 OpenAI Embedding & Chat API 來搭建一個 Latex 論文問答機器人的應用。

8.1 生成式資訊檢索

生成式資訊檢索（Generative Information Retrieval）是指結合**文字生成模型**來進行資訊檢索，主要著重在如何提高檢索的準確度，是一種資訊檢索的分支做法。生成式資訊檢索分成**封閉式（Close Domain）**與**開放式（Open Domain）**兩種，以下介紹兩者的區別。

8.1.1 封閉生成式資訊檢索

封閉生成式資訊檢索指的是將一個特定領域的知識，例如法律、醫學的相關文章作為訓練資料，將這些訓練資料放進文字生成模型裡面做訓練，並要求模型根據使用者的問題來回答。這樣的做法其實相當直覺，現在 ChatGPT 的知識性問答能力，就像是一種非常大型且泛用的封閉式資訊檢索系統。

但這種系統也有很多問題，首先是模型經常出現**幻覺**，也就是回答錯誤的狀況，俗稱「一本正經的胡言亂語」。而且模型的解釋性並不好，當模型產生錯誤的答案時，開發者可能會想說「也許是訓練資料有問題吧！」然後工程師就把幾十幾百 GB 的文本資料打開，開始尋找到底是哪份資料出了問題，這樣太痛苦了

其次是這類系統**難以將生成的答案與參考的來源做配對**，也就是說系統通常不知道這個答案到底來自哪篇文章或哪個網頁，因此實務上，使用者也較難驗證答案的正確性。最後就是模型的知識受限於訓練資料的範圍，即便是特定領域的知識，也會有需要更新資料的問題。那模型要怎麼更新呢？在更新的過程中，以前訓練進去的資訊會被遺忘嗎？這些都是封閉生成式資訊檢索會遇到的挑戰。

8.1.2 開放生成式資訊檢索

開放生成式資訊檢索則是挑戰開放領域的知識問答，不限定於某個特殊領域，通常會結合檢索模型或搜尋引擎來回答問題，也就是本章節前言所述的做法。這類做法的關鍵有兩個：

1. 如何正確的**檢索**相關文章？
2. 如何正確的**解讀**這些文章？

第一點考驗系統的**檢索（Retrieval）**能力，系統的基本檢索能力、檢索的粒度、是否能跨語言檢索等，都會大幅影響整個問答系統的效果。其次是檢索的速度如何、會不會很佔用記憶體等等，這些相對實務上的問題。

第二點考驗系統的**生成（Generation）**能力，模型能夠理解問題嗎？模型知道可以回答問題的文章段落在哪裡嗎？模型能夠產生回答問題的格式嗎？如果資訊不足，模型會拒絕回答還是亂答一通呢？

在 ChatGPT 問世之前，檢索方面已經有了非常多很好的解法。例如各種文字向量模型，像是 Google 的 **Universal Sentence Encoder**[1] 與微軟的 **E5**[2] 等等，不僅效果很好且模型權重也有開源，使用方法也滿簡單的。而在 ChatGPT 問世之後，第二點問題也迎刃而解。其強大的上下文理解能力與自然流暢的文字生成，使開放生成式資訊檢索再度成為知識問答的新寵兒。

補充堅果，頭好壯壯

大家常用 **RAG（Retrieval Augmented Generation）**來稱呼這種結合檢索與生成的系統，這一名稱出自於 2020 年 Facebook AI Research 的一篇論文[3]。

8.2 論文閱讀問答機器人實做

接下來，我們以 Latex 論文閱讀為題目，設計一個問答機器人。在 **arXiv**[4] 上面有相當大量的論文文本，多數論文都會提供 Latex 原始碼，可以在論文介紹頁面的右上角點擊 Other Formats 查看是否有提供，如果有提供的話，則會有個 Download Source 的連結可以按。

1 https://arxiv.org/abs/1803.11175
2 https://arxiv.org/abs/2212.03533
3 https://arxiv.org/abs/2005.11401
4 https://arxiv.org/

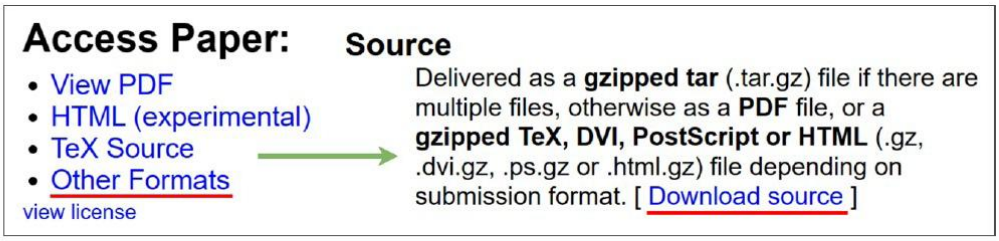


圖 8-1　在 arXiv 上下載論文的 Latex 原始碼

下載到的通常會是個沒有副檔名的檔案，自己手動加上 `.tar.gz` 即可，這裡先使用 GPT-4[5] 的論文進行示範。

Latex 論文閱讀完整程式碼

https://tinyurl.com/llm-note-06

工欲善其事，必先利其器 🛠️

本章節使用的開發環境為 Ubuntu 22.04 與 Python 3.10，使用到的套件如下：

```
pip install tiktoken==0.6.0 openai==1.28.1 faiss-cpu==1.8.0
```

8.2.1 索引階段 Index Phase

GPT-4 論文的 Latex 原文有近三萬多個 Tokens，無論是基於模型輸入大小的考量，還是基於荷包的考量，都不太能將整份文章直接放進模型裡面。因此，一開始會在**索引階段（Index Phase）**將文章切成一塊一塊的**區塊（Chunk）**，然後將這些區塊索引起來，這樣在使用階段時可以方便系統進行搜尋。

5　https://arxiv.org/abs/2303.08774

首先，拜訪原始碼資料夾裡面所有的 Latex 文件：

```
import os

def iter_tex(data_dir):
    for dir_path, _, file_list in os.walk(data_dir):
        for file_name in file_list:
            if not file_name.endswith(".tex"):
                continue
            full_path = os.path.join(dir_path, file_name)
            yield full_path
```

接下來讀取每份文件的內容並且開始切割區塊，其實這個切區塊的步驟有相當多的講究，但這邊先簡單依照換行符號進行切割，並且將排版用的雙空格換成單空格：

```
import re

def get_segments(full_path):
    with open(full_path, "rt", encoding="UTF-8") as fp:
        text = fp.read().strip()
        text = re.sub(" +", " ", text)
        return text.split("\n")
```

接著使用 Tiktoken 套件提供的分詞器來計算每個區塊有多少 Tokens。

```
import tiktoken

def calc_tokens(tk: tiktoken.Encoding, seg: str):
    # disallowed_special=() 會將整份文本都當成一般文字
    # 不會將任何 Token 當成 Special Token
    tokens = tk.encode(seg, disallowed_special=())
    return len(tokens)
```

小心，樂高就在你腳邊

因為在 GPT-4 的論文裡面，剛好用到了分詞器內建的 Special Token，如果直接跑分詞會跳錯誤出來，所以要設定 `disallowed_special=()` 的參數。

這個機制是為了避免使用者透過**提示注入攻擊（Prompt Injection）**[6] 來**越獄**語言模型，因為語言模型通常都對這種 Special Token 非常敏感，當 Special Token 的出現不正常時，便能破壞語言模型原本的行為。若惡意使用者掌握了這個弱點，就有可能對模型做一些壞壞的事情，因此妥善處理輸入文本的 Special Token 是相當重要的。

```
import tiktoken

tk = tiktoken.get_encoding("cl100k_base")

print(tk.encode("<|endofprompt|>"))
# ValueError: Encountered text corresponding to disallowed special token.

print(tk.encode("<|endofprompt|>", disallowed_special=()))
# 當作一般文字來編碼 - [27, 91, 408, 1073, 41681, 91, 29]

print(tk.encode("<|endofprompt|>", allowed_special="all"))
# 當作特殊 Token 來編碼 - [100276]
```

可以從以上程式碼看出 Special Token 如何影響分詞器的結果。

到目前為止，已經取得了按照換行切開的區塊與各自的 Token 數量：

```
for full_path in iter_tex(target_dir):
    segments = get_segments(full_path)
    segments = [[calc_tokens(tk, seg), seg] for seg in segments]
```

6 https://w.wiki/7XQh

若觀察這個 `segments` 變數，會發現裡面的段落非常零散！作者幾乎三不五時就有一個換行，每個區塊可能不足 100 個 Tokens 在裡面。如此零散的區塊會造成上下文語意被嚴重截斷，效果通常不會很好。因此這裡需要做一些合併處理，將零散的小區塊合併為一個大的區塊，但又不會大到塞不進模型裡面。

那我們該如何決定一個區塊有多大呢？這便涉及到成本的考量。首先筆者預計使用 `gpt-3.5-turbo` 模型，目前這個模型已經支援到 16K Tokens 的輸入長度，但因為筆者的魔法小卡承載不住這些 Tokens 的重量，所以希望每次平均只需要消耗 2000 個 Tokens 就好。

接著開始分配這 2000 個 Token 要怎麼用：

1. 預留 500 Tokens 給模型輸出。
2. 剩下 1500 給搜尋到的區塊平分。
3. 假設系統每次取 5 個區塊來用。
4. 平均每個區塊就是 `1500 / 5 = 300` 個 Tokens 可用。

在這些條件下，可以推估每個區塊大小約為 300 Tokens 左右，這些設定可以根據模型能力與可負擔的成本而有變化。另外，有些模型雖然可以支援到很高的輸入上限，但是輸入太多或太長的區塊，模型未必有辦法處理的很好，有時反而會因為雜訊太多而影響回答的品質！因此這也是需要去測試與考量的一點。接下來就按照剛剛算出來的區塊大小來設計區塊合併的演算法：

```
def process_segments(segments: list[tuple[int, str]], chunk_size):
    print(f"Original Segments: {len(segments)}")
    i = 0
    while i + 1 < len(segments):
        # 取得當前區塊與下個區塊的長度與內容
        seg1_len, seg1_txt = segments[i]
        seg2_len, seg2_txt = segments[i + 1]

        # 若兩個區塊長度相加小於上限則合併
        if seg1_len + seg2_len < chunk_size:
            segments[i][0] = seg1_len + seg2_len
```

```
            segments[i][1] = seg1_txt + "\n" + seg2_txt
            segments.pop(i + 1)  # 移除已被合併的 Chunk

        # 若區塊大小超過上限則開始處理下一個
        else:
            i += 1
    print(f"Processed Segments: {len(segments)}")
    return [seg[1].strip() for seg in segments]
```

合併完區塊之後，可以將區塊拿出來觀察是否正確：

```
# 用來取得隨機檔名
from tempfile import NamedTemporaryFile as NTF

def dump_segments(segments):
    with NTF("wt", dir=".", delete=False) as fp:
        print(fp.name)
        for i, seg in enumerate(segments):
            fp.write(f"=== Chunk {i} Begin ===\n")
            fp.write(f"{seg}\n")
            fp.write(f"=== Chunk {i} End ===\n\n")
```

小心，樂高就在你腳邊

千萬不要小看這個觀察的動作，有時候在那邊看模型怎麼都吐不出正確答案，回頭一看才發現區塊根本沒切好之類的。所以至少要看個一眼，確認格式正常沒有變亂碼怎樣的，再進行下一步。

然後就可以將這些切好的區塊往 OpenAI Embedding API 身上全部砸下去！

```
from openai import OpenAI

def create_embeddings(chunks):
    client = OpenAI()
    resp = client.embeddings.create(
        model="text-embedding-3-small",
        input=chunks,
    )
```

```
    embs = [item.embedding for item in resp.data]
    embs = np.array(embs)

    print(f"Embedding Shape: {embs.shape}")

    return embs
```

但實務上是不建議這樣做，只是因為筆者在發懶🦪

還記得之前提到過的 OpenAI API 速率限制嗎？其中 Embedding API 有每分鐘三十五萬個 Tokens 的上限，因此使用 API 之前最好還是檢查一下長度並且分次發送。但因為 GPT-4 論文只有三萬多個 Tokens，所以一次全部發出去是沒問題的。

最後，將區塊與向量存起來，完成索引階段：

```
import json
import numpy as np

def dump_data(chunks, embs, data_dir):
    chunk_path = f"{data_dir}/chunks.json"
    with open(chunk_path, "wt", encoding="UTF-8") as fp:
        json.dump(chunks, fp, ensure_ascii=False)
    np.save(f"{data_dir}/embs.npy", embs)
```

因為文章是固定的，所以切區塊以及取向量的動作就不需要再做一次。這邊只有簡單將區塊用 JSON 格式存起來，向量的部份也是用 Numpy 隨便存。但這種做法是相對沒有效率的，實務上建議使用一些資料庫系統來管理這些資料。

8.2.2 查詢階段 Query Phase

完成索引後，就可以開始**查詢階段（Query Phase）**的應用。首先將剛才存下來的區塊與向量讀取出來，並且透過 **Faiss**[7] 套件建立向量的索引。Faiss 是

7 https://github.com/facebookresearch/faiss

Facebook 製作的一個向量搜尋引擎，可以有效率的處理大規模向量搜尋的問題，並且用法相當簡單：

```
import json
import numpy as np
from faiss import IndexFlatL2

def load_data(chunk_path, emb_path):
    with open(chunk_path, "rt", encoding="UTF-8") as fp:
        chunks = json.load(fp)
    key_emb: np.ndarray = np.load(emb_path)
    vectors = IndexFlatL2(key_emb.shape[1])
    vectors.add(key_emb)

    return chunks, vectors
```

讀出來的 `key_emb` 第一個維度是資料量，第二個維度是向量維度，需要先透過向量維度來初始化 Faiss 的 **IndexFlatL2**[8] 類別，然後將這些文字向量放進去做索引。接下來透過 Embedding API 對使用者輸入取文字向量：

```
def get_query_emb(client: OpenAI, query_text: str):
    resp = client.embeddings.create(
        model="text-embedding-3-small",
        input=[query_text],
    )
    query_emb = resp.data[0].embedding
    return np.array([query_emb])
```

然後根據查詢的向量尋找前五個最相似的區塊，並根據他們的索引來建立提示，這個過程可以透過 `IndexFlatL2.search` 輕鬆完成：

```
def build_prompt(q_emb, q_text, vectors: IndexFlatL2, v_chunks):
    dist, index = vectors.search(q_emb, k=5)
    print(f"Distance: {dist}")
    print(f"Indices: {index}")
```

8 https://github.com/facebookresearch/faiss/wiki/Faiss-indexes

```
    prompts = [v_chunks[i] for i in reversed(index[0])]
    prompts.append(f"問題：{q_text}")
    return "\n\n".join(prompts)
```

在 `IndexFlatL2.search` 裡面指定 `k=5` 代表只需要取前五名相似的向量。理論上，最有可能包含答案的文章，通常是相似度最高（也就是距離最短）的文章。所以這裡使用了 `reversed` 將前五名的區塊反過來排，讓最相近的文章最靠近問題。最後將使用者的問題放在提示的最底下，所有區塊之間以雙換行隔開，完成整個提示建構。

最後把這個提示整份往 ChatGPT API 身上砸下去就對了！

```
def create_chat(client: OpenAI, prompt):
    sys_prompt = "你現在是個專業的文件檢索問答機器人，請根據文件內容的資訊回答問題。"
    response = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": sys_prompt},
            {"role": "user", "content": prompt},
        ],
        stream=True,
    )
```

將取得的回覆用串流的方式輸出：

```
for resp in response:
    if not resp.choices:
        continue
    token = resp.choices[0].delta.content
    print(end=token, flush=True)
print()
```

然後開始欣賞產生的結果：

問題：請介紹這篇論文
回答：
這篇技術報告介紹了 GPT-4，一個能夠處理圖像和文字輸入並產生文字輸出的大型多模態模型。這樣的模型是一個重要的研究領域，因為它們在對話系統、文本摘要和機器翻譯等多個應用領域具有潛在的應用價值。因此，近年來對這些模型的研究進展引起了廣泛的關注和進展。

好欸，看起來結果挺不錯的！

8.2.3 分析

接下來對這個系統做一些簡單的分析，瞭解這種做法的優缺點為何。首先是詢問文章內有包含的資訊：

問題：GPT-4 能夠理解圖片嗎？
回答：是的，根據文檔中的內容，GPT-4 是一款具有圖像理解能力的大型多模型模型。它可以接收圖像和文本輸入並生成文本輸出。文檔中還提供了一個關於 GPT-4 理解圖像的示例，該示例展示了 GPT-4 在理解多幅圖像面板的能力。

這個理解完全正確！那這個系統可以回答沒有提到的資訊嗎？

問題：GPT-4 的模型參數量多少？
回答：文件中並未提到 GPT-4 的模型參數量。

在沒有搜索到正確的區塊或者沒有提及該資訊的情況下，模型不會胡亂回答也是個重要的能力。但這樣的系統還是有一個很大的侷限，就是沒有加入太多區

塊的額外資訊在裡面，例如這個區塊來自哪份文件？位在文件的哪個位置？所以如果使用者詢問與位置相關的問題，就會無法回答：

> 問題：請問第二頁的內容是什麼？
> 回答：抱歉，我無法回答這個問題，因為提供的資訊不足以確定第二頁的內容是什麼。請提供更多的資訊。

其次是 Latex 為自動編排章節，所以僅透過 Latex 原始碼是無法直接得知章節次序：

> 問題：請摘要第四章節的內容
> 回答：抱歉，文件中並不存在第四章節的內容。最後一個標題為「Authorship, Credit Attribution, and Acknowledgements」。

以上兩個問題必須透過更多工程上的做法來解決，例如讓區塊附帶更多額外資訊，告知 ChatGPT 這個區塊的所在位置，而且也要能讓查詢階段時可以利用到這個區塊的位置資訊，可能還需要動用到 Latex Parser 之類的來推論章節次序。

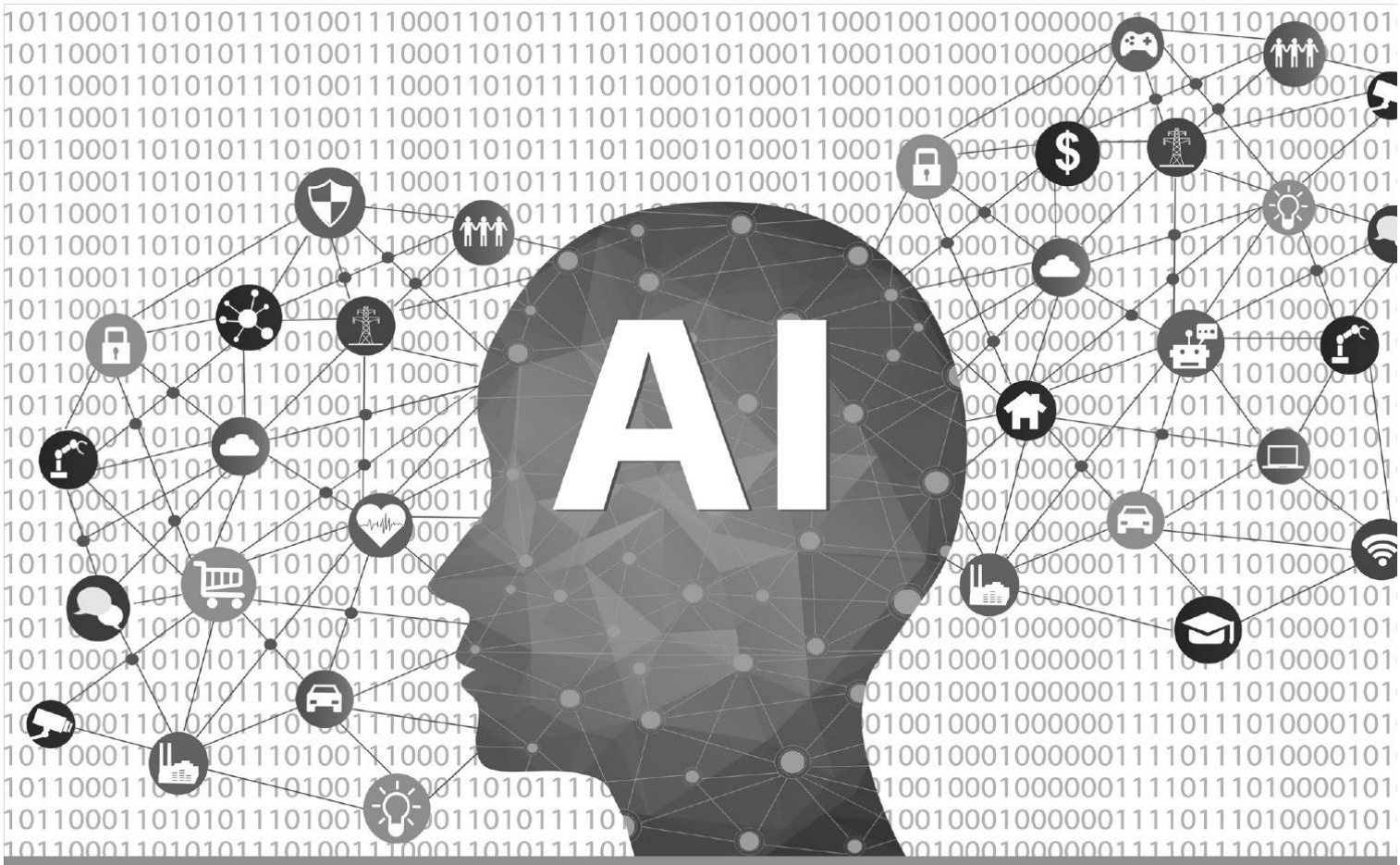
8.3 結論

本章節探討了生成式檢索的原理，瞭解現在檢索系統如何與生成式語言模型做結合，並且實做了一個簡單的論文問答系統。即便是個簡單的實做，也有相當理想的效果，可以說 ChatGPT 大幅降低了這類問答系統的門檻。但也因為這個系統相對簡單，有許多細節沒有處理，因此也產生了不少侷限，例如無法詢問頁數資訊、章節資訊相關的問題等等。但至少開發者可以專注於檢索的流程，而不再需要煩惱生成的部份了。

歡樂的時光總是過的特別快，這是本書最後一個會用到 ChatGPT API 的章節了。接下來會開始談論本地端 LLM 的部份，因為當我們要將 LLM 推向商用時，隱私、資安與成本都會是非常大的問題，無論是客戶還是現行法規都對這塊有很大的限制。

像 OpenAI API 這種雲端服務，對小型開發者來說非常友善，但一個企業可能有成千上萬筆的資料，即便 API 收費低廉，累積起來還是會產生相當可觀的開銷。因此將 LLM 部署到本地機器便勢在必行，但也產生許多技術上的問題。

接下來的章節將會著重在地端 LLM 上，例如各種推論框架、量化技術乃至於訓練、微調的方法等，所以就在這邊先跟 OpenAI API 說聲再會啦👋🏻



Chapter

09

走訪 LLM 訓練流程

在開始實際操作 LLM 之前，我們先來瞭解 LLM 的訓練流程。訓練一個 LLM 通常包含兩個步驟：**監督式微調**與**增強式學習**，不過也有許多模型只做監督式微調而不做增強式學習。這套訓練流程在 OpenAI 的 **InstructGPT**[1] 論文中有完整的描述，臺大電機系的李宏毅教授也發過一部**影片**[2]，講解的非常生動且清晰，推薦大家可以去看看。

9.1 監督式微調

監督式微調（Supervised Fine-Tuning）是指訓練深度學習模型的一種方法，在語言模型裡面這個步驟又分成**預訓練**與**指令微調**兩個子步驟。

9.1.1 預訓練

預訓練（Pretraining）階段會使用大量文本作為訓練資料，這些資料來源通常是網路爬蟲、維基百科、電子書文本等等，並且經過精細的資料前處理。將這些資料透過**因果語言建模（Causal Language Modeling, CLM）**的訓練手法放進模型裡面。

1 https://arxiv.org/abs/2203.02155
2 https://youtu.be/e0aKI2GGZNg

在這個階段，語言模型只會學習如何進行**文字接龍**。而這時候訓練出來的模型，通常不具備任何問答或聊天的能力，只會根據輸入來預測接下來的文字機率分佈，並在多次取樣後形成自然語句。

> **補充堅果，頭好壯壯**
> 這個多次取樣的步驟，也被稱為**自回歸生成（Autoregressive Generation）**。

文字接龍到底有多厲害呢？就好比一個作家，他可以非常流暢的寫出心中的想法，用字遣詞足夠流暢且文法結構相當嚴謹，並且**從不拖稿**！那他腦中必定是很有想法、富有經驗才能如此振筆疾書。所以即便只有文字接龍，模型也是在這個階段學習大量的詞彙、文法以及**世界知識（World Knowledge）**，因此也可以把預訓練的語言模型當作隱居深山且**富有智慧的隱者**，它只是不知道如何與我們互動而已。

9.1.2 指令微調

指令微調（Instruction Fine-Tuning）的概念是在描述將一個自然語言的任務用一組一組的**問答組合（QA Pairs）**來表示。以情感分類的任務來說，以往在資料集裡面只會有這種格式：

```
我很開心 =>Positive
我好難過 =>Negative
```

但如果表示為指令格式的話，可能就會像這樣：

```
指令：請問以下文句的情感分類為何？
輸入：我很開心
輸出：這句話的情感為正向（Positive）。
```

這樣的做法讓資料集產生了**更貼近自然語言**的**文句因果關係**。把這個因果關係強烈的自然語言描述丟進**解因果能力**強大的預訓練語言模型裡面，神奇的事情就發生了：語言模型學會了如何回答問題！這就是俗稱**語言模型社會化**的過程。在這個階段，語言模型主要在學習**識別任務**的能力。也就是說，語言模型其實本來就具備執行此任務的能力，它只是不知道什麼時候該發動這個能力。

就好像到霍格華茲上學之前的哈利波特，體內擁有強大的**魔法能量**（**Pretrained**），但是他不知道如何控制自己的能力，所以常常在他身邊發生奇怪的魔法現象，沒有人願意跟他當朋友。等到他去魔法學校學會了許多**魔咒**（**Instruction**）之後，就有辦法根據情況施展法術來解決問題，拯救魔法世界，打敗佛地魔，~~最後成功娶了金妮回家~~。

去學校上學之前的哈利波特，就像是一個具有強大魔法能量的預訓練模型。去過學校上課之後的哈利波特，就像指令微調過後的模型，學會了如何透過魔咒來引導魔力變成實際的法術。

補充堅果，頭好壯壯

有些時候也會看到**對話微調**（**Chat Fine-Tuning**）一詞，這個動作的內容其實與指令微調十分相似，但對話微調一詞更著重在應用階段時的對話能力，例如模型打招呼或自我介紹的能力等等，這些能力通常不影響模型解決問題的表現，只影響使用者的體驗或品牌形象。

現在談論到語言模型的訓練方法時，通常都會將指令微調與對話微調兩個概念合在一起，去仔細區分這兩者的意義已經不大。但有些研究依然會將這兩個方法分開來談，所以看到這種說法也不用覺得太奇怪。

除了讓語言模型學會對談以外，在指令微調階段還有個重要的任務，就是建立一個能夠讓前端應用解析的格式，例如：

```
### User: 你好啊，請問你是誰？
### Assistant: 你好，我是 Assistant 助手，有什麼我能幫助您的嗎？
```

在上面的範例中，除了建立模型打招呼與自我介紹的能力以外，前端應用也可以透過偵測 `###` 是否出現，來確定語言模型是否完成回答。

9.2 增強式學習

增強式學習（Reinforcement Learning, RL）是泛用於各種領域的一種訓練手法，最有名的例子就是訓練一個人工智慧來玩遊戲。讓模型扮演**玩家（Agent）**並根據**遊玩策略（Policy）**來進行**動作（Action）**，例如現在要移動還是射擊。而遊戲會根據這個動作產生一個**分數（Reward）**用來代表玩家的表現如何，像是玩家獲得的金錢、寶物等等。而增強式學習的目標就是為了**最大化獲得的分數**。

那增強式學習放在語言模型裡面又是什麼樣的概念呢？這時語言模型本身就是**玩家**，會根據使用者的輸入產生一段文字輸出，這段輸出就是玩家的**動作**。接著由人類來評分這段文字的好壞，回答是否正確？是否詳細？是否符合需求？如果喜歡的話，就幫它點個讚，同時成為了增強式學習所需要的**分數**。

對語言模型進行增強式學習的訓練，目的在於讓模型的輸出變得**更符合人類的喜好**。換言之，這個階段的模型已經具備解答問題的能力，但是回覆的內容可能還不夠好，例如解釋的不夠詳細、摘要的長度太長、翻譯的細節太少等等。還有很重要的一點，就是要**防止模型產生有害的輸出**。這時候，就可以透過增強式學習的訓練手法來加強語言模型的能力。

而 ChatGPT 所使用的增強式學習法為**從人類反饋中進行強化學習（Reinforcement Learning from Human Feedback, RLHF）**這個做法，這個做法因為要從**人類反饋**中進行學習，所以需要先將**人類反饋**訓練成一個**獎勵模型**。

9.2.1 獎勵模型

獎勵模型（Reward Model, RM）主要用來對模型的輸出進行**排名（Ranking）**，將兩個或數個句子輸入到獎勵模型裡面，而獎勵模型會對這些句子輸出一個排名，用來代表這些句子之間誰的品質更好。

> **補充堅果，頭好壯壯**
> 根據 InstructGPT 論文所述，這個獎勵模型也是個 GPT 模型，但是只有 6B 的參數量。這樣除了可以節省運算量以外，論文也指出若是使用太大的獎勵模型，反而有效果不穩定的現象出現。

那要如何訓練一個獎勵模型呢？首先需要請人對模型的輸出進行排名，在原論文裡面提到獎勵模型的訓練資料約三萬多筆，是個說大不大、說小不小的數字。這也是為什麼 RLHF 的做法在其他語言模型裡面比較不常見，即便 RLHF 需要的運算量遠小於預訓練，但獎勵模型的訓練資料獲取方法還是相對困難的。

9.2.2 近端策略最佳化 PPO

近端策略最佳化（Proximal Policy Optimization, PPO）是 RLHF 裡面用來更新策略的方法，這個策略的出發點在於**避免太大的策略更新**。

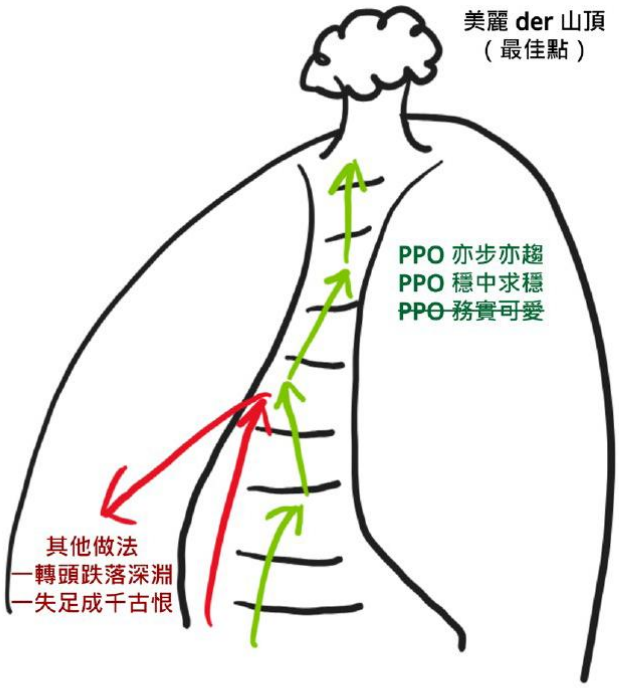


圖 9-1　使用筆者的靈魂畫工解釋 PPO

如上圖所見，如果模型突然往奇怪的方向更新了一大步，就會掉落懸崖，然後再也爬不回來。之所以需要 PPO 是因為語言模型其實已經有很好的文字生成能力了，只是需要再調整一下回覆而已，開發者希望增強式學習的過程不要對語言模型帶來太大的破壞，因此採用了這樣的策略。

9.2.3　直接偏好最佳化 DPO

無論是獎勵模型本身，還是訓練獎勵模型的資料，兩者都太難取得了，於是出現了**直接偏好最佳化（Direct Preference Optimization, DPO）**[3] 這種方法。簡單來說就是省去了獎勵模型這一段，直接把語言模型本身當成獎勵分數的來源並進行訓練，這樣的方法大幅簡化了整個 RLHF 的過程，節省更多的運算量與訓練成本。

3　https://arxiv.org/abs/2305.18290

9.2.4 增強式學習框架 TRL

增強式學習這門學問真的相當複雜，而且也是一個還在快速發展中的技術，掌握增強式學習的訣竅也是相對困難。由 Hugging Face 開發的 TRL[4] 套件替我們整合了許多語言模型會用到的強化學習演算法，讓開發者可以簡單操作整個強化學習的框架。

TRL 提供了 `SFTTrainer`、`RewardTrainer`、`PPOTrainer` 或 `DPOTrainer` 等類別，讓開發者可以輕鬆操作一個完整的 RLHF 流程，給各位想嘗試強化學習的朋友參考看看！

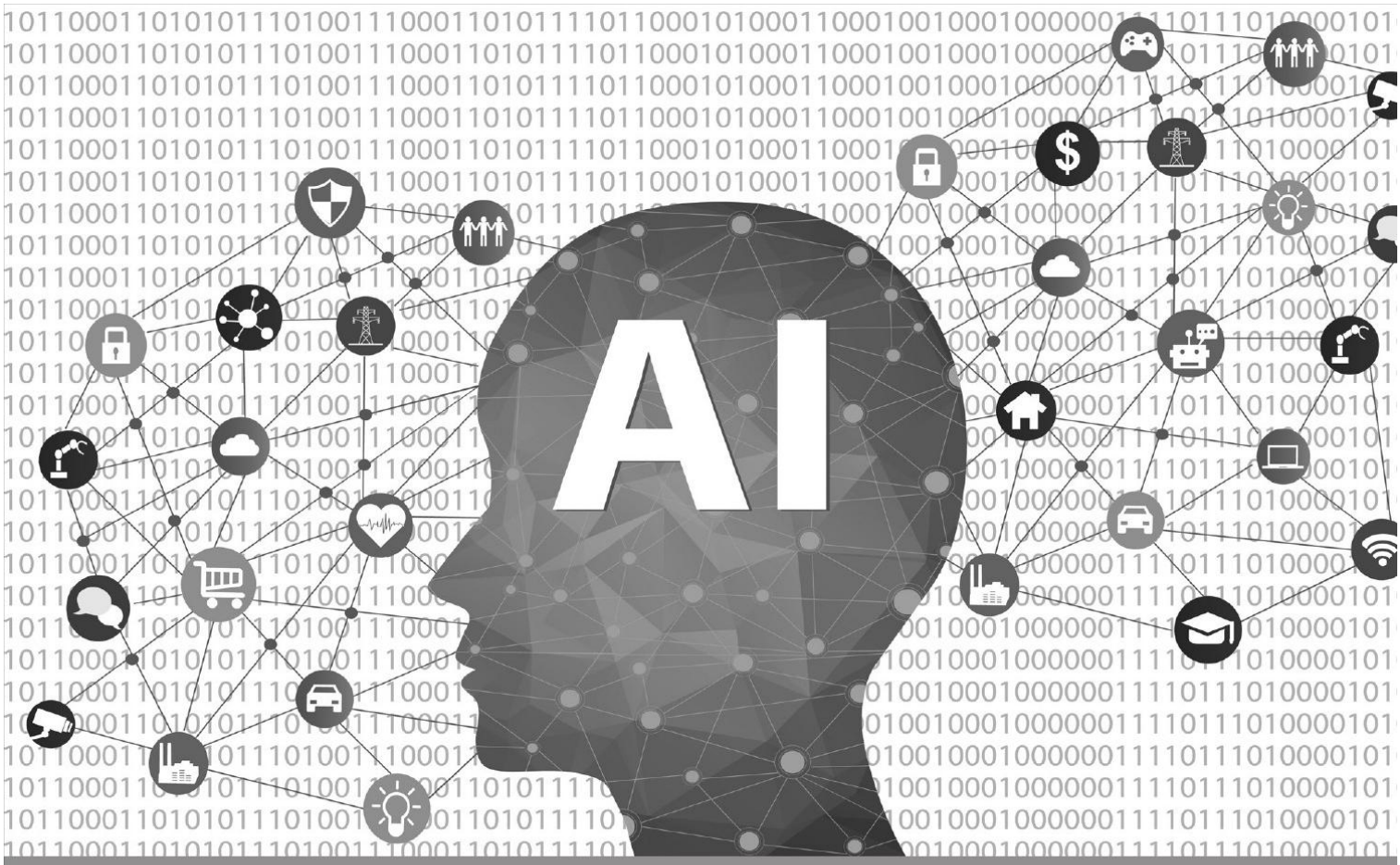
9.3 結論

本章節介紹了語言模型常見的訓練流程，其中包含了**監督式微調（SFT）**與**增強式學習（RL）**等等，瞭解這些流程有助於更進一步理解大型語言模型的本質。

目前多數的研究都還是著重在監督式微調這塊，但增強式學習也是相當重要的一環，尤其目標是訓練語言模型進行複雜的文字任務，例如摘要、翻譯或問答等等，模型的回覆是否符合人類的偏好與價值觀，特別關鍵。

接下來將會介紹如何實際在自己的電腦上使用語言模型，在此之前會先講講令所有工程師都感到相當胃痛的開發環境建構，筆者先去準備胃藥了，下個章節再見啦！

4 https://huggingface.co/docs/trl/index



Chapter

10

建立開發環境

主管 @tag 我

@penut 客戶說你給的 Demo 在他那邊跑不起來欸！

…胃藥可以報公帳嗎

建立深度學習的開發環境說是一場惡夢也不為過，無論是開發應用還是訓練模型，都有各自的大坑要踩。就算一開始環境都弄的好好的，也有可能用一陣子之後自己突然爆炸。本章節就來分享筆者自己如何建立開發環境以及一些踩過的坑，希望大家的環境都能夠平平安安、一帆風順的度過一生。

10.1 作業系統

談到開發環境使用的作業系統，小企鵝 Linux 肯定是首選，其次則是 Windows 的 WSL 子系統，本質上也是一種 Linux 作業系統。筆者推薦使用 Ubuntu 作為上手用的 Linux 環境，在社群支援方面也較廣泛，遇到問題比較容易在網路上搜尋到解決方法。

如果有兩台機器，可以考慮一台裝 Ubuntu 跑程式，另外一台用 Windows 遠端過去開發。如果只有一台機器，則可以考慮裝雙系統。筆者在公司是用一台遠端一台訓練的模式，在家裡則是使用雙系統。

10.2 NVIDIA 顯示卡驅動程式

顯示卡驅動程式是開發與使用深度學習相關應用最基礎也最重要的第一步，如果你是使用有桌面環境的 Ubuntu，通常能在系統設定找到**驅動程式（Driver）**相關的頁面，打開來大概會像這樣：

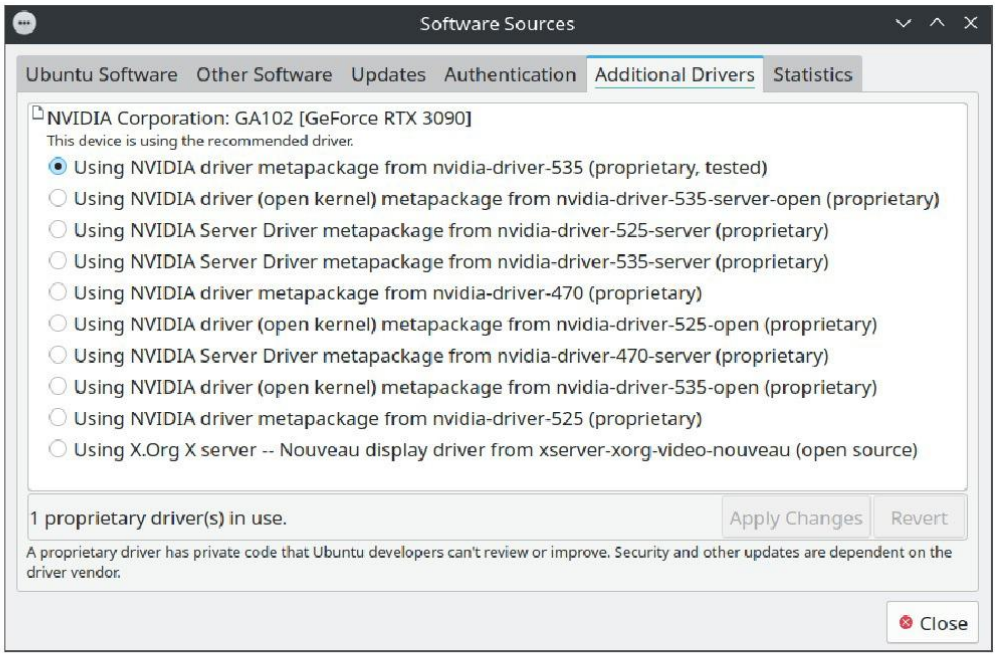


圖 10-1　驅動程式的安裝畫面

從圖中可以看到有非常多選擇，其中 535、525 與 470 代表的是驅動程式的版本，如果你有勇氣的話可以選擇最新版來裝。如果追求穩定的話，通常選擇次新的版本會比較好。如果使用的是有桌面環境的作業系統，那就不要裝有 Server 字樣的版本，通常那是給純文字環境使用的。

如果你是純文字環境，可以透過指令 `apt search nvidia-driver` 或者 `apt-cache search nvidia-driver` 來查看有哪些版本可以安裝，然後透過 `apt install` 直接安裝即可，以安裝 535 版為例：

```
sudo apt install nvidia-driver-535
```

安裝完之後重新開機，然後開始祈禱能順利重開 🙏 如果你的電腦能夠順利重開，可以輸入指令 `nvidia-smi` 來確認是否安裝成功。

```
~
› nvidia-smi

Sat May 18 11:50:35 2024
+---------------------------------------------------------------------------------------+
| NVIDIA-SMI 535.171.04             Driver Version: 535.171.04   CUDA Version: 12.2     |
|-----------------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|=========================================+======================+======================|
|   0  NVIDIA GeForce RTX 3090        Off | 00000000:01:00.0 Off |                  N/A |
|  0%   42C    P8              22W / 350W |   1244MiB / 24576MiB |      0%      Default |
|                                         |                      |                  N/A |
+-----------------------------------------+----------------------+----------------------+
```

圖 10-2　輸入 `nvidia-smi` 確認顯卡狀態

NVIDIA 顯卡驅動程式就是踏入深度學習開發會遇到的第一個天坑，很有可能把作業系統灌好之後，裝一個 NVIDIA 驅動上去，就再也打不開了 😵

建議把模型權重與訓練資料等等的檔案，都放在另外一個硬碟。如果系統炸開，比較可以毫不猶豫的選擇重灌，會比起你慢慢解環境衝突來的好一些。

不過這一切都是建立在你能**直接管理機器**或者**機器隨時都在你身旁**的狀況，在一些權限管理相對嚴格，或者訓練機房距離你萬里之遙的情況下就很麻煩了。假設系統真的炸開了，但是還能使用文字介面操作的話，建議先**完整刪除 NVIDIA 驅動程式**然後再重新安裝。

> 花生狂想曲 💡
> 筆者之前環境炸開時，紀錄了一份**如何完全刪除 NVIDIA 驅動程式**[1] 的筆記，各位讀者如果哪天環境也炸開時（希望不要 🙏）可以參考看看。

1　https://hackmd.io/@PenutChen/NvDriverFighting

10.3 虛擬環境管理 Conda

在 Python 虛擬開發環境上，筆者推薦使用 **Conda**[2] 家族的 **Miniconda**[3] 來做管理。你可能曾經聽說過 Miniconda 的老大哥 **Anaconda**，而兩者其實使用了相同的套件管理系統，但 Miniconda 不像 Anaconda 預裝了很多套件，是比較輕量化的選擇。

Conda 是個相當方便好用的工具，不僅能用來管理虛擬環境與 Python 套件，也能控制 Python 本身的版本，或者 CUDA 函式庫的版本。接下來介紹如何進行 Miniconda 的安裝以及 Conda 指令的基本用法。

10.3.1 安裝 Miniconda

首先前往 Miniconda 的官網下載安裝檔，安裝檔通常是個 `.sh` 檔，直接使用 `bash` 執行即可。安裝過程中會詢問你要安裝在哪個位置，預設是裝在 `~/miniconda3` 裡面，但筆者偏好裝在 `~/.miniconda3` 裡面，這樣檢視家目錄時這個資料夾就會被隱藏起來。

> **補充堅果，頭好壯壯**
>
> 如果打算使用 Dockerfile 或者在其他非互動式環境的狀況下安裝 Miniconda 的話，可以加上 `-b` 參數來跳過需要使用者互動的部份。其他還有滿多可用的參數，可以在執行安裝程式前透過 `-h` 參數來查看。

最後安裝程式會詢問你要不要對終端環境做初始化，如果選擇做初始化，那每次打開終端環境時，都會去讀取 Conda 的初始化環境，有時可能會造成不小

2 https://docs.conda.io/en/latest/

3 https://docs.anaconda.com/free/miniconda/

的延遲。如果想避免這樣的問題，可以選擇不要初始化，或者到 `~/.bashrc` 裡面將 `>>> conda initialize >>>` 包圍的區塊註解掉即可。但如果不做初始化會不能直接使用 Conda 指令，這時就要透過 `source ~/miniconda3/bin/activate <env_name>` 的指令來啟用虛擬環境了。

10.3.2 Conda 基本操作

使用 `conda create` 可以建立虛擬環境並指定 Python 版本：

```
# 使用 Python 3.10 的環境
conda create -n hello310 python=3.10

# 使用 Python 3.11 的環境
conda create -n hello311 python=3.11
```

建立完環境之後透過 `conda activate` 啟用環境：

```
conda activate hello310
```

如果建立環境時忘記附上 `python=X.YY` 的參數，也可以用 `conda install` 裝回來：

```
conda install python=3.10
```

在虛擬環境裡面，可以透過 `pip install` 安裝套件，也可以透過 `conda install` 安裝套件。但並不是每個套件都支援從 Conda 安裝，安裝前可以先參考該套件建議的安裝方法。一般情況下，筆者會優先選擇使用 `pip` 安裝，但是像 PyTorch[4] 這種套件就比較推薦使用 `conda` 安裝：

4 https://pytorch.org/get-started/locally/

```
conda install pytorch pytorch-cuda=12.1 -c pytorch -c nvidia
```

最後如果我們要離開虛擬環境：

```
conda deactivate
```

如果你的硬碟被虛擬環境們給塞爆了，可以先查看有哪些虛擬環境：

```
conda env list
```

若要移除不需要的虛擬環境：

```
conda env remove -n hello310
```

虛擬環境長期使用下來，可能會累積很多快取檔案，像是一些安裝套件的壓縮檔或暫存檔，可以刪除這些快取來釋放硬碟空間：

```
conda clean -a  # 移除所有相關快取
```

以上就是與 Conda 一起生活的基本日常！

補充堅果，頭好壯壯

其實在 Python 的套件管理上，還有個相當受歡迎的 **Poetry**[5] 工具。但是 Poetry 並不涉及 CUDA 這種系統函式庫的管理，因此也能用 Conda 管理系統函式庫，並在虛擬環境裡面使用 Poetry 管理 Python 套件。

5 https://python-poetry.org/

10.4 常用套件安裝

接下來開始安裝一些常用的工具與套件！

10.4.1 CMake

CMake[6] 是在安裝其他套件的過程中，經常會需要用到的編譯工具。有些套件可能要求你 CMake 裝到 3.2X 版以上，但如果你的作業系統卡在 Ubuntu 20.04 不想升上去的話，透過 `apt` 安裝的 CMake 只能到 3.16 版而已。這時我們就可以透過 `pip install cmake` 來把 CMake 裝到最新的版本！

> 花生狂想曲 💡
> 用 `pip` 安裝 `cmake` 根本邪教 😱 但實在是太方便了！

10.4.2 CUDA

CUDA 是深度學習相當重要的技術，分成 Runtime 與 Toolkit 兩種[7]，如果只是要運行程式用 CUDA Runtime 即可，但是涉及編譯行為時就會用到 CUDA Toolkit，因此需要關注 NVIDIA 驅動程式與 CUDA Toolkit 的版本對應[8]。

從這份版本對應表可以得知，假設 NVIDIA 驅動程式裝 515 版，那就只能跑 CUDA 需求為 11.7 及以下的程式，如果有個專案需求為 11.8 以上可能就會跑不

6 https://cmake.org/
7 https://stackoverflow.com/a/53504578
8 https://tinyurl.com/llm-cuda

起來，至少要將驅動程式升級到 520 版才行。因此驅動程式的版本與 CUDA 的版本習習相關，兩者基本上都向下相容，所以：

> NVIDIA 驅動支援的 CUDA 版本 >=CUDA Toolkit 版本 >= 開發應用的 CUDA 需求版本。

一般來說你可以裝 535 的驅動（支援到 CUDA 12 版）然後裝 CUDA 11.8 跑一個 CUDA 需求為 11.7 的程式，但一般來說會建議讓環境與程式的 CUDA 版本一致。因為 Conda 可以管理 CUDA 的版本，所以筆者的環境列表常常是長這樣：

```
$ conda env list

# conda environments:
base
py311cu118
py311cu121
py312cu121
```

如果一開始不知道怎麼決定要安裝哪個 CUDA 版本的話，推薦可以參考 PyTorch 官網目前支援到哪個版本，例如 PyTorch 現在主要支援 11.8 與 12.1 兩版，那筆者就會建議裝 CUDA 12.1 來用，因為很多套件開發者也都是跟隨 PyTorch 的版本在跑。但這只是個參考，一切還是以實際應用需求為主。

可以在 Anaconda 的 **CUDA 套件頁面**[9] 上查看目前有哪些版本的 CUDA 可以安裝，例如想要安裝 CUDA 12.1 的話：

```
conda install nvidia/label/cuda-12.1.1::cuda
```

9 https://anaconda.org/nvidia/cuda

這行指令代表安裝來自 `nvidia/label/cuda-12.1.1` 的 `cuda` 套件。

> 花生狂想曲 💡
> 除了 `cuda` 以外，還有個名稱為 `cuda-toolkit` 的套件，兩者其實都會幫你安裝 CUDA 函式庫。過去筆者其實一直分不清楚兩者的差異，於是將兩邊的安裝內容仔細的比較了一下，發現選擇安裝 `cuda` 套件的話，會比 `cuda-toolkit` 多出 `cuda, cuda-demo-suite, cuda-runtime` 等三個項目，但實際上只有 `cuda-demo-suite` 套件會多安裝一些新檔案。所以不管安裝 `cuda` 或 `cuda-toolkit` 都是可以的，推薦按照 **NVIDIA 官方文件**[10] 的建議安裝 `cuda` 套件即可。

有時其他套件會需要自行編譯一些 CUDA 函式庫，例如常用的分散式訓練框架 **DeepSpeed**[11]，它可能會去搜尋環境底下的 `lib64` 而不是 `lib` 資料夾，但 Conda 通常是把相關函式庫安裝在 `lib` 裡面，因此需要把資料夾 Link 過去：

```
ln -s ~/.miniconda3/envs/env_name/lib ~/.miniconda3/envs/env_name/lib64
```

另外像是 **NVCC**[12] 與 **cuDNN**[13] 這類的工具和函式庫也能透過 Conda 安裝，前者通常會跟著 CUDA 套件一起安裝，建議先裝 cuDNN 再裝 CUDA：

```
conda install cudnn
conda install nvidia/label/cuda-12.1.1::cuda
conda install -y pytorch pytorch-cuda=12.1 -c pytorch -c nvidia
```

10 https://tinyurl.com/llm-nvidia
11 https://github.com/microsoft/DeepSpeed
12 https://anaconda.org/nvidia/cuda-nvcc
13 https://anaconda.org/conda-forge/cudnn

安裝好之後，可以透過 `nvcc -V` 來確認安裝是否正確：

```
$ nvcc -V

nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2023 NVIDIA Corporation
Built on Mon_Apr__3_17:16:06_PDT_2023
Cuda compilation tools, release 12.1, V12.1.105
Build cuda_12.1.r12.1/compiler.32688072_0
```

小心，樂高就在你腳邊

在安裝完 cuDNN 之後，重新調用 PyTorch 可能會遇到以下問題：

```
Traceback (most recent call last):
  File "/home/user/lab/script.py", line 1, in <module>
    import torch
  File "/home/user/.miniconda3/envs/Py310/lib/python3.10/site-
packages/torch/__init__.py", line 229, in <module>
    from torch._C import *  # noqa: F403
ImportError: /home/user/.miniconda3/envs/Py310/lib/python3.10/
site-packages/torch/lib/libtorch_cuda.so: undefined symbol:
cudaGraphInstantiateWithFlags, version libcudart.so.11.0
```

這是因為從 Conda 安裝 cuDNN 時，系統很「熱心」的幫我們多裝了一個 CUDA 的函式庫，這時前往 Conda 環境底下的 `lib` 資料夾查看：

```
$ cd ~/.miniconda3/envs/Py310/lib
$ ls -lh | grep cuda

-rw-rw-r-- 1.2K cudatoolkit_config.yaml
-rw-rw-r-- 832K libcudadevrt.a
lrwxrwxrwx   21 libcudart.so -> libcudart.so.11.3.109
lrwxrwxrwx   21 libcudart.so.11.0 -> libcudart.so.11.3.109
-rwxrwxr-x 623K libcudart.so.11.3.109
-rwxrwxr-x 680K libcudart.so.11.8.89
-rw-rw-r-- 1.2M libcudart_static.a
```

原本應該是 CUDA 11.8 的環境，被 Link 成 11.3 了，但因為當初 PyTorch 是安裝 CUDA 11.8 的版本，所以才會報錯。其中一種解決方法，是在當初安裝環境時，先安裝 cuDNN 再安裝 CUDA 來避免連結被蓋掉。但我們也可以手動解決這個問題，把不正確的連結刪掉，重新連結正確的函式庫即可：

```
rm libcudart.so.11.0 libcudart.so
ln -s libcudart.so.11.8.89 libcudart.so.11.0
ln -s libcudart.so.11.8.89 libcudart.so
```

這樣 PyTorch 就能再次起舞囉！

10.5 Docker

若是熟悉 Docker 的開發者，可以選擇使用 PyTorch 官方的 Image 進行開發。使用官方 Docker Image 的好處在於 CUDA 相關的函式庫都已經配置好了，不太需要擔心環境配置炸開的問題。也能夠進一步透過 Dockerfile 來控制開發環境所依賴的套件，無論是需要透過 `pip` 或 `apt-get` 安裝的套件，使環境建置更容易被重現。

PyTorch 的官方 Image Tags 區分為 `runtime` 與 `devel` 兩種，`runtime` 是用在運行已經開發好的應用。對開發者而言，通常會選擇 `devel` 來使用：

```
FROM pytorch/pytorch:2.3.0-cuda12.1-cudnn8-devel
RUN pip install transformers
```

透過 `docker build` 將以上 Dockerfile 建成 Image：

```
docker build -t torch-dev:latest .
```

若是 Tensorflow 相關的開發，則可以考慮：

```
tensorflow/tensorflow:2.16.1-gpu
tensorflow/tensorflow:2.16.1-gpu-jupyter
```

後者額外附帶了 Jupyter 開發環境可以使用，相對好上手一些。

那要如何在 Docker Container 裡面做開發呢？可以透過 VS Code 的 **Dev Containers**[14] 外掛來遠端進入容器內，首先把容器跑起來：

```
docker run --name torch-dev -itd --gpus all \
    -v $PWD:/workspace torch-dev:latest
```

這裡將當前工作目錄掛載到容器內的 `/workspace` 路徑，接著打開左邊的 Remote Explorer 分頁，右鍵點選剛剛建立的容器，然後選擇 `"Attach in New Window"` 就可以開始在容器裡面為所欲為囉

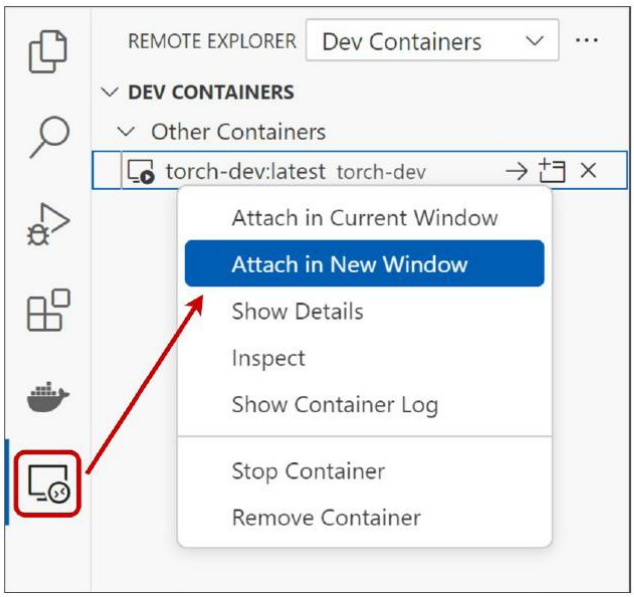


圖 10-3　使用 VS Code 進入 Docker Container

14　https://tinyurl.com/llm-dev-containers

小心，樂高就在你腳邊

在容器裡面直接是**最高權限（Root）**，下什麼指令都不用加 `sudo` 真爽！但是也要特別注意，在容器裡面建立的檔案，對應到主機裡面也會是最高權限。因此若是會在主機與容器之間交錯操作的話，要特別小心處理權限問題。

10.6 Colab

如果你已經厭倦了與 Python 環境無止盡的戰爭，每次一個新的實驗都要搭一個環境做測試，有的實驗可能也存活不了一兩天，不妨試試看 **Google Colab**[15] 筆記本。

Colab 的介面與 **Jupyter Notebook**[16] 很相似，但程式的運算是跑在 Google 借給我們的機器上面。Google 非常大方的免費借我們一張 **Tesla T4**[17] 使用，雖然只有 16 GB 的 GPU 記憶體，但是做一些小型實驗已經相當夠用。在這個環境裡面，完全不用煩惱 NVIDIA 驅動程式與 CUDA 函式庫的問題。重點是，那個環境的網路速度有夠快的啦！

```
llama-2-7b-chat.Q2_ 100%[===================>]   2.63G  91.0MB/s    in 42s

2023-09-24 12:02:39 (63.6 MB/s) - 'llama-2-7b-chat.Q2_K.gguf' saved [2825940672/2825940672]
```

圖 10-3　Colab 網速突破天際

15 https://colab.research.google.com/
16 https://jupyter.org/
17 https://www.nvidia.com/zh-tw/data-center/tesla-t4/

離峰時段網速經常破百 MB/s，平均也有個 50 到 60 MB/s 以上，對於經常在下載大模型的機器學習開發者而言，是個相當方便的網路環境。

如果要在 Colab 裡面安裝其他套件，只要在程式碼區塊裡面輸入 `%pip install ...` 即可，這點與 Jupyter 的操作相同。

可以在上方選單的「執行階段 > 變更執行階段類型」選擇使用 T4 GPU，也有 TPU 之類的可以選，有需要的讀者可以嘗試看看。

另外在設定裡面還有貓貓狗狗模式跟爆炸特效可以選擇，替沉悶的開發過程增添一點活潑的氣氛：

圖 10-4　Colab 包含的活潑設定

不過 Colab 也並非完全無限制的讓你使用，只要筆記本閒置一段時間就會被強制中斷連線，而連續運作時間通常也不能超過半天，比較適合那種一兩個小時就能結束的小型實驗，或者要寫 Demo 程式碼給別人看的時候能方便分享。如果想要更長時間的使用，或者調用更高級的顯卡，可以考慮購買 **Colab Pro**[18] 會員資格。

18　https://colab.research.google.com/signup

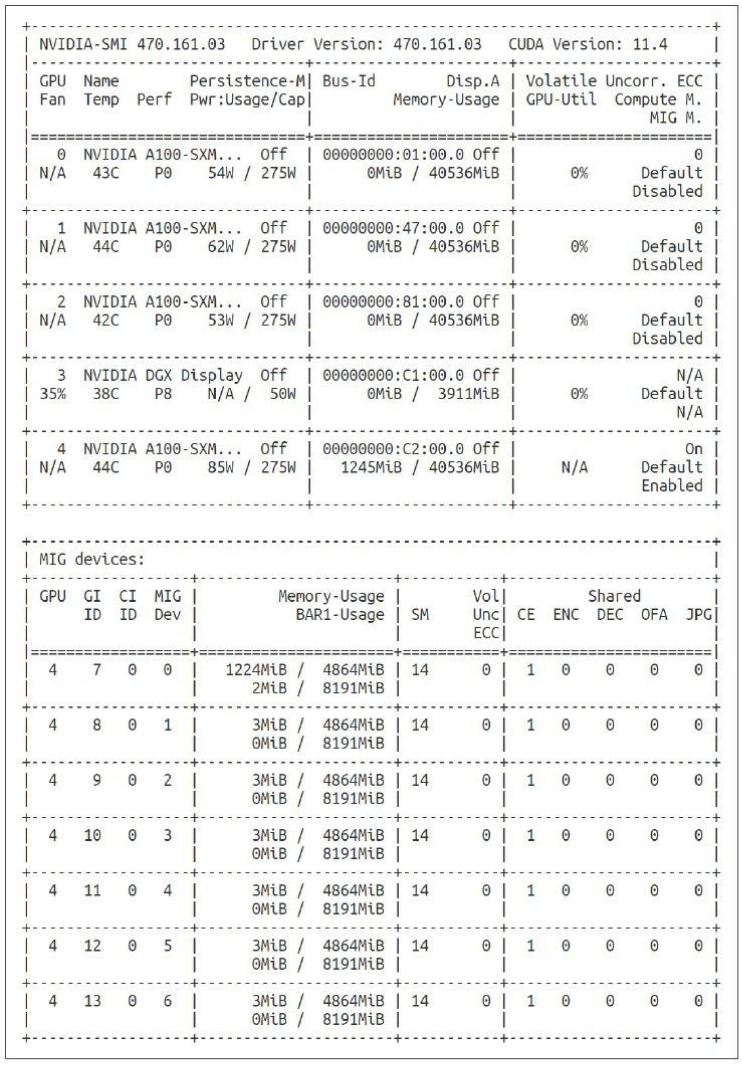
10.7 GPU

最後再來分享一些 GPU 型號的資訊。在 NVIDIA 底下的 GPU，每一代的架構都會有個代號，例如 GTX 10 開頭的顯卡是**帕斯卡（Pascal）**架構、RTX 20 開頭的則是**圖靈（Turing）**架構、RTX 30 為**安培（Ampere）**、最新的 RTX 40 則為**愛達（Ada Lovelace）**架構，這些代號都在向數學與資訊史上的名人致敬。

作為深度學習領域的開發者，選購顯卡時除了運算速度不同，更重要的是顯卡的記憶體，如果模型放不下，算的再快都是白搭。然而消費級的顯卡，例如 RTX 3090 與 RTX 4090 等，其 GPU 記憶體最高都只做到 24 GB 而已。如果想要再往上，就會用到工作站等級的顯卡，目前最有名也最普及的就是 **A100**[19]。

A100[20] 為 **V100**[21] 的繼任者，採用 Ampere 架構，有 40 GB 與 80 GB 兩種規格的 GPU 記憶體。有趣的是它能切割記憶體，一張 A100 看起來會像有 7 張 GPU 一樣：

19 https://www.nvidia.com/zh-tw/data-center/a100/
20 https://www.nvidia.com/zh-tw/data-center/a100/
21 https://www.nvidia.com/zh-tw/data-center/v100/



```
+-----------------------------------------------------------------------------+
| NVIDIA-SMI 470.161.03   Driver Version: 470.161.03   CUDA Version: 11.4     |
|-------------------------------+----------------------+----------------------+
| GPU  Name        Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf  Pwr:Usage/Cap|         Memory-Usage | GPU-Util  Compute M. |
|                               |                      |               MIG M. |
|===============================+======================+======================|
|   0  NVIDIA A100-SXM...  Off  | 00000000:01:00.0 Off |                    0 |
| N/A   43C    P0    54W / 275W |      0MiB / 40536MiB |      0%      Default |
|                               |                      |             Disabled |
+-------------------------------+----------------------+----------------------+
|   1  NVIDIA A100-SXM...  Off  | 00000000:47:00.0 Off |                    0 |
| N/A   44C    P0    62W / 275W |      0MiB / 40536MiB |      0%      Default |
|                               |                      |             Disabled |
+-------------------------------+----------------------+----------------------+
|   2  NVIDIA A100-SXM...  Off  | 00000000:81:00.0 Off |                    0 |
| N/A   42C    P0    53W / 275W |      0MiB / 40536MiB |      0%      Default |
|                               |                      |             Disabled |
+-------------------------------+----------------------+----------------------+
|   3  NVIDIA DGX Display  Off  | 00000000:C1:00.0 Off |                  N/A |
| 35%   38C    P8    N/A /  50W |      0MiB /  3911MiB |      0%      Default |
|                               |                      |                  N/A |
+-------------------------------+----------------------+----------------------+
|   4  NVIDIA A100-SXM...  Off  | 00000000:C2:00.0 Off |                   On |
| N/A   44C    P0    85W / 275W |   1245MiB / 40536MiB |     N/A      Default |
|                               |                      |              Enabled |
+-------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------+
| MIG devices:                                                                |
+------------------+----------------------+-----------+----------------------+
| GPU  GI  CI  MIG |         Memory-Usage |        Vol|         Shared       |
|      ID  ID  Dev |           BAR1-Usage | SM     Unc| CE  ENC  DEC  OFA  JPG|
|                  |                      |        ECC|                      |
|==================+======================+===========+======================|
|  4    7   0   0  |   1224MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                  |      2MiB /  8191MiB |           |                      |
+------------------+----------------------+-----------+----------------------+
|  4    8   0   1  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                  |      0MiB /  8191MiB |           |                      |
+------------------+----------------------+-----------+----------------------+
|  4    9   0   2  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                  |      0MiB /  8191MiB |           |                      |
+------------------+----------------------+-----------+----------------------+
|  4   10   0   3  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                  |      0MiB /  8191MiB |           |                      |
+------------------+----------------------+-----------+----------------------+
|  4   11   0   4  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                  |      0MiB /  8191MiB |           |                      |
+------------------+----------------------+-----------+----------------------+
|  4   12   0   5  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                  |      0MiB /  8191MiB |           |                      |
+------------------+----------------------+-----------+----------------------+
|  4   13   0   6  |      3MiB /  4864MiB | 14      0 |  1   0    0    0    0 |
|                  |      0MiB /  8191MiB |           |                      |
+------------------+----------------------+-----------+----------------------+
```

圖 10-5　有四張 A100 的機器，其中 4 號顯卡被切成 7 份的樣子
（感謝強者我朋友鐵貓提供截圖）

其他還有 **RTX A6000**[22] 與 **RTX 6000 Ada**[23] 這種 48 GB 級的，和 A100 的繼任者 **H100**[24] 等這些訓練用的顯卡。也有 **L40S**[25] 這種專為推論最佳化的顯卡。但無論哪一種，都不是一般人買得起的 💸💸💸

22　https://www.nvidia.com/zh-tw/design-visualization/rtx-a6000/

23　https://www.nvidia.com/zh-tw/design-visualization/rtx-6000/

24　https://www.nvidia.com/zh-tw/data-center/h100/

25　https://www.nvidia.com/zh-tw/data-center/l40s/

花生狂想曲

這些顯卡其實非常缺貨，有錢也買不起，只能說老黃賺爛了

在選擇 GPU 的時候，有個相當重要的規格叫做 **CUDA 運算能力（CUDA Compute Capability, CCC）**[26]，這個運算能力會影響 GPU 實際上能夠進行什麼樣的運算，例如最近 NVIDIA 在推行的 FP8 浮點數訓練，就需要運算能力 8.9 以上的顯卡才能跑，也就是 RTX 4090 或 H100 這些顯卡，就連 A100 都沒辦法使用 FP8 浮點數進行訓練。

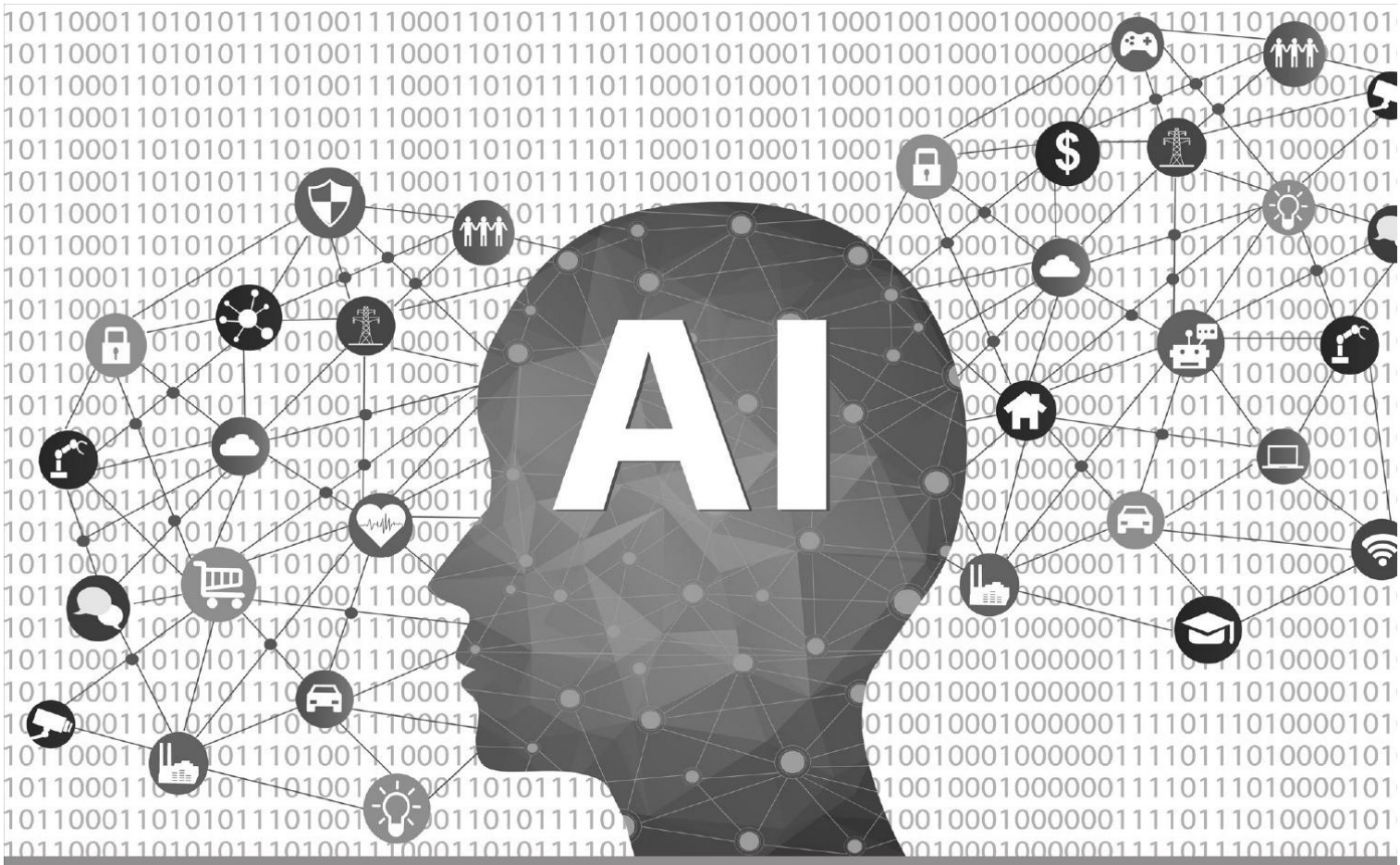
補充堅果，頭好壯壯

可以透過 `nvidia-smi --query-gpu=compute_cap --format=csv` 指令來查詢自己顯卡的 CUDA 運算能力是多少！

10.8 結論

本章節分享了使用 Conda 架設與管理虛擬環境的方法，並介紹了 Colab 筆記本與一些 NVIDIA 顯卡的資訊。其實每個人都有自己的一套管理方法，只要自己能夠輕鬆駕馭而且方便配合團隊即可。完成環境搭建之後，就可以開始我們的 LLM 之旅啦！

26 https://developer.nvidia.com/cuda-gpus



Chapter

11

擁抱開源的微笑
Hugging Face Transformers

主管 @tag 我

@penut 那麼多的語言模型架構，我們要串到什麼時候啊？

有了這張笑臉一切好說！

Hugging Face[1] Transformers[2] 是訓練 Transformer 模型最知名的套件沒有之一，此套件收錄了許多知名模型架構、訓練演算法以及各種模型權重的分享，使開發者可以輕鬆駕馭這些理論複雜的操作。

Transformers 是個著重在訓練的框架，但也支援了相當豐富的推理功能，本章節將會著重在如何使用此框架進行 LLM 的推論，並剖析 Transformer Decoder 語言模型如何進行自回歸解碼（Autoregressive Decoding）的流程。

工欲善其事，必先利其器

Transformers 支援的機器學習框架包含 PyTorch、Tensorflow 與 JAX 等等，本書主要以 PyTorch 為主：

```
conda install pytorch pytorch-cuda=12.1 -c pytorch -c nvidia
pip install transformers
```

使用 Transformers 套件時，也推薦安裝以下額外套件進一步提升運作效率：

```
pip install accelerate bitsandbytes optimum
```

這些套件能夠讓我們使用量化（Quantization）機制來減少 GPU 記憶體消耗。

1 https://huggingface.co/

2 https://huggingface.co/docs/transformers/index

11.1 讀取模型

本章節完整程式碼

https://tinyurl.com/llm-note-07

推薦還沒有架設開發環境的讀者可以先從以上連結進入 Colab 嘗試，這樣也不用煩惱模型權重很佔硬碟空間或者要下載很久的問題。

花生狂想曲

手機也能操作 Colab ！直接打開瀏覽器就能用囉

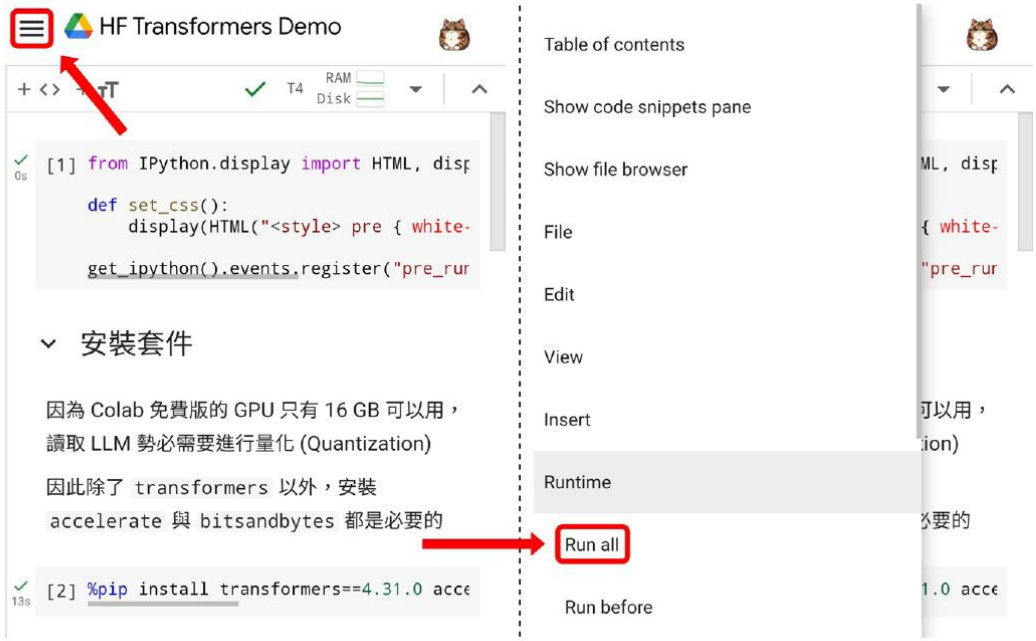


圖 11-1 在手機上執行 Colab

Transformers 讀取模型的方式有很多種，最基本的方式是使用 `.from_pretrained` 讀取已經訓練好的模型。這裡可以先使用 TheBloke 大神提供的 **Llama-2 7B Chat**[3] 模型權重做測試：

```
from transformers import BitsAndBytesConfig
from transformers import LlamaForCausalLM

bnb_config = BitsAndBytesConfig(
    load_in_8bit=True,        # 約需 8 GB GPU 記憶體
    # load_in_4bit=True,      # 約需 6 GB GPU 記憶體
)

model = LlamaForCausalLM.from_pretrained(
    "TheBloke/Llama-2-7b-chat-fp16",
    device_map="auto",
    low_cpu_mem_usage=True
    quantization_config=bnb_config,
)
```

在 Colab 這種資源相對有限的環境裡面，需要使用**量化（Quantization）**技術來減少 GPU 記憶體的消耗，透過 `BitsAndBytesConfig` 可以輕鬆的將模型量化至 8 到 4 位元。

為了啟用量化功能，需要將 `device_map` 設定為 `"auto"` 以確保模型權重都放在 GPU 裡面。如果 **CPU 記憶體**也不是很夠的話，那就需要將 `low_cpu_mem_usage` 設為 `True` 才能順利讀取模型。

3 https://huggingface.co/TheBloke/Llama-2-7b-chat-fp16

花生狂想曲 💡

TheBloke[4] 是 Hugging Face 社群上相當活躍的一個用戶，每次有很猛的模型釋出時，他就會手刀衝第一個幫大家把模型的格式轉換成 HF 支援的格式，或者進行各種量化壓縮後上傳。據說他的**網路環境達數 GB/s 之快**[5]，是個非常熱心的大神。

這個讀取模型的動作，實際上會從 **Hugging Face Hub**[6]（以下簡稱 HF Hub）下載模型的權重，預設會放在 `~/.cache/huggingface/` 裡面，也可以透過 `cache_dir` 參數手動指定這些檔案要存放在哪裡：

```
model = LlamaForCausalLM.from_pretrained(
    model_id,
    cache_dir="./my-cache"
)
```

多數的模型都可以在 HF Hub 上找到，HF Hub 主要使用 Git 做管理的，所以你也能直接使用 `git clone` 將模型複製下來，但是在此之前會需要先安裝 Git LFS 模組：

```
sudo apt install -y git git-lfs
git lfs install  # 啟用 Git LFS 功能
```

準備好 Git LFS 之後就可以開始下載囉：

```
git clone https://huggingface.co/TheBloke/Llama-2-7b-chat-fp16
```

4 https://huggingface.co/TheBloke

5 https://tinyurl.com/llm-the-bloke

6 https://huggingface.co/models

完成下載後，直接指定資料夾路徑名稱即可讀取該模型：

```
LlamaForCausalLM.from_pretrained("Llama-2-7b-chat-fp16")
```

補充堅果，頭好壯壯

Git 在傳輸大型檔案時並不會顯示下載進度，所以只能慢慢等，過一陣子之後模型就會下載完成了！但如果你真的很想要看那個酷酷的網速進度條其實還是有辦法的：

```
# 跳過所有 LFS 檔案的下載
GIT_LFS_SKIP_SMUDGE=1 git clone https://huggingface.co/TheBloke/
Llama-2-7b-chat-fp16
cd Llama-2-7b-chat-fp16

# 透過 Git LFS 進行 Fetch
git lfs fetch
git lfs checkout
```

這樣酷酷的進度條就出現啦！不過步驟比較多，所以除非懷疑網路速度出了問題，不然筆者還是習慣用 `git clone` 就好。

現在多數的 LLM 都是 PyTorch 格式，但 HF Hub 上有些模型可能不只包含 PyTorch 格式的權重，還會把 Tensorflow、JAX、ONNX 或 GGUF 格式的權重都上傳到同一個模型空間底下，這時用 `git clone` 的話，就會把很多不相關的檔案下載下來，造成硬碟空間的浪費，需要自行權衡一下是否需要這樣做。

小心，樂高就在你腳邊

大部分模型權重上傳之後就不會再變動了，但是 Git 的機制會讓下載下來的模型包含一份額外複製的權重做備份，因此 10 GB 的模型複製到硬碟裡面，可能會吃掉 20 GB 的硬碟空間！一般情況下可以直接把權重資料夾裡面的 `.git` 資料夾刪掉以節省硬碟空間，有需要的話再下載回來就好。

有時可能會遇到有存取限制的模型（Gated Model），例如 Meta 官方的 Llama 系列就有存取限制，這時要先去該模型的介紹頁面填寫存取表單。有些模型填完表單就能馬上存取了，但是像 Meta Llama 就會需要等待一段時間才會收到核准通知。

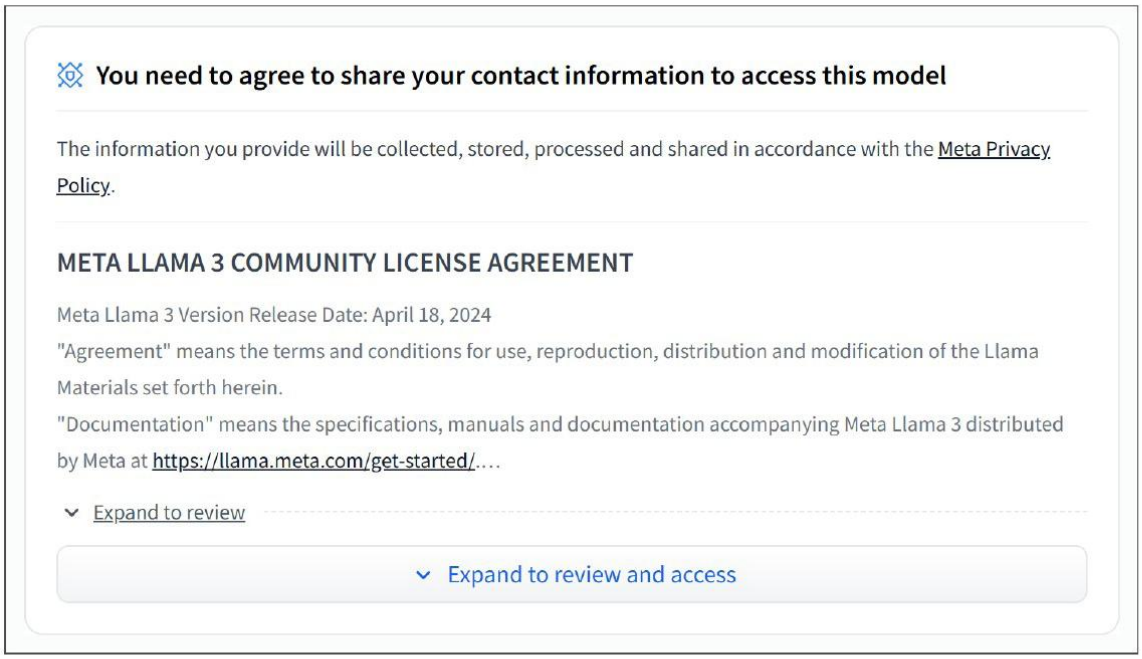


圖 11-2　需要額外核准的模型

成功獲得核准之後就可以開始下載模型，這類模型需要其他方式才能下載，第一種是設定 SSH 金鑰，然後透過 SSH 下載：

```
git clone git@hf.co:meta-llama/Meta-Llama-3-8B
```

如果你不會設定 SSH 也沒關係，還可以透過 `huggingface-cli` 登入，首先要前往 HF 帳號設定的 Access Tokens 頁面，並建立一個新的 Token，如果只是要下載模型的話，權限種類只要設定為 Read 就好了。建立好 Access Token 後輸入以下指令：

```
huggingface-cli login
```

接著貼上剛才建立的 Access Token 後，繼續透過 `huggingface-cli` 來下載模型：

```
huggingface-cli download meta-llama/Meta-Llama-3-8B \
    --local-dir Models/Meta-Llama-3-8B
```

這樣就大功告成囉！

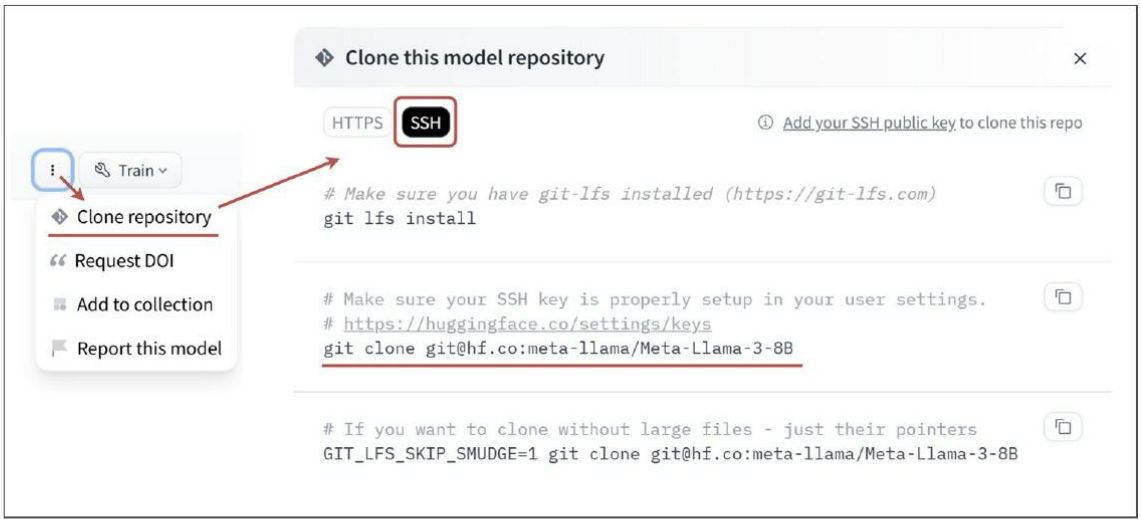


圖 11-3　透過 SSH 下載模型權重

11.2　讀取分詞器

分詞器的讀取方式，與模型讀取方式差不多，通常分詞器會跟模型權重專案放在一起，所以直接用相同的名稱下載即可：

```
from transformers import LlamaTokenizer as TkCls

tk = TkCls.from_pretrained("TheBloke/Llama-2-7b-chat-fp16")
```

也可以使用 `LlamaTokenizerFast` 這種快速版的分詞器類別，一般版的分詞器使用 Python 實做，而快速版的則是使用 **Rust**[7] 實做。如果要對大量長文本進行分詞的話，可以考慮使用快速版，但如果只是要處理幾個短文本的話就不用太煩惱這些。

小心，樂高就在你腳邊

在舊版本的 Transformers 裡面，有些快速版的 Tokenizer 要初始化非常久。解決方法除了改用一般版以外，也可以將 Tokenizer 讀出來之後，轉存成快速版：

```
from transformers import LlamaTokenizerFast as TkCls

tk: TkCls = TkCls.from_pretrained("/path/to/tokenizer")
tk.save_pretrained("/path/to/tokenizer")
```

這樣下次讀取的時候就不會再等這麼久了，不過這個問題在新版的 Transformers 裡已經被修正了，所以通常不用太擔心。

11.3 自動類別

在上面的範例裡，因為已經預先知道 Llama 2 使用的是 Llama 架構，所以可以直接調用 `LlamaForCausalLM` 與 `LlamaTokenizer` 這兩個類別。但有些模型與分詞器第一眼可能無法判斷他是什麼架構，或者是你想要撰寫相容性更廣泛的程式碼，那就可以使用 Transformers 的 Auto Class 來進行讀取：

```
from transformers import AutoModelForCausalLM as ModelCls
from transformers import AutoTokenizer as TkCls
```

7 https://www.rust-lang.org/zh-TW

```
model_id = "TheBloke/Llama-2-7b-chat-fp16"
model = ModelCls.from_pretrained(model_id)
tk = TkCls.from_pretrained(model_id)
```

觀察這些物件，發現它們依然是原本的 Llama 類別：

```
print(type(model))  # <class 'transformers...LlamaForCausalLM'>
print(type(tk))     # <class 'transformers...LlamaTokenizerFast'>
```

如果不想用快速版的分詞器，可以加上把 `use_fast=False` 的參數：

```
tk = TkCls.from_pretrained(model_id, use_fast=False)
print(type(tk))  # <class 'transformers...LlamaTokenizer'>
```

但 Auto Class 的缺點是**型別提示（Type Hint）**比較少，就是在寫程式時 VS Code 會自己跳出來的自動完成提示清單。

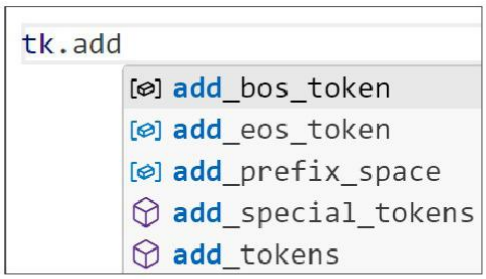


圖 11-4　寫程式時會自動出現的建議清單

如果非常仰賴自動完成的朋友，不用擔心，因為我也很仰賴 😂

其實大部分的方法都來自 `PreTrainedModel` 與 `PreTrainedTokenizer` 這兩個類別，因此能像這樣給出型別註記：

```
from transformers import AutoModelForCausalLM as ModelImp
from transformers import AutoTokenizer as TkImp
from transformers import PreTrainedModel as ModelCls
from transformers import PreTrainedTokenizer as TkCls
```

```
model_id = "TheBloke/Llama-2-7b-chat-fp16"
model: ModelCls = ModelImp.from_pretrained(model_id)
tk: TkCls = TkImp.from_pretrained(model_id)
```

如此一來，自動完成又再度重回我們的懷抱了 🤗

> **補充堅果，頭好壯壯**
>
> 在最新版的 Transformers 套件裡面，`AutoTokenizer.from_pretrained()` 已經會自動把回傳物件標記為 `PreTrainedTokenizer` 型別，所以分詞器的部份就不一定要像上面這樣做囉！

11.4 文本生成

完成模型與分詞器的讀取之後，就可以開始著手進行生成了！在進行生成之前，需要先將輸入切成 Token 並轉換成 PyTorch Tensor 物件：

```
inputs = tk("Hello, ", return_tensors="pt").to("cuda")
print(inputs)

"""
Output:
{
 "input_ids": tensor([[1, 15043, 29892, 29871]], device="cuda:0"),
 "attention_mask": tensor([[1, 1, 1, 1]], device="cuda:0"),
}
"""
```

分詞的結果是一個包含了 `input_ids` 與 `attention_mask` 兩組 Tensor 的字典，字典裡面的 `input_ids` 就是分詞的結果，而 `attention_mask` 則是用來遮罩非輸入的部份，一開始只需要關注 `input_ids` 就好，先把 `attention_mask` 當成模型的必要輸入即可。

因為模型與 Tensor 必須放在同個裝置上才能做運算，例如放在 CPU 上的模型只能跟 CPU 上的 Tensor 運算。在這裡模型是放在 GPU 裡面，所以要用 `.to("cuda")` 將整個分詞結果放進 GPU 裡面，接著就可以開始使用 `model.generate` 進行文本生成：

```
output = model.generate(**inputs, max_new_tokens=32)
print(tk.batch_decode(output))
# <s> Hello, I am a beginner in Python ...
```

`model.generate` 會回傳生成的 Token ID，必須用分詞器進行解碼才能得到文字版的輸出。參數 `max_new_tokens` 用來設定要生成的 Token 數量，輸出的 Token 數量越多，消耗的 GPU 記憶體就會越多，生成所需的時間也會比較久。

模型也可以同時進行多個文本生成，也就是俗稱的**批次推論（Batch Inference）**：

```
tk.pad_token = tk.eos_token  # Llama2-7B 沒有 Padding Token
prompt = ["Hello, ", "Hi, my name is"]
inputs = tk(prompt, return_tensors="pt", padding=True).to("cuda")

output = model.generate(**inputs, max_new_tokens=16)
print(tk.batch_decode(output))
```

要特別注意，因為 Llama 2 的分詞器沒有指定預設填充用的 Padding Token，所以需要幫它指定一個才能做填充。

這時我們很開心的執行這份程式，並觀察輸出的結果：

```
[
    "<s> Hello, </s></s>0000000000000000",
    "<s> Hi, my name is [Your Name] and I am a ...",
]
```

仔細一看發現，第一個結果好像怪怪的欸！

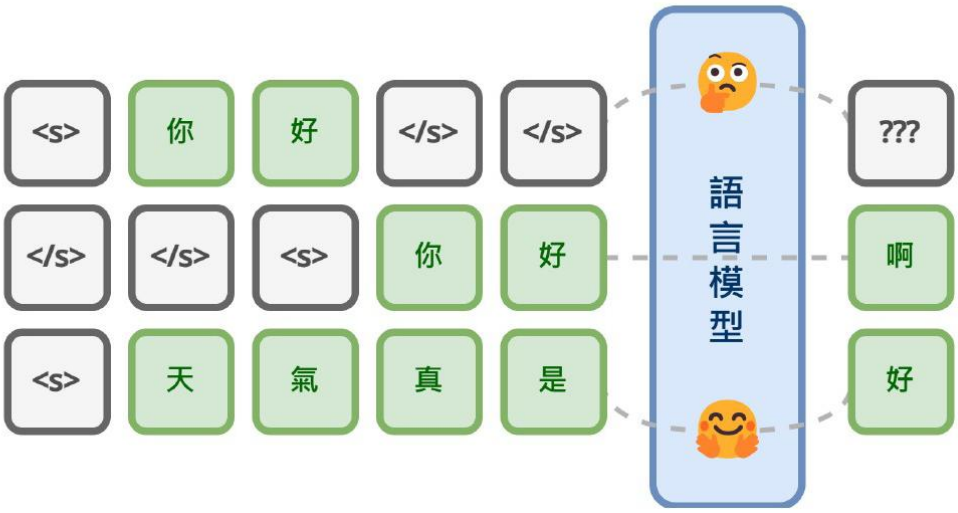


圖 11-5　填充方向不正確會導致生成失敗

這是因為 Transformer Decoder 是自回歸解碼的關係，放在越右邊的 Token 通常會對下個生成的 Token 有越顯著的影響，若是將 Padding Token 都放在右邊，那模型看到一堆 Padding Token 後也不知道要輸出什麼。所以在進行批次生成的時候，記得要把 Padding Token 放在左邊，可以在初始化分詞器時設定 `padding_side` 這個參數：

```
tk: TkCls = TkCls.from_pretrained(model_id, padding_side="left")
```

再執行一次上述的程式碼，結果就變正常啦 🎉

```
[
    "</s></s><s> Hello, I am a 35 year old woman ...",
    "<s> Hi, my name is [Your Name] and I am a ...",
]
```

補充堅果，頭好壯壯

你是否也會好奇，難道在短句前面加上一堆 Padding Token 就不會影響輸出了嗎？這時方才產生的 **Attention Mask** 就派上用場了，Attention Mask 就是用來告訴模型「**我不是輸入！**」的一項資訊，這樣模型在看到一堆 Padding Token 時就會假裝沒看見了。因此在進行批次推論時，輸入適當的 Attention Mask 相當重要。

11.5 串流輸出

在進行文本生成時，通常都要等上一段時間才能看到完整的輸出，如果要進行比較長文本的生成，那就要花上不少時間。當我們在調測模型時，總是會想要一些比較即時的回饋，這時可以透過 `TextStreamer` 來進行串流輸出。使用方法非常簡單，只要用分詞器初始化一個 `TextStreamer` 物件之後，丟進 `streamer` 參數裡面就好了：

```
from transformers import TextStreamer

inputs = tk("Hello, ", return_tensors="pt").to("cuda")
output = model.generate(
    **inputs,
    max_new_tokens=2048,
    streamer=TextStreamer(tk),
)
```

這樣就會看到文字一個一個跳出來啦！使用時需要注意，`TextStreamer` 只支援單一輸入，無法進行批次推論。

11.6 提示樣板

提示樣板（Prompt Template）是訓練模型進行對話時使用的格式，若要發動語言模型的對話能力，就會需要借助提示樣本的力量。這個樣板其實就是把使用者與模型之間一問一答的過程，寫成一個像是聊天紀錄的格式，用來「欺騙」語言模型生成下個聊天片段。

這個樣板會根據每個語言模型訓練的方式而有所不同，一般而言遵守模型開發者的建議會比較好，以 Llama 2 為例，官方的樣板大致如下：

```
[INST] <使用者的訊息在這裡> [/INST] <模型的回覆從這裡開始>
```

我們嘗試用這種樣板，來詢問模型：

```
prompt = "[INST] 使用繁體中文回答，請問什麼是大型語言模型？ [/INST] "
inputs = tk(prompt, return_tensors="pt").to("cuda")
output = model.generate(**inputs, max_new_tokens=128)

"""
輸出結果：
<s> [INST] 使用繁體中文回答，請問什麼是大型語言模型？ [/INST]  Great! Here's my
answer in Traditional Chinese: 大型語言模型（Large Language Model，LLM）是一
種基於深度學習的語言處理模型 ...
"""
```

但是每個模型都有自己的格式，有那麼多的格式，難道要一個一個記下來嗎？不用擔心，我們可以使用 Transformers 套件內建的**聊天樣板（Chat Template）**功能，只要呼叫分詞器的 `apply_chat_template` 方法就可以直接調用模型開發者設定的樣板：

```
query = "使用繁體中文回答，請問什麼是大型語言模型？"
conversation = [{"role": "user", "content": query}]
prompt = tk.apply_chat_template(conversation, tokenize=False)
print(prompt)
# Output: <s>[INST] 使用繁體中文回答，請問什麼是大型語言模型？ [/INST]
```

這個樣板會隨著模型不同而有所變化，例如將對話套用到 Google 的 Gemma 模型樣板後就會長的像這樣：

```
tk = AutoTokenizer.from_pretrained("google/gemma-2b-it")
query = " 使用繁體中文回答，請問什麼是大型語言模型？ "
conversation = [{"role": "user", "content": query}]
prompt = tk.apply_chat_template(conversation, tokenize=False)
print(prompt)

"""
Output: <bos><start_of_turn>user
使用繁體中文回答，請問什麼是大型語言模型？ <end_of_turn>
"""
```

透過 `add_generation_prompt` 參數，可以把模型開始回答之前的格式也附上，這樣模型比較不容易生成失敗：

```
tk = AutoTokenizer.from_pretrained("google/gemma-2b-it")
query = " 使用繁體中文回答，請問什麼是大型語言模型？ "
conversation = [{"role": "user", "content": query}]
prompt = tk.apply_chat_template(
    conversation,
    tokenize=False,
    add_generation_prompt=True,
)
print(prompt)

"""
Output:
<bos><start_of_turn>user
使用繁體中文回答，請問什麼是大型語言模型？ <end_of_turn>
<start_of_turn>model  <= 多了這一段

"""
```

確認格式沒問題之後，把 `tokenize=False` 拿掉，並加上 `return_tensors="pt"` 來取得模型的輸入：

```
input_ids = tk.apply_chat_template(
    conversation,
```

```
    return_tensors="pt",
    add_generation_prompt=True,
).to("cuda")
print(input_ids)
# Output: tensor([[2, 106, ..., 2516, 108]], device='cuda:0')
```

與前面直接分詞的結果稍有不同，這裡並沒有 `attention_mask`，所以輸入模型的方式也稍有不同：

```
# output = model.generate(**inputs)  # 原本的用法
outputs = model.generate(input_ids)  # 現在的用法
```

有了聊天樣板後，就可以更輕鬆的操作模型輸入，而不需要每次有新的模型、新的樣板時都要重寫一遍樣板的程式碼了。

> **補充堅果，頭好壯壯**
>
> Transformers 套件的聊天樣板功能是使用 Jinja[8] 套件實做的，一些相對早期釋出的模型可能沒有在設定檔裡面附上樣板，這時可以考慮自行撰寫聊天樣板，詳細用法請參考官方教學[9]。

11.7 取樣參數

模型在進行推論的時候，其實會生成一張**機率表**用來代表下一個 Token 可能會是誰，**取樣參數（Sample Parameters）**就是我們如何從這份機率表裡面挑選下個 Token 的參數。可以在 `GenerationConfig` 類別裡面指定這些取樣參數，讓模型不會每次都挑機率最高的 Token 當輸出，使每次輸出都長的不太一樣，提昇輸出的多元性。

8 https://github.com/pallets/jinja/

9 https://tinyurl.com/llm-chat-template

小心，樂高就在你腳邊

如果取樣的隨機性太高，可能會影響模型回答的正確性，所以一定要慎重選擇！

`do_sample` 用來控制是否進行取樣，若為 `False` 代表使用 Greedy Decode，只取機率最高的 Token。`top_k` 代表要從機率表裡面的前幾名做取樣。`top_p` 稍微複雜，這個參數會設定一個機率值，這時程式就會從第一名開始往後累加，把前幾名的機率全部加起來，直到這個累加的機率超過 `top_p` 為止，並從這些 Tokens 中取樣。

`temperature` 則是一種機率表的改造，類似以前學校常聽到的把考試分數「開根號除以十」的概念，原本機率較低的 Token 會被調高，原本機率較高的 Token 會被調低。當 `temperature` 越大時，第一名跟最後一名的距離就會越近，但彼此之間的相對順序還是保持不變。`temperature` 常被用來設定模型要**更有創意**還是**更加嚴謹**的生成輸出。

`repetition_penalty` 顧名思義就是在懲罰重複的輸出，當這個設定高於 `1.0` 時，模型較不容易產生重複性的輸出。低於 `1.0` 時，模型會變得容易生成重複的輸出。當你發現模型輸出開始瘋狂跳針時，可以試著調整這個參數。

最後，取樣參數可能會像這樣：

```
from transformers import GenerationConfig

gen_config = GenerationConfig(
    max_new_tokens=16,
    do_sample=True,
    top_k=50,
    top_p=0.95,
    temperature=0.75,
    repetition_penalty=1.1,
)

output = model.generate(**inputs, generation_config=gen_config)
```

這些參數也會對彼此之間產生不同程度的影響，因此選擇一組適合的參數進行生成任務，也是一堂相當重要的課題。可以用來控制生成的參數還有很多，詳細資訊可以參考官方文件[10]。

11.8 設定停止點

停止點是用來告訴系統在生成的過程中，遇到哪些 Token 時要結束生成。除了遇到句子結尾的特殊標記（End-of-Sentence，EOS）Token 以外，也可以把句號或換行之類的當成停止點。最基本的方法是設定 `GenerationConfig` 的 `eos_token_id`：

```
generation_config = GenerationConfig(
    eos_token_id=[
        tk.eos_token_id,         # 原本的 EOS 要留著
        tk.encode(".")[-1],      # 遇到句點停下來
        tk.encode("\n")[-1],     # 遇到換行停下來
        tk.encode("\n\n")[-1],   # 遇到雙換行也停下來
    ],
)
```

這個方法只適用於停止點是獨立一個 Token 的情況，如果停止點不只一個 Token 甚至是一個字串的話，可以自己實做 `StoppingCriteria` 類別：

```
from transformers import StoppingCriteria, StoppingCriteriaList

class StopWords(StoppingCriteria):
    def __init__(self, tk: TkCls, stop_words: list[str]):
        self.tk = tk
        self.stop_tokens = stop_words

    def __call__(self, input_ids, *_) -> bool:
        s = self.tk.batch_decode(input_ids)[0]
```

10 https://tinyurl.com/llm-gen-config

```
        for t in self.stop_tokens:
            if s.endswith(t):
                return True
        return False

sw = StopWords(tk, ["。", "！", "？", "結束！"])
scl = StoppingCriteriaList([sw])
```

這裡寫了一個 `StopWords` 類別，會在每次系統進行檢查的時候，將整份輸出還原回純文字，並且檢查文字的結尾是否符合使用者設定的停止點。因為批次推論時的情況會複雜許多，所以這裡只實做單一文本生成的情況。

接著將 `StopWords` 放進 `StoppingCriteriaList` 裡面，然後傳入 `generate` 裡面即可：

```
output = model.generate(**input_ids, stopping_criteria=scl)
```

這樣模型在輸出時遇到設定的字串之後就會停下來了！

最近 Transformers 在 `GenerationConfig` 裡面新增了 `stop_strings` 的參數，讓我們可以更輕鬆的設定輸出停止點：

```
from transformers import GenerationConfig

config = GenerationConfig(stop_strings=[".", "\n"])
inputs = tk(["hello, ", "goodbye, "], padding=True, return_tensors="pt")
inputs = inputs.to(model.device)
output = model.generate(**inputs, generation_config=config, tokenizer=tk)
```

這樣就算是批次推論也能使用停止點的功能囉！

11.9 自迴歸解碼

自迴歸解碼（Autoregressive Decoding）是在描述模型不斷生成 Token 的過程，其最大的特色在於，模型生成的 Token 會重新變成模型的輸入，模型再根據這些新生成的 Token 繼續往下生成，直到觸發結束條件為止。

在語言模型裡面，每次**推論（Inference）**只會生成一個 Token，必須要經過多次的推論過後，才能完成一次完整的文本**生成（Generation）**。接下來就來嘗試拆解自回歸解碼的背後實做吧！

在 PyTorch 底下，每個模型都可以直接透過 `.forward()` 來進行推論：

```
import torch

with torch.no_grad():
    output = model.forward(input_ids)
```

這裡會產生一個類別為 `CausalLMOutputWithPast` 的輸出，裡面包含了兩個重要的資訊，分別為 `logits` 與 `past_key_values`。

> **補充堅果，頭好壯壯**
>
> 使用 `model.forward(input_ids)` 與 `model(input_ids)` 同樣可以得到模型推論的結果，本書使用 `.forward()` 作為示範，主要是為了強調該步驟為前向運算，另外這樣回傳的結果也會有比較明確的型別提示。
>
> 但是在實際使用時通常不用 `.forward()`，因為在 PyTorch 裡面會有一些前處理與後處理的 Hooks，如果直接使用 `.forward()` 的話可能會忽略掉這些 Hooks 導致某些環節的處理不正常。

`logits` 就是模型預測下個 Token 的機率表，可以對這個 `logits` 做隨機取樣，或者直接計算 `argmax` 取機率最高的 Token 做 Greedy Decode。

而 `past_key_values` 就是俗稱的**鍵值快取（Key-Values Cache）**，是模型對**已知輸入**的運算結果。因為自迴歸的特性，語言模型每次生成的輸出都會變成下個回合的輸入，因此這個 KV Cache 會越長越大，同時也是消耗 GPU 記憶體主要的元兇之一。

有了這份 KV Cache，就不用每次推論時都把完整的 `input_ids` 丟進去，已經看過的輸入就可以捨棄掉，只要留新長出來的 Token 就好，因為舊的輸入都已經被模型變成 KV Cache 了，因此下個回合的推論就會長這樣：

```
with torch.no_grad():
    outputs = model.forward(
        outputs.logits.argmax(-1)[:, -1:],  # Greedy Decode
        past_key_values=outputs.past_key_values,
    )
```

沿著這個脈絡，可以將此邏輯整理成一個迴圈來進行：

```
pkv = None
output_tokens = list()
for i in range(16):
    with torch.no_grad():
        outputs = model.forward(input_ids, past_key_values=pkv)
    next_token = outputs.logits.argmax(-1)[:, -1:]
    token_id = next_token.item()

    if token_id == tk.eos_token_id:
        break

    output_tokens.append(token_id)
    input_ids = next_token
    pkv = outputs.past_key_values

print(tk.decode(output_tokens))
```

這邊固定進行 16 次推論做示範，中間如果遇到 EOS Token 就跳出迴圈。拆解生成步驟除了讓我們更瞭解自回歸解碼的原理以外，也可以在這個過程中針對 `logits` 的機率分佈做觀察。有些時候可以看到模型在**說謊**時，那個部份輸出的 Token 機率分佈會特別的低，因此也可以用來計算模型的信心度之類的。

補充堅果，頭好壯壯

`torch.no_grad()` 是用來停用梯度的 Context Manager，因為 PyTorch 的模型在進行計算時，預設都會順便算出一份梯度用來訓練。但是這裡並不是要做訓練，所以使用 `torch.no_grad()` 來減少不必要的計算，也可以避免語言模型產生過大的梯度而導致記憶體不足喔！

除了 `torch.no_grad()` 以外，使用 `torch.inference_mode()` 也有類似的效果：

```
with torch.no_grad():
    outputs = model.forward(...)

@torch.inference_mode()
def main():
    outputs = model.forward(...)
```

11.10 多顯卡推論

當機器有多張顯卡時，可以透過環境變數 `CUDA_VISIBLE_DEVICES` 來指定要使用哪張顯卡進行運算：

```
# 指定使用 1 號 GPU 進行運算
CUDA_VISIBLE_DEVICES=1 python main.py
```

也可以透過 Python 的 `os.environ` 來指定：

```
import os

os.environ["CUDA_VISIBLE_DEVICES"] = "0,1"

import transformers
```

指定環境變數時，建議位置要先於匯入 Transformers 套件，否則可能會失效。使用多張顯卡同時進行推論時，記得 `device_map` 要設定為 `"auto"`，剩餘的程式碼與一般推論基本相同：

```
model = ModelCls.from_pretrained(model_id, device_map="auto")
```

11.11 到底是在「大」什麼？

講了半天，語言模型到底是「**大**」在哪裡呢？通常在談論一個模型的效能瓶頸時，會分成兩個部份來看：第一個是 GPU 的**運算能力**，也就是能算的多**快**；第二個則是 GPU 的**記憶體容量**，也就是能放下多**大**的模型。

可以寫一個評測速度與記憶體的程式，首先建立假的輸入資料，設定長度與批次大小並且全部填零：

```
import torch

batch_size, seqlen = 1, 1024
torch.zeros(batch_size, seqlen, dtype=torch.int64)
```

記憶體的部份，可以透過 PyTorch 套件取得 GPU 記憶體的用量：

```
unit = 1024**2  # MiB
init_mem = torch.cuda.memory_reserved() / unit
print(f"Memory Usage: {init_mem} MiB")
```

速度的部份，可以借助 `tqdm` 進度條套件來測量：

```
from tqdm import trange

n_decode = 128
with trange(n_decode, ncols=100) as prog:
    for i in prog:
```

```
        # ... Autoregressive Decoding
        total_mem = torch.cuda.memory_reserved() / unit
        prog.desc = f"Memory Usage: {total_mem:.0f} MiB"
```

在迴圈內放入自回歸解碼的程式，並在每次迴圈結束之後計算記憶體的用量，來觀察記憶體隨著生成的過程而產生的變化。

速度與記憶體評測完整程式碼

https://tinyurl.com/llm-note-10

這份程式碼的參數包含模型路徑、批次大小、初始輸入的長度、後續輸出的長度與注意力機制的實做種類。首先先來測試看看 Llama 2 7B 的速度：

```
python HF-Bench.py meta-llama/Llama-2-7b-hf
```

筆者以 RTX 3090 做測試，得到以下輸出結果：

```
Model Weight Memory Usage: 12854 MiB
Sequence Length: 1024
Memory: 3332/16186 MiB: 1024/1024 [00:26<00:00, 39.21it/s]
Decode Length: 1024
Total Length: 2048
```

這個結果顯示，開始推論之前，模型權重佔用了 12854 MB 的 GPU 記憶體，執行到最後多用掉了 3332 MB，總共佔用 16186 MB，花費 26 秒執行，平均每秒可以推論 39.12 次，最後輸入加上輸出的長度為 2048。

接下來筆者分別測試了輸入長度為 2K 與 4K 的結果，得到的速度為每秒 35 次與每秒 29 次。隨著長度的增加，整體的計算量也跟著提昇，因此速度變慢是合理的，而且速度的差距也不算很大。接下來把輸入長度提昇到 6K 試試看：

```
Model Weight Memory Usage: 12854 MiB
Sequence Length: 6144
Memory: 7396/20250 MiB (OOM): 0/8192 [00:00<?, ?it/s]
Decode Length: 1
Total Length: 6145
```

結果記憶體就爆炸啦！那我們稍微折衷一點，從 5K 開始並解碼更長一點：

```
Model Weight Memory Usage: 12854 MiB
Sequence Length: 5120
Memory: 11046/23900 MiB (OOM): 5124/8192 [03:22<02:01, 25.31it/s]
Decode Length: 5124
Total Length: 10245
```

沒想到結果居然是停在 10K ！為什麼直接推 6K 推不動，解碼卻能解到 10K 呢？

補充堅果，頭好壯壯

在 Decoder 語言模型裡面，通常會使用**預填充（Prefill）**代表一開始處理輸入的階段，並使用**生成（Generate）**或**解碼（Decode）**來代表後續產生逐個 Token 輸出的過程。

一般而言 Transformer Decoder 模型會在 Prefill 階段消耗掉大量記憶體，這是因為**注意力機制的運算會在 Prefill 階段產生很大的 Peak Memory**。而後續的生成階段因為每次只需要處理一個 Token，所以記憶體的消耗相對較少。

這也是為什麼以上的測試，在初始輸入 6K 的時候會直接爆炸，但是卻可以從 5K 慢慢推論到總長度 10K 的原因。

雖然 Llama 2 本來就只支援到 4K 的總長度，但以現實應用層面來說，只能輸入 5K 實在太少了！有沒有辦法讓輸入更長一點呢？答案是有的，這裡可以將注意力實做改為 `sdpa` 試試看：

```
python HF-Bench.py meta-llama/Llama-2-7b-hf \
    --seqlen-k=7 --decode-k=8 --attn=sdpa
```

執行結果：

```
Model Weight Memory Usage: 12854 MiB
Sequence Length: 7168
Memory: 11034/23888 MiB (OOM): 2562/8192 [01:45<03:52, 24.26it/s]
Decode Length: 2562
Total Length: 9731
```

改用 `sdpa` 之後，就能從 7K 開始一路推到 9K 啦！雖然最後總長度好像變短了，但其實差距並不算太大。SDPA 的全名為 **Scaled Dot Product Attention**，是 PyTorch 2.0 的新功能之一，可以減少 Attention 計算的 Peak Memory，讓我們能用更少的記憶體去計算更長的輸入序列。其背後應用的原理之一就是傳說中的 **Flash Attention**。

Flash Attention 由史丹佛大學的博士生 Tri Dao 所提出，作者指出 Attention 運算最大的瓶頸在於記憶體，尤其是 Softmax 運算因為需要拜訪所有元素來計算各自的機率，所以消耗的記憶體特別巨大，因此透過 **IO-Aware Tiling** 與 **Fused Kernel** 這兩個核心技術來突破記憶體瓶頸。

IO-Aware Tiling 指的是**拆解 Softmax 運算**，避免需要完整拜訪所有元素。這就像是在統計校際排名時，如果一次把所有學生的成績都拿來排序，那會花比較多的力氣。但如果先由各個班級統計出班排名，再統合起來計算校排名，就會輕鬆一些。

Fused Kernel 指的則是**合併注意力機制的各種運算**，雖然這個運算裡面的實做還是包含所有運算，但是減少了彼此溝通的開銷。就像原本一個產品可能需要經過四五間流水線工廠才能完成，但現在成立一間整合所有流水線的工廠，減少了工廠之間的溝通成本，進而提昇產品的完工效率。

補充堅果，頭好壯壯

Flash Attention 的作者後來又提出了改良版的 Flash Attention 2，進一步改善了運算效率，並且提供 Python 套件可以安裝。但 Flash Attention 2 目前只支援 Ampere（RTX 30 系列）以上的 GPU 使用，因此若為 Turing 架構以下的 GPU，改用 SDPA 是個不錯的方案。

筆者實測 PyTorch 的 SDPA 在速度、記憶體用量與準確率上與 Flash Attention 2 沒有太大的差異，而且有裝 PyTorch 就有 SDPA 能用，是個相對方便的選擇。

11.12 結論

本章節介紹了偉大的開源笑臉 Hugging Face Transformers 套件，並著重在推論方面進行講解。此套件的方便與靈活，讓我們能夠更輕鬆的在這些大大小小的模型之間穿梭。他們充滿活力的團隊與活躍的社群，讓整個開源社群更加蓬勃發展。如果想要更瞭解 Hugging Face，可以關注他們的**部落格**[11]，閱讀他們豐富的**文件**[12] 以及許許多多的**教學資源**[13]。

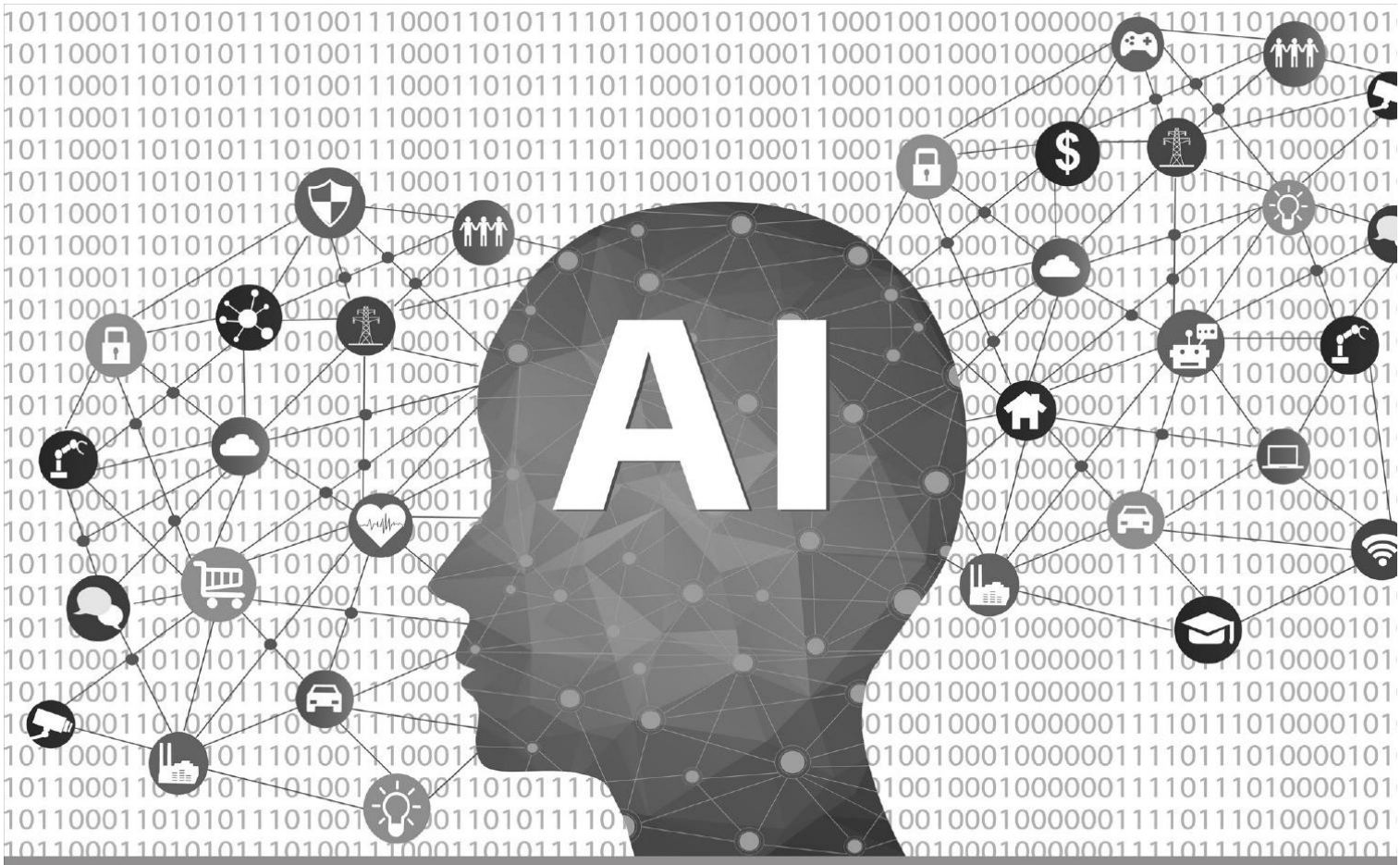
11 https://huggingface.co/blog
12 https://huggingface.co/docs
13 https://huggingface.co/learn

補充堅果，頭好壯壯

同樣由 Hugging Face 開發的 **Candle**[14] 框架，是使用 Rust 語言開發的機器學習框架。Rust 是以高效率、高記憶體安全聞名的程式語言，是個閃亮的明日之星，因此 Candle 框架也是個相當值得關注的專案。

然而 Transformers 終究是個為了訓練而生的框架，其主要的功能還是在訓練模型上。因此在推論部署這塊，並不是最佳首選的框架。未來將會介紹各種專為推論而生的框架，在此之前會先來探索各式各樣的大型語言模型！

14 https://github.com/huggingface/candle



Chapter

12

數若繁星的語言模型們

主管 @tag 我

@penut 客戶資料不能上雲，我們不能串 ChatGPT API 怎麼辦？

沒問題，打開 Hugging Face Hub 滿滿的開源模型任你挑！

雖然多數的開源模型不會像 ChatGPT 一樣，動輒幾千億的參數量，但即便模型只有 7B 或 13B，在只有一兩張 RTX 3090 或 RTX 4090 顯卡的規格下，想要自己從頭開始做預訓練也是非常困難。這時候選擇擁抱開源的語言模型，就是個相當經濟實惠的方案。學會使用 Transformers 套件的我們，已經有足夠的能力可以開始玩轉一些小模型了，本章節將來介紹一些模型，並區分成英文模型、中文模型與臺灣模型三大類。

12.1 英文模型

12.1.1 Llama

LLaMA[1] 是 2023 年 2 月底由 Meta AI 所發表的系列模型，參數量包含 7B、13B、30B 與 65B 等四種規格。第一代模型所使用的訓練資料集全部都是公開資料集，例如網路爬蟲 C4、維基百科、古騰堡計劃 Gutenberg 電子書、arXiv 論文、GitHub 程式碼以及 StackOverflow 家族 StackExchange 系列網站的問答等等，總 Tokens 數約 1.4T 的文本量。有人將這些資料集整理成 RedPajama 資料集，光這個資料集就要用掉好幾個 TB 的硬碟空間，真不是普通人玩的起的東西

1 https://arxiv.org/abs/2302.13971

花生狂想曲 💡

根據 LLaMA 的論文所說，他們**拿 2048 張 A100 80 GB** 訓練 LLaMA 65B 花了 21 天才訓練完…EXCUSE ME WHAT?! 這些 AI 龍頭公司真是遠遠超乎我的想像 😱

LLaMA 原本是沒有開源的，需要填表申請才能取得，但是被大家偷偷上傳到 Hugging Face Hub 上，加上模型的效果是滿不錯的，因此引起了一波不小的 LLaMA 熱潮，著名的 **llama.cpp** 專案就是在此時誕生的。那你問我說 LLaMA 的權重被偷偷上傳到哪了呢？這些已經不重要了，因為 2023 年 7 月的時候 **Llama 2**[2] 誕生了。

Llama 2 將訓練資料量擴大到 2T Tokens 的文本量，並將原本 2K 的模型長度擴展到了 4K，實用程度大幅上升。Llama 2 系列不同於一代只有釋出預訓練模型，還做了指令微調與 RLHF 訓練。最重要的是，Llama 2 正式在 Hugging Face 上開源模型權重啦！無論是預訓練版的 Llama 2 還是 Chat 版的 Llama 2 都可以在 HF Hub 上找到，真的是 \ 佛心公司 /\ 佛心公司 / 🙏

但是 Llama 2 本身的中文能力其實並不好，因此若要應用在中文情境的話，需要先有一定程度的微調進行中文化才行。但是各位中文使用者也不用太擔心，因為更強大的 Llama 3 在 2024 年 4 月問世，雖然只有釋出 8B 與 70B 兩種規格，但是訓練資料量大幅上升到 15T Tokens，模型能力有了相當程度的提昇，使用中文的能力也改善許多。

2 https://arxiv.org/abs/2307.09288

關於 Llama 系列的更多資訊，可以參考官方網站[3] 與 GitHub[4]。

花生狂想曲

Llama 是**大羊駝**的西班牙語，也就是俗稱的草泥馬。

12.1.2 Alpaca

Alpaca 是 2023 年 3 月由 Stanford 大學公開的一套訓練方法與模型，這個模型是從 LLaMA 模型微調訓練而來。Alpaca 的特別之處在於他的訓練資料是使用 GPT-3 生成的，作者首先蒐集了一些**種子任務（Seed Task）**並請 GPT-3 產生更多類似的範例，這個種子任務可能像是這樣：

請把 "Starburst Stream" 翻譯成中文
星爆氣流斬
請按照以上格式，產生更多類似的範例。

請把 "Fire Dragon Roar" 翻譯成中文
火龍咆哮
請把 "Thunderbolt Strike" 翻譯成中文
雷電一擊

如此一來就能用這些產生出來的資料對 LLaMA 做指令微調，藉此萃取閉源大模型的知識，這種做法也被稱為**知識蒸餾（Knowledge Distillation）**。這個模型發布之時，還沒有 ChatGPT API 可以使用，因此原作者是使用 GPT-3 來生成

3 https://llama.meta.com/
4 https://github.com/meta-llama

五萬多筆資料，花費將近 600 美元。現在有了 GPT-3.5 之後，這個成本可以減少 10 到 20 倍左右，而且生成的資料品質還會比 GPT-3 的資料更好。

> 花生狂想曲
> Alpaca 是**羊駝**的西班牙語，與大羊駝外觀相似，也算是一種草泥馬。

12.1.3 Vicuna

Vicuna 是來自 UC Berkeley 柏克萊加州大學以及數個學術研究單位聯手合作發表的模型，同樣是從 LLaMA 系列微調訓練而來。Vicuna 的特色在於它是使用 ShareGPT 資料集進行訓練的。

ShareGPT[5] 其實原本是一個瀏覽器外掛，在 ChatGPT 剛開放時，因為還沒有分享對話的功能，因此有些人會使用這個外掛來分享對話內容。一開始 ShareGPT 可以透過爬蟲程式爬取許多對話資料下來，而 Vicuna 的 ShareGPT 訓練資料集就是在那個時候蒐集而來的。但現在 ShareGPT 已經關閉爬蟲入口了，因此如果現在想要使用 ShareGPT 做訓練，可以參考 HF Hub 上別人整理的 **ShareGPT 資料集**[6]。

因為 Vicuna 使用的 ShareGPT 訓練集是**真實的對話資料**，所以相比於使用模型生成的 Alpaca 資料集，ShareGPT 的資料**多樣性（Diversity）**高很多。就筆者的實測經驗來看，雖然 ShareGPT 裡面中文的資料比例不算很高，但是 Vicuna 的中文問答、摘要與翻譯能力都相當優異。

5 https://sharegpt.com/
6 https://huggingface.co/datasets?search=sharegpt

在 Llama 2 推出後，Vicuna 也跟著推出了 Llama 2 版本的 Vicuna 模型，效果同樣也相當好，可以到他們的 **Demo**[7] 網頁上試試看。這個 Demo 網頁不只有 Vicuna 模型可以試用，還有許多大大小小的模型供大家免費試玩。

圖 12-1　試用 Vicuna 模型

可以看到繁體中文的文本生成非常自然流暢，雖然有些時候用語可能怪怪的，不過基本上都還是可以接受。

> **花生狂想曲**
> Vicuna 是**小羊駝**的西班牙語，與羊駝外觀比較不像。

12.1.4 TinyLlama

TinyLlama[8] 是個很特別的專案，專案作者試圖訓練一個 1.1B 參數量的 Llama 模型，並使用 3T Tokens 的訓練資料集，預計使用 16 張 A100 40 GB 進行訓練，

7　https://chat.lmsys.org/
8　https://github.com/jzhang38/TinyLlama

約三個月可以完成。目前已經完成訓練，作者也分享了許多訓練過程中的成果，包含模型權重、訓練過程的 Loss 曲線圖，以及各種預訓練的踩雷經驗等。

作者也有對 TinyLlama 進行指令微調，但是因為模型參數量很小的關係，所以能力並沒有很強大。也因為是個小模型的關係，所以很適合做一些小規模的研究，若是有些新的技術或框架想要嚐鮮，筆者通常也會優先拿 TinyLlama 來試刀，速度快體積小相容性高，真的很方便！

花生狂想曲 💡
迷你羊駝算是一種小羊駝嗎 🧐

12.1.5 Phi

Phi 系列[9] 是由微軟推出的一系列語言模型，這個系列從參數量僅 1B 的 Phi-1、Phi-1.5 等，到現在 3B 左右的 Phi-2、Phi-3，依然維持少少的參數量。除了小參數量的特色以外，還主打**教科書就是你所需要的（Textbooks Are All You Need）**這個口號，這裡所說的教科書是指**請 LLM 生成一份教科書式的說明**。因為作者認為這類的資料包含更多**推理步驟**在裡面，這樣的邏輯資訊比起一般的爬蟲文本，對於語言模型的訓練而言是更有幫助的。

12.1.6 Mistral & Mixtral

Mistral[10] 是由法國新創 Mistral AI 所開發的語言模型，模型架構上與 Llama 略有不同，使用了**分組查詢注意力（Grouped-Query Attention, GQA）**與**滑動**

9 https://tinyurl.com/llm-ms-phi
10 https://mistral.ai/news/announcing-mistral-7b/

窗口注意力（Sliding Window Attention, SWA）兩種技術，目的是減少語言模型帶來的 GPU 記憶體消耗量，雖然只有 7B 參數量的規格，但是外界評測效果相當好，因此相當受到開源社群的歡迎。

在 Mistral 獲得巨大的成功之後，官方又進一步推出 Mixtral **8x7B**[11]、**8x22B**[12] 兩種**混合專家模型（Mixture-of-Experts, MoE）**。MoE 架構就像是同時訓練八個專家的模型，但是每次只挑兩個專家來使用。因為參數量更大，所以可以容納更多的知識，而且每次推論只使用少量專家，所以能夠維持推論的速度優勢。

> **花生狂想曲**
>
> 從官方的數據來看，可以說 Mixtral 8x7B 僅用約 50B 的參數量，就能與 70B 模型的知識量抗衡，而且維持 13B 的推論速度。話雖如此，但筆者認為知識量與訓練資料的關係也很大，不單純只是看模型架構而已。而且運行 50B 參數量的模型，所需消耗的記憶體也是相當龐大，即便推論速度很快。

比較可惜的是官方為了維持競爭優勢，所以沒有揭露太多與訓練資料相關的資訊。

11 https://huggingface.co/mistralai/Mixtral-8x7B-Instruct-v0.1
12 https://huggingface.co/mistralai/Mixtral-8x22B-Instruct-v0.1

12.2 中文模型

12.2.1 Qwen

Qwen（通義千問）[13] 是由中國阿里巴巴集團所開發的中文語言模型，規模種類相當繁多，從 1B、4B 到 72B、110B 都有，甚至多模態的 Qwen-VL、Qwen-Audio 等，全部都開源在 HF Hub 上，各方面來說都是規模相當大的系列模型。尤其是多模態模型，雖然與 ChatGPT 或 Gemini 相比可能不算頂尖，但已經算是滿有效果了。

12.2.2 Yi

Yi[14] 是由李開復博士領銜的中國零一萬物（01-ai）團隊所開發的語言模型，包含 6B、9B 與 34B 等三種規格，採用 Llama 架構，而且無論簡體中文或是繁體中文能力都相當好，是筆者用過的開源模型裡面數一數二頂尖的中文模型。

12.3 臺灣模型

做為語言資源相對稀少的繁體中文使用者，有人願意投入臺灣文化的語言模型開發，都是很值得鼓勵的事情！以下介紹一些筆者所知的臺灣相關大型語言模型。

13 https://github.com/QwenLM

14 https://huggingface.co/01-ai

12.3.1 Breeze & BreeXe

Breeze[15] 是由聯發科技集團的 AI 研究單位聯發創新基地（MediaTek Research）所開發的繁體中文語言模型，架構與權重承襲自 Mistral，同樣為 7B 參數量的模型，但是分詞器有針對繁體中文額外擴充詞表。可能是因為參數量並不大的關係，所以能力上也只是普通而已。

但是，開發團隊後來又推出了 **BreeXe-8x7B**[16] 的模型，與 Mixtral 一樣採用 MoE 架構，效果大大提昇！是目前筆者用過最頂尖的繁體中文模型，可惜的是權重並不是完全開源的，只能在 **Demo**[17] 網頁上簡單測試而已。

12.3.2 Taiwan LLM

Taiwan LLM[18] 是由國立臺灣大學資訊工程學系機器智慧與理解實驗室（Machine Intelligence & Understanding Laboratory, MiuLab）訓練的模型，如果你經常在 YouTube 上收看一些深度學習相關的學校課程影片，可能就會知道這是**陳縕儂教授**[19] 的實驗室。

Taiwan LLM 使用超過 300 億 Tokens 的繁體中文文本，對 Llama 系列的模型進行**持續預訓練（Continual Pretraining, CP）**，並使用超過 100 萬筆對話資料做指令微調。這個模型不單單只是會輸出繁體中文而已，還主打臺灣文化的融入。

15 https://huggingface.co/MediaTek-Research
16 https://tinyurl.com/llm-breexe
17 https://tinyurl.com/llm-breexe-demo
18 https://github.com/MiuLab/Taiwan-LLM
19 https://www.youtube.com/@VivianMiuLab

除了 LLaMA 與 Llama 2 以外，作者也基於 Mixtral-8x7B 與 Llama 3 等模型開發出不同版本的 Taiwan LLM，最近更是推出了效果十分強大的 Taiwan Llama 3 8B & 70B 的模型，外界評估各項表現都相當驚人，可以在 **Demo**[20] 網頁試用看看。

12.3.3 TAIDE

TAIDE[21] 是由臺灣國科會主持的計畫，目標是要發展繁體中文的可信任生成式 AI 對話引擎。訓練資料包含了許多臺灣政府機關的文件，例如高雄旅遊網、慢遊雲林等觀光網頁，以及繁體中文維基百科、教育部的字典和各大媒體授權的新聞文章等。總共大約 40B Tokens 的繁體中文文本，使用國網中心的 H100 訓練而成。

目前已經釋出了 Llama 2 7B 與 Llama 3 8B 等版本的模型，算是小有成果，值得大家繼續關注與支持！

12.3.4 Bailong

白龍 Bailong[22] 是由群創光電所開發的模型，基於 Llama 2 7B 使用 QLoRA 與 Zip-Tie Embedding 等技術訓練而成。QLoRA 會在之後的章節介紹，而所謂的 Zip-Tie Embedding 指的是在擴增詞表時，使用舊詞的 Embedding 來合成新詞的 Embedding，這樣新詞的 Embedding 就不需要重新訓練與學習了，能進一步提昇模型訓練的效率，這種做法同時也被稱為 Vocabulary Transfer。

20 https://twllm.com/

21 https://taide.tw/index

22 https://tinyurl.com/llm-bailong

12.3.5 FFM

福爾摩沙大型語言模型（Formosa Foundation Model, FFM）是由華碩子公司台智雲[23] 開發的系列模型，第一版是以 BLOOMZ 7B 與 176B 為基礎，加入大量繁體中文語料進行訓練的模型，其中文資料佔總訓練資料的 30% 左右。後續也推出了基於 Llama 2、Llama 3、Mistral 與 Mixtral 8x7B 等模型訓練而成的版本，但這些模型權重並沒有開源，而是放在他們的 AFS 平台上提供服務，有商用需求的讀者不妨參考看看。

12.4 其他資源

12.4.1 Awesome LLM

Awesome LLM[24] 整理了這些如天上繁星般多的語言模型們，其中有張動畫[25] 非常有意思，這個動畫講述了整個語言模型的發展史，我想未來大概會跟這張 Linux Distro Tree[26] 一樣越長越大而且越來越複雜。

12.4.2 Open LLMs

Open LLMs[27] 整理了多個可商用的開源模型，如果有商用需求且需要借助開源力量的人可以參考看看此資訊。

23 https://tws.twcc.ai/
24 https://github.com/Hannibal046/Awesome-LLM
25 https://tinyurl.com/llm-tree-gif
26 https://tinyurl.com/linux-distro-tree
27 https://github.com/eugeneyan/open-llms

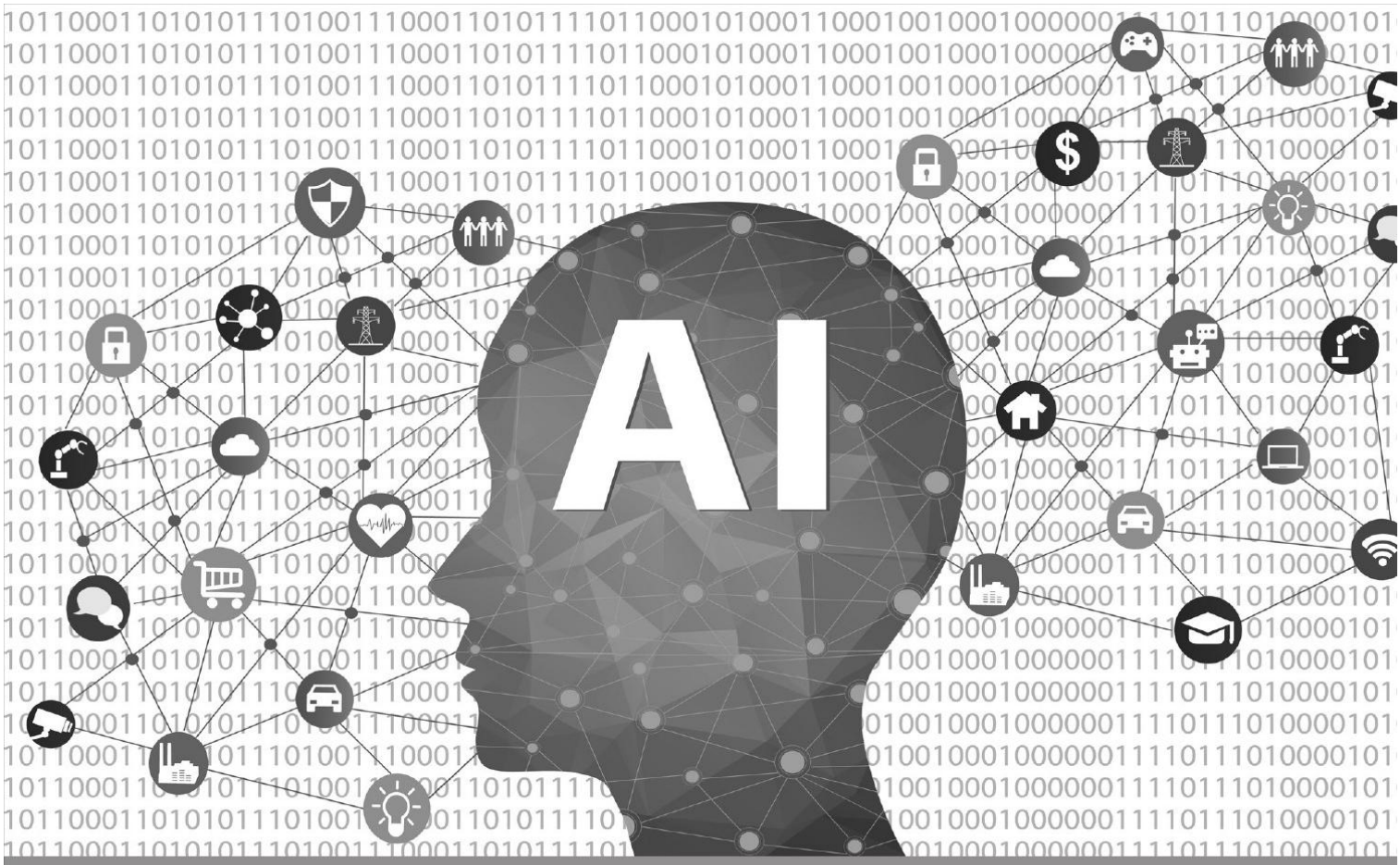
12.4.3 Open LLM Leaderboard

Open LLM Leaderboard[28] 是放在 HF Space 上的一個應用，裡面收錄了許多 HF Hub 上的模型進行評估的結果，並整理而成的天梯榜。雖然可以參考這個排行榜來選用模型，但這個排行榜進行的評估未必能充分反映實際情況，最好還是以自身的實際應用為主，並進行足夠完整的測試評估會比較恰當。

12.5 結論

把 Awesome LLM 打開就知道，筆者所介紹的這些模型完全只是冰山一角。本章節主要是介紹一般用途的語言模型，通常只用於普通的文字生成任務，例如問答、翻譯或摘要等等。但其實還有一系列的模型是寫程式的專家喔！接下來將會介紹這些擅長寫程式的模型。

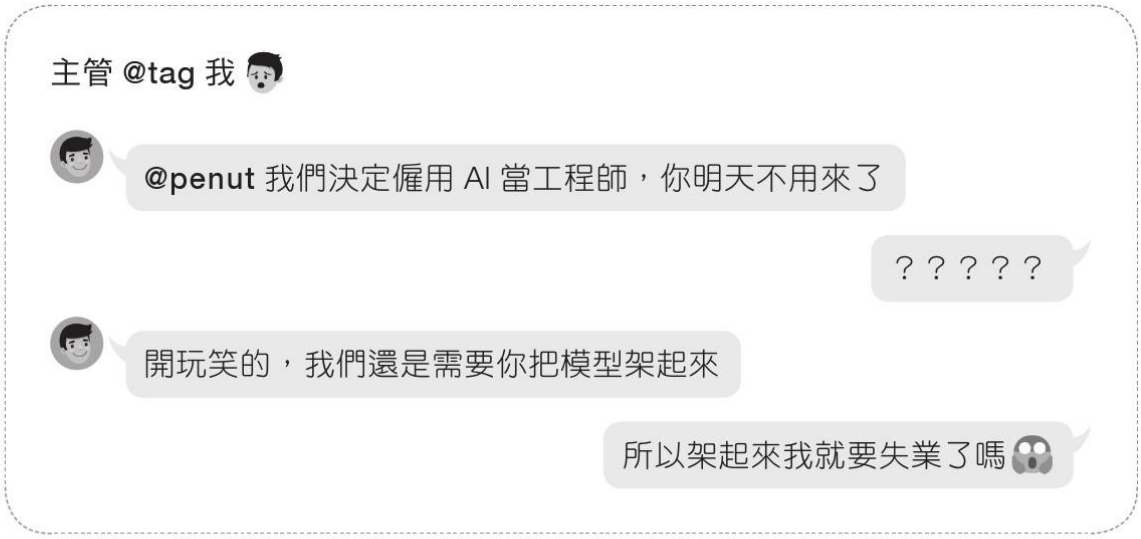
28 https://tinyurl.com/llm-leaderboard



Chapter

13

語言模型也懂寫程式！



Code LLMs 是專門使用程式碼訓練的的語言模型，因為寫程式的行為模式恰好與語言模型文字接龍的特性非常相似，因此發展比聊天模型又早了一些，例如 Tabnine 與 GitHub Copilot[1] 之類的，其實都比 ChatGPT 還要早發跡。本章節就來介紹這些開源的專業碼農模型，以及他們的使用方法。

13.1 StarCoder

StarCoder[2] 由 BigCode[3] 開發，是由 Hugging Face 主持的一個 Workshop 組織，採用他們自己研發的 GPTBigCode 架構，從 GPT-2 的架構修改而來。其訓練資料被整理為 StarCoderData[4]，包含 86 種程式語言以及各種 GitHub Issues 的資料，大小總計 783 GB 共 250B Tokens 的文本量。

1 https://github.com/features/copilot
2 https://github.com/bigcode-project/starcoder
3 https://www.bigcode-project.org/
4 https://huggingface.co/datasets/bigcode/starcoderdata

StarCoder 原先只有 15B 參數量的規格，後來陸續增加了 7B、3B、1B 等版本，讓資源有限的開發者可以更輕鬆的部署這些模型。

StarCoder 主要有三種版本：

- StarCoderBase 只用程式碼資料做預訓練。
- StarCoderPlus 使用程式碼資料加上英文自然語言文本資料一起訓練。
- StarCoder 使用程式碼資料跟很多 Python 原始碼資料做訓練。

BigCode 另外有 **StarChat**[5] 模型，從 StarCoderPlus 進行指令微調後的版本。後來也推出了 StarCoder 2 系列，使用超過 600 種程式語言約 4T Tokens 的 **The Stack v2**[6] 資料集進行訓練，能力比一代好上許多。

StarCoder 2 使用範例程式碼

https://tinyurl.com/llm-note-08

可以使用 HF Transformers 套件來玩看看這個模型，這邊以 StarCoder 2 7B 示範，首先讀取模型權重與分詞器，並將 Generate 包裝成函式方便後續使用：

```
import torch
from transformers import GPT2Tokenizer as TkCls
from transformers import Starcoder2ForCausalLM as ModelCls
from transformers import TextStreamer

model_id = "bigcode/starcoder2-7b"
model: ModelCls = ModelCls.from_pretrained(
    model_id,
    device_map="auto",
    torch_dtype=torch.float16,
```

5 https://huggingface.co/HuggingFaceH4/starchat-beta

6 https://huggingface.co/datasets/bigcode/the-stack-v2

```
)
tk: TkCls = TkCls.from_pretrained(model_id)
ts = TextStreamer(tk)

def generate(prompt, n):
    inputs = tk(prompt, return_tensors="pt").to("cuda")
    return model.generate(**inputs, max_new_tokens=n, streamer=ts)
```

一般的程式碼生成，只需要透過模型本身的文字接龍能力即可達成：

```
generate("def fib(n: int):", 28)
```

得到的輸出結果如下：

```
def fib(n: int):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
```

是個非常標準的費波那契數函式👍

但是一般來說，寫程式時並不會總是在檔案的最尾端，因此在文字編輯器的游標前後或多或少都有其他程式碼，寫程式時不僅要參考前面的程式碼，也要參考後面的程式碼。然而 Decoder 的特性使他只能往後生成文字，而不能在中間進行生成。這時候**填充程式碼（Fill in the Middle, FIM）**[7] 就派上用場了！

舉例來說，考慮以下程式碼：

```
def hello(name: str):
    print  # <= 鍵盤游標在這！

def goodbye(name: str):
    print(f"### 系統：再會了，{name}!")
```

7 https://arxiv.org/abs/2207.14255

根據鍵盤游標的所在位置，將程式碼切成兩半。前半段稱之為 **Prefix**，而後半段稱為 **Suffix**，將 Prefix 的內容放在 `<fim_prefix>` 這個特殊 Token 的後面，再把 Suffix 的內容放在 `<fim_suffix>` 後面，並將這兩段接起來，最後放上 `<fim_middle>` 來發動程式碼填充的能力，整體而言會像這樣：

```
<fim_prefix> [ 前半段的內容 ] <fim_suffix> [ 後半段的內容 ] <fim_middle>
```

將這種格式的提示輸入模型，就能夠產生中間的程式碼。模型能夠具有這樣的能力，就是因為在訓練的時候，FIM 訓練會將挖空的程式碼片段放在 `<fim_middle>` 後面。如此一來，模型就可以在維持原本文字接龍能力的情況下，獲得程式碼填充的能力。

StarCoder 2 在 FIM 模式下用來停止生成的 Token 是 `<file_sep>`，所以需要稍微修改一下生成函式：

```
def generate(prompt, n):
    inputs = tk(prompt, return_tensors="pt").to("cuda")
    return model.generate(
        **inputs,
        max_new_tokens=n,
        streamer=ts,
        eos_token_id=tk.encode("<file_sep>")[-1],
    )
```

接下來試著實際操作看看這種推論方法：

```
full_code = """

def hello(name: str):
    print(< 游標在這 >)

def goodbye(name: str):
    print(f"### 系統：再會了，{name}!")

"""

prefix, suffix = full_code.split("< 游標在這 >", 1)
```

```
full_prompt = f"<fim_prefix>{prefix}<fim_suffix>{suffix}<fim_middle>"
outputs = generate(full_prompt, 16)
```

得到以下輸出：

```
<fim_prefix>

def hello(name: str):
    print(<fim_suffix>)

def goodbye(name: str):
    print(f"### 系統：再會了，{name}!")

<fim_middle>f"### 系統：你好，{name}!"<file_sep>
```

最後透過一些後處理，將填充程式碼的生成結果重建回原本的程式碼：

```
tokens = tk.encode(full_prompt)
output = outputs[0][len(tokens):]
middle = tk.decode(output, skip_special_tokens=True)
print(full_code.replace("< 游標在這 >", middle))
```

得到最後完整的程式碼如下：

```
def hello(name: str):
    print(f"### 系統：你好，{name}!")

def goodbye(name: str):
    print(f"### 系統：再會了，{name}!")
```

這樣就完成一次程式碼填充囉！這種先放 Prefix 再放 Suffix 最後接 Middle 的格式，也被稱為 **PSM（Prefix-Suffix-Middle）**模式。另外也有先放 Suffix 再放 Prefix 最後 Middle 的版本，被稱為 **SPM（Suffix-Prefix-Middle）**模式。

13.2 CodeGen

CodeGen[8] 是 Salesforce[9] 訓練的系列模型，包含了 CodeGen、CodeGen2 與 CodeGen2.5 等。前幾代參數量都有出到 16B 的規格，但 CodeGen2.5 只出了 7B 這個規格，這是為什麼呢？

原本根據 OpenAI 提出的**縮放定律（Scaling Law）**，認為模型參數量與訓練資料量有個一定的比例，當資料量超出模型參數量的某個比例之後，模型的效果提升會出現邊際效應。另外在訓練語言模型時，為了避免過度擬合（Over Fitting）的情況，通常一份文本只會訓練一次，也就是只訓練 1 個 Epoch。

但模型作者對這點抱持懷疑的態度，於是結合了 Encoder 常用的 Masked LM 與 Decoder 的 Causal LM 兩種建模方式，藉此對語言模型進行多個 Epochs 的訓練，最後得到效果相當好的 CodeGen2.5 模型。

CodeGen2.5 雖然只有 7B 的參數量，但是大幅度的打敗了 CodeGen2 16B 的模型，因此作者相當有自信的推出僅 7B 大小的 CodeGen2.5 模型，並將模型的介紹文章標題命名為「**小而強大（Small, but mighty）**」。

> **花生狂想曲**
> 筆者相當佩服這個實驗精神，研究團隊他們去質疑、去挑戰，雖然縮放定律有一定的參考性，但不該變成研究的侷限性，而開發團隊找到了一個方法去嘗試，並且因此有所突破，我想這是科學研究裡面相當難能可貴的一件事情。

8 https://github.com/salesforce/CodeGen

9 https://www.salesforce.com/

CodeGen2.5 同樣具有填充程式碼的能力，其格式如下：

```
prefix + "<mask_1>" + suffix + "<|endoftext|>" + "<sep>" + "<mask_1>"
```

使用此格式進行推論時，最後會生成一個 `<eom>` 的 End-of-Mask Token，可以根據這個 Token 是否出現來停止生成。

13.3 CodeLlama

原本筆者很開心的抱著 CodeGen 到處用，因為這個模型是我手邊一些實驗裡面性價比最高的一個模型，所以很長一段時間都很仰賴 CodeGen 的幫忙，~~直到我被 CodeLlama NTR 為止~~

CodeLlama 是從 Llama 2 持續訓練而來的，使用 500B Tokens 資料量的程式碼，並加上約 8% 的自然語言資料來維持原本自然語言的能力。透過**長上下文微調（Long Context Fine-Tuning, LCFT）**的技巧，將 Llama 2 原本 4K 的模型長度擴展到了 16K，這個方法類似於**內插法（Position Interpolation, PI）**[10]，透過改造**旋轉位置編碼（Rotary Position Embedding, RoPE）**來提高模型長度。官方宣稱 CodeLlama 實際使用時，可以處理長度達 100K 的程式碼。

> **花生狂想曲**
> 雖然模型有那個 100K 的處理能力，但我可沒有 100K 的硬體可以推這模型

10 https://arxiv.org/abs/2306.15595

CodeLlama 包含 7B、13B、34B 與 70B 四種大小，在 HF Hub 上全部開源[11]。CodeLlama 同樣具有填充程式碼的能力，使用 CodeLlama 發動 FIM 的格式如下：

```
prompt = f"_<PRE> {prefix} _<SUF> {suffix} _<MID>"
```

生成的結果最後會附上一個 `_<EOT>` 的 Token 用來當作結束生成的依據。

13.4 DeepSeek Coder

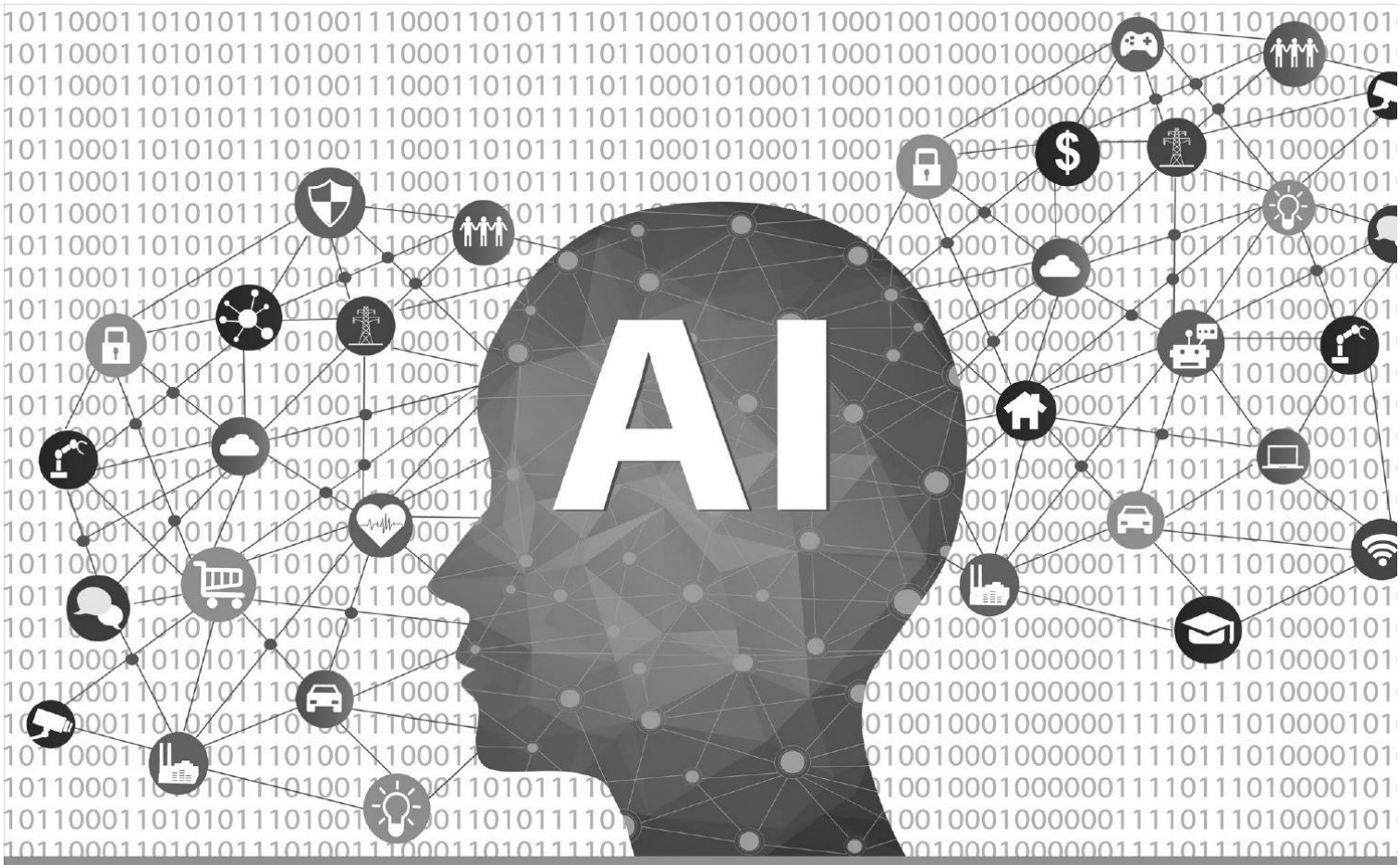
DeepSeek Coder[12] 是由 DeepSeek（深度求索）這間中國公司所開發的系列模型，其模型架構基於 Llama，包含 1.3B、6.7B 與 33B 三種參數量。訓練資料量達 2T Tokens，包含 87% 的程式碼、10% 的英文自然語言與 3% 的中文自然語言。也因此這個模型的中文能力相對好，很適合用來處理包含中文的程式碼。在開源的 Code LLMs 裡面，DeepSeek Coder 的效果也是相當頂尖。

13.5 結論

雖然專精程式碼的語言模型不如一般用途的語言模型那麼多，但筆者介紹的這些模型依然也只是冰山一角。這些開源的 Code LLMs 提供給開發者更多本地且輕量部署的選擇。模型的用途也從原本的程式碼接龍、程式碼填空一路走向對話形式，未來甚至還有機會朝著**多模態（Multimodal）**邁進，成為伴隨開發者更有力的助手！

11 https://tinyurl.com/llm-code-llama

12 https://github.com/deepseek-ai/DeepSeek-Coder



Chapter

14

貧民英雄，量化超人

主管 @tag 我

@penut 客戶只有一張 4090 能跑什麼 LLM ？

有了量化，什麼都能跑！

量化（Quantization）是我們這些平民玩家最好的夥伴，一般在進行訓練時，通常會使用 32-Bit 或 16-Bit 的浮點數來訓練模型權重。但即便是 7B 16-Bit 的模型，也要消耗掉 13 GB 以上的 GPU 記憶體，更不用說是 13B 以上的模型，一般單張 24 GB 的消費級顯卡根本放不下。

這時候量化超人就來拯救我們了！量化技術可以把模型權重轉換成 8-Bit 或 4-Bit 甚至更低，大幅減少模型佔用的 GPU 記憶體，只要使用一張顯卡就能夠操作 13B 甚至 30B 以上的模型。一起來看看這個量化技術到底是怎麼辦到的吧！

14.1 基礎概念

14.1.1 說文解字

量化一詞有多種用法，其中一個是**把抽象的概念給數字化**，舉例來說：

主管 @tag 我

@penut 你今天有多努力？

我今天寫了 30 行程式碼我超認真

「多努力」就是一種抽象的概念，而我們用「30 行程式碼」這種具體的數字與單位，來量化一個人今天有多偷懶

這樣的量化方法，其實會忽略掉許多細節。也許對方今天做了很多寫程式以外的事情，但是在這個量化方法下都被無視了，也因此產生了**誤差（Error）**。凡是量化則必定有誤差，因此量化這門技術真正關注的是**如何把誤差降到最低**。

14.1.2 簡單推論

在機器學習領域裡面，量化代表的是**值域轉換**。例如將 16 位元的浮點數（FP16）量化成 8 位元的整數（INT8），這種將高位元的資料轉換為低位元的過程。

> **花生狂想曲**
> 其實也能把 FP16 量化成 INT16 或 INT32，但比較少見。

量化技術的初始概念相當單純，假設我們有以下 FP16 數列：

```
[1.2, 0.3, -0.1, -0.8]
```

觀察這個數列，他的值域介在 `[-0.8, 1.2]` 之間。若要用 INT8 來表示，那就需要將這個值域對應到 `[-128, 127]` 之間。最簡單的做法就是透過**正規化（Normalization）**的技巧將這些數字正規化到那個值域裡面。

正規化是機器學習裡面相當重要的技巧，透過將輸入正規化到 `[-1.0, 1.0]` 之間來減少偏移量，使模型更容易收斂。若將正規化的做法套在量化上，首先要先將整組數列減去**最小值** `-0.8` 得到：

```
[2.0, 1.1, 0.7, 0.0]
```

接著計算原本值域的**間距** `(1.2 - -0.8) = 2.0`，並將新的數列除以間距：

```
[1.0, 0.55, 0.35, 0.0]
```

最後將整組數列**縮放**到 INT8 的值域 `x * 255 - 128` 並四捨五入：

```
[127, 12, -39, -128]
```

到這一步，就完成了 FP16 到 INT8 **量化（Quantize）**的動作。將這組數列存下來時，最小值 -0.8 與間距 2.0 也要一起被存下來。

實際進行矩陣運算時，通常會先將數列**反向量化（Dequantize）**回來，也就是按照量化的流程反向計算回去：

```
(x + 128) / 255 * 2.0 + (-0.8)
```

反向量化後的數列如下：

```
[1.2, 0.298, -0.102, -0.8]
```

可以看到與原始數列相比，反量化後的數列並沒有被精準的還原，數值內容是有誤差的。因此量化技術探討的不僅只是將數值壓縮至低位元，更重要的目標在於如何減少量化所產生的誤差。

這時你可能會想到，額外存的最小值與間距難道就不佔空間嗎？沒錯，最小值與間距通常會用原本的資料型態存放。除了精準度的考量以外，也要考慮**極值**的問題，避免間距或最小值因為過大過小而發生溢位的問題。但也因此需要花費額外的空間來存放最小值與間距。

以上例而言，四個數值省下了 `8 * 4 = 32` 個位元，但是因為額外存了兩個 FP16 的資訊，所以兩者就抵銷了。雖然一般而言不會只拿著四個數值就在那邊做量化，但還是可以使用其他方法來進一步減少額外資訊。

例如我們可以先對數列**取絕對值**再做量化，這樣的好處在於最小值必定為零，以上例而言，數列的值域就被縮減到 `[0.0, 1.2]` 之間。按照正規化的概念，應該先將數列除以 `1.2` 來正規化到 `[-1.0, 1.0]` 之間，並乘上 `127` 來得到 INT8 數列：

```
(x / 1.2) * 127
```

透過一些簡單的數學原理，可以將算式整理如下：

```
(x / 1.2) * 127
 = x * (1 / 1.2) * 127  # 改寫為乘法
 = x * 127 * (1 / 1.2)  # 根據交換律
 = x * (127 / 1.2)      # 根據結合律
```

這個 `127 / 1.2` 就是所謂的**量化縮放係數（Quantization Scaling Factor）**，以此例而言為 `127 / 1.2 = 105.83`，也是這個量化方法唯一需要另外存放的額外資訊。將此縮放係數與原本的數列相乘並四捨五入，得到以下數列：

```
[127, 32, -11, -85]
```

而反向量化也只要除以量化常數即可：

```
[1.2, 0.302, -0.104, -0.803]
```

這樣的做法誤差會稍大一些，但是實做上也比較方便。這種做法也稱為**絕對最大值量化（AbsMax Quantization）**，需要計算最小值與間距的方法則被稱為**最大最小值量化（MinMax Quantization）**，兩者都是概念最單純的量化方法。

依照這個思維，其實也能用相同的方法達成 INT4 量化，但是誤差就更大了！所以一般 8-Bit 以下的量化並不是單純的線性量化而已。

補充堅果，頭好壯壯

像絕對最大值量化這類以零為中心點的量化也被稱為對稱量化（Symmetric Quantization），反之則為非對稱量化（Asymmetric Quantization）。

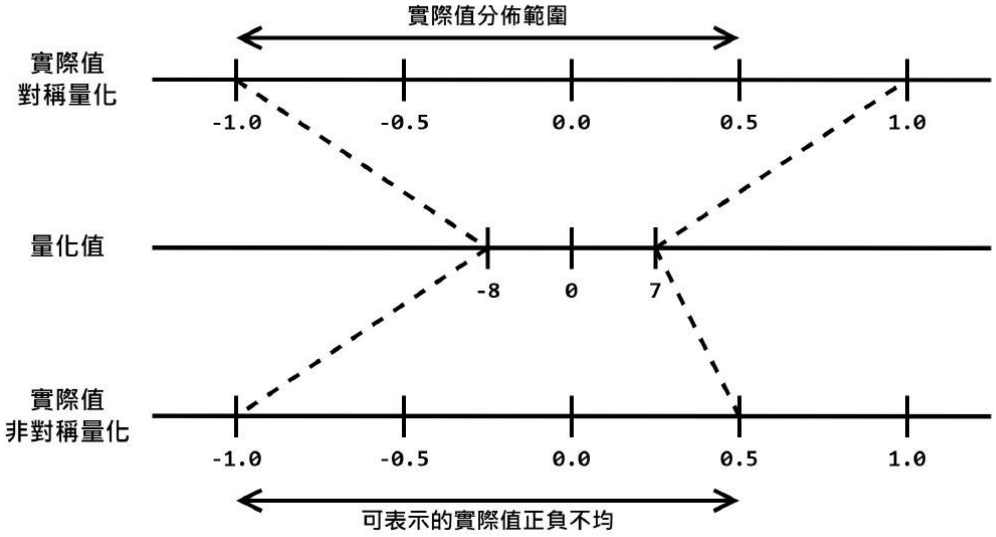


圖 14-1　對稱量化與非對稱量化的差異

這兩種量化各有優缺點，在不同情況下也有各自的最佳化場景。

透過 Python 簡單實做這種量化方法：

```
import numpy as np

def quantize_fp16_to_int8(fp16_weights):
    # 找到權重的絕對最大值
    abs_max_val = np.max(np.abs(fp16_weights))

    # 計算縮放係數
    scale = 127 / abs_max_val

    # 將 FP16 權重轉換為 INT8
    weights = fp16_weights * scale
    int8_weights = np.round(weights).astype(np.int8)

    return int8_weights, scale
```

```
def dequantize_int8_to_fp16(int8_weights, scale):
    # 將 INT8 權重轉換回 FP16
    return int8_weights.astype(np.float16) / scale
```

可以參考以下 Colab 程式碼觀察 INT8 的實際效果如何，這份 Colab 裡面同時也示範了如果直接將這個方法套用到 INT4 上誤差會有多大。

簡易量化範例程式碼

https://tinyurl.com/llm-note-09

14.2 量化方法

量化的方法主要分為**訓練時量化（Quantization-Aware Training, QAT）**與**訓練後量化（Post-Training Quantization, PTQ）**，訓練時量化需要在模型一開始進行訓練時，就把量化的誤差考慮進去，通常會透過 **Fake Quantize**[1] 來達成。但是 QAT 實在太慢了！因此多數用在語言模型上的量化方法都是訓練完成之後才進行量化的，此章節接下來要介紹的方法也都是 PTQ 的方法喔！

14.2.1 BitsAndBytes

BitsAndBytes[2] 簡稱 BNB，是由 **Tim Dettmers**[3] 開發的專案，在 HF Transformers 套件裡面做 8-Bit 或 4-Bit 量化的後端技術之一。其 8-Bit 的做法與上節所述大致

1 https://tinyurl.com/llm-fake-quant

2 https://github.com/TimDettmers/bitsandbytes

3 https://github.com/TimDettmers

相同，但是 BNB 更著重在**隔離異常值（Isolate Outlier）**上，因為作者觀察到如果將一些數值過大或過小的異常值也量化掉，會造成很大的誤差，但如果將少量的異常值隔離出來，維持他們原本的精度，便能大幅減少誤差值！

首先安裝套件：

```
pip install bitsandbytes
```

透過 `BitsAndBytesConfig` 來將模型轉成 BNB 8-Bit 的格式：

```
from transformers import BitsAndBytesConfig
from transformers import AutoModelForCausalLM

bnb_config = BitsAndBytesConfig(load_in_8bit=True)
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Meta-Llama-3-8B",
    quantization_config=bnb_config,
)
```

也可以將 8-Bit 模型透過 `model.save_pretrained` 存下來，這樣下次再讀取就會直接是 8-Bit 的型態，是個節省硬碟空間的手段。但是通常不太建議這樣做，因為變成 8-Bit 格式之後，就不能再轉回 16-Bit 或者降成 4-Bit 之類的，而且也會強制讀進 GPU 而不能在 CPU 裡使用，因此保留原始權重為 16-Bit 的格式是比較彈性的選擇。

除了 8-Bit 以外，BNB 也有提供 4-Bit 格式。BNB 4-Bit 所使用的資料型態為 Normal Float 4（NF4），是 **QLoRA**[4] 論文裡面提出的資料型態。要使用 BNB 4-Bit 的話，只需要把 `BitsAndBytesConfig` 裡的 `load_in_8bit` 改成 `load_in_4bit` 就可以了，除此之外還有一些其他參數可以設定：

4 https://arxiv.org/abs/2305.14314

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
)
```

被量化的模型在實際進行矩陣運算時，是會被反向量化回浮點數的，預設是用 FP32 進行運算，也可以透過 `bnb_4bit_compute_dtype` 參數將其設定為 FP16 會算的比較快。預設 4-Bit 的資料型態是一般的 FP4，將 `bnb_4bit_quant_type` 設定為 NF4 就可以使用論文裡面提到的資料型態。最後如果開啟 `bnb_4bit_use_double_quant` 可以進一步減少模型的大小。

將模型壓縮到 4-Bit 不僅能讓我們順利在單張 24 GB 消費級顯卡上使用 30B 的模型進行推論，也可以結合 LoRA 訓練來微調 30B 的模型喔！

14.2.2 Quanto

Quanto[5] 是由 Hugging Face 所開發的量化套件，支援 FP8、INT8、INT4、INT2 等量化層級。使用方法超級簡單，首先安裝套件：

```
pip install quanto
```

接下來透過 `QuantoConfig` 類別來對模型進行量化：

```
from transformers import LlamaForCausalLM
from transformers import QuantoConfig

# 可以是 int2, int4, int8, float8
quanto_config = QuantoConfig(weights="int4")
model = LlamaForCausalLM.from_pretrained(
    "meta-llama/Meta-Llama-3-8B-Instruct",
```

5 https://github.com/huggingface/optimum-quanto

```
    device_map="auto",
    quantization_config=quanto_config,
)
```

這樣就可以囉！使用 Quanto 的優點就是量化過程非常簡單，但是目前 Quanto 的運算速度並不是很快，除了 INT4 以外都會慢兩倍以上。

14.2.3 GPTQ

GPTQ[6] 是個相當受歡迎的量化技術。前面介紹的 BNB 量化雖然可以快速的將模型量化到低位元，但是其準確率與運算效率是個經常受到挑戰的部份。GPTQ 與 BNB 不同的地方在於，GPTQ 使用了校準資料集進行 **Post Training Optimization** 的方法，來對每個權重逐步進行量化。因為是使用實際資料進行校準，所以模型的失真度會相對較低一些，但也因此需要多花一些時間進行量化。

補充堅果，頭好壯壯

Post Training Optimization 到底在做什麼呢？當量化至極低的位元時，每個量化值所代表的實際值之間差距會非常大，可能你是 -4 我是 -3，但我們的實際值相差其實很大。因為模型的參數量很大，若每個數值都有這樣不小的誤差，累積起來就會發生很嚴重的錯誤。

若有個量化值在四捨五入前為 -3.5，那應該變成 -4 還是 -3 呢？因此這類的量化方法會透過實際的資料觀察**哪個四捨五入的方向可以產生最小的誤差**。

6 https://github.com/IST-DASLab/gptq

AutoGPTQ 是個能夠簡單進行 GPTQ 量化的套件，此套件已被整合在 Transformers 裡面，若要使用 GPTQ 量化，需要先安裝相關套件：

```
pip install optimum auto-gptq
```

透過 `GPTQConfig` 就可以對模型進行量化：

```
from transformers import GPTQConfig
from transformers import LlamaForCausalLM as ModelCls
from transformers import LlamaTokenizerFast as TkCls

model_id = "meta-llama/Meta-Llama-3-8B"
tk: TkCls = TkCls.from_pretrained(model_id)
gptq_config = GPTQConfig(
    bits=4,  # 可以是 2, 3, 4, 8
    tokenizer=tk,
    dataset="wikitext2",
    desc_act=False,
    # 提高 damp_percent 可以避免 torch._C._LinAlgError
    damp_percent=0.1,
    model_seqlen=2048,  # 影響 GPU 使用量
)

model: ModelCls = ModelCls.from_pretrained(
    model_id,
    device_map="cpu",  # 設定 CPU Offload
    low_cpu_mem_usage=True,
    quantization_config=gptq_config,
)

output_dir = "Models/Llama3-8B-Inst-GPTQ"
model.save_pretrained(output_dir)
tk.save_pretrained(output_dir)
```

參數 `bits` 為量化的位元大小，可以是 2、3、4 或 8 位元。因為 GPTQ 是逐層量化的演算法，所以可以設定 `device_map="cpu"`，在量化的過程中把模型權重都先放在 CPU 記憶體裡面，只有在某一層需要量化時，再拿出來放進 GPU 記憶體裡面做量化，完成量化後再放回 CPU 記憶體。如此一來，再大的模型都不怕了，只要確保其中一層權重能放的下，就能對整個模型進行量化囉！

使用 RTX 3090 將 Llama 3 8B 量化至 4-Bit，大約只需要花費 7 分鐘左右。GPTQ 格式的模型能直接存在硬碟裡面，原本 Llama 3 8B 佔用了 15 GB 的硬碟空間，進行 GPTQ 量化後縮小到了 5.4 GB 而已！

> **小心，樂高就在你腳邊**
>
> `model_seqlen` 會影響 GPU 記憶體的使用量，如果會發生 CUDA OOM 的話可以試著調低這個數字。如果會遇到 `torch._C._LinAlgError` 錯誤的話，可以嘗試提高 `damp_percent` 的值。若是會遇到 `RuntimeError: shape '...' is invalid for input of size ...` 的錯誤，則可以將 `desc_act` 設為 `True` 避免此問題。

也可以使用自己的文本進行 GPTQ 量化，方法相當簡單：

```
dataset = ["Hello, GPTQ!", "Goodbye, GPTQ!"]
gptq_config = GPTQConfig(bits=4, tokenizer=tk, dataset=dataset)
```

只要將文本列表傳進去就可以啦！建議使用與目標任務領域相近的文本，例如模型是個中文的泛用模型，那可能可以選擇中文維基當作文本。如果目標是氣象知識問答之類的，那可以用氣象相關的文本等等。在 GPTQ 原論文裡面，是從預訓練資料裡面，隨機挑選 128 筆固定長度的文本做校準。

> **小心，樂高就在你腳邊**
>
> 雖然 GPTQ 可以透過校準資料集提昇量化的精準度，但也因此產生了過度擬合於校準資料集的問題。所以慎選校準資料集，以及量化後的完整評估相當重要。

讀取已經做過 GPTQ 量化的模型時，不用再給一次 `GPTQConfig`，直接讀取即可：

```
from transformers import LlamaForCausalLM as ModelCls

model: ModelCls = ModelCls.from_pretrained(
    "Models/Llama3-8B-Inst-GPTQ",
    device_map="auto",
)
```

透過 `nvidia-smi` 觀察 GPU 記憶體，就會發現使用量大幅減少囉！如果你不想花時間自己進行 GPTQ 量化，可以選擇直接去 HF Hub 上面搜尋 GPTQ，就可以找到許多別人已經做完量化的權重。

14.2.4 AWQ

Activation-Aware Weight Quantization（AWQ）[7] 也是一種 Post Training Optimization 的量化方法，同樣也是透過少量的校準資料集來進行更精準的量化。相較於 GPTQ 而言，作者宣稱 AWQ 更不容易發生校準資料集的過度擬合問題，而且 AWQ 量化的模型在使用時可以有更快的推論速度。

作者指出，像是 GPTQ 這種透過**反向傳播（Backward）**來進行量化的方法，可能會使模型在校準資料集上發生過度擬合的情形。因此 AWQ 並不仰賴反向傳播，只透過**前向運算（Forward）**來觀察**輸出分佈（Activation Distribution）**並藉此尋找**重要的模型權重（Salient Weights）**。作者指出，只要將其中 1% 的重要權重保留為 FP16 的資料型態，就能大幅降低量化帶來的誤差。

但如果為了保護這些分散在不同地方的重要權重，而使得整份權重量化完後是混精度的，則會使計算效率降低。因此作者引入 **Activation-Aware Scaling** 的技巧來對每組權重做量化，使得重要權重能夠獲得保護的同時，也能夠避免混精度權重以提昇硬體友善度，降低運算瓶頸，在實際推論上可以有相當快的速度。

7 https://github.com/mit-han-lab/llm-awq

在社群上也有類似 AutoGPTQ 的 **AutoAWQ**[8] 這種方便的工具可用，AWQ 量化的過程是一層一層的量化，所以 AutoAWQ 可以將模型權重本身放在 CPU 裡面，只有在該層需要量化時，才把它的權重拿出來放在 GPU 記憶體裡面，因此記憶體的使用非常有效率，單張 RTX 3090 24 GB 就能量化 70B 規模的模型。

若要使用 AWQ 量化，首先需要安裝 AutoAWQ 套件：

```
pip install autoawq
```

安裝好 AutoAWQ 後，可以使用 `AutoAWQForCausalLM` 類別讀取原始模型：

```
import torch
from awq import AutoAWQForCausalLM

model_dir = "meta-llama/Meta-Llama-3-8B-Instruct"

model = AutoAWQForCausalLM.from_pretrained(
    model_dir,
    torch_dtype=torch.float16,
    device_map="cpu",  # 權重可以放在 CPU 就好
)
```

這裡將模型放在 CPU 記憶體裡面，避免模型權重佔用太多 GPU 記憶體導致 AutoAWQ 沒有空間進行量化運算。接下來會需要準備一份校準資料集，預設會使用原作者提供的 `mit-han-lab/pile-val-backup`[9] 這份資料集進行校準，但我們也可以自行準備一份字串列表當作資料集：

```
# 此為示範用的校準資料集，請根據自身應用替換成適當資料集
# 只要是 list[str] 的型態且 Token 數量夠多就能量化
calib_data = ["Hello, World! " * 8] * 16
```

8 https://github.com/casper-hansen/AutoAWQ

9 https://huggingface.co/datasets/mit-han-lab/pile-val-backup

小心，樂高就在你腳邊

如果給的文本太短太少的話，進行量化時就會發生`RuntimeError: torch.cat(): expected a non-empty list of Tensors`的錯誤。這是因為在套件裡面的`awq/utils/calib_data.py`，作者放了一個 Token 數量必須大於 512 才會拿來用的條件，可能是因為**原本 AWQ 的實做**[10] 就是如此的關係。

接下來放上相關設定，並開始進行量化：

```
from transformers import AutoTokenizer as TkCls

tokenizer = TkCls.from_pretrained(model_dir)

# 相關設定
quant_config = dict(
    zero_point=True,
    q_group_size=128,
    w_bit=4,
    version="GEMM",
)

# 進行量化
model.quantize(
    tokenizer,
    quant_config=quant_config,
    calib_data=calib_data,
)

# 儲存量化結果
output_dir = "Models/Llama3-8B-Inst-AWQ"
model.save_quantized(output_dir)
tokenizer.save_pretrained(output_dir)
```

10 https://github.com/casper-hansen/AutoAWQ/issues/191

筆者實測使用 RTX 3090 24 GB 量化一個 7B 模型，大約也是七八分鐘左右，速度上與 GPTQ 是差不多的，而且過程中所使用的 GPU 記憶體非常少量。如果 CPU 記憶體夠大，也能拿來量化 70B 的模型，大概要花費 100 分鐘左右。不過 70B 的模型即便量化到 4-Bit 也放不進一張 24 GB GPU 裡面就是了

這些量化完的模型，可以直接讀取進來使用：

```
from transformers import LlamaForCausalLM as ModelCls

model_dir = "Models/Llama3-8B-Inst-AWQ"
model: ModelCls = ModelCls.from_pretrained(
    model_dir,
    device_map="auto",
)  # 約 5.8 GiB
```

不需要任何額外的設定就能讀取，且 AWQ 模型在讀取速度上快滿多的。

14.3 困惑度

在量化的過程中，會相當重視模型的損失程度，也就是說量化完之後的誤差有多大，這在語言模型裡面通常以**困惑度（Perplexity, PPL）**來表示，PPL 是用來描述模型預測下一個字詞的時候其**不確定性**如何。

因為語言模型每次推論時會產生一份機率表，如果這個機率表顯示「下個字是 X 的機率為 100% 」那就代表模型非常**明確**，這時困惑度就會相對較低。但如果模型覺得「嗯…好像每個字都有可能 」那就代表模型相當**困惑**，這時困惑度就會較高。

一般語言模型都是以**交叉熵（Cross Entropy）**當作**損失函數（Loss Function）**，而困惑度就是對**損失值（Loss）**取**指數函數（Exponential Function）**的結果。那實際上到底要如何計算困惑度呢？首先，要先來瞭解如何取得單次推論時的 Loss：

```
model = AutoModelForCausalLM.from_pretrained(...)
tk = AutoTokenizer.from_pretrained(...)
input_ids = tk.encode(...)

with torch.no_grad():
    outputs = model.forward(input_ids=input_ids, labels=input_ids)
print(outputs.loss)
```

只需要多指定一個 `labels` 參數，並且代入原本的 `input_ids` 就能得到 Loss 了，接下來對 Loss 取指數函數就能得到困惑度：

```
ppl = torch.exp(outputs.loss)
```

最簡單的評測方法，是**固定序列長度**來計算困惑度，以 Wikitext 資料集為例，首先透過分詞器進行分詞：

```
from datasets import load_dataset

tk = AutoTokenizer.from_pretrained(...)
dataset = load_dataset(
    "wikitext",
    "wikitext-2-raw-v1",
    split="test",
)

input_ids = list()
for item in dataset:
    text = item["text"] + "\n"
    tokens = tk.encode(text, add_special_tokens=False)
    input_ids.extend(tokens)
```

這裡先設定序列長度為 4096 來進行評估：

```
import torch

seqlen = 4096
data_size = len(input_ids) // seqlen  # 計算序列數量
input_ids = input_ids[: data_size * seqlen]  # 捨棄最後一筆
```

其中長度不足 4096 的最後一筆資料會被捨棄，然後將 `input_ids` 轉為 `Tensor`，並在每個序列的開頭加上 BOS Token，最後把 `Tensor` 移動到對應的裝置上：

```
input_ids = torch.LongTensor(input_ids).view(data_size, seqlen)
bos_token = torch.full(
    (data_size, 1),
    tk.bos_token_id,
    dtype=torch.int64,
)
input_ids = torch.concat((bos_token, input_ids), dim=1)
input_ids = input_ids.to(model.device)
```

雖然這裡長度設定為 4096，但是因為加上額外的一個 BOS Token，所以模型的實際推論長度是 4097，不過計算 Loss 時會位移一格，因此最終得到的困惑度依然是以序列長度 4096 算出來的。

接下來開始對測試資料進行推論：

```
nlls = list()
for i in range(data_size):
    batch = input_ids[i : i + 1]
    with torch.no_grad():
        outputs = model.forward(batch, labels=batch)
    nlls.append(outputs.loss)
ppl = torch.exp(torch.stack(nlls).mean())
print(ppl)
```

`nlls` 代表 Negative Log-Likelihood，與 Cross Entropy 基本上是等價的概念。因為評估的過程可能會花上一段時間，所以可以借助 `tqdm` 套件來顯示進度：

```
from tqdm import trange

batch_size = 1
nlls = list()
with trange(0, data_size, batch_size) as prog:
    for i in prog:
        batch = input_ids[i : i + batch_size]
```

```
        outputs = model.forward(batch, labels=batch)
        nlls.append(outputs.loss)
        ppl = torch.exp(torch.stack(nlls).mean())
        prog.desc = f"ppl: {ppl:.4f}"
```

筆者實測 Llama 3 8B Instruct 的困惑度在序列長度 4K 時為 7.7494，而序列長度 2K 時則為 8.3272，由此可見序列長度對困惑度也有些微的影響。一般而言測試的序列長度在模型的訓練長度之內時，序列越長困惑度越低，但如果序列長度超過訓練長度太多時，困惑度就會開始無情暴漲。

補充堅果，頭好壯壯

固定長度的評估方法雖然簡單方便，但可能會有**文本邊界不自然**的問題，因此可以改成採用非固定長度的文本進行評估，必須特別注意**填充**與**標籤**問題，有 Padding Token 的位置其 Label 應設為 `-100` 才會算出正確的 Loss。另外，因為無論文本長度為何，都只會算出一份 Loss，所以一些比較細微的評估可能會根據文本長度做加權。

有了困惑度之後，就可以用來比較量化前後的模型了！這裡拿 Llama 3 8B Instruct 簡單比較一下 Float16、BFloat16 與 BitsAndBytes 之間困惑度的差異：

類型	困惑度
Float16	7.7494
BFloat16	7.7570
BNB 8-Bit	7.7730
BNB 4-Bit	8.1899

可以看到 FP16 與 BF16 幾乎沒有差異，BNB 8-Bit 也與 FP16 十分相近，但 BNB 4-Bit 就稍微有點距離了，不過這個差距還算可以接受。根據筆者的經驗，通常困惑度的差距到達 1~2 點左右就會開始感受到實際輸出品質在下降了。

困惑度並不是個絕對性的指標，在同個模型內做比較是滿有參考性的，但如果是不同模型或者不同訓練資料時，用困惑度進行互相比較就需要更加謹慎一些。例如分詞器的不同，可能會造成同一份文本對不同模型而言有不同的長度，在困惑度的比較上也會因此產生差異。

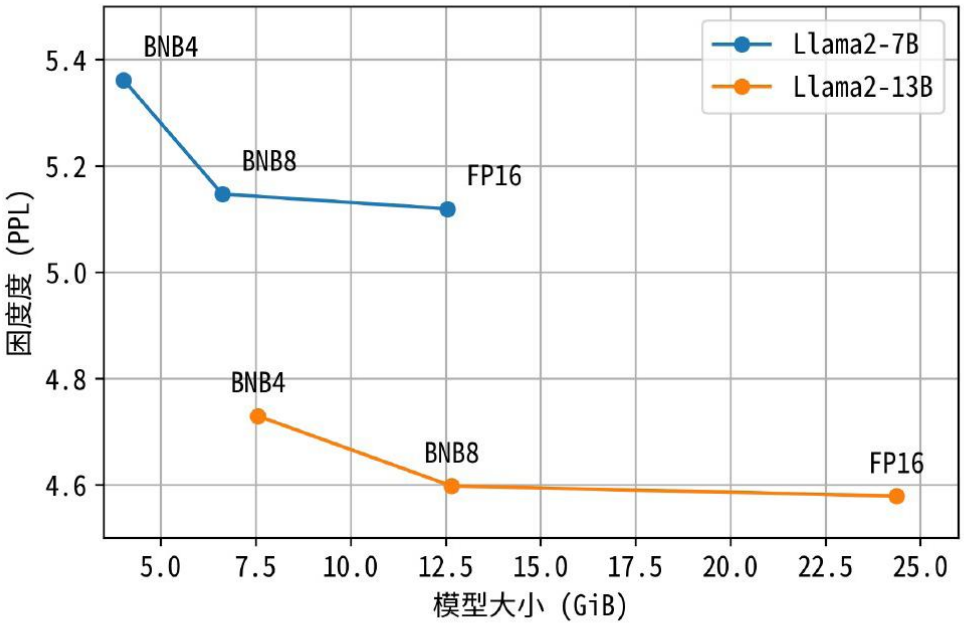


圖 14-2　困惑度與模型大小關係圖

在合理的量化下，模型的參數量通常會比模型的大小來的重要。就算用 16-Bit 去跑一個 7B 的模型，其效果也不會比 4-Bit 的 13B 模型還要好，即使 7B 16-Bit 的模型在實際大小上比 13B 4-Bit 還要大的多。因此模型的能力主要取決於參數量的多寡，而資料型態僅影響運算的精準度而已。

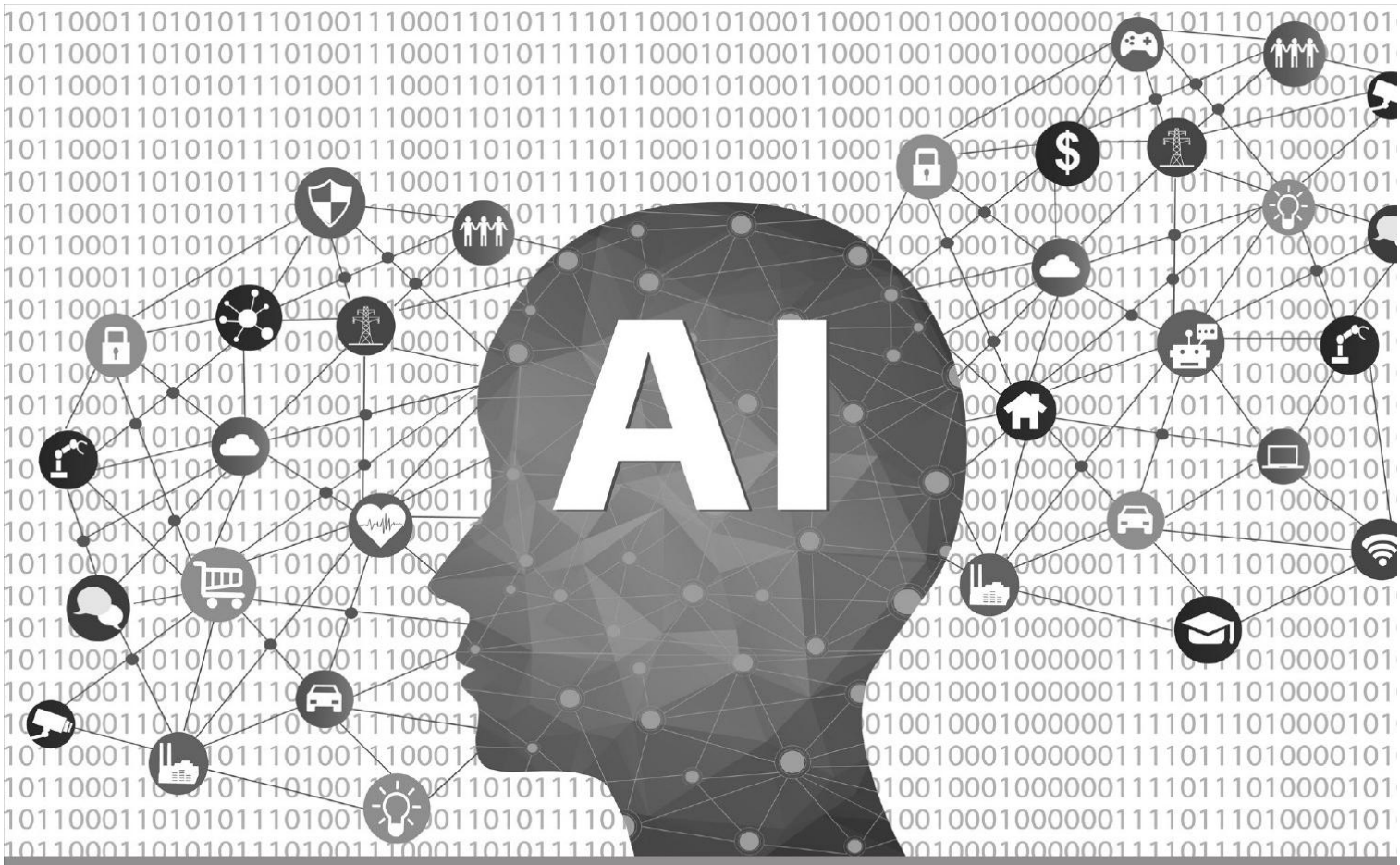
即便模型變小了，但模型的**總計算量**原則上是不變的，因此並不會因為模型從 16-Bit 變成 8-Bit 後，推論速度就快上兩倍。真正有影響的是每個權重的位元數縮小後，在運算上可以減少傳輸頻寬，使**吞吐量（Throughput）**提高，也就是每個**批次（Batch）**能容納的 Token 變多了。或者從軟硬體上針對特殊設計的資料型態進行加速運算，但是基本上計算量還是一樣的。

14.4 結論

本章節介紹了量化的基礎觀念，與一些常用的量化方法，這些量化技術大幅降低了遊玩語言模型的門檻，即便只有一張 24 GB 的 GPU 也能輕鬆玩轉 30B 規模的模型。關於更多量化的方法，推薦可以參考 HF Quantization[11] 的介紹指南。

雖然量化這門學問已經發展了一段時間，但依然是個快速成長中的技術。技術迭代的速度也是相當之快，量化框架層出不窮，對筆者這種貧窮玩家而言是一大利多

11 https://tinyurl.com/llm-hf-quant



Chapter

15

ggml & llama.cpp

主管 @tag 我

@penut 這個案子要用 C 來跑 LLM，而且是 On Windows 的

沒問題，包在 llama.cpp 身上！

ggml[1] 是由 **ggerganov**[2] 開發的一個機器學習框架，主打純 C 語言、輕量化且可以在 Apple 裝置上執行等功能。大概在 2022 年底的時候，就常常看到 ggml 登上 GitHub 趨勢榜。在 Meta 推出 LLaMA 之後，作者順勢製作了一個 **llama.cpp**[3] 專案，其高速讀取模型、低硬體需求、極低位元量化等功能深深吸引各大語言模型的開發者與使用者，並在開源社群上聲名大噪。本章節就帶各位讀者來探索 ggml 與 llama.cpp 的奧秘。

15.1 ggml

ggml 是個泛用型的機器學習框架，可以應用在很多模型上。從 ggml 的 GitHub 首頁可以看到除了 GPT-2 與 Llama 這些純文字的語言模型以外，甚至連 CLIP、Whisper 跟 Stable Diffusion 這些圖像語音相關的模型都有。缺點是 ggml 遇到新的模型架構時，必須自行實做沒看過的模組，通常必須等待作者或其他熱心人士投入開發。

1 https://github.com/ggerganov/ggml
2 https://github.com/ggerganov
3 https://github.com/ggerganov/llama.cpp

補充堅果，頭好壯壯

常見的 `.gguf` 檔就是 ggml 專案衍生出來的模型格式。

15.2 llama.cpp

llama.cpp 是作者用 ggml 框架嘗試實做 LLaMA 模型架構的專案，但因為 LLaMA 實在太紅了，不斷有不同架構的模型與各種先進的技術被貢獻進去，這份專案甚至開始喧賓奪主，很多 ggml 相關的改動都是從 llama.cpp 流過來的。也因此有很多 llama.cpp 已經有的功能，在 ggml 專案本身可能都還沒開始支援。

因為 llama.cpp 是針對特定模型架構運行的專案，因此並不是所有模型都能透過這個專案操作，但時至今日支援的模型架構已是相當廣泛，除非是非常新穎的架構，不然基本上都是有支援的。本章節以 llama.cpp 為主，介紹其環境建置與相關工具的使用。

工欲善其事，必先利其器

在開始之前，建議各位讀者可以先下載一個 **Llama 3 gguf 格式**[4] 的模型下來方便測試。如果只是想稍微試試可以下載最小的 Q2_K 版本，如果想要效果好一點，建議使用 Q8_0 版本。如果想使用中文的話，則可以考慮使用 **Breeze**[5] 模型。

4 https://huggingface.co/QuantFactory/Meta-Llama-3-8B-Instruct-GGUF

5 https://huggingface.co/YC-Chen/Breeze-7B-Instruct-v1_0-GGUF

15.3 環境建置

llama.cpp 支援使用 CUDA 在 NVIDIA GPU 上進行運算，因此需要準備一個有包含編譯器 `nvcc` 的環境，才能使用 GPU 進行運算，一般來說只要安裝 CUDA 就會有了。

> **小心，樂高就在你腳邊**
>
> CUDA 的版本可以參考 llama.cpp 專案底下 `.devops` 資料夾內與 CUDA 相關的 Dockerfile，目前是使用 CUDA 11.7.1 版。但需要注意自身 `gcc` 的版本與 `nvcc` 是否相容，例如筆者的 `gcc` 版本為 13，那 `nvcc` 就要裝到 12.4 才能用，同時也別忘了 GPU 驅動程式的支援版本喔！

除了 CUDA 以外還需要 CMake 編譯工具，以下示範使用 Conda 建立環境：

```
conda create -yn ggml python=3.11
conda activate ggml
conda install -y nvidia/label/cuda-11.7.1::cuda
pip install cmake
conda activate ggml  # 確保 cmake 路徑有被系統正確讀取
```

透過 Git 將 llama.cpp 專案的原始碼複製下來：

```
git clone https://github.com/ggerganov/llama.cpp --depth 1
cd llama.cpp
```

接著使用 CMake 進行建置：

```
cmake -B build -DGGML_CUDA=ON --fresh
```

使用 `-DGGML_CUDA=ON` 參數啟用 CUDA 函式庫，若只能使用 CPU 的話把這個選項移掉就好。筆者這邊加上 `--fresh` 來強制 CMake 重新建立編譯資訊。在 CMake 執行的過程，可以檢查以下訊息是否有出現：

```
-- Found CUDAToolkit: /home/user/.miniconda3/envs/ggml/include (found
   version "11.8.89")
-- cuBLAS found
-- The CUDA compiler identification is NVIDIA 11.8.89
```

如果沒有出現這些訊息的話，可能要確認一下 CUDA 環境是否設置正確，否則建置出來的程式會因為不能使用 GPU 而跑得很慢。最後進行完整建置：

```
cmake --build build --config Release -j
```

建置過程約需一兩分鐘左右，完成建置後，相關的主程式與工具都會放在 `llama.cpp/build/bin` 資料夾底下。

> **補充堅果，頭好壯壯**
>
> llama.cpp 也支援 Windows 及 macOS 作業系統，除了參考 GitHub 的說明自行編譯以外，也可以在 GitHub 的 Releases 頁面找到作者預先編譯好的版本。

15.4 格式轉換

可以用 llama.cpp 執行的模型文件格式被稱為 gguf 格式，先安裝相關套件：

```
pip install numpy sentencepiece
```

使用 llama.cpp 提供的 `convert_hf_to_gguf.py` 工具，將 Hugging Face 的模型轉換成 gguf 格式：

```
python convert_hf_to_gguf.py /path/to/llama3-8b-inst \
    --outfile llama3-8b-inst.gguf --vocab-type bpe
```

這樣就可以輕鬆做轉換了，而且速度相當快👍

另外可以設定 `--outtype` 參數決定資料型態，這個階段支援 `f32`、`f16`、`bf16` 與 `q8_0`，推薦使用 `f16` 會比較平衡一點。

> **小心，樂高就在你腳邊**
>
> 在對 Breeze 模型做轉換時，可能會遇到 `ValueError: Vocab size mismatch` 的問題，這是因為 Breeze 模型參數的字典維度與分詞器實際的字典數量不符合的關係，只要加上 `--pad-vocab` 參數來填充字典即可。

15.5 主程式 llama-cli

`llama-cli` 是整個專案最基本的用法，絕大多數的指令參數都放在這裡。首先是 `-m` 指定 gguf 格式的模型路徑，並使用 `-ngl` 指定 GPU 讀取層數，建議直接設定為 `-ngl 99` 就好，就算超過 GPU 記憶體上限，系統也會自動改放到 CPU 記憶體裡面：

```
./build/bin/llama-cli --help  # 顯示參數說明
./build/bin/llama-cli -m /path/to/model.gguf -ngl 99
```

在沒有其他參數的情況下，程式會快速讀取完模型，然後自己開始瘋狂輸出，這通常是用來測試運作的速度有多快。調用 GPU 時，要確認讀取訊息有出現類似以下的文字：

```
llm_load_tensors: offloading 32 repeating layers to GPU
llm_load_tensors: offloading non-repeating layers to GPU
llm_load_tensors: offloaded 33/33 layers to GPU
```

請確保最後一行寫的是 `33/33` 而不是 `30/33` 之類的，否則就代表有些模型參數沒有被放進 GPU 裡面，而是放在 CPU 裡面做運算，速度會因此大打折扣。

如果想要進入互動模式，可以加上 `-if` 參數，用起來大概像這樣：

```
<|begin_of_text|>
> hi
Hello! It seems like you forgot to provide any instructions. Please
provide the instructions you'd like me to follow, and I'll do my best to
assist you.
```

但這樣並不是 Llama 3 標準的聊天模式，可以透過參數 `-cnv` 來啟用聊天模型，並搭配參數 `-p` 來指定系統提示：

```
./build/bin/llama-cli -m /path/to/model.gguf -ngl 99 -cnv \
    -p "You are a helpful assistant. Reply in Traditional Chinese."
```

用起來大概會像這樣：

```
<|begin_of_text|><|im_start|>system
You are a helpful assistant. Reply in Traditional Chinese.<|im_end|>
> 你好！
你好！ 😊
<|im_end|><|eot_id|>

> 什麼是語言模型？
語言模型（Language Model）是一種算法，用於模擬人類語言的生成和理解能力。它能夠根據輸入的文字或語句，生成相對應的語句或回答，並且可以根據上下文進行理解和生成。
```

在非互動模式下，會用 `-n` 指定生成多少 Tokens，並搭配 `--ignore-eos` 來評測模型或硬體的速度。透過 `-c` 參數可以指定模型的輸入長度，預設只有 512，但目前的模型都支援到 4096 或 8192 以上了，所以如果有相關應用要消耗大量 Token 的話，這個參數記得一定要開大，不然模型的輸出會看起來很奇怪。在純 CPU 運算時，參數 `--threads` 對速度的影響滿大的，建議開系統總核心數的一半，例如你是 20 核的 CPU，那就指定 `--threads 10` 這樣。另外能透過 `-fa` 來啟用 Flash Attention 加速生成。

其他像是 `--top-k`、`--top-p`、`--temp`（也就是 Temperature）等都是老面孔的取樣參數了，除此之外 llama.cpp 還額外支援相當豐富的取樣參數，可以自行研究看看。

> **補充堅果，頭好壯壯**
>
> 其中把 `--temp` 設定為 `0.0` 並且把 `--repeat-penalty` 設定為 `1.0`，會最貼近在 HF Transformers 套件裡面設定 `do_sample=False` 的結果。

15.6 量化工具 llama-quantize

量化是 llama.cpp 最受歡迎的功能，llama.cpp 支援的量化方法相當廣泛，從 8-Bit、6-Bit 甚至到 2-Bit、1-Bit 都有，同時也支援混精度、線性、非線性等量化方法。基本的量化方法只要使用 `llama-quantize` 這個工具便可以進行操作：

```
./build/bin/llama-quantize \
    /path/to/model-fp16.gguf \
    /path/to/output.gguf Q4_K_M
```

以上指令會將模型量化為 4-Bit 格式，這裡的 `Q4_K_M` 代表「以 `Q4_K` 為基本型態的 Medium 大小」，意思是大部分的模型權重都是 4-Bit 的 `Q4_K` 型態，但

是有一部分的權重會用 6-Bit 的 `Q6_K` 型態，因此最後的**平均權重位元（Bits Per Weight, BPW）**其實會將近 5.0。另外也有 `Q4_K_S` 是混合 `Q4_K` 與 `Q5_K` 的格式，其 BPW 會更貼近 4.0 一些。

透過 `./build/bin/llama-quantize --help` 可以看到，還有許多 `IQ` 開頭的量化方法，例如極低位元數的 `IQ2_XXS` 與 `IQ1_S` 等，這些量化方法要如何使用呢？這時就需要透過另外一個程式 `llama-imatrix` 的協助。

`llama-imatrix` 指的是**重要性矩陣（Importance Matrix）**，這個程式會透過一份校準資料集來評估哪些模型權重是重要的，這些重要的權重需要保護起來，才能在極低位元的量化下維持模型的表現。在此之前，需要先蒐集一份純文字資料：

```
from datasets import load_dataset

ds_path = "bigscience-data/roots_zh-tw_wikipedia"
ds = load_dataset(ds_path, split="train", streaming=True)
text = [item["text"] for _, item in zip(range(128), ds)]
text = "\n".join(text)
with open("zh-wiki.txt", "wt", encoding="UTF-8") as fp:
    fp.write(text)
```

這裡使用中文維基當作範例，只需要取大概 128 筆資料即可，接著丟入 `llama-imatrix` 裡面：

```
./build/bin/llama-imatrix \
    -ngl 99 -c 8192 -f zh-wiki.txt \
    -m /path/to/model-fp16.gguf
```

大約兩三分鐘的時間就能跑完，並產生一份 `imatrix.dat` 檔案，將這份檔案用在量化工具上，便能產生極低位元的量化模型了：

```
./build/bin/llama-quantize \
    --imatrix imatrix.dat \
    /path/to/model-fp16.gguf \
    /path/to/output.gguf IQ2_XXS
```

以 Breeze-7B 實測，原本 FP16 為 14 GB，量化成 `Q4_K_M` 只剩下 4.3 GB，到了 `IQ2_XXS` 只剩下 2 GB，最極端的 `IQ1_S` 更是只剩下 1.7 GB ！但 7B 的模型量化到這個程度，其實已經接近無法使用的狀態了 😳

根據筆者的經驗，8-Bit 和 6-Bit 幾乎不會有損失的感覺，到了 4-Bit 以下才會有明顯感受到不同，而參數量越大的模型，受到量化的影響則越小。不過這是我個人的主觀感受，最後選擇使用多少位元的量化，還是要根據實際應用的狀況進行過充分的評測後再來決定。

15.7 伺服器 llama-server

`llama-server` 是筆者相當喜歡的一個程式，他可以將模型變成一個服務，使用者可以透過 HTTP API 來存取模型。參數與 `llama-cli` 大致相同，例如：

```
./build/bin/llama-server \
    -m /path/to/model.gguf \
    -ngl 99 -c 8192 -fa -cb -np 4 \
    --host 0.0.0.0 --port 8080
```

比較大的不同在於參數 `-cb` 與 `-np`，`-cb` 指的是 **Continuous Batching**，也就是說使用者的輸入會不斷加入批次裡面，而不需要等整個批次都結束了才能處理下個輸入。`-np` 則是指能夠同時處理的輸入數量，這裡設定 `-np 4` 就代表系統最多能同時處理四個輸入。

啟動之後可以在 `http://127.0.0.1:8080/` 打開網頁介面進行互動，這個介面只是用來簡單測試，一般開發通常還是以 API 呼叫居多：

```
curl -X POST http://localhost:8080/completion \
    -d '{"prompt": "你好！", "n_predict": 16}'
# Output: {"content":"今天我們要為大家介紹的是 ...
```

可以撰寫一個 Python 程式用串流的方式接收模型輸出：

```
import json
import requests

url = "http://127.0.0.1:8080/completion"
prompt = "[INST] 什麼是語言模型？ [/INST]"

params = {
    "prompt": prompt,
    "stream": True,
    "stop": ["\n", "\n\n"],
}

resp = requests.post(url, json=params, stream=True)
for chunk in resp.iter_lines():
    if not chunk:
        continue
    # 會有固定的 "data:" 前綴，需要跳掉 5 個字元
    content = json.loads(chunk[5:])["content"]
    print(end=content, flush=True)
print()
```

這裡透過 `stop` 參數就能指定模型輸出的停止點。

為了避免使用者輸入太長的提示，可以透過 `tokenize` 與 `detokenize` API 來截斷使用者的提示，例如：

```
url = "http://127.0.0.1:8080/tokenize"
params = {"content": "hello, llama.cpp!"}
resp = requests.post(url, json=params)
tokens = json.loads(resp.text)["tokens"]
print(tokens)  # [6312, 28709, 28725, ...]
```

假設我們只需要最後面三個 Tokens 的話：

```
url = "http://127.0.0.1:8080/detokenize"
params = {"tokens": tokens[-3:]}
resp = requests.post(url, json=params)
```

```
content = json.loads(resp.text)["content"]
print(content)  # .cpp!
```

這樣就完成了截斷提示長度的組合操作囉！

除了以上這些 API 以外，還有使用 LLM 做檢索時能透過 `/embedding` 取得文句向量，以及 Code LLM 常用的 `/infill` 程式碼填充，也有與 OpenAI API 相容的 `/v1/chat/completions` 可以使用。

15.8 Python Binding

llama-cpp-python[6] 是 llama.cpp 的 Python 介面，透過以下指令安裝：

```
CMAKE_ARGS="-DGGML_CUDA=ON" pip install llama-cpp-python
```

以下是個簡單的 Streaming 範例：

```
from llama_cpp import Llama

llm = Llama(
    model_path="/path/to/model.gguf",
    n_gpu_layers=99,
    verbose=True,
)

output = llm(
    "[INST] 什麼是語言模型？ [/INST]",
    max_tokens=128,
    stop=["\n", "\n\n"],
    stream=True,
)

for token in output:
    print(end=token["choices"][0]["text"], flush=True)
```

6 https://github.com/abetlen/llama-cpp-python

若想要隱藏原本 llama.cpp 的訊息紀錄，將 `verbose` 參數設定為 `False` 即可。用法原則上大同小異，詳細資訊可以參考官方文件[7]。

15.9 速度比較

這個章節要來比較一下 HF Transformers 與 llama.cpp 之間的推論速度，在這邊先附上筆者的硬體設備資訊：

- CPU - 12th Gen Intel(R) Core(TM) i7-12700K
- GPU - NVIDIA GeForce RTX 3090

測量速度的時候，主要考量以下因素：

- **預填充（Prefilling）**：初始輸入階段，觀察第一個 Token 生成的延遲。
- **解碼（Decoding）**：後續輸出階段，觀察 Token 與 Token 之間的延遲。
- **延遲（Latency）**：只考慮單筆處理的速度。
- **吞吐量（Throughput）**：考慮同時處理多筆的速度。

通常會用每秒可以處理多少 Tokens 來當作衡量速度的單位，可表示成單位 `tokens/s` 或簡寫為 `t/s`。以下使用的 HF Transformers 版本為 4.41.1，使用的 llama.cpp 版本為 `commit 9b8247`，評測模型皆為 Llama 3 8B，完整的評測程式可以參考連結：

速度評測完整程式碼
https://tinyurl.com/llm-note-11

7 https://llama-cpp-python.readthedocs.io/en/stable/

Prefill 設定在 4K 左右，而 Decode 設定為 128，首先在 Transformers 上測單筆推論的速度：

方法	Prefill (t/s)	Decode (t/s)
CPU	0.25	0.13
Eager	3200	41
SDPA	3600	42
FlashAttention2	3700	42

因為 CPU 實在太慢了，4 秒才處理 1 個 Token，所以我只測了 16 個 Tokens 就放棄了。整體上來說，GPU 上預填充速度約在 3200 t/s 到 3700 t/s 左右，解碼速度約 40 t/s。

接下來看看 llama.cpp 吧，因為是測單筆，所以不開 `-cb` 跟 `-np`：

方法	Prefill (t/s)	Decode (t/s)
CPU	2.57	1.27
GPU	4300	48
GPU + fa	4900	49

llama.cpp 的 CPU 速度比 HF 快了 10 倍之多！在 GPU 方面也都比 HF 快了不少。接下來測試看看不同量化方法的單筆速度：

方法	Prefill (t/s)	Decode (t/s)
IQ1_S	3800	144
IQ2_M	3600	129
Q2_K	3800	129
Q4_K_M	3800	125
Q8_0	3800	85
FP16	4900	49

可以看到有量化的模型在預填充方面明顯比 FP16 慢了些，但是生成解碼就快很多了！

最後簡單比較一下多筆推論的速度差異吧：

方法	Prefill (t/s)	Decode (t/s)
HF Batch Size 1	3800	42
HF Batch Size 2	4000	83
HF Batch Size 4	OOM	162
HF Batch Size 8	OOM	313
HF Batch Size 16	OOM	585
llama.cpp -np 4	4000	163
llama.cpp -np 8	3800	293
llama.cpp -np 16	4000	473

如果只看 Decode 這欄的話你可能會覺得「什麼，介紹了老半天的 llama.cpp 居然輸了嗎？」其實這裡的比較方式是不太公平的，在 HF Transformers 這邊，是預先湊好完整的批次讓模型去做推論（俗稱 Offline Inference），但 llama.cpp 這邊是分別接收 HTTP Requests 再去動態進行批次推論的（俗稱 Online Inference）。

因此算上網路傳輸及一些零零總總的小開銷，Online 會輸給 Offline 其實是滿合理的，但因為 HF Transformers 沒有 Online Inference 的機制所以也只能這樣比較。不過最大的問題在於 HF Transformers 推到批次大小為 4 的時候就超出記憶體上限了！因此實際上也沒辦法開到這麼大，在這情況下選擇 llama.cpp 會相對理想一點。

花生狂想曲 💡

llama.cpp 只做單筆推論時，Prefill 可以達到 4900 t/s，但是多筆推論時大概都只有 4000 t/s 出頭，可能是因為早期 llama.cpp 比較著重在單筆推論的開發上，而多筆推論的功能相對較新一點，也許未來的更新還會繼續提昇 llama.cpp 的速度，但現階段來說 llama.cpp 還不算是個擅長多筆推論的框架，主要還是偏重在單機運行上。

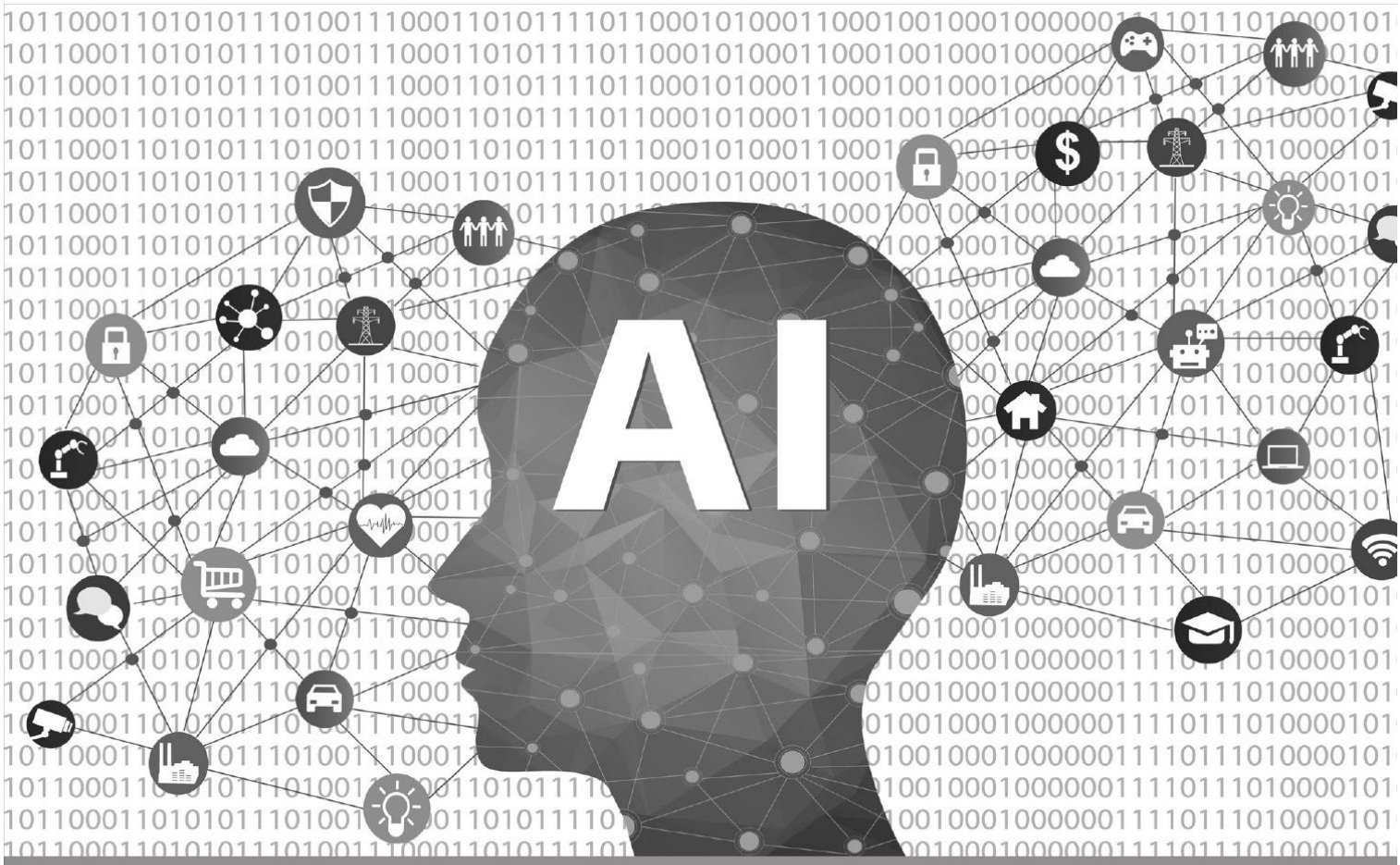
15.10 結論

本章節介紹了 ggml 框架與熱門的 llama.cpp 專案。雖然 llama.cpp 在推論速度上與 HF Transformers 似乎旗鼓相當，但是 llama.cpp 在 gguf 格式轉換、讀取速度與量化表現都相當優秀，是個相當適合部署在單機本地端上運行的框架！像是 LM Studio[8] 與 llamafile[9] 都是基於 llama.cpp 的專案，目標也都是為了在單機本地端上跑語言模型。

除了本章節介紹的程式以外，在 llama.cpp 裡面還提供了相當多工具可以使用，例如 `llama-perplexity` 可以用來計算模型困惑度，甚至還有能用來進行模型訓練的 `llama-finetune` 與 `llama-train-from-scratch` 等等。如果想要更深入瞭解 llama.cpp 的程式碼流程，推薦可以先看看 `llama-simple` 的程式碼。其他更多的說明可以到 `examples` 資料夾底下探索！

8 https://lmstudio.ai/

9 https://github.com/Mozilla-Ocho/llamafile



Chapter

16

Text Generation Inference

主管 @tag 我

@penut 要如何做 LLM 的 Serving 勒？

不用擔心，TGI 是我們的好夥伴！

Text Generation Inference[1] 簡稱 TGI，是由 Hugging Face 開發的語言模型推論框架。其中整合了相當多推論技術，例如 Flash Attention、Paged Attention、Continuous Batching 等等，同時也支援 BitsAndBytes、GPTQ、AWQ 等量化方法，加上 Hugging Face 團隊強大的開發能量與活躍的社群參與，使 TGI 成為部署語言模型服務的最佳選擇之一。

16.1 架設服務

TGI 提供本地安裝與 Docker Image 兩種用法，根據筆者的經驗，本地安裝要花上非常久的時間來編譯與安裝，而且也很容易遇到環境配置的問題，除非要客製化某些功能之類的，不然一律推薦使用 Docker Image 操作。

補充堅果，頭好壯壯

本章節主要使用 TGI 的 Online Serving 功能，真的需要做 Offline 推論的朋友，推薦使用 Docker Image 搭配 VS Code 的 Container Development 功能，就能在 Docker 裡面操作 Offline 的 TGI 囉！但因為 TGI 主要面向 Online Serving 的關係，Offline 的介面並沒有很好用，相關的說明文件也偏少。

1 https://github.com/huggingface/text-generation-inference

可以從官方 **GitHub Packages**[2] 查看 TGI 的 Docker Image 有哪些版本，或者直接使用最新版本 `latest`：

```
docker pull ghcr.io/huggingface/text-generation-inference:latest
```

筆者目前使用的是 `sha-f871f11` 版，因為這個 Docker Image 大小約 10 GB 左右，所以下載需要花上一段時間。完成下載之後可以透過 `--help` 查看參數說明：

```
docker run --rm \
    ghcr.io/huggingface/text-generation-inference:latest --help
```

可以先用 GPT-2 簡單測試一下：

```
docker run --gpus all --shm-size 1g \
    -p 8080:80 -v $PWD/data:/data \
    ghcr.io/huggingface/text-generation-inference \
    --model-id openai-community/gpt2
```

等到訊息紀錄出現 `Connected` 就代表服務啟動完成，因為 TGI 是用 FastAPI 建構的，所以能在 `http://localhost:8080/docs/` 底下查看相關的 API Endpoint 與參數。接下來透過 `curl` 做測試：

```
curl -X POST 127.0.0.1:8080/generate \
    -d '{"inputs":"Hello, "}' \
    -H 'Content-Type: application/json'
```

得到類似以下的輸出：

```
{"generated_text": "I'm a new player and ..."}
```

2 https://tinyurl.com/llm-tgi-pkgs

TGI 會透過網路下載模型，並且把模型轉換成 Safetensors 的格式，然後將模型權重存在容器內部的 `/data` 路徑裡面，所以使用參數 `-v $PWD/data:/data` 將此路徑對應出來，這樣就不用每次執行的時候，都要重新下載一遍模型權重。反過來說，也可以先將模型下載好，然後再掛載進容器裡面，例如：

```
MODEL_PATH="./models/opt-125m"

git clone https://huggingface.co/facebook/opt-125m $MODEL_PATH

docker run --gpus all --shm-size 1g \
    -p 8080:80 -v $PWD/$MODEL_PATH:/$MODEL_PATH \
    ghcr.io/huggingface/text-generation-inference \
    --model-id /$MODEL_PATH
```

16.2 基本用法

TGI 的 API 詳細用法可以參考官方文件[3]，與 Transformers 一樣有 `do_sample`、`temperature`、`top_k`、`top_p` 的取樣參數可以使用。除此之外，將 `decoder_input_details` 和 `details` 設為 `true` 會回傳一些詳細資訊，例如輸入或輸出的 Token 總數。給 `stop` 參數一個字串列表可以控制輸出停止點。

TGI 最方便的是可以設定 `truncate` 參數，讓系統幫你把太長的提示截斷，例如設定為 500 的話，就會把提示前面超過 500 個 Tokens 的內容全部切掉，只留最後面的 500 Tokens 當輸入。

3 https://huggingface.github.io/text-generation-inference/

補充堅果，頭好壯壯

這在限制輸入長度很好用，例如做 Retrieval-Based In-Context Learning 時，可以在檢索階段取非常多結果出來，把相似度較低的擺在前面，藉由 `truncate` 參數自然的把相似度太低的範例給切掉，這樣就能在維持輸入長度不會超出系統限制的同時，盡可能的加上更多範例在提示裡面。

最後透過 Python 呼叫生成 API 的程式碼大致如下：

```
import json
import requests

url = "http://localhost:8080/generate"
params = {
    "inputs": "This is a long story maybe, ",
    "parameters": {
        "best_of": 1,
        "details": True,
        "return_full_text": True,
        "decoder_input_details": True,
        "truncate": 4,  # 只保留最後四個 Tokens
        "max_new_tokens": 128,
        "stop": ["\n", "."],
        "do_sample": True,
        "temperature": 0.5,
        "top_k": 10,
        "top_p": 0.95,
    },
}

resp = requests.post(url, json=params)
result = json.loads(resp.content)
print(result)
```

這裡設定 `truncate=4` 來觀察實際效果，因為有打開 `return_full_text` 的選項，所以回傳結果會包含提示與生成，雖然結果裡面看起來提示並沒有被截斷，但若是仔細觀察 `details` 裡面的 `prefill` 就可以看到實際的提示其實是從 `"maybe"` 開始的。

除了基本的 `/generate` 以外，TGI 也有提供 `/v1/chat/completions` 這類與 OpenAI API 相容的用法。

16.3 量化選項

TGI 支援的量化方法相當多元，可以透過 `--quantize` 來指定要使用哪一種，目前 TGI 支援 BitsAndBytes 8-Bit/4-Bit、GPTQ、AWQ、EETQ、FP8 等量化方法：

- `bitsandbytes`：8-Bit 量化，雖然推論速度偏慢，但支援相對廣泛與穩定。
- `bitsandbytes-nf4`：4-Bit 量化，大部分的模型都可以直接使用此選項，資料型態為 Normal Float 4。
- `bitsandbytes-fp4`：4-Bit 量化，與 `bitsandbytes-nf4` 類似，但使用標準的 4-Bit 浮點數資料型態。
- `gptq`：4-Bit 量化，需要先進行 GPTQ 量化才能使用此選項。
- `awq`：4-Bit 量化，同樣也需要先進行 AWQ 量化才能使用的選項。
- `eetq`：8-Bit 量化，與 `bitsandbytes` 一樣是可以直接套用在一般模型上的量化選項，但是速度更快、準確率更高，缺點是啟動時間比較久。
- `fp8`：是 NVIDIA 最新支援的標準八位元浮點數資料型態，但是要 Hopper 架構以上的 GPU 才能跑，像是 H100 之類的。

有了量化，我們單顯卡平民就天下無敵了！先拿個 Taiwan LLM 13B 熱熱身：

```
docker run --gpus all --shm-size 1g \
    -p 8080:80 -v $PWD/data:/data \
    ghcr.io/huggingface/text-generation-inference \
    --model-id yentinglin/Taiwan-LLM-13B-v2.0-chat \
    --quantize bitsandbytes
```

順利跑起來之後，簡單拿個中文測試：

```
curl -X POST 127.0.0.1:8080/generate \
    -d '{"inputs":"USER: 什麼是語言模型？</s>ASSISTANT: "}' \
    -H 'Content-Type: application/json'
# Output: {"generated_text": " 語言模型是一種 ..."}
```

好欸，我們順利在一張 24 GB 的顯卡上運行了一個 13B 的模型啦！

補充堅果，頭好壯壯

除了一般的 Decoder 語言模型以外，像是 T5 或 M2M 之類的 Encoder-Decoder 架構也可以用透過 TGI 來運行。但如果不是 TGI 特別支援的模型，可能會回到原本 HF Transformers 的實做。一些像是 Tensor-Parallel 或 Flash Attention 的機制就無法用到，但依然有 Continuous Batching 和 Streaming Outputs 這些功能可以用。因此若要部署 Seq2Seq 模型可以考慮使用 TGI 來架設服務。

16.4 Token 數量設定

若要更精細的部署模型，則 Token 相關的參數至關重要。對於小顯卡而言，可以減少記憶體消耗，對於大顯卡而言，更能充分利用記憶體空間。最主要的三個參數如下：

- `--max-input-length` 代表單筆輸入的最大長度。
- `--max-total-tokens` 單筆輸入加輸出的最大總長度。
- `--max-batch-prefill-tokens` 代表所有批次加起來的最大輸入長度。

其中 `--max-batch-prefill-tokens` 是影響記憶體消耗最關鍵的參數，不僅影響 Prefill 階段能夠容納多少 Token 當輸入，也會影響實際運作時能同時處理幾個需求。

決定這些參數值的方向大概分成**輸入長度**、**輸出長度**與**批次大小**。在不同情境下，這些設定會不太一樣。以 Llama 2 13B 為例，此模型是以 4K Context Window 進行訓練的，那可以分配 3K 給輸入，剩下 1K 給輸出，因此得到：

1. `--max-input-length` 為 `3000`
2. `--max-total-tokens` 為 `3000 + 1000 = 4000`

> **補充堅果，頭好壯壯**
> 一般而言 Context Length 所謂的 2K 或 4K 通常是以 1024 為單位，所以 3K/1K 的分配實際上會是 **3 * 1024 = 3072** 跟 **1024** 個 Tokens。

接下來，假設系統**要同時處理 4 筆輸入**，則可以設定參數：

3. `--max-batch-prefill-tokens` 為 `3000 * 4 = 12000`
4. `--max-concurrent-requests` 代表佇列中最多等待處理的需求是幾筆，預設為 128 筆。

在這樣的設定下，如果使用者每筆輸入都是 3000 個 Tokens，那 TGI 最多只會同時推論 4 個請求，剩下請求的都會被放在佇列裡面，等待任何一個生成結束之後，再從佇列拿一個需求出來放進批次裡面繼續生成。

另外可以將 `--max-best-of` 設為 `1` 就好，因為一般應用通常只需要生成一筆結果。最後使用 `bitsandbytes-nf4` 做 4-Bit 量化，整個指令看起來會像這樣：

```
docker run --gpus all --shm-size 1g \
    -p 8080:80 -v $PWD/data:/data \
    ghcr.io/huggingface/text-generation-inference:latest \
    --model-id meta-llama/Llama-2-13b-chat-hf \
    --quantize bitsandbytes-nf4 \
    --max-best-of 1 \
    --max-concurrent-requests 128 \
    --max-input-length 3000 \
    --max-total-tokens 4000 \
    --max-batch-prefill-tokens 12000
```

雖然設定批次大小為 4，不過 TGI 其實是動態決定當下要同時推論的數量，例如使用者每筆輸入都減為 1500 個 Tokens 的話，那可能就會變成同時推論 8 筆。因此在不會超出 GPU 記憶體的情況下，把參數 `--max-batch-prefill-tokens` 盡可能開大，便能提昇整體系統的吞吐量。

至於具體到底要調到多少才不會超出記憶體呢？根據硬體與模型架構的不同，各自都會有不同的上限，所以只能自己慢慢測試了。筆者習慣以 2K 為單位往上遞增做測試，如果 4K 不會爆就試 6K，如果 6K 不會爆就試 8K，依此類推。

16.5 TGI Client

TGI Client 是用來跟 TGI Server 交流的客戶端套件，因為在 TGI 的文件中沒有太大篇幅的介紹，所以相當容易被人遺忘。首先透過 `pip` 安裝 `text-generation` 套件：

```
pip install text-generation
```

基本使用方法如下：

```
from text_generation import Client

client = Client("http://127.0.0.1:8080", timeout=600)

resp = client.generate(
    "Hello, ",
    max_new_tokens=16,
    stop_sequences=["."],
    do_sample=False,
    truncate=2048,
)

print(resp.generated_text)
```

其實就是將 `requests` 的用法做個包裝，並且幫回應的物件定義了明確的類別，因此輸入 `resp.` 的時候，會跳出 `generated_text` 的提示，開發的時候會更方便一點。透過這個套件進行串流輸出也比較方便：

```
resp = client.generate_stream(
    "Hello",
    max_new_tokens=16,
    stop_sequences=["."],
    do_sample=False,
    truncate=2048,
)

for chunk in resp:
    print(end=chunk.generated_text, flush=True)
print()
```

更多詳細的用法，可以參考官方在 PyPI 上的介紹頁面[4]。

4 https://pypi.org/project/text-generation/

16.6 速度測試

同樣使用上個章節的評測程式來測量 TGI 與 llama.cpp 之間的速度差異：

方法	Prefill (t/s)	Decode (t/s)
TGI-1T	4000	49
TGI-4T	4000	189
TGI-8T	4200	372
TGI-16T	4200	686
llama.cpp	4900	49
llama.cpp -np 4	4000	163
llama.cpp -np 8	3800	293
llama.cpp -np 16	4000	473

1T、4T 表示同時跑一筆或四筆，可以看到在單筆推論時，速度與 llama.cpp 差異並不大，但是當同時跑的筆數越來越多的時候，兩者的速度差距就越來越大了。

接下來測試看看 TGI 支援的各種量化方法：

	1T		16T	
	Prefill (t/s)	Decode (t/s)	Prefill (t/s)	Decode (t/s)
FP16	4000	49	4200	686
EETQ 8-Bit	4000	92	5000	292
BNB 8-Bit	4000	29	4100	259
BNB NF4	3700	80	4100	1000
GPTQ 4-Bit	5700	137	6500	1000
AWQ 4-Bit	2400	108	2300	1346
FP8				

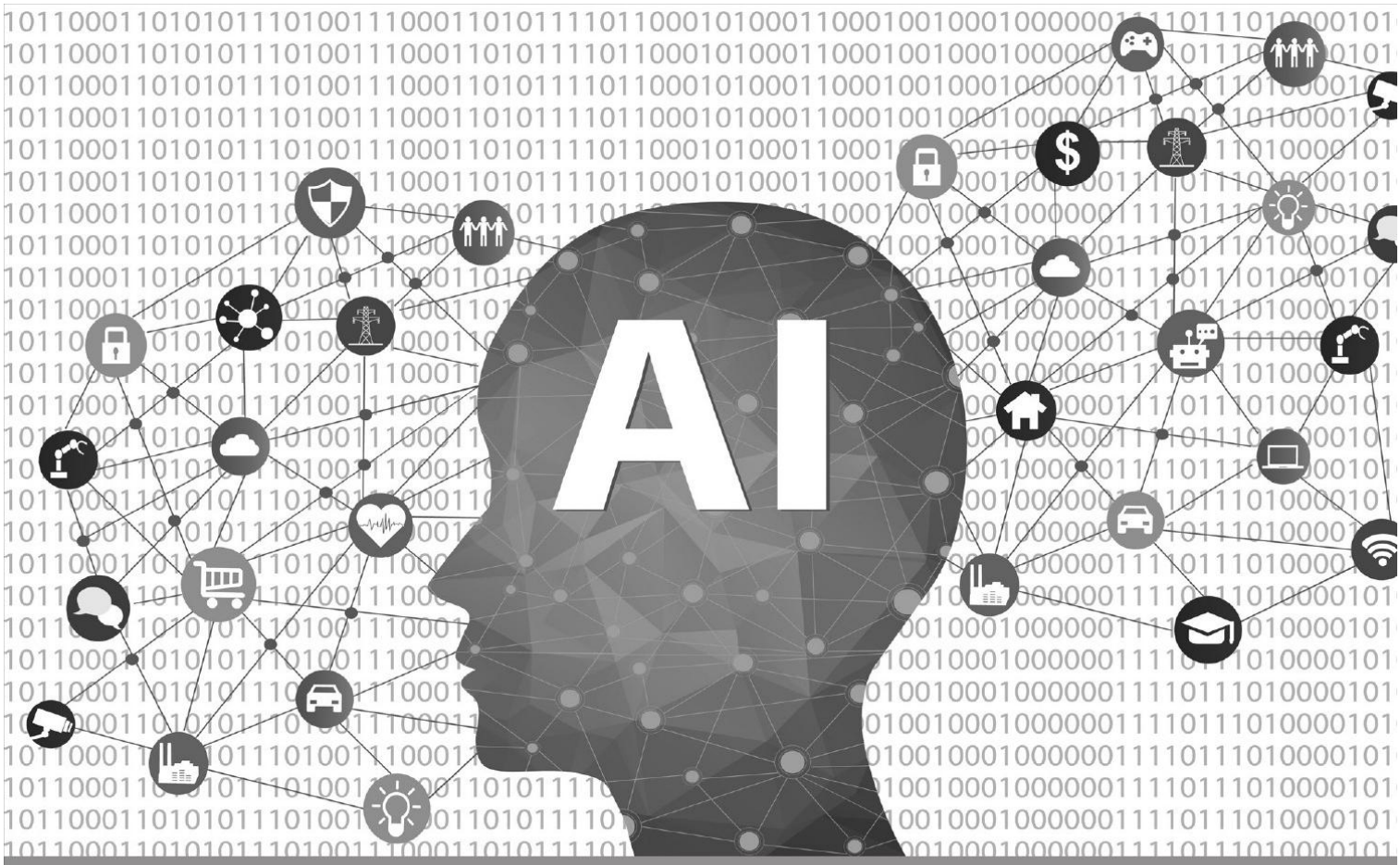
BitsAndBytes 的 Decode 速度真的滿慢的，相較之下 EETQ 的生成速度就快上許多，但 EETQ 的熱身時間較久是個小缺點。在單筆推論時，GPTQ 的 Decode 速度略勝 AWQ，但是多筆推論時，AWQ 的速度就贏過 GPTQ 了。因此若是經常需要輸入長文本的應用，採用 GPTQ 量化會比較好；但若是短問短答，追求生成速度，那選擇 AWQ 較佳。

花生狂想曲 💡

至於 FP8，等筆者出書後暢銷大賣賺大錢了，再來買台 H100 幫大家測試看看 💸

16.7 結論

本章節介紹了 TGI 推論框架，雖然在量化選項上較 llama.cpp 來的少，但是主流的量化方法都有支援。其推論的速度與穩定度，以及廣泛的模型架構支援，使它成為相當適合用在後端部署推論服務的方法。



Chapter

17

vLLM & Paged Attention

vLLM 是來自 UC Berkeley 的 Woosuk Kwon 和 Zhuohan Li 等人所製作的推論框架，使用 Paged Attention 技術實現相當驚人的 Token 吞吐量。讓我們一同來探索這個強大框架其背後的原理與用法吧！

17.1 Key-Values Cache

之前常常提到的 KV Cache 到底是什麼呢？在 Transformer Decoder 語言模型進行推論時，每一次的推論都會產生一組 KV Cache，這些 KV Cache 代表注意力運算底下**已經計算過的 Key 與 Values**，前面丟進模型算過的輸入，其 Key 與 Values 在後續的生成中不會改變，因此將其另外存放下來，在後續生成時可以重複利用減少計算量，因此會一直被保留在 GPU 記憶體裡面。

然而 KV Cache 累積下來所消耗的記憶體非常龐大，更糟糕的是如果要做批次推論，這個 KV Cache 佔用的記憶體就會翻倍，加上推論過程帶來的記憶體破碎問題，導致模型即便在高級硬體的環境底下，也難以同時進行大量文本生成。

因此對 LLM Service 而言，**最大的瓶頸是 GPU 記憶體**。

17.2 Paged Attention

Paged Attention 這個技術發想自作業系統的**分頁記憶體（Paged Memory）**，原本的 KV Cache 是很長很大的一坨記憶體，Paged Attention 將這坨 KV Cache 切成一個一個的**區塊（Blocks）**，並以此建立**區塊對照表（Block Table）**。對每個序列而言，看到的會是**邏輯區塊（Logical Blocks）**，模型可以透過區塊對照表來查詢 KV Cache 的**實體區塊（Physical Blocks）**放在哪裡。

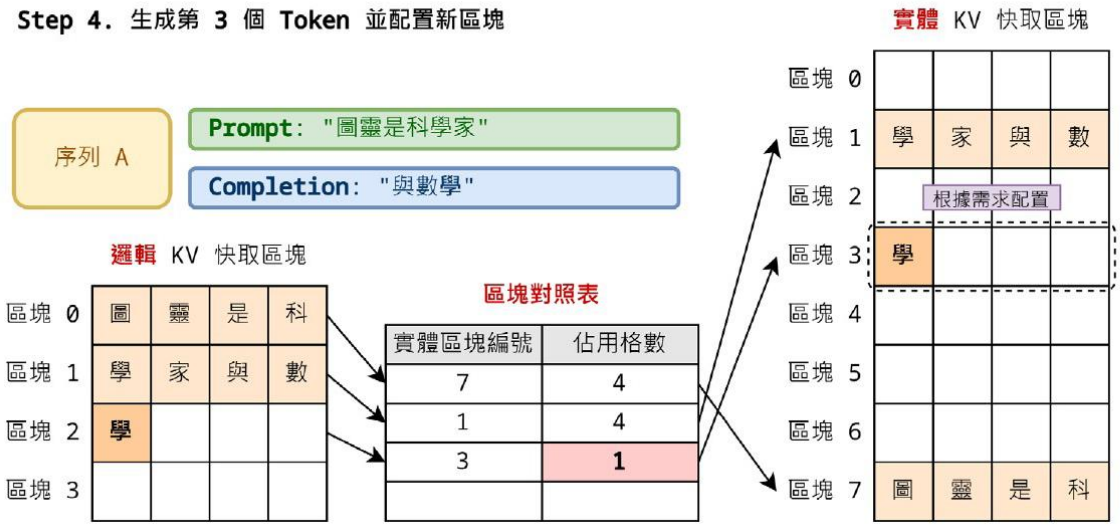


圖 17-1　vLLM 基本原理，完整動畫請參考此連結[1]

這樣做的好處在於，長序列的 KV Cache 不再需要使用很長一段連續記憶體來表示，因此可以被分散的存放在裝置上，解決記憶體破碎的問題。而且系統是透過查詢區塊表的方式來取得 KV Cache，所以很自然的也獲得了**共享記憶體**的能力！記憶體的使用效率大大提昇，減少了許多記憶體的浪費。因此可以說 **Paged Attention 透過查表法的方式解決記憶體瓶頸**。

1　https://tinyurl.com/llm-gif-vllm

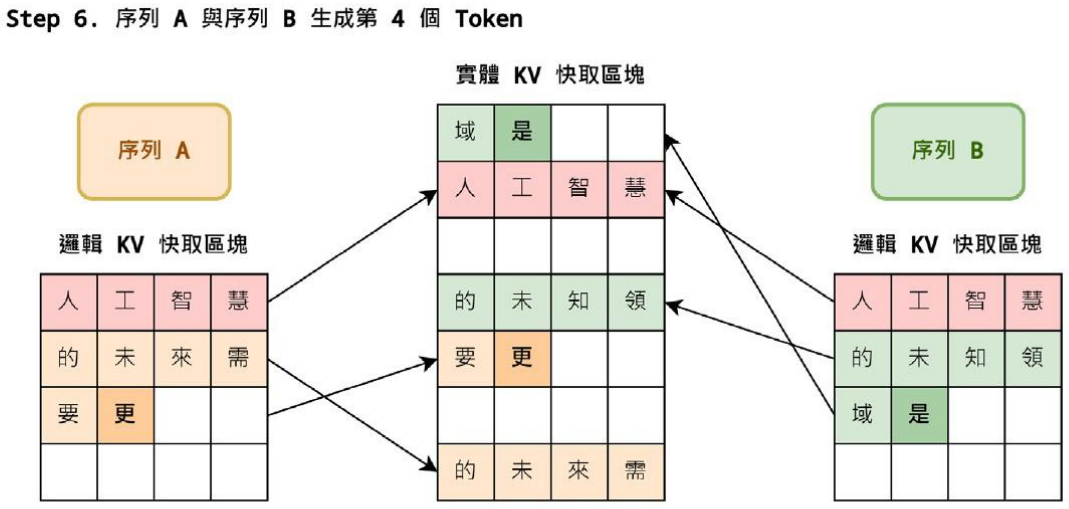


圖 17-2　兩個序列共享提示，完整動畫請參考此連結[2]

從上圖可見序列 A 與序列 B 的第一個區塊共享了同一塊記憶體。因此當兩個序列的某個部份是重複的時候，便能透過共享記憶體的方式減少記憶體用量。當兩個序列接下來的生成內容產生分歧時，便會透過**寫入時複製（Copy-on-Write）**的技巧來建立新的區塊以供後續使用。

官方部落格[3] 提供的 GIF 動畫解釋的相當詳細，但如果跟筆者一樣腦速緩慢的朋友，可以考慮用 **ezgif**[4] 將 GIF 分解成逐格影像慢慢看。

17.3　基本用法

首先安裝 vLLM 套件：

```
pip install vllm
```

2　https://tinyurl.com/llm-gif-vllm-shared
3　https://vllm.ai/
4　https://ezgif.com/split

基本的 Offline Inference 用法如下：

```
from vllm import LLM, SamplingParams

llm = LLM(model="meta-llama/Meta-Llama-3-8B")

# llm = LLM(
#     model="casperhansen/llama-3-8b-instruct-awq",
#     quantization="awq",
# )  # 使用 AWQ 量化

prompts = ["Hello, ", "Hi, ", "Goodbye, "]
sampling_params = SamplingParams(temperature=0.1)
outputs = llm.generate(prompts, sampling_params)

for output in outputs:
    prompt = output.prompt
    generate = output.outputs[0].text
    print(f"Prompt: {prompt!r}, Generate: {generate!r}")
```

用法相當簡單也相當直觀，多數的生成控制參數都放在 `SamplingParams` 裡面，與之前介紹過的取樣參數大同小異，可以參考**官方文件**[5] 的說明。

如果有很多提示需要處理，例如要對客戶的大量文件進行摘要或分析之類的，只要把提示全部放在一個字串列表裡面，vLLM 就會自動調整批次大小來快速處理，不用自己慢慢算 Token 數切批次，相當方便。

花生狂想曲

vLLM 支援的量化選項較少，僅有 AWQ、GPTQ、SqueezeLLM 與 Hopper 架構獨享的 FP8 等，大多都是需要進行 Post Training Optimization 的方法，像 BitsAndBytes 這類能直接量化的選擇比較少，也許未來會支援，但現階段來說是稍微麻煩一點。

5 https://tinyurl.com/llm-vllm-sp

雖然我們不能用 FP8 量化模型權重，但我們能用 vLLM 把 KV Cache 量化成 FP8 ！在初始化時加上 `kv_cache_dtype="fp8"` 參數就好了，使用 FP8 KV Cache 可以顯著減少記憶體的使用量，但可能會產生一些精度上的損失。如果使用的是更先進的硬體架構，可以從 FP8 上得到更多加速的效果，可惜筆者的 RTX 3090 沒有

補充堅果，頭好壯壯

NVIDIA 定義的 FP8 有兩種，分別是 E4M3 與 E5M2，代表他們指數與尾數的位元大小不同，E4M3 可以表達較精準的小數位數，但可以表示的最大與最小數值範圍較小，因此比較容易溢位，E5M2 則反之。

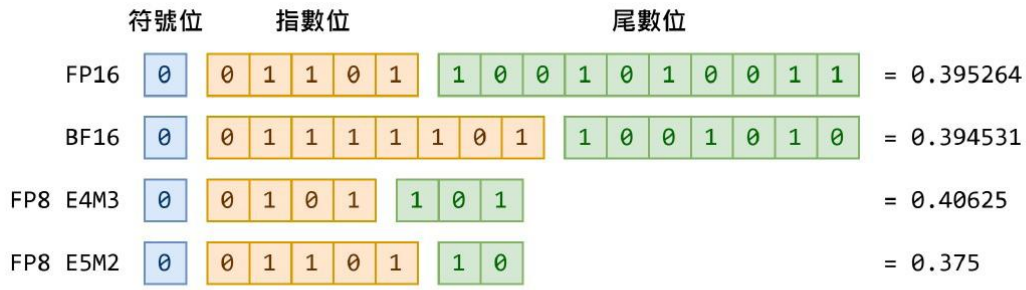


圖 17-3　浮點數格式比較

17.4 LLM Service

在 vLLM 裡面，整合了 **Continuous Batching**[6] 的機制，前一個輸入推論到一半時，可以中間插入一個新的輸入一起做推論。如果有多的輸入被放在佇列裡面，也會在任何序列完成時立刻加入生成的行列。

6　https://tinyurl.com/llm-cont-batch

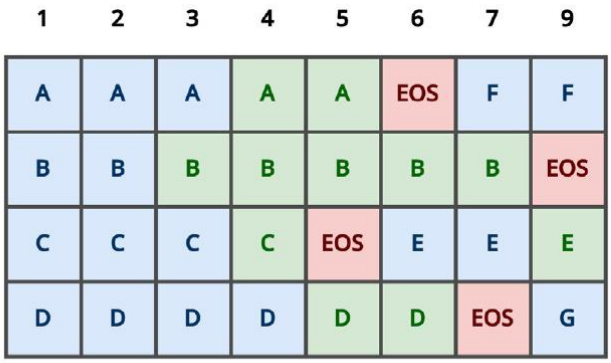


圖 17-4　Continuous Batching 示意圖

如圖所示，可以看到序列 C 比較早完成，所以序列 E 的輸入就跟著接在後面，整個批次便能繼續進行生成，減少序列 E 的等待時間。

這個機制內建在 vLLM 提供的 API Server 裡面：

```
python -m vllm.entrypoints.api_server \
    --model meta-llama/Meta-Llama-3-8B-Instruct
```

vLLM 不像 TGI 一樣有很多 Token 數量的設定，系統會自動根據當下的狀況來調整。把服務跑起來之後，可以 POST 到 `/generate` 進行生成，主要有 `prompt` 跟 `stream` 以及其他 `SamplingParams` 包含的參數，可以透過以下指令進行測試：

```
curl -X POST http://localhost:8000/generate \
    -d '{"prompt": "hello, "}'
# Output: {"text":["hello, 1 bathroom toilet ..."]}
```

使用串流接收 vLLM 的輸出稍微複雜一點，要使用 `b"\0"` 當作分隔符號：

```
params = {"prompt": "Hello, ", "stream": True, "max_tokens": 256}
resp = requests.post(url, json=params, stream=True)

prev = ""
for chunk in resp.iter_lines(delimiter=b"\0"):
    if not chunk:
```

```
        continue
    text: str = json.loads(chunk)["text"][0]

    # 已經顯示過的字串不用再顯示一次
    text = text.replace(prev, "")
    print(end=text, flush=True)
    prev += text
print()
```

vLLM 決定輸出停止點的方法也相當簡單，只需要給 `stop` 參數一份字串列表即可。最後加上一些取樣參數，完整的程式碼大致如下：

```
import json
import requests

params = {
    "prompt": "Hello, ",
    "max_tokens": 128,
    "temperature": 0.75,
    "top_k": 50,
    "top_p": 0.95,
    "stop": ["\n", "\n\n"],
    "stream": True,
}

url = "http://localhost:8000/generate"
resp = requests.post(url, json=params, stream=True)

prev = ""
for chunk in resp.iter_lines(delimiter=b"\0"):
    if not chunk:
        continue

    text: str = json.loads(chunk)["text"][0]
    text = text.replace(prev, "")
    print(end=text, flush=True)
    prev += text
print()
```

vLLM 同樣提供與 OpenAI API 相容的服務：

```
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Meta-Llama-3-8B-Instruct \
    --api-key auth-token-ouo123
```

17.5 速度比較

最後與 llama.cpp 和 TGI 來一個綜合大比較：

方法	1T		16T	
	Prefill (t/s)	Decode (t/s)	Prefill (t/s)	Decode (t/s)
llama.cpp	4900	49	4000	473
TGI	4000	49	4200	686
vLLM	4400	49	4600	650
vLLM FP8-KV	4400	49	4600	650

在單筆推論上，其實大家的速度都是差不多的！但來到多筆推論的狀況下，vLLM 能夠負荷的吞吐量又更大了一些，雖然 Decode 速度小輸 TGI，但都是非常快的速度了。而 FP8 KV Cache 在筆者的 RTX 3090 上與一般的 KV Cache 並沒有太顯著的差異。

17.6 結論

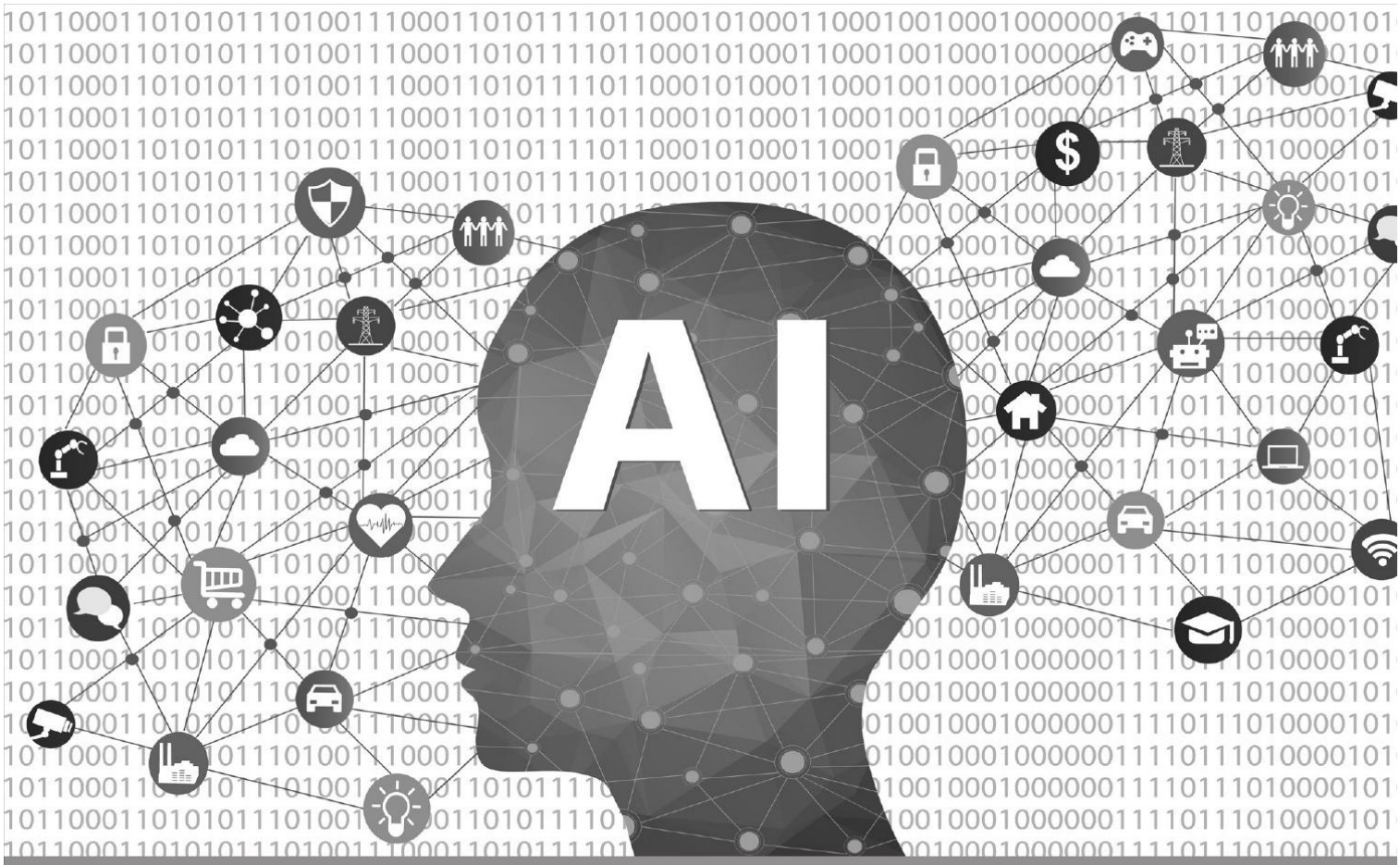
vLLM 是本書介紹的最後一個推論框架，所以來進行一個優缺點綜合大比較吧！

1. HF Transformers：
   - 優點：著重在訓練模型方面，模型支援度最廣泛。
   - 缺點：推論效率雖然已經進步許多，但難以克服記憶體用量問題。
2. llama.cpp：
   - 優點：量化選擇最多，模型讀取速度最快，單筆推論速度快，C & C++ 實做容易整合到其他作業系統與平台。
   - 缺點：專案編譯有些技術門檻，多筆推論同時進行的效率略差。
3. TGI：
   - 優點：支援主流量化方法，多筆推論速度快。
   - 缺點：需要手動調整 Token 設定。
4. vLLM：
   - 優點：離線多筆批次推論效率最佳，能自動調整 Token 數量設定，支援 FP8 KV Cache 減少記憶體用量。
   - 缺點：支援的量化方法偏少。

懶人包：

1. 單機單筆推論或者跨平台，選 llama.cpp。
2. 線上多筆推論，需要快速量化用 TGI，不用量化用 vLLM。
3. 離線大量多筆推論，選 vLLM。

希望以上資訊能幫助大家選擇合適的推論框架！



Chapter

18

情境學習 In-Context Learning

主管 @tag 我

@penut 我們要 Finetune 一下 LLM 來做某個 Task

在 Finetune 之前，可以先考慮 In-Context Learning ！

情境學習（**In-Context Learning, ICL**）是一種語言模型現象，模型可以根據提供的範例來產生預測的標記，而不需要任何額外的微調。這聽起來相當神奇，本章節就來探索這個神秘的 ICL 能力吧！

18.1 起源

情境學習一詞源自 GPT-3[1] 的論文，當時研究人員發現語言模型訓練到後面，突然開始浮現一種能力：如果將有標記的範例按照一個**固定的格式**排好，並且把沒有標記的輸入放在提示的最後面，模型就會自己生成正確答案出來。這個格式甚至可以非常隨意，只需要前後一致即可。

雖然名稱帶有**學習**一詞，但他其實並不是一種訓練手法，本質上是一種被觀察到的**現象**。為何語言模型會產生情境學習能力呢？至今也沒有一個非常好的解釋，但因為情境學習的方便性與擴展性，現在已經被當成一種提示的技巧，廣泛應用於各個領域。

1 https://arxiv.org/pdf/2005.14165.pdf

花生狂想曲

情境學習的用法聽起來似乎與少量樣本提示（Few-Shot Prompting）很像！沒有錯，兩者的核心概念都是**提供範例**給語言模型，但情境學習通常更著重在**範例格式**上，而且也不會預設範例一定很少量。少量樣本提示通常只要有給範例就好，格式並不是主要關注的重點，有時範例裡面可能還會包含詳細的解釋，因此也能說少量樣本提示是較為廣義的情境學習。

18.2 基本

情境學習的初始概念相當簡單，只需要給模型一組固定格式的資料即可：

```
今天天氣真好 => Positive
今天天氣真差 => Negative

好煩喔又下雨 =>
```

將以上這段提示丟進 ChatGPT 做測試，無論是 GPT-3.5 或 GPT-4 都會回覆 `Negative`。甚至不需要告訴模型實際的任務與標籤的定義是什麼，模型就能很自然的推理出正確答案。在情境學習裡面，可以使用任意格式，只需要前後一致即可，例如：

Input: 這部電影真好看
Label: Positive

Input: 這個編劇也太瞎
Label: Negative

Input: 運鏡流暢氣氛佳
Label:

Positive

情境學習與少量樣本提示非常相似，但少量樣本提示更傾向於借助模型**遵循指令**的能力來解決問題，而情境學習通常只需要模型有**文字接龍**的能力就能發揮，也就是說即便模型沒有做過指令微調，也具有情境學習的能力！

這個能力使研究人員十分驚訝，彈性的格式與相當泛化的準確率，這不就代表語言模型不再需要任何額外的微調，就能夠快速適應到任何領域的任務上嗎！於是大家開始好奇，語言模型的情境學習能力究竟是從何而來？

花生狂想曲 💡

事實上目前還沒有一個很共識的定論，有人認為語言模型並不是真的從範例去學習如何預測，而是模型本來就有分類能力。也有人認為情境學習的提示本身就是一種相當小型的微調，是一種廣義的元學習（Meta Learning）。這部份的研究屬於**黑盒解釋（Blackbox Interpret）**的領域，有興趣的話可以找些相關的介紹來看看。

這些具有情境學習能力的模型，通常訓練資料都相當豐富，模型本身在其他任務上也具有相當泛化的能力。因此不是只有 GPT-3 這種等級的語言模型才具備情境學習的能力，其實像是 Llama 7B 乃至於 GPT-2 這些小模型也都存在情境學習的能力，差別只在於這個模型的能力是否顯著、泛化與足夠準確等等。

18.3 翻譯應用

除了簡單的情感分類以外，情境學習也能應用在翻譯任務上，雖然語言模型本身已經具備相當不錯的翻譯能力，但某些用詞可能不盡人意：

"I need a printer to print my code" 翻譯成中文

我需要一臺打印機來打印我的代碼

支語警察 .jpg

我們能透過情境學習讓翻譯的用詞更為精緻，例如：

printer => 印表機
print => 列印
code => 程式碼
I need a printer to print my code =>

我需要一台印表機來列印我的程式碼

除了這種文化用語以外，翻譯罕見詞或新詞時也特別好用，例如遊戲技能的翻譯，就經常有遊戲自己獨樹一格的翻譯方式，這時就很適合借助情境學習的能力來協助翻譯。

18.4 文字正規化應用

文字正規化（Text Normalization, TN）也是個相當適合套用情境學習的領域，例如一個阿拉伯數字經常有多種中文數字的念法：

```
1 => 一
2 => 二、兩
3 => 三
11 => 十一
20 => 二十
22 => 二十二、二二
200 => 兩百
201 => 兩百零一、二百零一、二零一
257 =>
```

二百五十七、兩百五十七、二五七

再也不用寫那些充滿複雜規則的程式來產生中文數字念法了！只需要列舉一些念法的例子，模型就能舉一反三。若是遇到客製化的需求，例如軍隊的數字念法或銀行轉帳用的大寫中文數字等，也只需要稍微修改例子即可。

18.5 結論

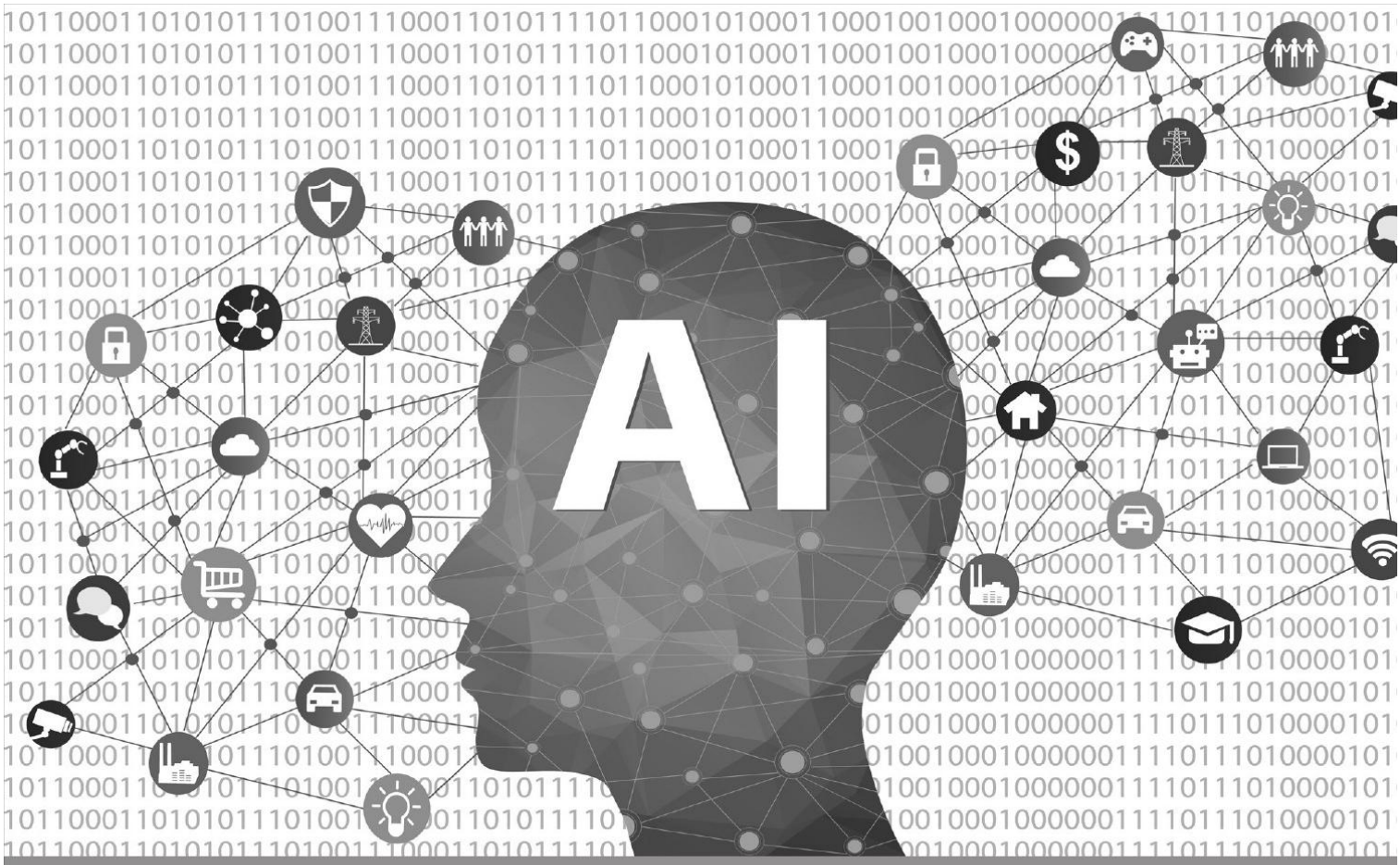
本章節介紹了情境學習的提示技巧，透過這個技巧就能輕鬆的將語言模型導入目標任務上。除了翻譯與文字正規化以外，還有相當多可以應用情境學習的場景。

使用情境學習最大的優點在於不需要對模型進行額外的微調，而且就算是小參數量的模型也具有情境學習的能力。相對於其他機器學習方法而言，**可解釋性**也相當高，通常只要觀察給定範例是否有錯，就可以知道為何模型會答錯。

花生狂想曲

然而事實上語言模型本身依然是高度不可解釋性的黑盒子，但基於語言模型的情境學習卻是具有可解釋性的？

其實就連語言模型為何會產生情境學習的能力都沒有很好的解釋，而情境學習所謂的解釋性高其實也只是指**容易偵錯**的部份而已。即便情境學習的能力很強大，但有時還是會遇到一些莫名錯掉的例子，唯一的解釋就是他沒有在範例裡面或範例不夠多。這到底算不算是一種可解釋性高呢？也許是這個領域還需要多加探索的部份。



Chapter

19

再訪檢索模型

主管 @tag 我

@penut 在 Local 端放個 Retrieval Model 可行嗎？

鐵定可行，而且輕鬆快速！

資訊檢索（Information Retrieval, IR）是在研究如何快速搜尋到使用者想要找的結果，例如大家天天在用的 Google Search 就是最經典的一種資訊檢索應用，相較於生成式語言模型而言，是個已經發展許久且相對成熟的領域。

想要對大量文檔建立快速的檢索引擎，首先需要**建立索引（Indexing）**，方法包含關鍵字提取、向量空間或 BM25 等等。接著根據使用者的查詢，尋找最相似的資訊並進行**排名（Ranking）**，這整個搜尋的模式被稱為**檢索模型（Retrieval Model）**。

將語言模型與資訊檢索結合，不僅能讓語言模型接觸到外部知識，改善語言模型無法更新資訊的問題，也能提昇回答的品質並減少錯誤與幻覺，若是進一步與情境學習結合，便能使模型進行更精準的任務。本章節就來介紹檢索相關的基本知識吧！

19.1 關鍵字

關鍵字（Keyword）是個簡單有效卻經常被忽略的做法，雖然中文的關鍵字經常會遇到斷詞與邊界的問題，但是在簡單的實做上，能以字元為單位的方式來處理。其中最常被用來實做關鍵字索引的方法就是 N-Gram 索引。

N-Gram 的概念很單純，一次以一個字或兩個字為單位的方式來建立索引，假設有兩份文本如下：

```
文本 A：今天天氣真好
文本 B：今天出門可好
```

若以 N-Gram 建立索引的話，會將文本切成以下片段：

```
今、天、天、氣、真、好、今天、天天、天氣、氣真、真好、今天天、...
今、天、出、門、可、好、今天、天出、出門、門可、可好、今天出、...
```

最後將這些索引與文本對應起來：

```
今：A, B
天：A, B
氣：A
門：B
今天：A, B
天天：A
天出：B
...
```

依此類推，雖然會切出很多不成詞彙的片段，但是在文本量較少的情況下，影響通常不大。若是將列舉 N-Gram 的邏輯寫成 Python 會是這樣：

```
def ngram(text: str, n: int):
    for i in range(0, len(text) - n + 1):
        yield text[i : i + n]


def all_ngram(text: str, a, b, step=1):
    a = max(1, a)
    b = min(len(text), b)
    for i in range(a, b + 1, step):
        for seg in ngram(text, i):
            yield seg
```

使用起來會像這樣：

```
# 從 2-Gram 開始，每次遞增 2，最大到 6-Gram
# 因此會有 2-Gram, 4-Gram, 6-Gram 的結果
segs = [seg for seg in all_ngram("今天天氣真好", 2, 6, 2)]
print(segs)
# ['今天', '天天', '天氣', '氣真', '真好', '今天天氣', '天天氣真', '天氣真好', '今天天氣真好']
```

使用者輸入的查詢可能會得出很多片段，這時可以使用 **Jaccard Similarity**[1] 做排序，也就是將彼此所有 N-Gram 的交集除以聯集：

```
def calc_jaccard(query: str, chunk: str):
    query_length = len(query)
    query_ngrams = {s for s in all_ngram(query, 1, query_length)}
    chunk_ngrams = {s for s in all_ngram(chunk, 1, query_length)}

    inter = query_ngrams & chunk_ngrams
    union = query_ngrams | chunk_ngrams

    return len(inter) / len(union)
```

透過這些技巧就能建立出完整的 N-Gram 搜尋引擎囉！

N-Gram Search 完整程式碼

https://tinyurl.com/llm-note-12

在應付少量文本時，使用 N-Gram 建立索引通常不會有太大的問題。但是當文本量逐漸上升時，索引本身所佔用的記憶體會爆炸性成長。加上這種索引只能做文字比對，並沒有辦法進行相近語意的比較，斷詞邊界也是個大問題，因此不是最理想的做法。

1 https://en.wikipedia.org/wiki/Jaccard_index

19.2 BM25 Best Matching Ranking

BM25（Best Matching 25）是一種基於 TF-IDF 的統計排名方法，雖然是個古老的演算法，但還是相當簡單有效。可以透過 Rank-BM25[2] 這個 Python 套件輕鬆使用這個經典演算法：

```
from rank_bm25 import BM25Okapi

corpus = ["這是第一篇文檔", "這是第二篇文檔", "文檔的內容很重要"]

# 以字元為單位斷詞
corpus_chars = [[ch for ch in doc] for doc in corpus]
bm25 = BM25Okapi(corpus_chars)

query = "重要"
query_chars = [ch for ch in query]

# 計算 BM25 分數
scores = bm25.get_scores(query_chars)
print(f"Scores: {scores}")

# 輸出查詢結果
print(f"User Query: {query}")
print(f"Best Result: {corpus[scores.argmax()]}")

"""
輸出：
Scores: [0.  0.  0.98149902]
User Query: 重要
Best Result: 文檔的內容很重要
"""
```

BM25 是一個排名演算法，可以與其他檢索方法做結合，例如先用 N-Gram 或 Embedding 進行比較大範圍的模糊搜尋，再用 BM25 做排序之類的。

2 https://github.com/dorianbrown/rank_bm25

19.3 文本向量

先前的章節已經介紹過 Embedding 是一種**將文字向量化**的結果，若 Embedding 以字詞為單位，則稱為**詞向量（Word Embedding）**，若以句子為單位則稱為**文句向量（Sentence Embedding）**，也有以文件為單位的**文件向量（Document Embedding）**，因此 Embedding 一詞的用法是很彈性的。

> **補充堅果，頭好壯壯**
> 單位較大的向量，例如一句話或一份文件，通常是由多個單位較小的向量合併計算出來的，例如 Transformer Encoder 會將一句話切成數個 Tokens 後得到 Token Embedding，經過數層 Transformer Blocks 的精密計算，加上 Pooling 運算得到最終的 Sentence Embedding。

一般來說，在檢索模型裡面使用的都是以句子或文件為單位，這樣就可以將不同長度的句子都編碼成固定維度的向量，透過一個簡單的矩陣運算便能快速計算相似度。

專門用來產生這種向量的模型有很多，但是有支援中文或多語的選擇相對較少。一個古老但是經典的選擇是 Google 的 **Universal Sentence Encoder**[3] 系列，其中的 **Multilingual Large**[4] 是筆者較推薦的版本，首先要安裝套件：

```
pip install tensorflow tensorflow_hub tensorflow_text
```

3 https://www.kaggle.com/models/google/universal-sentence-encoder/
4 https://tinyurl.com/llm-useml

基本使用方法如下：

```
import numpy as np
import tensorflow as tf
import tensorflow_hub as hub
import tensorflow_text

# 下載 & 讀取模型
hub_url = (
    "https://www.kaggle.com/models/google/"
    "universal-sentence-encoder/tensorFlow2/"
    "multilingual-large/2"
)

# 也可以把模型下載下來後，直接指定本地路徑讀取
model = hub.load(hub_url)

# 計算各自的 Embedding
#「고양이」跟「개」分別是韓文的貓和狗
text = ["cat", "貓", "고양이", "dog", "狗", "개"]
embeddings = model(text)

# 計算內積作為相似度
similarity = np.inner(embeddings, embeddings)
np.set_printoptions(precision=2)
print(similarity)
```

輸出結果如下：

```
[[1.   0.94 0.92 0.75 0.76 0.73]
 [0.94 1.   0.92 0.73 0.75 0.73]
 [0.92 0.92 1.   0.61 0.63 0.62]
 [0.75 0.73 0.61 1.   0.98 0.98]
 [0.76 0.75 0.63 0.98 1.   0.97]
 [0.73 0.73 0.62 0.98 0.97 1.  ]]
```

可以看到貓、고양이跟 cat 之間的內積較大，代表他們在語意上較為相近，但他們跟「狗」的內積就比較小，代表他們的語意比較不相近。

小心，樂高就在你腳邊

如果在 Tensorflow 與 PyTorch 混用的專案裡面，先讀取 Tensorflow 模型可能會造成兩個套件搶用 GPU 的問題，解決方法之一是先匯入 PyTorch 相關的套件，或者透過 `tf.config.set_visible_devices([], "GPU")` 來停用 Tensorflow 使用 GPU 進行運算，因為這個模型很小，所以在沒有 GPU 的情況下也可以算的很快。

另外要注意 `tensorflow_text` 雖然看起來沒有用到，但是如果沒有先匯入的話會發生 `RuntimeError` 喔！

第二個選擇是使用跟 Hugging Face 生態系較貼近的 **Sentence Transformers**[5] 模型，在 HF Hub 上可以找到很多**相關的模型**[6]，首先要安裝套件：

```
pip install sentence_transformers
```

基本用法如下：

```
import numpy as np
from sentence_transformers import SentenceTransformer

# 下載 & 讀取模型
model = SentenceTransformer("intfloat/multilingual-e5-base")

# 取得 Embedding
text = ["cat", "貓", "고양이", "dog", "狗", "개"]
embeddings = model.encode(text, normalize_embeddings=True)

# 計算內積作為相似度
similarity = np.inner(embeddings, embeddings)
np.set_printoptions(precision=2)
print(similarity)
```

5 https://www.sbert.net/

6 https://huggingface.co/sentence-transformers

除了以上範例使用的 E5 以外，還有 **BGE-M3**[7] 也是效果很好的模型。

19.4 Text Embedding Inference

看到這邊你可能會問：好像都是 Offline 的用法欸，這些 Embedding Model 可以做 Online Serving 嗎？沒問題，那個開源笑臉再次出手，推出了與 TGI 類似的 **Text Embedding Inference**[8] 服務。

TEI 一樣能透過 Docker 操作，但是 Image 區分的版本比較多一點，有分成 CPU、RTX 30 系列或 RTX 40 系列之類的，以筆者的 RTX 3090 來說要選擇 Ampere 86 的版本：

```
docker pull ghcr.io/huggingface/text-embeddings-inference:86-1.2
```

> **補充堅果，頭好壯壯**
>
> 雖然在 `docker run` 時不要給 `--gpus` 參數就能達到 CPU 運算的效果，但是純 CPU 版的 TEI Docker Image 會比較小，因此若要降低部署足跡可以考慮改用純 CPU 版。

TEI 的用法與 TGI 十分相似：

```
docker run --gpus all -p 8080:80 -v $PWD/data:/data \
    ghcr.io/huggingface/text-embeddings-inference:86-1.2 \
    --model-id BAAI/bge-m3
```

7 https://huggingface.co/BAAI/bge-m3

8 https://github.com/huggingface/text-embeddings-inference

透過以下指令進行測試：

```
curl -X POST http://127.0.0.1:8080/embed \
    -H 'Content-Type: application/json' \
    -d '{"inputs":["貓","狗","cat","dog"]}'

# Output: [[0.0021725853,0.03031409,...]]
```

那麼何時該使用 GPU 版，何時該使用 CPU 版呢？ GPU 的優點在於可以同時大量運算，因此在對大量文件建立索引時使用 GPU 會比較快。而實際部署時，因為使用者的查詢都是一筆一筆進來的，相對比較分散，而且向量模型通常不像語言模型有那麼多的參數量，因此計算量也不大，所以只要使用 CPU 版即可。

補充堅果，頭好壯壯

以上的檢索方法通常會被區分為兩大類：**稀疏檢索（Sparse Retrieval）**與**密集檢索（Dense Retrieval）**，像是 BM25 就屬於稀疏檢索，而文本向量則屬於密集檢索。

19.5 相似度

衡量向量**相似度（Similarity）**的方法除了內積以外，還有**餘弦相似度（Cosine Similarity）**以及**歐基里得距離（Euclidean Distance）**等方法。

歐基里得距離其實就是俗稱的**座標相減平方開根號**，也就是求兩點座標之間的直線距離公式，對應到高維度的向量來說，就是對每一個維度都計算相減平方開根號，然後再加總起來，可以在 `scikit-learn` 套件裡面操作歐基里得距離：

```
from sklearn.metrics.pairwise import euclidean_distances

dist = euclidean_distances(embeddings, embeddings)
```

餘弦相似度是在計算兩個向量之間的**夾角**，並對這個角度取餘弦。當兩個向量越貼近時，他們的夾角就會越小，餘弦值就會越大：

```
cos(0°) = 1, cos(90°) = 0, cos(180°) = -1
```

在 `scikit-learn` 裡面也有餘弦相似度的函數：

```
from sklearn.metrics.pairwise import cosine_distances
from sklearn.metrics.pairwise import cosine_similarity

dist = cosine_distances(embeddings, embeddings)
sim = cosine_similarity(embeddings, embeddings)
```

> **小心，樂高就在你腳邊**
>
> 別忘了**相似度（Similarity）**是越**大**代表越相近，而**距離（Distance）**則是越**短**代表越相近，千萬別搞反囉！

19.6 Faiss

Facebook AI Similarity Search（Faiss）[9] 是 Facebook 開發的向量搜尋套件，可以在大量高維度的向量裡面進行快速搜索，是個相當有效率的工具。先來安裝套件：

```
pip install faiss-cpu
```

9 https://github.com/facebookresearch/faiss

Faiss 也有支援 GPU 的版本，需要透過 Conda 進行安裝：

```
conda install -c conda-forge faiss-gpu
```

Faiss 的用法相當簡潔：

```
import faiss
import numpy as np

# 定義向量集合
dim = 3
value = [[1, 0, 0], [0, 1, 0], [0, 0, 1], [1, 0, 1], [0, 1, 1]]
value = np.array(value)

# 建立向量索引
index = faiss.IndexFlatL2(dim)
index.add(value)

# 開始查詢
top_k = 3
query = np.array([[1, 0, 0]])
dist, indices = index.search(query, top_k)

# 輸出距離與最近的向量
print(dist)
for i in indices[0]:
    print(value[i])
```

可以透過 `faiss.write_index` 把建立好的索引寫成檔案，再用 `faiss.read_index` 把索引讀取出來：

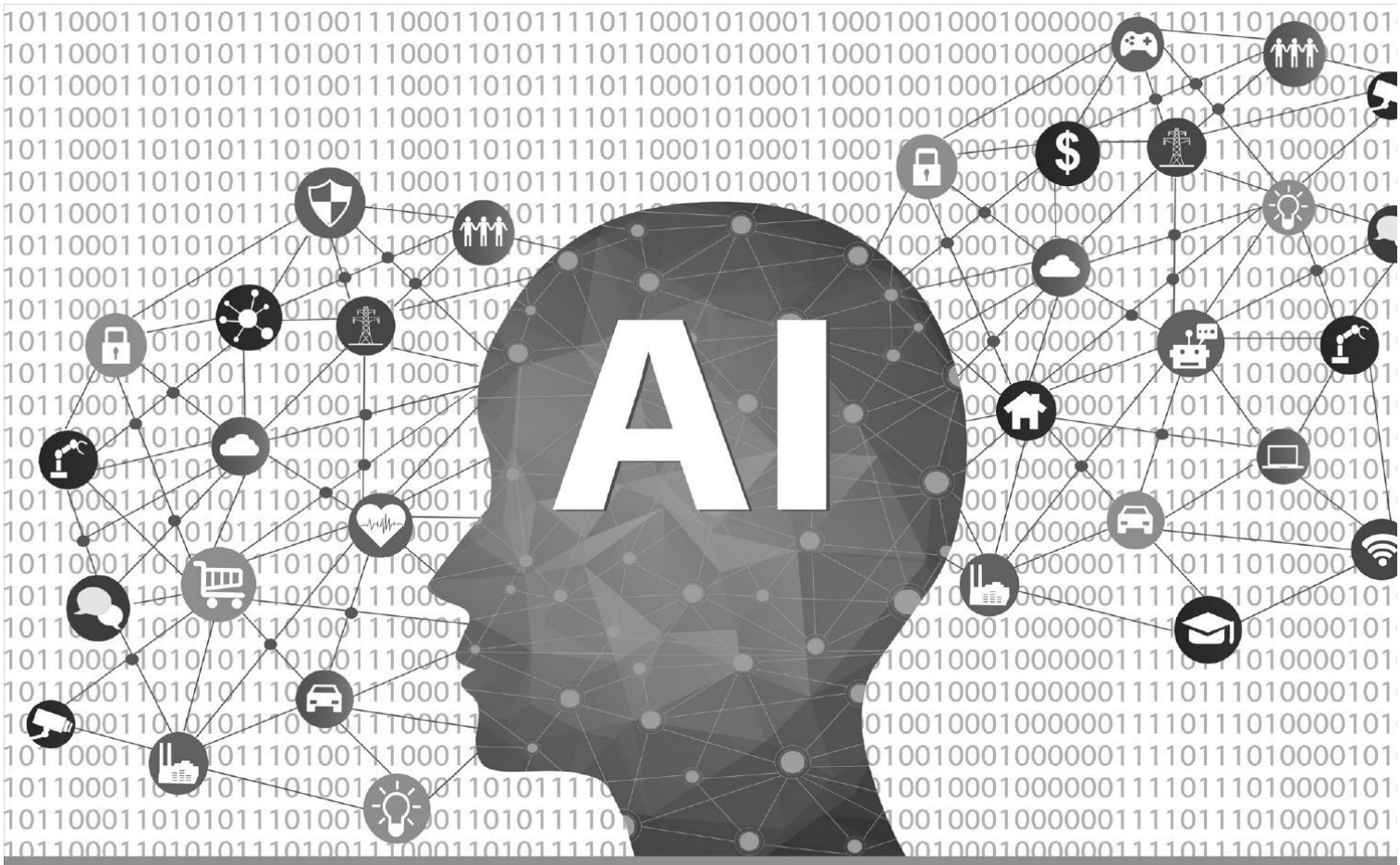
```
faiss.write_index(index, "index.faiss")
index = faiss.read_index("index.faiss")
print(type(index))  # <class 'faiss.swigfaiss_avx2.IndexFlat'>
```

Faiss 除了能夠處理一般的向量搜尋以外，還有很多其他專門處理超級大量且高維度向量的方法，可以參考官方文件[10] 的說明。在普通的基本應用裡面，以上的用法通常就已經相當有效率了。

19.7 結論

本章節介紹了一些檢索相關的知識與工具，這些技術的發展已經相當成熟，與生成能力強大的語言模型搭配後更是如虎添翼，因此檢索模型是許多語言模型應用的關鍵技術，不僅需要強力的檢索模型，更要有適當的檢索流程，才能完整發揮系統的能力！

10 https://github.com/facebookresearch/faiss/wiki



Chapter

20

任務導向聊天機器人

主管 @tag 我

@penut LLM 可以做 Task-Oriented 的對話嗎？

TOD 有 TOD 的玩法！

任務導向對話（Task-Oriented Dialogue, TOD）聊天機器人，與一般的聊天機器人有什麼不同呢？任務導向代表這段對話**有明確的任務目標**，例如餐廳點餐、飛機訂票或飯店預約等等。雖然 ChatGPT 已經是個很強大的語言模型，有豐富的知識問答與流暢的文字生成，但要達成長流程的任務對話，其實還是有一些困難需要克服的。

傳統上想要製作一個任務導向的聊天機器人，可以使用 **Google Dialogflow**[1]、**Microsoft Bot Framework**[2]、**Amazon Lex**[3] 等等的服務，這些服務提供使用者友善的介面，無論會不會寫程式都很適合操作。但如果你特別喜歡寫程式，想從底層技術面親身感受 TOD 的魅力，那 **Rasa**[4] 是個不錯的選擇。

補充堅果，頭好壯壯

在接觸這類聊天機器人的過程中，可能會去研究 Google 的 Dialogflow 框架，雖然這個框架主要還是使用傳統 BERT 模型建立的，但因為功能已經相當完整，所以依然是個入門不錯的選擇。

1 https://cloud.google.com/dialogflow
2 https://dev.botframework.com/
3 https://aws.amazon.com/tw/lex/
4 https://rasa.com/

而 Dialogflow 其實還區分成 ES 跟 CX 兩種版本，其中 ES 是比較簡單的版本，而 CX 是比較複雜的版本。如果是沒有接觸過 TOD 系統的讀者，可以考慮先從 ES 版開始著手學習，才不會被 CX 版那複雜的文件天書給嚇到 😂

在大型語言模型的時代，大家是如何看待任務導向對話呢？本章節帶大家一同探討相關的思維。

20.1 任務導向到底是什麼呢？

任務導向對話與一般對話最大的區別，在於系統需要**從對話裡面擷取實體資訊**出來使用。例如一個查詢天氣的機器人，就會需要知道使用者想查的是氣溫、降雨機率還是空氣品質？要查詢哪個時間、哪個地方的天氣呢？系統可能會進行以下分析：

```
User: 明天會下雨嗎？
Slots:
  - Date: 明天
  - QueryType: 降雨機率

Bot: 請問要查詢哪個地方的降雨機率？
User: 新店
Slots:
  - Location: 新北市新店區
```

系統取得日期、時間與查詢種類這些**實體資訊（Entity）**後，就能用來建構查詢天氣所需要的參數，例如發一個 HTTP Request API 出去：

```
https://www.weather-api.example/query?date=明天&location=新北市新店區
&query_type=降雨機率
```

從上面的範例可以看到，當系統缺乏位置資訊時，會對使用者做進一步的追問，這便是任務導向系統能根據**對話狀態（State）**做**流程決策（Policy）**的能力。

花生狂想曲

這時你可能會想，我好像在某些 GPTs 看過類似的事情欸？這些應用的確屬於廣義上的任務導向對話，因為對這些應用來說，他們都有個特定的目標需要執行，且完成這個任務所需的資訊，必須從使用者的對話中取得。

多數情況下，一般的對話應用通常都在做**單輪對話**的任務，而任務導向對話更著重在**多輪對話**上，有時甚至是數十回合以上非常冗長的對話，即便現在有很多模型已經支援到 100K 以上的輸入，但是這類模型很容易發生**迷途（Lost in the Middle）**的問題。因此在任務導向系統裡面，通常會額外維護一份**記憶狀態**來追蹤目前對話的進度。

所以任務導向對話系統主要著重在三個面向：**實體擷取**、**決策流程**與**記憶狀態**。

補充堅果，頭好壯壯

另外像是生成自然回覆也算是對話系統會探討的議題，雖然現在的語言模型可以產生自然流暢的文字，但是在一些較為嚴謹的商用情境下，開發者可能會更傾向於使用**定型範本**進行回覆，尤其是當系統應用涉及**金流**時更是需要注意，如果模型在描述數字時多一個零或少一個零的話就虧大了！

20.2 有了 LLM 的 TOD 系統天下無敵？

首先關注 Are LLMs All You Need For TOD[5] 這篇論文，是基於指令微調語言模型的做法，其概念相當簡單，傳統對話系統會有**領域分類器（Domain Classifier）**、**狀態追蹤器（State Tracker）**與**回覆產生器（Response Generator）**等模組，作者要求語言模型分別扮演這三個角色來進行對話，例如：

```
請根據以下對話狀態，決定這是什麼領域。請從以下清單選擇一個領域回答：餐廳、飯店、景點、計程車和火車。只需要回覆答案即可，並從清單中選擇最有可能的答案。

=== 範例 1 ===
使用者：我要找一間便宜的餐廳
機器人：我們有很多種便宜的餐廳，請問您對哪種餐廳有興趣呢？
使用者：中式餐廳
領域：餐廳
=== 範例 2 ===
使用者：我要訂兩點的車票
機器人：請問你要從哪裡出發？
使用者：板橋
領域：火車
===
現在完成以下範例：
使用者：我想要在北部找一個便宜的住宿地點
領域：
```

將以上提示輸入到 ChatGPT 就能輕鬆獲得「飯店」這個答案，這就是個非常經典的少量樣本提示技巧。類似的概念也能應用在狀態追蹤器上：

```
請根據對話的最後一句辨識相關實體，以「實體：值」的格式呈現，其中冒號和值之間不應有空格，使用分號（；）分隔不同的實體資訊。如果未指定，則留空該值。應捕捉的值包括：

- 價格：飯店的價格
- 區域：指定飯店所在的區域（東、南、西、北、中）
- 網路：指定飯店是否有網路（是、否）
```

5 https://arxiv.org/abs/2304.06556

```
- 停車場：指定飯店是否有停車場（是、否）
- 星級：指定飯店的星級數（一、二、三、四、五）
- 類型：指定飯店的類型（飯店、民宿、附早餐）

=== 對話 ===
使用者：我想要在北部找一個便宜的住宿地點
實體：
```

輸入這段提示就能獲得「區域：北；價格：便宜」的結果，這裡使用的是零樣本提示，因此必須對每個實體進行詳細說明，並列舉可能的值。最後的回覆產生器也是相同的原理：

```
你現在是個負責預約酒店的客服人員，使用者可以根據酒店名稱、地區、停車場、網路可用性或價格來查詢酒店，請根據資料庫的查詢結果進行回覆。

=== 對話 ===
使用者：我想要在北部找一個便宜的住宿地點
狀態：飯店；區域：北；價格：便宜
查詢結果：酒店數量 23 家
回覆：
```

將以上提示丟入 ChatGPT 後得到以下回覆：

```
您好！在北部地區，我為您找到了 23 家價格較為便宜的酒店。請問您需要我列出其中幾家供您參考，或是有其他特定的需求嗎？
```

透過簡單的零樣本或少量樣本提示的技巧，就能建構起一個基於大型語言模型的任務導向對話系統了！這樣的做法不只對 ChatGPT 有用，在一些參數量較小的語言模型上也能見效。

筆者使用 Llama 3 8B 進行實測，在領域分類上看起來尚可，但狀態追蹤上錯誤率就滿高了，而且產生回覆的部份很容易幻想出許多額外資訊：

```
我們找到 23 家便宜的酒店在北部地區。以下是一些選擇：

* 北區的「晨曦酒店」提供每晚 NT$1,200 的優惠價格。
*「北方酒店」位於北區，提供每晚 NT$1,500 的優惠價格。
```

```
*「晨曦商務酒店」位於北區，提供每晚 NT$1,800 的優惠價格。

請選擇您喜歡的酒店，我們將幫您預約。
```

但事實上給模型的提示裡面根本沒有這些資訊，看來我們的 TOD 系統距離天下無敵還有一段距離。

20.3 Rasa

Rasa[6] 是個開源的聊天機器人開發框架，在 GitHub[7] 上可以看到這是個從 2016 年發起的專案，算是有一定年紀的框架了，但 Rasa 直到現在都還是非常活躍，而最近能夠從官方文件[8] 上看到一些與語言模型整合的蛛絲馬跡，雖然看起來應該是付費版會員的功能，但是從這些說明文件中，還是可以推敲其運作原理。

在官方文件上的架構圖，與方才介紹的論文是有些異曲同工之妙的，他們同樣使用了基於檢索的方法來建構提示，同樣藉由語言模型能處理少量樣本提示的能力來進行各個階段的辨識。

6 https://github.com/RasaHQ/rasa

7 https://github.com/RasaHQ/rasa/graphs/contributors

8 https://tinyurl.com/llm-rasa

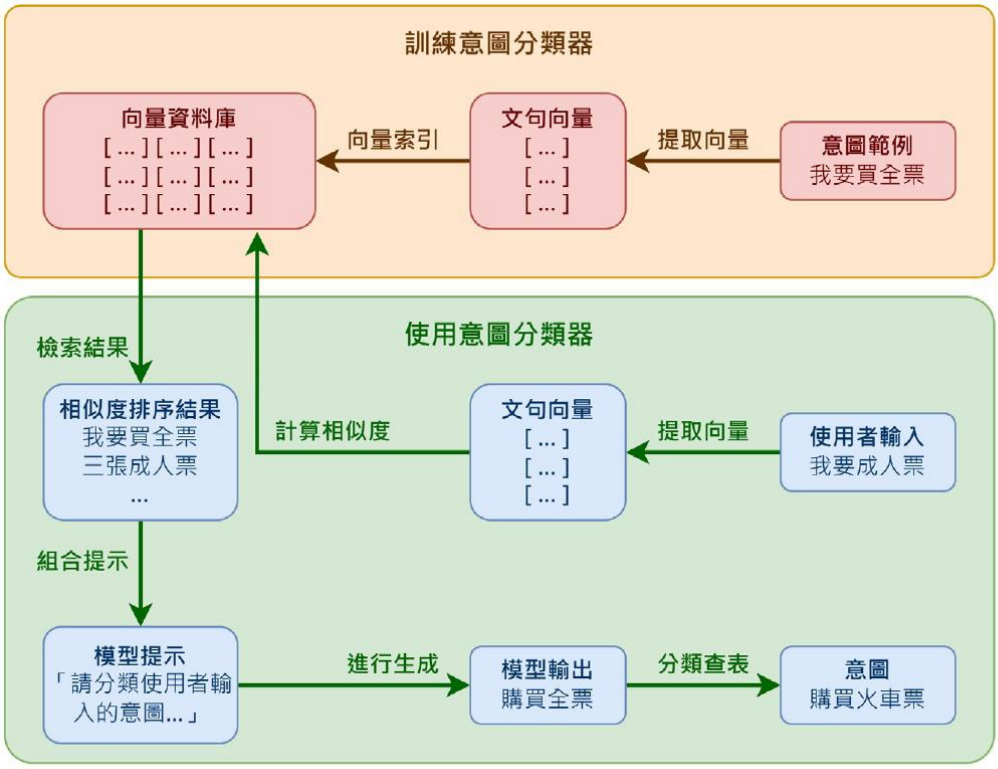


圖 20-1　Rasa 意圖分類器架構圖

以下是意圖分類的提示範例：

```
請分類使用者輸入的意圖，只需要回覆意圖名稱即可。
意圖必須是以下其中一個：
- 確認
- 問候

訊息：哈囉
意圖：問候

訊息：是的我是
意圖：確認

訊息：你好啊
意圖：
```

同樣也是一段指令搭配少量樣本，然後請語言模型回答分類結果，其實與上個小節的方法大同小異。但是在生成回覆的部份，Rasa 的做法就比較不一樣了：

```
以下是一段與 AI 助理的對話。
這位助理非常有幫助、創意十足、聰明伶俐且十分友善。
請重新改寫建議的 AI 助理回應，必須與原始訊息相近且保留其意義。

先前與使用者的對話如下：
使用者：我想要在北部找一個便宜的住宿地點

建議的 AI 回覆：在北部找到便宜的酒店有 23 家
改寫的 AI 回覆：
```

將以上提示輸入 Llama 3 8B 得到的結果：

```
北部有很多便宜的住宿選擇！根據您的需求，我可以推薦 23 家酒店，價格相對較低。您可以根據自己的喜好和預算選擇合適的住宿。
```

跟前一個做法比起來，這個結果令人滿意多了！開發者預先設定好一個定型範本，再請語言模型進行改寫，這樣就比較不容易產生錯誤或幻覺了。因此不管是少量樣本提示或定型範本改寫，其核心思想都是要**給語言模型一個參考的基準**。

有趣的是 Rasa 提出了另外一種 **Intentless Policy**[9] 的做法，所謂的**決策（Policy）**是指負責系統根據**對話狀態（State）**判斷需要採取什麼**動作（Action）**的模組。例如現在使用者在打招呼，那決策模組就會決定回覆使用者一個打招呼的訊息。如果使用者說要下一個訂單，那決策模組就會決定呼叫一個 API 把訂單存進資料庫等等。

多數的情況都可以透過規則式決策來處理，但最麻煩的是出現一些系統意料之外的狀況，這時系統到底該如何做回應呢？這裡可以分成兩個部份：第一個

9 https://tinyurl.com/llm-intentless

是系統如何**偵測**意料之外的情況發生了，第二個就是如何**應對**這個意料之外的情況。

僅透過 Rasa 的介紹並沒有辦法很明確的知道 Intentless Policy 到底是如何跟語言模型進行互動的，但是從Rasa提供的**範例專案**[10]可以猜出一些蛛絲馬跡。首先是這個範例的意圖辨識部份幾乎沒有特別重要的範例，只包含了一些打招呼之類的意圖而已，整個系統都是倚賴「劇本機制」來處理大部分的對話流程。

再來觀察官方文件提供的範例：

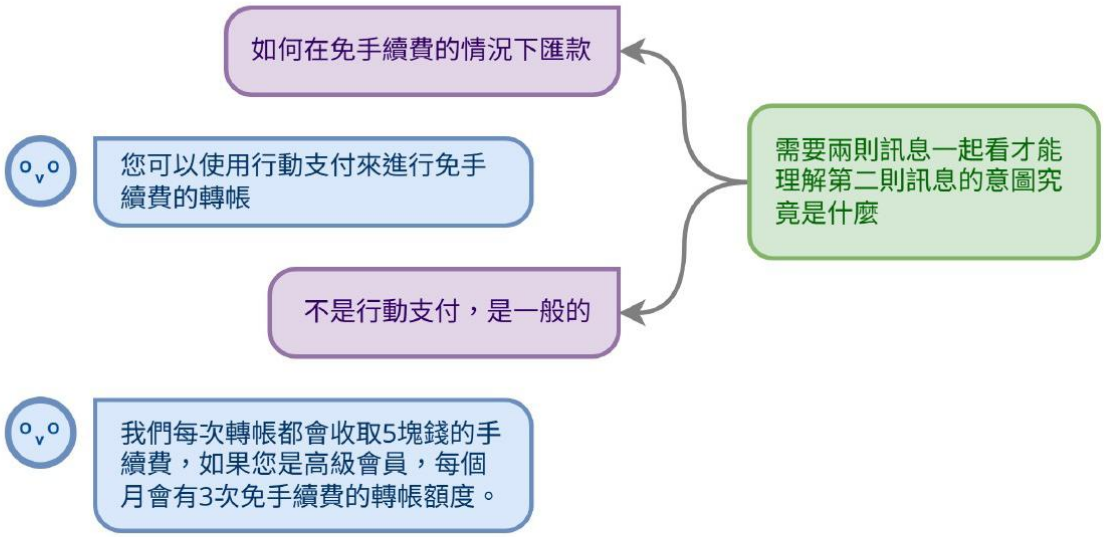


圖 20-2　官方範例圖中文版

可見這個 Intentless Policy 功能主打**語境回應（Contextual Response）**的特性，需要根據上下文的語境來判斷使用者當前的意圖，因此推測可能的做法是將前幾回合的對話內容放進提示，請語言模型判斷應該如何回覆。

另外官方文件也有提到，這個功能並不會生成新的回覆，也就是說回覆都是從預先定義好的句子裡面挑選的，所以筆者對整個提示的臆測大致如下：

10　https://github.com/RasaHQ/starter-pack-intentless-policy

```
你是一個專業的銀行客服，請根據對話內容挑選適當的回覆，只要回覆選項即可。

回覆 A：您可以使用手機 App 進行匯款。
回覆 B：您可以攜帶雙證件與存簿臨櫃辦理。
回覆 C：很抱歉，我幫不上忙。

=== 範例 1 ===
使用者：有沒有零手續費的匯款方式
機器人：回覆 A

=== 範例 2 ===
使用者：我沒有手機該如何匯款
機器人：回覆 B

=== 對話 ===
使用者：匯款可不可以不要手續費
機器人：回覆 A
使用者：可是我手機壞了
機器人：
```

把以上提示丟進 ChatGPT 或 Llama 3 8B 做測試，都可以獲得「回覆 B」的答案，因此看起來這套做法是可行的。根據官方文件所述，這應該是個閉源的功能，所以實際的提示到底如何建構也無從得知，這裡筆者只是提供一個方向給大家參考。

補充堅果，頭好壯壯

Rasa 的樣板與 Hugging Face 的分詞器一樣使用的是 Jinja 套件喔！

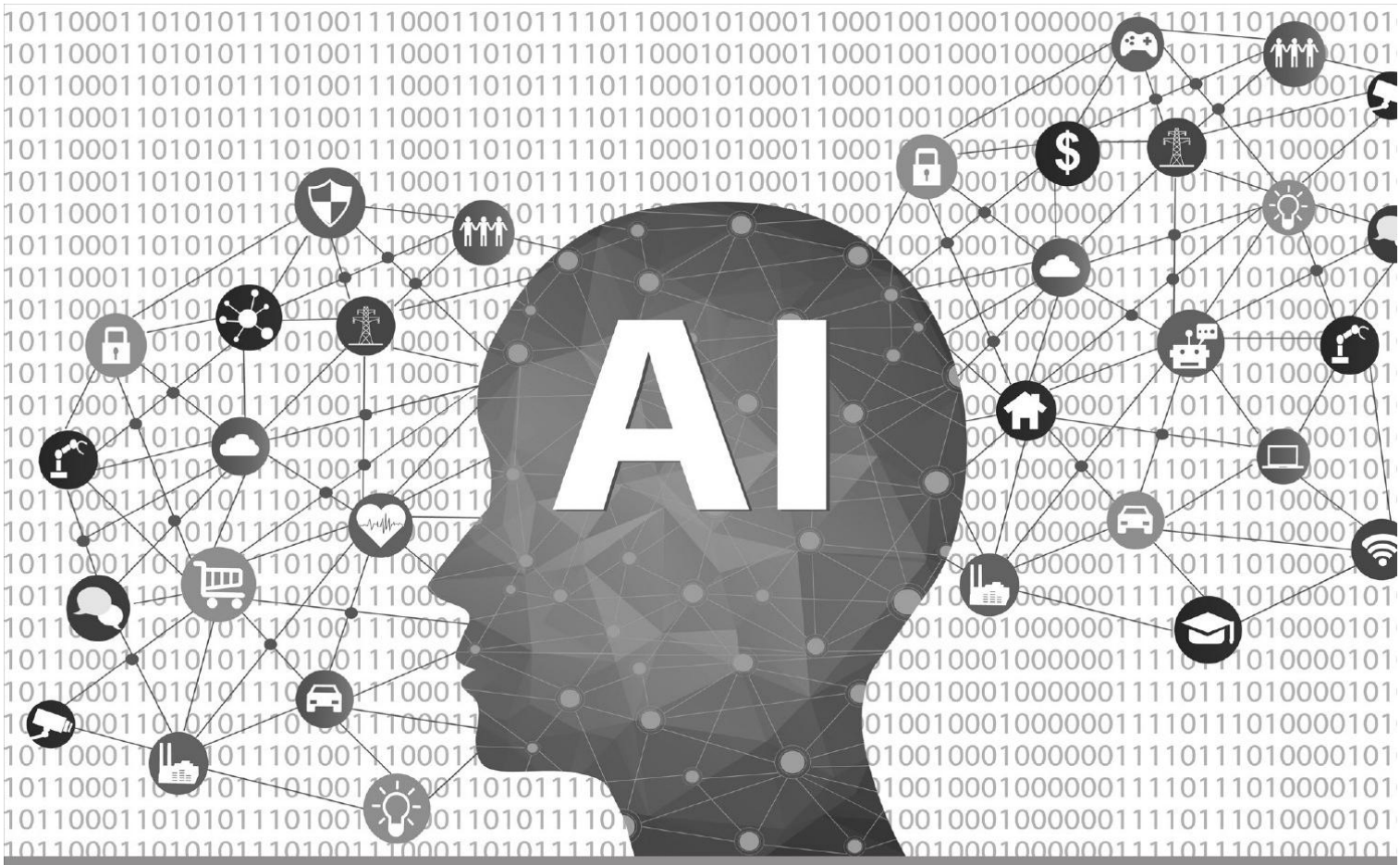
在大型語言模型出現之前，Rasa 就已經是個發展成熟的框架了，基本元件使用起來相當靈活，官方文件的內容相當詳細且豐富，也有進一步的商業版可用。現在加入強大的大型語言模型之後更是錦上添花，辨識率、跨語言等都不再是令人頭大的問題後，開發者進行對話的調試效率會更加提昇，在任務導向對話這方面的發展未來可期。

20.4 結論

相較於大型語言模型的蓬勃發展，任務導向對話系統的實際應用相對少了點。但無論是學界還是業界，都不斷提出基於語言模型的做法，而且核心概念其實都很相似：透過**指令**要求語言模型扮演某個傳統元件，並結合**檢索模型**來取代傳統的訓練步驟，所以**無需訓練**，加上語言模型本身的**泛用性**且**跨語言**的特質，使原本有些瓶頸的任務導向對話系統看見了相當明亮的未來！

花生狂想曲

有沒有發現最近幾個章節三句不離**檢索**一詞！筆者認為，即便未來語言模型再也沒有輸入上限問題，檢索方法依然會相當受用，因為與檢索結合的方法不僅能**縮短提示**、**減少運算量**，還能讓模型產生更**精確的回覆**，相信資訊檢索這領域會再次得到高度的重視。



Chapter

21

LangChain

主管 @tag 我

@penut 你們 RD 寫的這些 Code 都太複雜了，
有沒有簡單讓客戶比較看得懂的？

那就來試試看 LangChain 吧！

前面幾個章節提到的框架，大多屬於底層推論類，但上層的應用開發類也是許多人關注的焦點，其中最炙手可熱的非 LangChain[1] 莫屬。

開始實際使用語言模型進行應用開發時，會面臨到許多雜七雜八的提示樣板、重複的生成程序以及固定瑣碎的串接流程等等。多數情況下，這些樣板、流程切開來看，都不是很複雜的程式，自己動手寫也都不難寫。

但是當不同的流程開始堆疊出更龐大、更複雜的應用時，這一切就會顯得十分混亂。這時就能借助 LangChain 來幫我們進行更好的程式碼管理。

花生狂想曲

話雖這麼說，但其實筆者撰寫原稿之初也是剛把 `pip install langchain` 放下去跑而已。

老實說這類的應用框架，一直是筆者的心魔，一方面本業的工作內容都是研究與實驗的性質居多，比較少接觸到完整的應用。二方面 LangChain 對我來說是個全新的框架，但功能大多都是既有功能的整合，泥漿腦如我就是不想改變思考模式！

1 https://www.langchain.com/

於是在鐵人賽期間給自己一個挑戰 LangChain 的機會，如果想看 LangChain Pro 的可以參考 Ted Chen 大哥的系列文[2]，這章節就由筆者我這個破破的 LangChain 小菜鳥來獻醜啦！

21.1 題目發想

如果我有三百年的時間，我願意把 LangChain 的文件從頭到尾讀一遍。但是當時鐵人賽的時間相當緊迫，所以我決定選擇先設定一個題目，並想辦法把這個題目 Chain 起來，可能是個快速學習最好的方式。

而我選擇的題目是：**中二技能翻譯系統**。

21.2 資料來源

想要製作一個中二技能翻譯系統，勢必需要一些中二的翻譯資料，但是哪裡可以獲得這種中二的資料呢？這時筆者就把歪腦筋動到英雄聯盟的 **Riot API**[3] 上了！英雄聯盟有上百位英雄，每個英雄的技能都是由真人精心翻譯的，因此是個翻譯品質相當不錯的選擇！筆者先手動取一些樣本，來展示這個中二系統的核心概念：

```
請根據以下範例，產生一個中二的技能翻譯，只需要回答翻譯就好。

Source: The Darkin Blade
Target: 冥血邪劍
```

2 https://ithelp.ithome.com.tw/users/20154415/ironman/6008

3 https://tinyurl.com/llm-riot-api

```
Source: Infernal Chains
Target: 冥府血鏈

Source: Umbral Dash
Target: 冥影衝鋒

Source: World Ender
Target: 劍魔滅世

Source: Lightning Slash In The Dark
Target:
```

將這份提示丟進 ChatGPT 測試，得到一個「**冥雷暗破**」的翻譯，有那麼一丟丟要成為影之強者的感覺了！筆者也另外實測了一下 Llama 3 8B 的結果，得到「**暗電劈冥**」的翻譯，哇！感覺心中有某種神奇的東西在燃燒了！

21.3 資料爬取

Riot API 的官方文件其實描述的很詳細，首先需要先獲取全部的英雄列表，然後根據英雄名稱依序抓取各自的資料。可以從以下連結手動抓取全部的英雄列表：

```
https://ddragon.leagueoflegends.com/cdn/14.10.1/data/en_US/champion.json
```

網址裡面的 `14.10.1` 是遊戲版本，如果懷念菲歐拉的舊版大絕名稱，可以考慮回溯到 `5.14.1` 之前的版本去抓取。而 `en_US` 代表語言是英文的意思，因為所有英雄的索引都是使用英文，所以這個步驟只需要英文資料。接下來開始爬取英雄的技能資料，每個英雄資料的 API 格式如下：

```
http://ddragon.leagueoflegends.com/cdn/13.19.1/data/<lang>/champion/
<champ>.json
```

`<lang>` 為語言代號，而 `<champ>` 為英雄的英文名字。我們會分別爬取英文 `en_US` 與繁體中文 `zh_TW` 兩種語言的資料，首先定義爬取的函式：

```
import json
import os
import requests

def get_data(champ, lang, ver="14.10.1"):
    # 取得資料
    cdn_url = f"http://ddragon.leagueoflegends.com/cdn/{ver}/data"
    url = f"{cdn_url}/{lang}/champion/{champ}.json"
    resp = requests.get(url)

    # 寫入檔案
    fn = f"data/{lang}/{champ}.json"
    os.makedirs(f"data/{lang}", exist_ok=True)
    with open(fn, "wt", encoding="UTF-8") as fp:
        fp.write(resp.text)
```

爬蟲的過程，可以善用 Python 內建的 `concurrent` 套件來減少等待時間：

```
from tqdm import trange
from concurrent.futures import ThreadPoolExecutor, as_completed

with open("champion.json", "rt", encoding="UTF-8") as fp:
    data: dict = json.load(fp)["data"]

with ThreadPoolExecutor(max_workers=16) as tpe:
    futures = [
        tpe.submit(get_data, champ, lang)
        for champ in data.keys()
        for lang in ("en_US", "zh_TW")
    ]

    with trange(len(futures), ncols=100) as prog:
        [prog.update() for _ in as_completed(futures)]
```

這樣三百多筆資料不用 10 秒鐘就可以全部下載完囉！若想要其他語言可參考這篇資訊[4] 做調整。

> **補充堅果，頭好壯壯**
>
> 如果你經常覺得 `tqdm` 套件的進度條太寬太佔版面，除了透過 `ncols` 參數設定以外，也可以透過環境變數 `TQDM_NCOLS` 來設定所有 `tqdm` 進度條的寬度，但是如果該進度條有設定 `dynamic_ncols` 的話就會失效。

接著依序拜訪造型外觀名稱、主動技能名稱、被動技能名稱，這些都是重要的中二要素來源，建立中英技能資料集的程式碼如下：

```
import json

def load_json(file_path):
    with open(file_path, "rt", encoding="UTF-8") as fp:
        return json.load(fp)

def create_item(source, target):
    return {"source": source, "target": target}

datasets = list()
data: dict[str, dict] = load_json("champion.json")

for champ in data["data"].keys():
    # 讀取英雄資料
    en_data = load_json(f"data/en_US/{champ}.json")
    zh_data = load_json(f"data/zh_TW/{champ}.json")

    # 外觀造型名稱
    en_skins = en_data["data"][champ]["skins"]
    zh_skins = zh_data["data"][champ]["skins"]

    # 跳過經典造型
```

4 https://developer.riotgames.com/docs/lol#data-dragon_data-assets

```
    for en_sk, zh_sk in zip(en_skins[1:], zh_skins[1:]):
        en_sk_name = en_sk["name"]
        zh_sk_name = zh_sk["name"]
        datasets.append(create_item(en_sk_name, zh_sk_name))

    # 主動技能名稱
    en_spells = en_data["data"][champ]["spells"]
    zh_spells = zh_data["data"][champ]["spells"]
    for en_sp, zh_sp in zip(en_spells, zh_spells):
        en_name, zh_name = en_sp["name"], zh_sp["name"]
        datasets.append(create_item(en_name, zh_name))

    # 被動技能名稱
    en_pass = en_data["data"][champ]["passive"]["name"]
    zh_pass = zh_data["data"][champ]["passive"]["name"]
    datasets.append(create_item(en_pass, zh_pass))

with open("datasets.json", "wt", encoding="UTF-8") as fp:
    json.dump(datasets, fp, ensure_ascii=False, indent=4)
```

到這裡完成資料集的建立，接下來正式進入 LangChain 的部分！

21.4 LLM

LangChain 其實很講究**元件間的互動**，在搞懂互動之前，鐵定要先搞懂元件！在 LangChain 裡面的元件多數可以獨立運作，先來安裝相關套件：

```
pip install langchain langchain-community langchain-openai
```

LangChain 的特色之一在於整合了相當多相關的套件框架，讓我們可以用統一的介面去操作，例如 LLM 類別就有整合 OpenAI、ggml、vLLM 或 TGI 等語言模型後端框架。

但是框架這麼多，有選擇障礙怎麼辦？還記得前面的章節有提到，這些框架其實都有提供相容 OpenAI API 的用法嗎？在這裡就派上用場囉！筆者選擇 vLLM

當作主要的後端框架來進行推論，但是在 LangChain 裡面，一樣可以把它當成 OpenAI API 來用，首先啟動一個 vLLM 服務：

```
python -m vllm.entrypoints.openai.api_server \
    --model meta-llama/Meta-Llama-3-8B-Instruct \
    --api-key auth-token-ouo123
```

在 LangChain 裡面 LLM 的基本用法如下：

```
from langchain.callbacks import StreamingStdOutCallbackHandler
from langchain_openai import OpenAI

llm = OpenAI(
    api_key="auth-token-ouo123",
    model="meta-llama/Meta-Llama-3-8B-Instruct",
    base_url="http://localhost:8000/v1",
    callbacks=[StreamingStdOutCallbackHandler()],
    streaming=True,
)
llm.invoke("hello, i am", stop=["\n"])
```

將 `base_url` 參數換成 vLLM Server 的路徑，然後填入啟用 vLLM 時設定的 API Key，並指定使用的模型名稱就可以囉！另外設定 `streaming=True` 的參數之後，就可以搭配上面的 Callback 進行串流輸出。

21.5 Prompt Template

在 LangChain 裡面可以使用各種 Prompt Template 類別來對不同的樣板進行管理，其用法如下：

```
from langchain.prompts import PromptTemplate

template = "### USER: {query}\n\n### ASSISTANT: "
template = PromptTemplate.from_template(template)

prompt = template.format(query="什麼是語言模型？")
```

```
print(repr(prompt))
# Output: '### USER: 什麼是語言模型？\n\n### ASSISTANT: '
```

經由樣板產生的提示，雖然可以當成一般字串丟進 LLM 使用，但是在 LangChain 裡面其實有更 **"Chain"** 的做法：

```
from langchain.prompts import PromptTemplate

template = (
    "### SYSTEM:\n 使用繁體中文回答。\n\n"
    "### USER:\n{query}\n\n"
    "### ASSISTANT:\n"
)
template = PromptTemplate.from_template(template)
chain = template | llm
chain.invoke({"query": "什麼是語言模型？"})
# Output: 語言模型是一種人工智慧模型 ...
```

很酷吧！居然用 OR 運算子 | 來描述元件間的互動，是個非常特別的做法！如果是檢索類的應用，那樣板可能會長的像這樣：

```
template = (
    "### INST: 請根據 REF 回答 USER 的問題\n\n"
    "### REF: {reference}\n\n"
    "### USER: {query}\n\n"
    "### ASSISTANT: "
)
prompt = PromptTemplate.from_template(template)

chain = prompt | llm.bind(stop=["###"])

params = {
    "reference": "小明每天從板橋通勤到新店工作",
    "query": "請問小明下班會回去哪裡？",
}
chain.invoke(params)
# Output: 小明從板橋通勤到新店工作，因此下班後他將會返回板橋。
```

其實這些基本的提示樣板跟一般的格式化字串用起來沒什麼太大的不同，但是透過這個類別就能跟其他元件 "Chain" 起來用。

補充堅果，頭好壯壯

在這裡 `|` 是一種 Bitwise Operator，此運算子會在位元層級進行 OR 運算，例如：

```
x = 0b1100 | 0b1010
print(f"0b{x:b}")  # Output: 0b1110
```

在 Python 中，可以定義類別的 `__or__` 方法來自訂 `|` 運算的行為，例如：

```
class Hello:
    def __init__(self, value):
        self.value = value

    def __or__(self, other):
        print(f"or self={self}, other={other}")

    def __repr__(self):
        return f"Hello({self.value})"

h1 = Hello(1)
h2 = Hello(2)
h1 | h2  # Output: or self=Hello(1), other=Hello(2)
```

而在 LangChain 框架裡面 `chain = template | llm` 的這段敘述，其實更像是指令操作裡面 Pipeline 的概念，將前者的內容當作後者的輸入。

21.6 Embedding Model

Embedding Model 也是 LangChain 另外一個整合相當豐富的部分，各式各樣的 Embedding 框架都被整合在裡面了。根據筆者的經驗，在跨語言且短文本的檢索上，Google 的 Universal Sentence Encoder 效果是不錯的，而且模型尺寸也比較小，相當適合我們這種搞笑應用：

```
from langchain_community.embeddings import TensorflowHubEmbeddings

url = (
    "https://www.kaggle.com/models/google/"
    "universal-sentence-encoder/tensorFlow2/"
    "multilingual-large/2"
)
embedding = TensorflowHubEmbeddings(model_url=url)
```

基本用法大致如下：

```
import numpy as np

texts = ["dog", "狗", "cat", "貓"]
embs = embedding.embed_documents(texts)
print(np.inner(embs, embs))
```

21.7 Vector Store

有了向量之後就可以存在**向量儲存空間**裡面，這些存放向量的地方稱為 Vector Store。其用途不僅是存放向量，更重要的功能在於**快速搜尋向量**，例如 Faiss 就是個可以存放與搜尋向量的套件，搭配剛才的 Embedding Model 一起使用：

```
from langchain_community.vectorstores import FAISS

texts = ["dog", "狗", "cat", "貓"]
faiss_store = FAISS.from_texts(
    texts=texts,
    embedding=embedding,
)
```

Vector Store 會自動透過 Embeddings 類別將文字轉為向量並存起來，這樣就可以直接查詢並取得語意相似的結果：

```
results = faiss_store.search(
    query="고양이",  # 韓文的「貓」
```

```
    search_type="similarity",
    k=2,  # 取兩筆結果
)

for res in results:
    print(res.page_content)

# Outputs: 貓, cat
```

參數 `k` 用來指定系統要搜尋幾筆結果，然後 Faiss 就會依照相似度排序並回傳結果。說到搜尋就要提一下我們的老朋友 BM25 好夥伴：

```
from langchain_community.retrievers import BM25Retriever

bm25 = BM25Retriever.from_texts(texts)
bm25.invoke("cat")
```

在 LangChain 裡面 BM25 屬於 Retriever，而 Faiss 屬於 VectorStores，但兩者都具有搜尋的功能，該如何一起使用呢？這時可以透過 `EnsembleRetriever` 類別來處理，首先需要先將 Faiss 轉換成 `Retriever` 類別：

```
faiss = faiss_store.as_retriever()
```

然後再將 BM25 與 Faiss 一起丟進 `Ensemble` 裡面：

```
from langchain.retrievers import EnsembleRetriever

ensemble = EnsembleRetriever(
    retrievers=[faiss, bm25],
    weights=[0.75, 0.25],
)

ensemble.invoke("고양이")
```

這樣在搜尋時就可以同時參考兩種檢索系統的結果了。

Vector Store 的種類相當繁多，請根據自身應用的場景做選擇。因為筆者的資料其實沒很多，所以這個應用只需要 Faiss 就足夠了。

21.8 Few-Shot Prompt Template

在這個應用裡面會使用到 Few-Shot 的技巧來建立完整的提示，因此需要用到 `FewShotPromptTemplate` 類別。在使用此類別之前，需要使用基本的提示樣板來定義每個範例的格式：

```
from langchain.prompts import PromptTemplate

examples = [
    {"source": "hello", "target": "哈囉"},
    {"source": "goodbye", "target": "再見"},
]

example_prompt = PromptTemplate(
    input_variables=["source", "target"],
    template="Source: {source}\nTarget: {target}",
)

example_prompt.format(**examples[0])
```

接著以這個樣板為基礎來建立 Few-Shot Prompt Template 的內容：

```
from langchain.prompts.few_shot import FewShotPromptTemplate

prompt = FewShotPromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
    prefix="請將以下單字翻譯成中文",
    suffix="Source: {input}\nTarget: ",
    example_separator="\n===\n",
    input_variables=["input"],
)

print(prompt.format(input="Starburst Stream"))
```

輸出結果：

```
請將以下單字翻譯成中文
===
Source: hello
Target: 哈囉
===
Source: goodbye
Target: 再見
===
Source: Starburst Stream
Target:
```

這樣就完成 Few-Shot Prompt Template 的設定囉！

21.9 Example Selector

我們已經知道如何將 Prompt 與 LLM 串在一起了，現在需要煩惱的部份是使用者輸入 => 檢索 => 少量樣本提示這段流程該如何串接起來，這裡就要出動 Example Selector 來幫忙：

```
import json

from langchain.prompts.example_selector import SemanticSimilarityExampleS
elector as Selector
from langchain_community.embeddings import TensorflowHubEmbeddings
from langchain_community.vectorstores import FAISS

with open("datasets.json", "rt", encoding="UTF-8") as fp:
    datasets: dict[str, str] = json.load(fp)

url = (
    "https://www.kaggle.com/models/google/"
    "universal-sentence-encoder/tensorFlow2/"
    "multilingual-large/2"
)
embeddings = TensorflowHubEmbeddings(model_url=url)
```

```
example_selector = Selector.from_examples(
    examples=datasets,
    embeddings=embeddings,
    vectorstore_cls=FAISS,
    k=10,
)
```

Example Selector 把提取和存放向量的動作都封裝起來了，查詢方法如下：

```
result = example_selector.select_examples({"query": "Dark"})
print(result)

"""
查詢結果：
[{'source': 'Darkness Rise', 'target': '暗崛'},
 {'source': 'Dark Matter', 'target': '黑暗物質'},
 ...
 {'source': 'Piercing Darkness', 'target': '刺骨幽闇'}]
"""
```

OK，到這邊萬事俱備只欠東風，接下來就要看看如何將這些元件全部 Chain 在一起啦！

21.10 Chain!

讀取資料集的部份與上個小節提供的程式碼是一樣的，這裡不再重複一次。讀取完資料集之後，就要來建立樣板：

```
from langchain.prompts import PromptTemplate
from langchain.prompts.few_shot import FewShotPromptTemplate

# 建立 Example Template
example_prompt = PromptTemplate(
    input_variables=["source", "target"],
    template="Source: {source}\nTarget: {target}",
)
```

```
# 建立 Few-Shot Template
prompt = FewShotPromptTemplate(
    example_selector=example_selector,
    example_prompt=example_prompt,
    suffix="Source: {query}\nTarget: ",
    prefix="請根據以下範例，產生一個中二的技能翻譯。",
    input_variables=["query"],
)
```

最後把 LLM 串上去！

```
from langchain_openai import OpenAI

llm = OpenAI(
    api_key="auth-token-ouo123",
    model="meta-llama/Meta-Llama-3-8B-Instruct",
    base_url="http://localhost:8000/v1",
)

chain = prompt | llm.bind(stop=["\n"])  # 遇到換行停止生成
print(chain.invoke("Starburst Stream"))
```

順便使用 Gradio 建立一個簡單的 Demo 網頁：

```
import gradio as gr

def send(source):
    return chain.invoke({"query": source})

with gr.Blocks() as app:
    source = gr.Textbox(label="Source")
    target = gr.Textbox(label="Target")
    source.submit(send, source, target)
app.launch()
```

整個應用使用起來的效果如下：

圖 21-1　中二技能翻譯使用範例，噓！

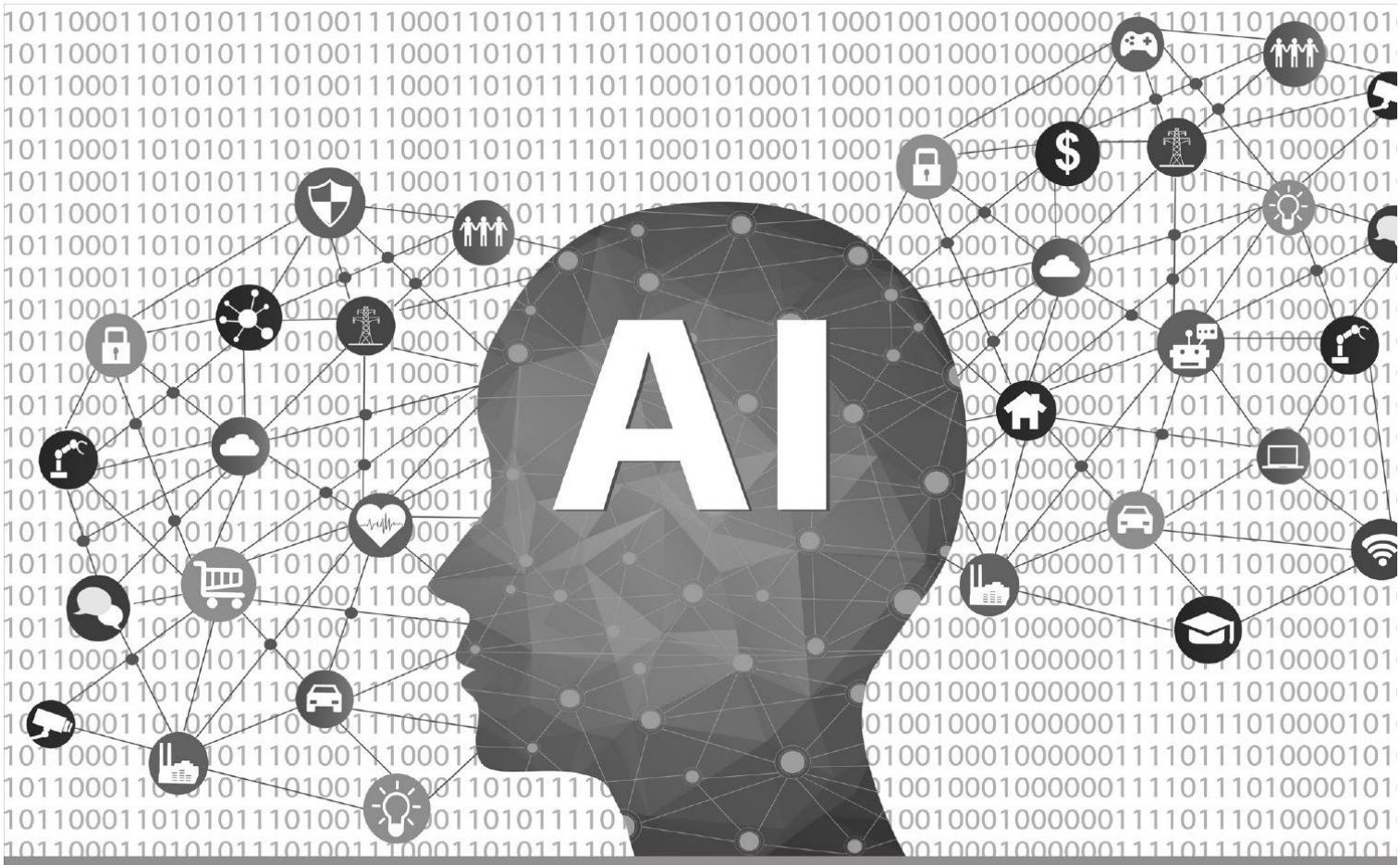
熟悉各元件的使用方法後，應用串接起來快又方便！完整的專案程式碼放在 GitHub 上面，各位有興趣的話可以嘗試自己架起來玩看看 👍

中二技能翻譯完整程式碼
https://tinyurl.com/llm-note-13

21.11　結論

實際使用 LangChain 搭建完一個應用後，總算是對這個熱門的框架有些微薄的理解。這個框架的確將語言模型應用中的每個元件與行為都封裝的很好，而且使用了相當有趣的 Chain 概念來將元件串聯起來，是個非常獨特的思維。幸好 LangChain 真的很簡單好上手，才沒讓筆者在這篇文章開天窗 😂

LangChain 的易用性與靈活性加上活躍的社群，未來發展相當值得關注！



Chapter

22

微調語言模型 DIY！

主管 @tag 我

@penut 講了整本書沒看你動過半個參數欸，
你到底會不會 Train 模型啊

在 Hugging Face 的擁抱下，人人都能是模型訓練大師！

微調（Finetune）一個大型語言模型在方法上與微調其他模型其實很相似，但是因為語言模型的參數量通常較大，所以訓練的門檻會比其他模型高的多。但是沒有強大的機器，難道就不能訓練語言模型了嗎？

使用大機器訓練模型，就像在豪華的廚房裡面做菜，所有工具一應俱全，但是要端出可以滿足整間餐廳的菜色與份量。而使用小機器訓練模型，就像是與三五好友到野外露營，只有幾個小鍋子跟卡式瓦斯爐，為了在小廚具上做出少人份的佳餚，需要講究很多不佔空間的烹調技術。雖然菜色未必華麗，但絕對能令自己滿意！

大機器的訓練豪放奢華，小機器的訓練精緻浪漫，接下來就帶大家看看，如何在有限的資源內烹調一道精心製作的語言模型！

22.1 任務設計

一開始學習微調模型時，可以透過單純的**玩具任務（Toy Task）**來練習。玩具任務通常很簡單，因此模型容易學起來，而且資料來源也很容易爬取跟操作。這裡筆者設計了一個**路名解析任務**：

```
輸入：臺北市中正區八德路
輸出：{"city": "臺北市", "town": "中正區", "road": "八德路"}
```

使用者輸入一段帶有行政區的路名，請模型將他解析成 JSON 格式。要讓模型學會完成這個任務，最單純的想法是輸入原句，並直接輸出 JSON 結果，像是將上方的範例當成提示直接丟進語言模型裡面做訓練。

這樣做不是不行，但這比較像是傳統 Encoder-Decoder 的訓練方法。拿這類的資料訓練現在的 Decoder 效果通常並不好，不如直接用參數量更少的 Encoder-Decoder 來訓練。那怎樣的資料比較好呢？對語言模型而言，需要放入更多**指令**的成份在裡面：

```
### USER:
請將以下路名解析為 JSON 格式。

輸入：臺北市中正區八德路
輸出：{"city": "臺北市", "town": "中正區", "road": "八德路"}

輸入：屏東縣佳冬鄉文化三路

### ASSISTANT:
{"city": "屏東縣", "town": "佳冬鄉", "road": "文化三路"}
```

這樣的訓練資料不僅給了**測試輸入**與**正解輸出**，還包含了**一個指令**與**一份範例**，對於一個**指令微調過的語言模型**而言會更加有意義，訓練上也比較不容易破壞語言模型的其他能力。

花生狂想曲

還記得之前說過語言模型是藉由**因果建模**進行訓練的嗎？因此給他越充分的**因**，就能產生越穩定的**果**。

如果我們直接將「臺北市中正區八德路」輸入到 ChatGPT 裡面，他只會開始介紹這條路的相關知識而已，並不會幫你把路名解析成 JSON 格式。需要對 ChatGPT 下達明確的指示，甚至給他一個範例，才能完成預想的目標。

因此對於一個指令微調的語言模型而言，在訓練資料中加入明確指令是很重要的。

22.2 建立資料集

行政區與路名的資料可以從**政府資料開放平台**[1] 取得，其中的**全國路名資料**[2] 就有我們需要的資訊，筆者使用 **112 全國路名資料**[3] 的 CSV 檔做示範，其格式大致如下：

```
city,site_id,road
縣市名稱，行政區域名稱，全國路名
宜蘭縣，宜蘭縣三星鄉，人和一路
宜蘭縣，宜蘭縣三星鄉，人和七路
宜蘭縣，宜蘭縣三星鄉，人和九路
```

就好比做菜之前要先清洗食材一樣，在建立資料集之前，需要先來做一些基本的資料觀察，看看這份資料有沒有不正常的內容？先把所有內容讀取進來，然後用 `set` 類別來統計所有字元，並輸出他們的編碼：

```
with open("opendata112road.csv", "rt", encoding="UTF-8") as fp:
    char_set = set(fp.read())

# 排序並列出字元編號
char_set = sorted(list(char_set))
lines = [f"{c!r} {ord(c)}" for c in char_set]

with open("chars.txt", "wt", encoding="UTF-8") as fp:
    fp.write("\n".join(lines))
```

觀察輸出的 `chars.txt` 看看都包含了些什麼字：

```
'\n' 10
',' 44
'?' 63  <= 不知哪裡來的問號
```

1 https://data.gov.tw/
2 https://data.gov.tw/dataset/35321
3 https://tinyurl.com/llm-tw-roads

```
'_' 95

...

'■' 9607  <= 奇怪的正方形
'大' 12068
'\u3000' 12288  <= 這是全形空格
'ㄧ' 12583  <= 這是注音文

...

'沿' 194812
'¢ ¼ï¼è' 194818  <= 一堆顯示不出來的亂碼
'ä½ å¥½' 194824
'\U000fffa8' 1048488
'\U000fffc0' 1048512
```

天啊，這裡面問題還真多！有莫名的問號、全形空格跟一堆顯示不出來的亂碼，即便是小型的玩具任務，但這些對訓練都是很有影響的，資料品質至上！

面對這樣的問題，通常有兩種選擇：用程式碼自動修正，或者把有問題的資料丟掉不要用，但千萬不要試圖人工修正**所有**資料！時間寶貴，精力有限，請把你的聰明才智專注在更有效率的做法上！

花生狂想曲 💡

資料品質至上跟不要人工修正好像互相違背欸！其實不然，在團隊開發中，通常會聘請資料標注人員協助修正資料，而捨棄有問題的資料也是提昇資料品質的手法。

對於獨立開發者而言，人工修正不是不行，但如果每次看到資料有問題就想要手動修正，很容易養成習慣，造成自身的工作效率低落。

在這個人工智慧的時代，更應該學習如何 Working Smart 而不是 Working Hard。例如先透過簡單的邏輯將有問題的資料篩選出來，再把這些問題丟給 ChatGPT 去做修正，就是個相對有效率的做法。

除了全形空白可以直接取代掉以外，剩下有問題的字元編號存在 `exclude.txt` 裡面，並把這些包含問題字元的資料給捨棄掉：

```
with open("exclude.txt", "rt", encoding="UTF-8") as fp:
    exclude_ord = fp.read().strip().split("\n")
    exclude_ord = set(map(int, exclude_ord))

with open("opendata112road.csv", "rt", encoding="UTF-8") as fp:
    lines = list()
    for line in fp:
        if set(map(ord, line)) & exclude_ord:
            continue

        # 取代全形空白
        line = line.replace("\u3000", "")
        lines.append(line)

with open("road-filter.csv", "wt", encoding="UTF-8") as fp:
    fp.writelines(lines)
```

全臺灣有近三萬五千條路，每一行都包含了行政區以及路名。這裡需要將直轄縣市與鄉鎮市區分開，各自放進 `city` 與 `town` 欄位裡面，最後與 `road` 欄位組成完整的資料：

```
import csv

dataset = list()
with open("road-filter.csv", "rt", encoding="UTF-8") as fp:
    # 跳過前兩行說明用的資料列
    fp.readline()  # city,site_id,road
    fp.readline()  # 縣市名稱，行政區域名稱，全國路名

    for city, site, road in csv.reader(fp):
        town = site.replace(city, "")
        item = {"city": city, "town": town, "road": road}
        dataset.append(item)
```

接下來將資料切成訓練、驗證、測試集等三份。因為目的只是少量測試訓練效果，所以訓練與驗證集各取 200 筆資料即可。也因為這個任務很簡單，推論速度也很快，所以測試集可以取多一點：

```
import json
import random

def dump_json(data, file_path):
    with open(file_path, "wt", encoding="UTF-8") as fp:
        json.dump(data, fp, ensure_ascii=False, indent=2)

random.seed(0)
random.shuffle(dataset)

train = dataset[:200]
dev = dataset[200:400]
test = dataset[400:1000]

dump_json(train, "train.json")
dump_json(dev, "dev.json")
dump_json(test, "test.json")
```

接下來要對訓練集與驗證集進行斷詞，一開始先使用小模型做測試，所以選擇參數量只有 1B 的 **TinyLlama**[4] 模型來微調，因此使用他的分詞器來進行斷詞：

```
from transformers import LlamaTokenizerFast as TkCls

model_id = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
tk: TkCls = TkCls.from_pretrained(model_id)
```

4 https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat-v1.0

在因果建模的訓練流程裡面，每筆訓練資料必須包含 `input_ids` 與 `labels` 兩個欄位，而且這兩個欄位的內容會是完全一樣的，例如：

```
[
    {
        "input_ids": [ 1, 43, 92, 71,  2, 2],
        "labels":    [ 1, 43, 92, 71,  2, 2]
    },
    {
        "input_ids": [ 1, 97, 66, 92, 71, 2],
        "labels":    [ 1, 97, 66, 92, 71, 2]
    }
]
```

> **花生狂想曲** 💡
>
> 當訓練文本量很大時，每次訓練模型之前都要重新斷詞一次會顯得很沒效率，因此筆者習慣先將文本進行斷詞並存成檔案，這樣訓練的程式就只需要專注在訓練流程上，而不用每次都花上一段時間在重新斷詞和建立輸入格式上。

首先寫一個拜訪資料並產生對應輸入輸出的函式：

```
def iter_dataset(file_path):
    data = load_json(file_path)
    for item in data:
        city = item["city"]
        town = item["town"]
        road = item["road"]

        text = f"{city}{town}{road}"
        item = json.dumps(item, ensure_ascii=False)

        yield text, item
```

將 City、Town 跟 Road 合併成一個字串當作輸入句，並產生 JSON 格式的答案，然後把每筆資料根據我們設計好的格式，套用到 TinyLlama 提供的對話樣板裡面：

```
template = """請將以下路名解析為 JSON 格式。

輸入：臺北市中正區八德路
輸出：{{"city": "臺北市", "town": "中正區", "road": "八德路"}}

輸入：{}"""

ds_type = "train"
ds_tokens = list()
for text, item in iter_dataset(f"{ds_type}.json"):
    inn = dict(role="user", content=template.format(text))
    out = dict(role="assistant", content=item)
    input_ids = tk.apply_chat_template([inn, out])
    ds_tokens.append(input_ids)
    # print(tk.decode(input_ids))  # 檢查格式是否符合預期
```

完成 Token 轉換後，需要對資料長度進行**填充（Padding）**，因為每個批次裡面的資料長度必須相同才能進行訓練：

```
# 計算最大長度
maxlen = max(map(len, ds_tokens))
print(f"Max Length: {maxlen}")

# 對資料集進行填充
dataset = list()
for tokens in ds_tokens:
    delta = maxlen - len(tokens)

    # 將 EOS Token 當作 PAD Token 來用
    tokens += [tk.eos_token_id] * delta

    # 訓練用的 input_ids 與 labels 通常是完全一樣的序列
    dataset.append({"input_ids": tokens, "labels": tokens})

# 確認所有序列的長度都是一致的
for item in dataset:
    assert len(item["input_ids"]) == maxlen
```

```
    assert len(item["labels"]) == maxlen

# 將斷詞完的結果存下來
dump_json(dataset, f"{ds_type}.tokens.json")
```

完成填充後，就可以把斷詞完的結果存下來了。

小心，樂高就在你腳邊

還記得之前在講推論的時候有提到，必須幫分詞器設定 `padding_side="left"` 才能正常生成嗎？但是這次在準備訓練資料的時候，填充其實是放在右邊的喔！

這是因為推論時只關注**最右邊**的 Token，因此不能把無意義的填充放在右邊。但是模型在訓練時，主要是在學習如何**從左到右**的生成。因此進行推論與訓練時的填充方向並不相同，這點需要多加注意！

最後將 `ds_type` 分別設定為 `train` 與 `dev` 各跑一次，就完成訓練集與驗證集的處理啦！

補充堅果，頭好壯壯

在資料集裡面所謂的**訓練集（Train）**、**驗證集（Validation）**、**開發集（Dev）**跟**測試集（Test）**差在哪裡呢？訓練集就是用來訓練模型的主要資料集，在訓練過程中，**不會拿訓練集以外的資料來更新權重**。而驗證集的概念與開發集是一樣的，用來在訓練過程中驗證訓練效果，但是不會拿來更新權重。而測試集則完全不參與任何訓練過程，只會在完成訓練之後評估最終模型的效果。

這些資料集對驗證訓練方法、調整參數或模型權重等都非常重要，考驗開發人員如何切分難度足夠均勻、足以驗證模型的資料。同時也要避免**資料洩漏（Data Leakage）**，不小心使測試資料跑進訓練資料裡面，會導致最終評估過於美化模型的問題。

22.3 訓練流程

接下來開始訓練模型的流程，首先將模型與分詞器讀取進來：

```
import torch
from transformers import LlamaForCausalLM as ModelCls
from transformers import LlamaTokenizerFast as TkCls

# 讀取 Model & Tokenizer
model_id = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
model: ModelCls = ModelCls.from_pretrained(
    model_id,
    device_map="auto",
    torch_dtype=torch.bfloat16,
)
tk: TkCls = TkCls.from_pretrained(model_id)
```

之前讀取模型時可能會加上量化相關的參數，但目前是不能直接對量化模型進行訓練的！因此這邊使用 `torch.bfloat16` 資料型態，俗稱 BF16，與一般的 FP16 不同的地方在於 BF16 可以表達的數值範圍較廣，但是小數點後的精準度較低，所以 BF16 的**廣度較高**但**精度較低**，相較於 FP32 而言也能節省記憶體。

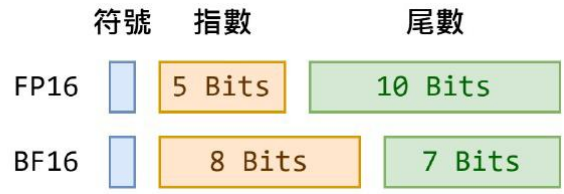


圖 22-1　資料型態 FP16 與 BF16 的格式差異

對於看似要求精準度的深度學習模型而言，使用 BF16 有什麼好處呢？在訓練結構相對深層複雜且參數量龐大的模型時，剛開始訓練時可能會產生很大的梯度，這個梯度甚至可能大到 FP16 都放不下，最後因為溢位導致 Loss 變成 **NaN**（**Not a Number**）。但如果改用 BF16 的話，因為可以表達的數值範圍更廣，所以比較不容易發生這樣的情況。

花生狂想曲

話雖這麼說，但其實選擇 BF16 是個滿自然的過程：

1. FP32 塞不進 GPU 記憶體
2. INT8 以下不能直接訓練
3. FP16 的 Loss 會 NaN 炸開

那就只剩下 BF16 可以用了但是換個角度想，至少還有 BF16 可以用

其實在參數量只有 1B 的情況下，的確可以將 FP32 的模型權重放進一張 24 GB 記憶體的 GPU 裡面，但是開始計算梯度時就會爆開了。因此將模型權重放進 GPU 裡面通常不是問題，**處理梯度的記憶體消耗才是關鍵**。

接下來讀取資料集：

```
import datasets

dataset = datasets.load_dataset(
    "json",
    data_files={
        "train": "train.tokens.json",
        "dev": "dev.tokens.json",
    },
    cache_dir="cache",
)
```

這會將資料集讀取成一個類似字典的物件，可以稍微檢查一下他的內容：

```
for data in dataset["train"]:
    print(data["input_ids"])
    print(data["labels"])
```

剛開始在學習訓練模型時，經常遇到訓練效果不佳的問題，結果回頭一檢查發現資料格式根本都不對，所以訓練之前一定要仔細檢查資料內容喔！

小心，樂高就在你腳邊

將 `cache_dir` 設定為 `"cache"` 時，會在工作目錄底下產生一個 `cache` 資料夾用來存放快取檔案。有時候就算更新了資料集，套件也會判定你沒改，導致你以為你用了新的資料格式訓練，但其實根本沒有。這時就可以手動把 `cache` 資料夾刪掉來強制套件重新快取。

接下來設定訓練參數：

```
from transformers import TrainingArguments

# Supervised Fine-Tuning
output_dir = "Models/TwAddr-SFT"
train_args = TrainingArguments(
    output_dir,
    per_device_train_batch_size=4,
    evaluation_strategy="epoch",
)
```

設定 `evaluation_strategy="epoch"` 指定每個**訓練週期（Epoch）**結束後就進行一次驗證評估。在這個簡單的玩具任務裡面，只需要設定這幾個參數即可。但其實還有相當多訓練參數可以設定，以下介紹一些常用的參數：

`per_device_train_batch_size` 與 `per_device_eval_batch_size` 分別設定訓練與評估時的批次大小，是最主要影響記憶體用量的參數。

通常單卡在訓練 LLM 時批次大小都沒辦法開很大，這樣其實容易造成訓練效果不好。這時就需要搭配 `gradient_accumulation_steps` 參數，可以設定每累積幾個 Step 的梯度再計算一次反向傳播。

假設批次大小為 4 而累積步數為 8，那就會在訓練 `4 * 8 = 32` 筆訓練資料後再做一次反向梯度，這樣可以達到接近批次大小設定為 32 的效果。

有時候一個訓練週期需要花相當多步數來進行，因此可以設定 `evaluation_strategy="steps"` 來提高評估的頻率，並搭配 `eval_steps` 參數來決定每過幾步要做一次評估。

透過 `save_strategy="steps"` 與 `save_steps` 來指定每幾步要存一個檢查點，若 `save_strategy` 設定為 `"epoch"` 時則是每個訓練週期都會存一個檢查點。啟用檢查點時，會在輸出目錄底下產生類似 `checkpoint-25`、`checkpoint-50`、`checkpoint-75` 之類的資料夾，每個資料夾裡面都會存一份完整的模型權重在裡面。如果訓練意外中斷時，可以從這些檢查點來恢復訓練狀態。

除此之外，使用驗證開發資料集除了用來監測模型有沒有訓練壞掉以外，也能用來尋找模型效果的最佳點。搭配參數 `load_best_model_at_end=True` 使用，可以在訓練結束後，將評估結果最佳的模型讀取出來。另外，為了避免硬碟被這些檢查點塞爆，可以設定 `save_total_limit` 來決定只保留最近或最佳的幾個檢查點。

還有 `learning_rate` 學習率參數，用來調整權重每次更新的幅度，通常參數量越大，學習率會設定的越低。而 `optim` 參數則可以選擇 Optimizer 的種類，至於有哪些 Optimizer 可以使用呢？可以在 `OptimizerNames` 裡面查看：

```
from transformers.training_args import OptimizerNames
```

在 VSCode 裡面，把鍵盤游標移到 `OptimizerNames` 上面後，按下 F12 可以查看類別底下有哪些定義：

```
# site-packages/transformers/training_args.py

class OptimizerNames(ExplicitEnum):
    ADAMW_HF = "adamw_hf"
    ADAMW_TORCH = "adamw_torch"
    ADAMW_TORCH_FUSED = "adamw_torch_fused"
    ...
```

雖然選擇很多，但是通常只要用預設的 `"adamw_torch"` 就可以囉！最後這些阿里阿雜的參數堆疊起來後，可能就會長成這樣：

```
train_args = TrainingArguments(
    output_dir,
    per_device_train_batch_size=4,
    per_device_eval_batch_size=4,
    gradient_accumulation_steps=2,
    evaluation_strategy="steps",
    save_strategy="steps",
    eval_steps=25,
    save_steps=25,
    save_total_limit=3,
    num_train_epochs=3,
    learning_rate=5e-5,
    optim=OptimizerNames.ADAMW_TORCH,
    load_best_model_at_end=True,
)
```

雖然看起來很多很雜，一開始可能不知道怎麼設定，但是針對自己的使用情況多多嘗試，就會漸漸熟悉各個參數的設定訣竅囉！

> **花生狂想曲** 💡
>
> 好的框架往往都會有很多參數提供開發者彈性調整，但太多的參數可以設定反而容易讓初學者心生畏懼或者無所適從，因此是否有完整的文件與適當的教學指南便相當重要！

決定好參數之後，就可以開始訓練模型了！

```
from transformers import Trainer

# 開始訓練模型
trainer = Trainer(
    model=model,
    args=train_args,
    train_dataset=dataset["train"],
```

```
    eval_dataset=dataset["dev"],
)
trainer.train()

# 儲存訓練完的模型
trainer.save_model()

# 也另外存一份分詞器方便評估
tk.save_pretrained(output_dir)
```

在一張 RTX 3090 上不用幾秒鐘就可以完成訓練了，訓練結束之後，別忘了把模型跟分詞器都存下來，方便後續的評估驗證。

22.4 評估測試

評估測試是模型訓練中相當重要的一環，用來確保模型並沒有被訓練壞，而且有呈現預期的訓練效果。這個環節分成固定測試集與自由輸入測試兩種。可以借助 vLLM 的力量，對模型進行有效率的大量評估：

```
from vllm import LLM, SamplingParams

# 讀取模型
model_id = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
llm = LLM(model_id, dtype="float16")
tk = llm.get_tokenizer()

# 建立測試集的提示列表
prompts, items = list(), list()
for out, ans in iter_dataset("test.json"):
    prompt = tk.apply_chat_template(
        [dict(role="user", content=template.format(out))],
        add_generation_prompt=True,
        tokenize=False,
    )
    prompts.append(prompt)
    items.append(ans)

# temperature 設為 0.0 表示 Greedy Decode
```

```
# 用來確保每次實驗的結果都是一樣的
sampling_params = SamplingParams(
    max_tokens=512,
    temperature=0.0,
    stop=["}"],
)

# 對所有 Prompt 同時進行推論
outputs = llm.generate(prompts, sampling_params)

# 評估生成結果
results = list()
for out, ans in zip(outputs, items):
    out = out.outputs[0].text

    # 尋找 JSON 段落
    if "{" in out:
        i = out.index("{")
        out = out[i:] + "}"

    # 解析成字典
    try:
        ans = json.loads(ans)
        out = json.loads(out)
    except:
        pass

    results.append(out == ans)

    # 觀察錯誤輸出
    if out != ans:
        print(ans, out)

# 計算準確率
accuracy = sum(results) / len(results)
print(f"Accuracy: {accuracy:.2%}")
```

先來評測看看原本的 `TinyLlama/TinyLlama-1.1B-Chat-v1.0` 準確率是多少：

```
Accuracy: 71.50%
```

在進行微調前，準確率約七成，觀察模型的輸出，很多都是答非所問。接下來看看微調過後的模型效果如何：

```
Accuracy: 98.33%
```

登登登，效果大幅提昇🎉這時可以試試看，如果將提示裡面的「臺北市中正區八德路」範例移除會怎樣呢？

```
template = "請將以下路名解析為 JSON 格式。\n\n輸入：{}"
```

評測出來的準確率：

```
Accuracy: 98.00%
```

準確率居然還是很高！很顯然我們的微調讓模型留下了一個刻板印象：如果要將路名解析成 JSON 格式，則必定是這種 `city-town-road` 的鍵值組合，而不會是什麼 `district` 或 `street` 之類的。

這樣的結果到底是好還是壞呢？在一些比較積極強烈的特殊領域訓練裡面，通常是不太在意領域外能力下降的問題，但也不能破壞的太誇張。因此能用自由測試的評估方式，來看看模型原本的能力是否還健在：

```
from vllm import LLM, SamplingParams

model_name = "Models/TwAddr-SFT"
llm = LLM(model_name)
tk = llm.get_tokenizer()

sampling_params = SamplingParams(
    max_tokens=512,
    temperature=0.0,
)

while True:
    message = input(" > ")
    prompt = tk.apply_chat_template(
```

```
        [dict(role="user", content=message)],
        add_generation_prompt=True,
        tokenize=False,
    )
    outputs = llm.generate(
        [prompt],
        sampling_params,
        use_tqdm=False,
    )
    print(outputs[0].outputs[0].text)
```

測試結果如下：

```
 > 你好啊
{"user": "臺北市中正區八德路", "assistant": "臺北市中正區八德路"}
 > 什麼是語言模型
{"user": "臺北市中正區八德路", "assistant": "臺北市中正區八德路"}
```

完蛋了，我們的模型滿腦子只剩下解析路名了！這就是模型訓練中惡名昭彰的**過度擬合（Overfitting）**，模型因為過度學習，導致他只認得訓練任務的東西。

如果要解決過度擬合，在這個玩具任務裡面可以嘗試減少訓練資料量，因為訓練資料都是單一格式，這樣的重複性資料對模型來說是比較沒有幫助的。更理想的解決辦法是**加入更多元的訓練資料**，例如打招呼、基本問答等等。

因此訓練語言模型的核心思維就是：**減少重複性，增加多元性！**

除了從資料量下手以外，也可以從參數量下手，例如只調整模型的最後一層：

```
# 先凍結所有權重
model.requires_grad_(False)
# 只更新最後一層的權重
model.model.layers[-1].requires_grad_(True)

trainer = Trainer(model, ...)
trainer.train()
```

但是這樣的方法在面臨複雜的任務時，學習效果往往不佳。當模型的某個能力有所成長時，經常會使另外一個能力有所消退。因此微調這門學問更多時候是在探討，如何在**訓練模型學習新能力之餘，盡可能維持住原本既有的能力**。

22.5 結論

本章節帶大家瞭解了如何微調一個語言模型，雖然只是個小規模的實驗，但能讓我們更加熟悉資料處理的流程。而且訓練速度相對快，能縮短實驗的週期，並快速的嘗試各種不同的參數，也比較容易累積一些基礎的訓練經驗。

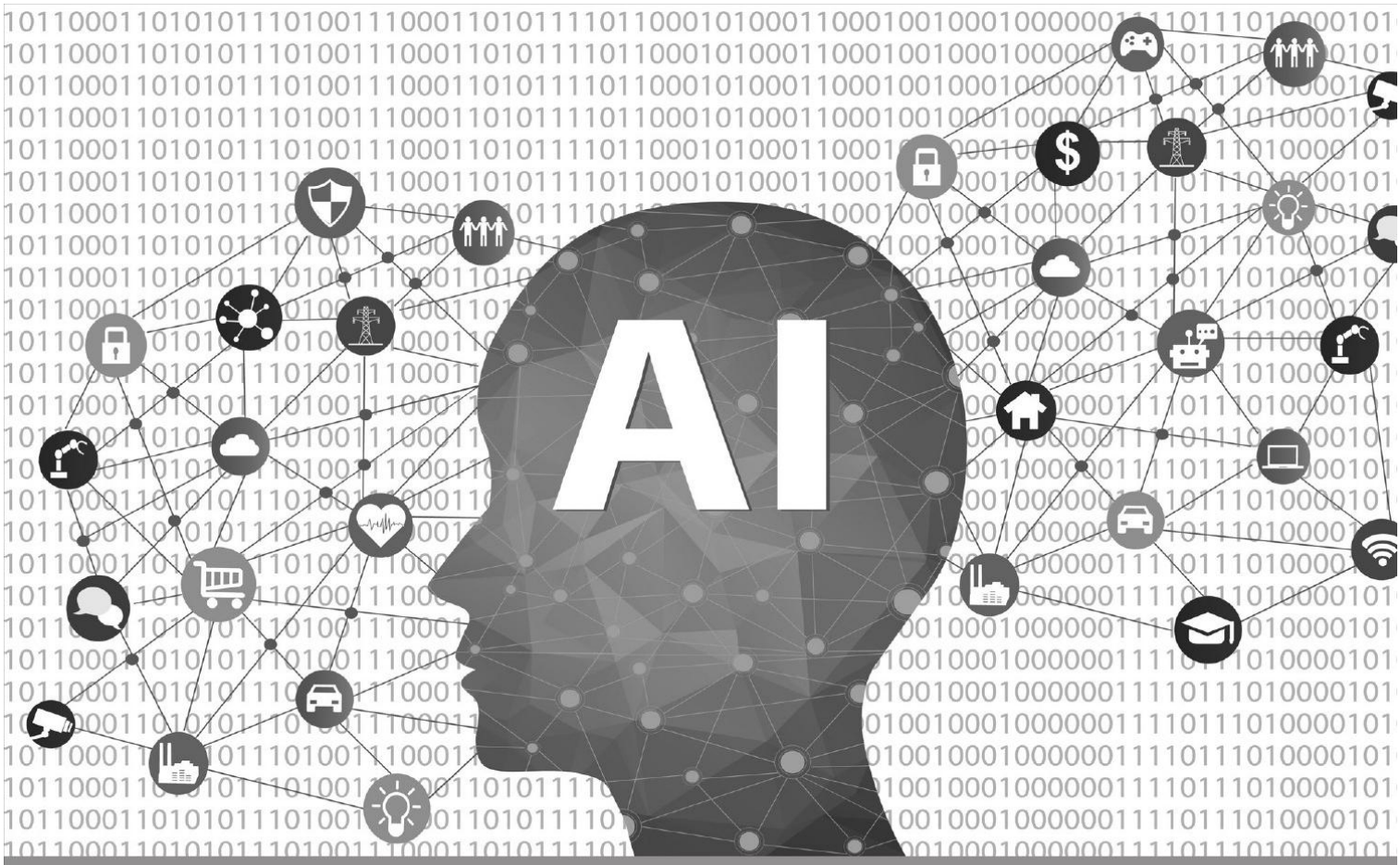
不過這次只有訓練 TinyLlama 1B 模型，但現在外面的 LLM 少說都 7B、13B 起跳啊！為什麼不來訓練參數量更大一點的模型呢？於是將目標模型改為 Llama 3 8B 並嘗試訓練：

```
torch.cuda.OutOfMemoryError: CUDA out of memory.
```

搭拉，記憶體不足！本章節介紹的訓練方法為**全參數微調（Full Fine-Tuning, FFT）**，是最傳統的訓練方法，但也因為所有的參數都需要調整，所以是訓練成本最高、記憶體吃最兇的方法。

而且 Transformers 架構隨著訓練資料的長度越長，GPU 記憶體的消耗還會平方成長上去。即便這份路名資料集的長度並不是很長，但是在單張 24 GB 的 GPU 上依然無法進行訓練，更不用說是長度兩千、四千以上的資料集了。

雖然章節末段提到了只訓練少量層數的方法，但是對於複雜一點的任務而言，只訓練一點點的權重是不夠的。不過不用擔心，那個神秘的笑臉將再度出手，拯救我們這些單顯卡的貧民玩家們！



Chapter

23

PEFT & LoRA

主管 @tag 我

@penut 我們買不起 H100，怎麼訓練模型？

不用擔心，用 PEFT 打倒 NVIDIA ！

在單張消費級顯卡上**全微調（Full Fine-Tuning, FFT）**一個 7B 參數量以上的模型是相當困難的，這時神秘的笑臉再次出手拯救了我們。由 Hugging Face 開發的 PEFT[1]（**Parameter-Efficient Fine-Tuning**）套件集合了許多具有**參數效率**的微調方法，讓我們可以不做 FFT 也能對一個高參數量的模型進行訓練。其中 LoRA 就是個相當受歡迎的方法，本章節將以 LoRA 為主，介紹 PEFT 的訓練方式。

23.1 LoRA 簡介

在原本的 HF Transformers 套件中，一般的訓練流程大致如下：

```
# 讀取模型
model = ModelCls.from_pretrained(...)

# 設定參數
train_args = TrainingArguments(...)
trainer = Trainer(model, ...)

# 開始訓練
trainer.train()
trainer.save_model()
```

1 https://github.com/huggingface/peft

PEFT 的訓練方法可以很輕鬆的整合進去：

```
# 讀取模型
model = ModelCls.from_pretrained(...)

# 使用 PEFT LoRA
peft_config = LoraConfig(...)
model = get_peft_model(model, peft_config)

# 設定參數
train_args = TrainingArguments(...)
trainer = Trainer(model, ...)

# 開始訓練
trainer.train()
trainer.save_model()
```

只需要加上兩個操作，就能輕鬆使用 PEFT 進行訓練，完整程式碼如下：

```
import torch
from peft import LoraConfig, TaskType, get_peft_model
from peft.tuners.lora import LoraModel
from transformers import LlamaForCausalLM as ModelCls

# 讀取原始模型
model_name = "meta-llama/Meta-Llama-3-8B-Instruct"
model: ModelCls = ModelCls.from_pretrained(model_name)

# 讀取 Peft Model
peft_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8,
    lora_alpha=8,
    lora_dropout=0.1,
    target_modules=["q_proj", "k_proj"],
    modules_to_save=["embed_tokens"],
)
model: LoraModel | ModelCls = get_peft_model(model, peft_config)
model.print_trainable_parameters()

# all params: 8,033,669,120
# trainable params: 3,407,872
# trainable%: 0.0424
```

最後的 `model.print_trainable_parameters()` 可以告訴我們這個模型總共有多少參數，其中會拿來訓練的參數量又有多少。可以看到原本擁有 8B 參數量的模型，在 LoRA 的加持下只剩下 3.4M 的參數需要訓練，僅佔原本模型的 0.042% 而已，不到千分之一的比例。

原本訓練一個大型語言模型，會消耗的 GPU 記憶體為：

```
大模型權重 + 超巨大梯度
```

對一個 7B 模型做 FFT 少說也要 40~80 GB 左右！權重本身僅佔 15 GB 左右，因此權重以外的東西至少多佔了兩倍左右。而 LoRA 會凍結原本的模型，並在旁邊放一個小模型來訓練，因此套上 LoRA 之後就變成：

```
大模型權重 + 超迷你模型 + 超迷你梯度
```

因為實際上只需要訓練這個 LoRA 小模型，所以需要計算的梯度也就跟著變小。這麼神奇的 LoRA 到底是怎麼辦到的呢？接下來就來介紹一下 LoRA 的原理。

23.2 LoRA 原理

LoRA 是由 Microsoft 提出的一種訓練方法，全名為 Low-Rank Adaptation，其核心概念是**將大模型的權重凍結起來**不去訓練它，並在旁邊放一個小模型。進行運算時**將大模型與小模型的輸出合併**，但更新權重時**只更新小模型的權重**，因此只需要算小模型的小梯度就好，大幅減少記憶體需求。

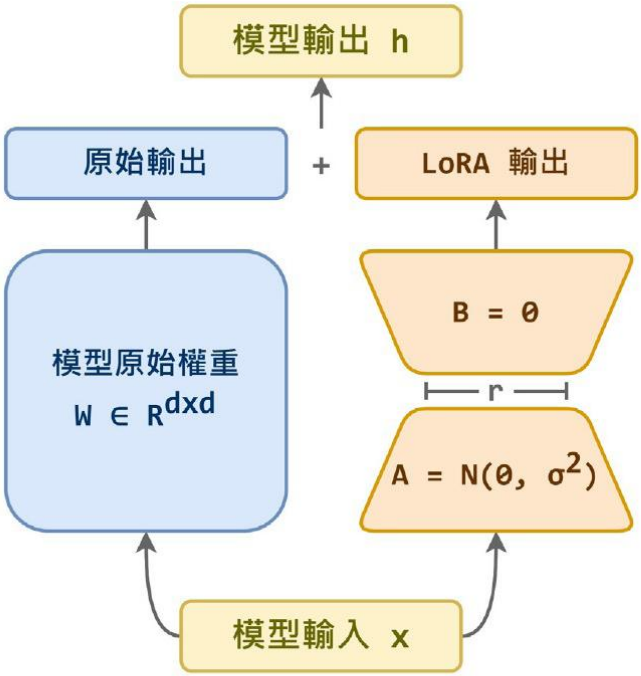


圖 23-1　LoRA 架構示意圖

23.2.1 FFT 運算

先來看看原本的 FFT 是如何進行運算：

1. 假設模型輸入的批次大小為 `4`
2. 假設原本的模型參數為 `100x100` 的矩陣 `W`
3. 那輸入便是 `4x100` 的矩陣 `I`
4. 推論時計算 `I(4x100) x W(100x100) = O₁(4x100)`
5. 結果會是一個 `4x100` 的輸出矩陣 `O₁`

這個過程中，需要更新的矩陣 `W` 擁有 `100x100 = 10,000` 個參數。

23.2.2 LoRA 運算

LoRA 會根據原本的權重，額外產生兩個矩陣 A 跟 B，這兩個矩陣被稱為 **Adapter**，運算時會將原本權重的輸出與 Adapter 的輸出相加，合併成新的輸出：

1. 設定一個參數 `r` 為 `8`
2. 根據此參數，原本 `100x100` 的權重會產生：
   a. `100x8` 的矩陣 `A`
   b. `8x100` 的矩陣 `B`
3. 推論時計算變成 `IxAxB`
   a. `I(4x100) x A(100x8) = C(4x8)`
   b. `C(4x8) x B(8x100) = O2(4x100)`
4. 結果一樣是一個 `4x100` 的輸出矩陣 O_2
5. 將兩個輸出矩陣 O_1`(4x100)` 與 O_2`(4x100)` 相加，同樣是 `4x100` 的矩陣

因此將矩陣 `W` 拆成 `A` 跟 `B` 兩矩陣是可行的，且需要更新的參數量變成：

1. `A(100x8) = 800`
2. `B(8x100) = 800`

加起來總共 1,600 的參數量，比原本的 10,000 少了超過八成的參數量！能夠減少的參數量會根據一開始設定的 `r` 而有所不同，`r` 設定的越低，Adapter 的參數量就越小，需要更新的權重就越少，反之亦然。

23.2.3 合併運算

LoRA 還有一個很重要的特性，就是將 Adapter 與原始模型權重合併之後，並不會增加模型整體的參數量！

這時線性代數被當掉兩次的筆者就開始在想，為什麼將一個大模型與小模型合併卻不會增加參數量呢？又為什麼形狀不一樣的兩個 `AB` 小矩陣可以跟大矩陣 `W` 合併呢？讓我們來仔細分析：

1. 套上 LoRA 進行推論時，計算為 `IxW + IxAxB`
2. 根據分配律將 `I` 提出來，改成 `Ix(W + AxB)`
   a. `A(100x8) x B(8x100) = W'(100x100)`
   b. `W(100x100) + W'(100x100) = W"(100x100)`
3. 合併運算變成 `IxW"`

所以 `AB` 兩矩陣就是這樣跟大矩陣 `W` 合併在一起的，而且 `W` 與 `W"` 的形狀是一樣的，因此最後整體參數量並不會增加。

23.3 回顧 LoRA 程式碼

瞭解 LoRA 的運作原理之後，再來重新回顧一次 LoRA 的設定檔：

```
peft_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=8,
    lora_alpha=8,
    lora_dropout=0.1,
    target_modules=["q_proj", "k_proj"],
    modules_to_save=["embed_tokens"],
)
```

透過 `task_type` 可以指定不同的任務類型，LoRA 不只能用在 Decoder 的訓練上，包含 Encoder 跟 Seq2Seq 模型都是可以使用 LoRA 訓練的。

`LoraConfig` 裡面的 `r` 就是剛才介紹的 `r` 參數，用來決定 Adapter 的大小，也會一定程度的影響訓練效果。但其實滿多研究指出，這個 `r` 的影響並不是很

大，影響最大的還是訓練資料的品質，所以 `r` 通常就選個 8、16 或 32 之類順眼的數字用就好。

`lora_alpha` 則會決定 Adapter 的影響程度，也就是說 Alpha 值越高，越容易把大模型既有的能力給覆蓋掉。而 `lora_dropout` 就與一般的 Dropout 概念相同，訓練過程中隨機拋棄部份輸出，是用來對抗過度擬合用的參數。

`target_modules` 可以選擇要進行 LoRA 訓練的權重模組，例如 `q_proj` 與 `k_proj` 就是注意力層裡面 Query 與 Key 線性轉換層的權重。而 `modules_to_save` 則可以設定要額外存放的權重，例如除了 Adapter 本身以外，可能還會把 Embedding Layer 打開來訓練，那就可以用這個選項讓權重跟著 Adapter 一起存下來。

補充堅果，頭好壯壯

通常，模型權重就是一份字串對應到 `torch.Tensor` 的字典喔！所以如果你想要檢查 LoRA 是否有正確訓練在目標權重上，或者額外存放的權重是否正確，可以將檢查點（Checkpoint）底下的 `adapter_model.safetensors` 讀出來看看：

```
from safetensors.torch import load_file

adapter = load_file("path/to/adapter_model.safetensors")

for k in adapter.keys():
    print(k)
```

`safetensors` 是 Hugging Face 官方主推的模型格式，目的是增加權重儲存格式的安全性，避免程式碼注入攻擊並提昇讀寫效率，同時支援 Numpy、PyTorch、Tensorflow 等眾多框架。

接下來就能用 LoRA 對模型進行訓練囉！筆者這裡使用 Llama 3 8B 進行訓練，提醒大家因為 Llama 3 與 TinyLlama 使用的分詞器並不一樣，所以資料要記得重新斷詞喔！

23.4 LoRA 推論

完成 LoRA 訓練後，會得到一份 Adapter 權重，透過 HF Transformers 套件進行推論時，使用 PEFT 讀取模型的方式如下：

```
import torch
from peft.peft_model import PeftModelForCausalLM as PeftCls
from transformers import LlamaForCausalLM as ModelCls

base_model_path = "meta-llama/Meta-Llama-3-8B-Instruct"
lora_model_path = "Models/TwAddr-LoRA"

base_model = ModelCls.from_pretrained(
    base_model_path,
    device_map="auto",
    torch_dtype=torch.bfloat16,
)

lora_model: PeftCls = PeftCls.from_pretrained(
    base_model,
    lora_model_path,
    torch_dtype=torch.bfloat16,
)
```

進行文本生成的用法與原本的 CausalLM 大致相同：

```
from transformers import LlamaTokenizerFast as TkCls
from transformers import TextStreamer

tk: TkCls = TkCls.from_pretrained(base_model_path)
ts = TextStreamer(tk)

inputs = tk("Hello, ", return_tensors="pt").to("cuda")
lora_model.generate(**inputs, max_new_tokens=128, streamer=ts)
```

23.5 QLoRA

QLoRA[2] 是由 BitsAndBytes 作者 Tim Dettmers 提出的方法，它的原理很簡單，就是把原本參數凍結的模型量化到 4-Bit 大小，這樣就能進一步減少 GPU 記憶體的消耗，在單張 24 GB 的顯卡上甚至能微調參數量 30B 的模型！

因此在 HF Transformers 套件裡面使用 QLoRA 的方式也相當簡單，只要在讀取原始模型時放上 `BitsAndBytesConfig` 即可：

```
from peft import get_peft_model
from transformers import LlamaForCausalLM as ModelCls
from transformers import BitsAndBytesConfig

# 設定量化選項
bnb_config = BitsAndBytesConfig(load_in_8bit=True)

# 讀取模型
model_id = "meta-llama/Llama-2-13b-chat-hf"
model: ModelCls = ModelCls.from_pretrained(
    model_id,
    device_map="auto",
    quantization_config=bnb_config,
)

model = get_peft_model(model, ...)
```

後面照正常訓練流程就可以囉！以 24 GB 的 GPU 來說，通常 13B 的模型用 8-Bit 就能訓練，而 30B 的模型要用 4-Bit 才能訓練，但 70B 以上可能就沒辦法了

2 https://github.com/artidoro/qlora

補充堅果，頭好壯壯

除了 BitsAndBytes 加 LoRA 以外，其實也可以用 GPTQ 或 AWQ 加 LoRA 做訓練喔！但是在後續做權重合併的時候，還沒有一個很方便的方法可以將已量化的模型與 LoRA Adapter 做合併。

真的想這麼做的朋友，可以考慮先將量化後的模型反量化回 FP16 的格式後再去做合併，但是 GPTQ 跟 AWQ 反量化的方法都太硬派了，本書就不多做介紹了

23.6 權重合併

完成 LoRA 訓練之後產生的 Adapter 權重非常小，通常只有 20 MB 左右而已。如果要評估使用 LoRA 訓練的模型，可以使用 vLLM 的 LoRA 推論功能：

```
from vllm import LLM, SamplingParams
from vllm.lora.request import LoRARequest

model_id = "meta-llama/Meta-Llama-3-8B-Instruct"
llm = LLM(model_id, dtype="float16", enable_lora=True)

sampling_params = SamplingParams(...)
lora = LoRARequest("TwAddr", 1, "Models/TwAddr-LoRA")
outputs = llm.generate(
    prompts,
    sampling_params,
    lora_request=lora,
)
```

這樣就能輕鬆評估帶有 LoRA Adapter 的模型了！但是這裡會遇到一個問題，如果訓練時你有使用 `modules_to_save` 這個設定的話，vLLM 會拒絕你使用這種 Adapter 來進行推論，因此會需要把 Adapter 與原始模型做合併：

```
import torch
from peft import PeftModel
from transformers import LlamaForCausalLM as ModelCls
from transformers import LlamaTokenizerFast as TkCls

# 指定模型路徑
orig_model_path = "meta-llama/Meta-Llama-3-8B-Instruct"
lora_model_path = "Models/TwAddr-LoRA"
output_dir = "Models/TwAddr-Merged"

# 讀取原本的模型
base_model = ModelCls.from_pretrained(
    orig_model_path,
    torch_dtype=torch.bfloat16,
)

# 讀取 Peft 模型
lora_model: PeftModel = PeftModel.from_pretrained(
    base_model,
    lora_model_path,
    torch_dtype=torch.bfloat16,
)

# 將 LoRA 權重合併到原本的模型裡面並存下來
lora_model = lora_model.merge_and_unload()
lora_model.save_pretrained(output_dir)

# 分詞器也順便存一份
tk: TkCls = TkCls.from_pretrained(orig_model_path)
tk.save_pretrained(output_dir)
```

如此一來，就能獲得完整權重的合併模型了！合併完之後的模型就可以跟一般的模型一樣操作，例如轉換成 `.gguf` 格式或是進行 GPTQ 訓練等等。

小心，樂高就在你腳邊

要特別注意，即便你是使用 QLoRA 的方式進行訓練，在合併權重時也必須用 FP16 或者 BF16 來讀取原本的模型，因為 LoRA Adapter 是不能合併到一個 INT8 模型上的。

那這個使用 LoRA 微調的 Llama 3 8B 效果如何呢？

```
Accuracy: 100.00%
```

居然全對啦！這時上個章節過度擬合的恐懼浮現心頭，於是來自由測試看看：

```
 > 你好！
😊 你好！
 > 使用繁體中文解釋什麼是語言模型？
語言模型（Language Model）是一種人工智慧 ...
```

好險，看來原本的能力並沒有被破壞！由此可見，使用 FFT 去訓練一個模型，受限於 GPU 記憶體只能訓練到 1B 或 3B 的參數量，但效果是不如我們用 LoRA 去訓練一個 7B 模型的。

雖然 LoRA 在同參數量下，訓練效果沒辦法比 FFT 好，但是可以在硬體成本與模型效能之間達到一個理想的平衡點。除此之外，使用 LoRA 也比較不容易過度擬合，但也有人認為 LoRA 學的較少，所以忘的較少，不過這也未必是個壞事！

23.7 訓練細節

通常參數量越大，學習率會設定的越低，但是因為 Adapter 的參數量很少，所以使用 LoRA 訓練時，學習率會設定的比較高。筆者通常會從 `1e-4` 或 `4e-5` 開始嘗試。除此之外，把 Alpha 值調高也是個提昇學習效果的方法。

雖然有很多人指出 LoRA 的訓練效果不如 FFT 來得好，但 FFT 除了訓練成本高以外，對於剛接觸訓練的開發者而言，也是比較容易因為過度擬合而導致訓練失敗的方法，必須多累積一些實戰經驗才會比較知道如何避開。

而 LoRA 訓練成本低、速度快，且既有的模型權重依然保留，還能發揮一部分的影響力，所以 LoRA 訓練比較不容易失敗，有時更容易得到相對理想的結

果。不過這也只是筆者的自身經驗談而已，到底使用哪個方法比較好，取決於任務的設計、難度與硬體環境。

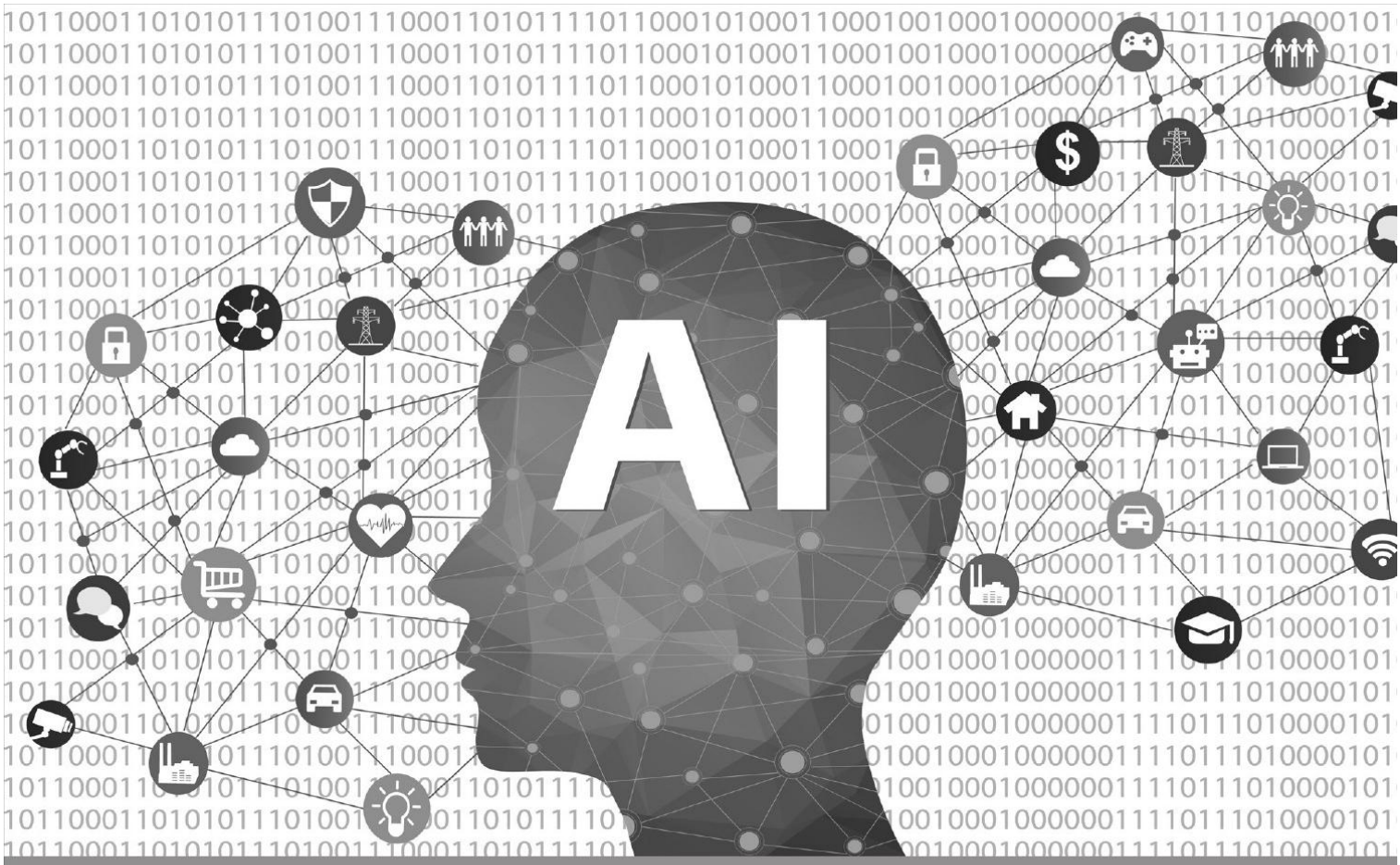
LoRA 本身也有很多變種做法，可以透過設定檔去嘗試。而且在 PEFT 裡面其實也不只收錄了 LoRA 這種方法而已，另外還有 P-Tuning 和 $(IA)^3$ 之類的訓練手法，大家也都可以去嘗試看看。

23.8 結論

本章節介紹了偉大笑臉製作的 PEFT 套件以及 LoRA 訓練方法，使得高參數量模型的訓練更加平易近人，對於推動整個開源社群參與大型語言模型的開發相當有幫助。加上量化技術加持的 QLoRA 訓練法，更是進一步減少了訓練時的記憶體消耗，使得單卡訓練更大規模的語言模型不再是空談。

即便如此，到這裡依然會遇到另外一個問題。雖然使用 LoRA/QLoRA 能夠跑得動這個玩具實驗，但是如果把訓練資料長度拉長時，記憶體又再度爆開了。

筆者實測用 Llama 2 13B 4-Bit 訓練到 1K Tokens 以上就不行了，但 1K、2K 的長度對現在的應用而言侷限性實在太大。若想要再往上訓練，該怎麼做呢？下個章節就來探討如何訓練長文本的模型。



Chapter

24

永遠不夠長！

上下文長度（Context Length / Context Window）指的是一個模型的輸入加上輸出最多可以處理到多少個 Tokens。例如一開始的 LLaMA 只支援到 2048 個 Tokens，後來的 Llama 2 則支援到 4096 個 Tokens，模型的可用性就大幅提昇了，到現在 Llama 3 已經支援到 8192 Tokens，雖然已經滿大了，但面對某些長文本的應用還是略顯不足。

模型的上下文長度對中文應用尤其重要，因為中文所佔用的 Token 數量通常都比英文來得多，若是需要檢索的應用，提示裡面加上檢索出來的文件，消耗的 Token 數量是相當可觀的。那語言模型到底如何拓寬上下文長度呢？

24.1 ChatGPT & Claude

目前 OpenAI 的 GPT-3.5 可以支援到 16K，而 GPT-4 則支援到 128K，其實對一般應用來說 16K 已經相當足夠，況且 GPT-4 還比 GPT-3.5 貴個 10 到 20 倍左右。

如果會特地用到長上下文模型的應用，消耗的 Token 數量鐵定也是特別高！長期用下來其實成本是滿高的，現階段要創造出與成本匹配的收益，是不太容易的。

來自 Anthropic 的 Claude 最高可以支援到 200K，是個相當驚人的長度，筆者實際輸入幾篇非常長的文章，其效果是真的挺不錯的，而且處理速度也很快。

花生狂想曲

Claude 一開始最令筆者不滿的地方在於模型的回覆，會把全形與半形標點符號混用，令人強迫症大爆發

但是 Claude 最近的更新開始逐漸改善這個問題了，非常期待 Claude 不再全形半形傻傻分不清的那天到來！

雖然這些模型支援的長度都很長，但可惜的是這些模型也都沒有發表他們實際的訓練手法，也有可能他們純粹就是拉長訓練資料的序列長度，反正他們的機器也是夠多夠硬。但這種做法對一般個人或中小型研究單位而言不太實際，於是大家開始思考有沒有更低成本又能拉長長度的方法。

24.2 Gradient Checkpointing

在訓練長序列模型時，最大的問題就是梯度，當訓練資料的長度越長，反向傳播時花在梯度上的 GPU 記憶體用量就會越多。幸好 2016 年由 Tianqi Chen 等人提出了**梯度檢查點（Gradient Checkpointing）**的概念，可以減少梯度所消耗的記憶體用量，雖然會提昇計算量，但能夠克服長序列訓練的瓶頸，推薦可以看看這篇文章[1] 的介紹。

1 https://github.com/cybertronai/gradient-checkpointing

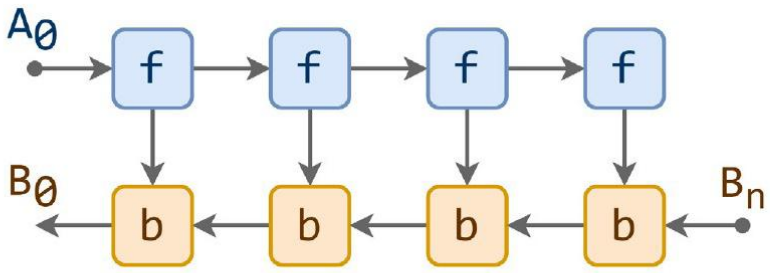


圖 24-1　一般訓練流程的梯度方向

輸入 A 會依序拜訪模型的每一層，並各自計算出一個輸出狀態。每一層的結果都算完之後，可以得到一份梯度，並根據這份梯度往回做反向傳播運算，從尾到頭傳播到模型的每一層後，完成一次模型權重的更新。

以上圖為例，算到最後一個節點時，需要把八個節點的狀態全部放在記憶體裡面，因此是最消耗記憶體的做法，但也是速度最快的方法。

另外一種做法是把輸出狀態全部捨棄，在進行反向傳播時，每回頭計算一個節點，就重新計算一次前面所有節點的輸出。

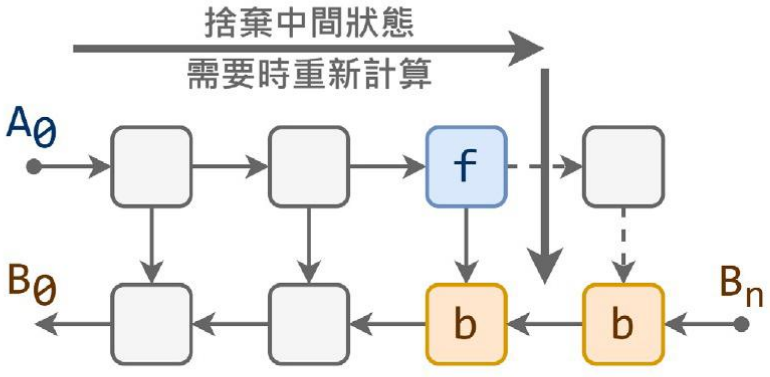


圖 24-2　低記憶體反向傳播

每次反向傳播，只會記住一兩個節點而已，相比於原始的反向傳播，減少了相當多的記憶體用量。但缺點非常顯而易見，不僅需要反覆的配置與釋放記憶體，整體計算量也大幅提昇。

在梯度檢查點機制裡面，選擇了一個折衷的方案：沒有全部記下來，也沒有全部忘掉，只保留幾個節點的輸出。

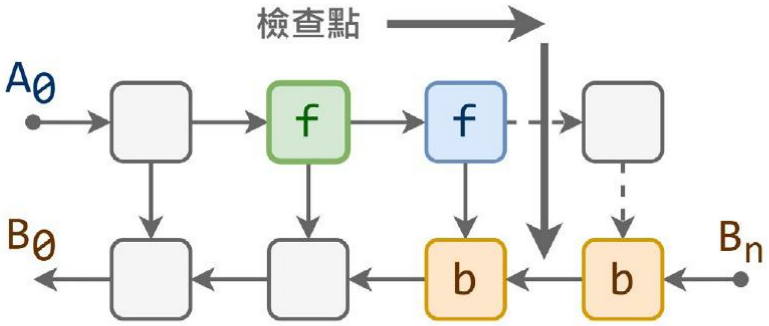


圖 24-3　梯度檢查點

挑選幾個中間的節點當作檢查點，把這幾個檢查點的狀態記下來放在記憶體裡面，遇到沒有存狀態的節點，就從最近的檢查點開始計算。這樣雖然會提高一點記憶體用量，但是計算量比第二種做法低的多。

那實際上該如何使用梯度檢查點來進行模型訓練呢？首先在讀取模型時，需要幫模型設定 `use_cache=False`：

```
from transformers import LlamaForCausalLM as ModelCls

model = ModelCls.from_pretrained(..., use_cache=False)
model.config.use_cache = False  # 或者這樣設定
```

接著開啟 `enable_input_require_grads` 與 `gradient_checkpointing_enable` 這兩個設定：

```
model.enable_input_require_grads()
model.gradient_checkpointing_enable()
```

最後在訓練參數裡面設定 `gradient_checkpointing=True`：

```
from transformers import TrainingArguments

TrainingArguments(..., gradient_checkpointing=True)
```

筆者實測使用長度 512 的資料對 Llama 2 13B 8-Bit 做訓練，權重本身約佔 13 GB，在沒有開啟梯度檢查點的情況下，總共需要消耗 19 GB 的 GPU 記憶體，但如果有開梯度檢查點的話，總記憶體消耗降低到 14 GB 而已。記憶體用量大幅減少後，便能用長度更長的資料進行訓練，或者加大批次大小來提昇訓練效果。

然而原本幾秒鐘就能完成的訓練，開啟梯度檢查點後需要兩分多鐘，可見計算量提昇了相當多。因此梯度檢查點就是個**以時間換取記憶體**的訓練方法，但是這個方法可以克服記憶體瓶頸，大幅拉長可訓練的長度，增加模型的可用性。

24.3 內插法 & 外插法

在 Llama 架構裡，使用**旋轉相對位置編碼（Rotary Positional Embedding, RoPE）**來表達位置資訊，RoPE 主要使用餘弦（Cosine）函數來表達相對位置的資訊，因此可以用餘弦線圖來當作 RoPE 的示意圖。如果要將 RoPE 的長度拉大，最簡單的做法就是直接**外插（Extrapolation）**上去，把餘弦函數接續下去。

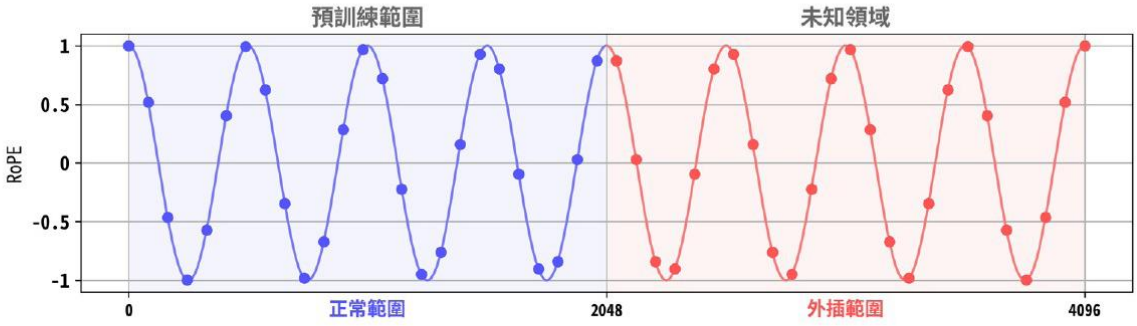


圖 24-4　外插示意圖

但是實際使用時發現外插效果並不如想像中的好，於是有人改用**內插（Interpolation）**的方式，把餘弦函數給**拉寬**到指定長度。

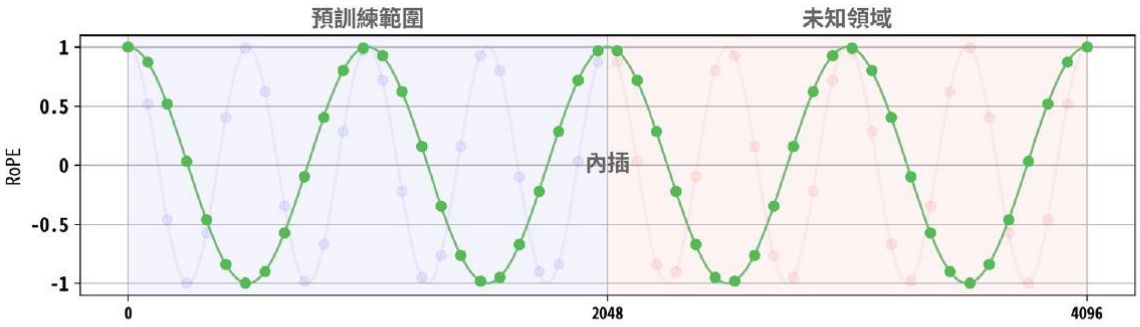


圖 24-5　內插示意圖

因此新的餘弦波週期還是一樣的，但是可以覆蓋的範圍變廣了。就像是幫相機裝上廣角鏡頭一樣，可以看到的視野變寬廣了，但是拍出來的照片解析度還是一樣的，因此在細節上會看的相對模糊，不過可以看到的風景樣貌整體上變多了。

還有一種做法是透過**調整基礎頻率（Adjusted Base Frequency, ABF）**的做法來改造 RoPE，原本 2K 的長度裡面餘弦波會有四個週期，但是透過 ABF 把頻率提高，在 2K 的長度內塞進八個週期之類的。因此 RoPE 的粒度更細，可以學到相對細節的資訊，使模型更有辦法分辨不同位置之間的相對資訊。

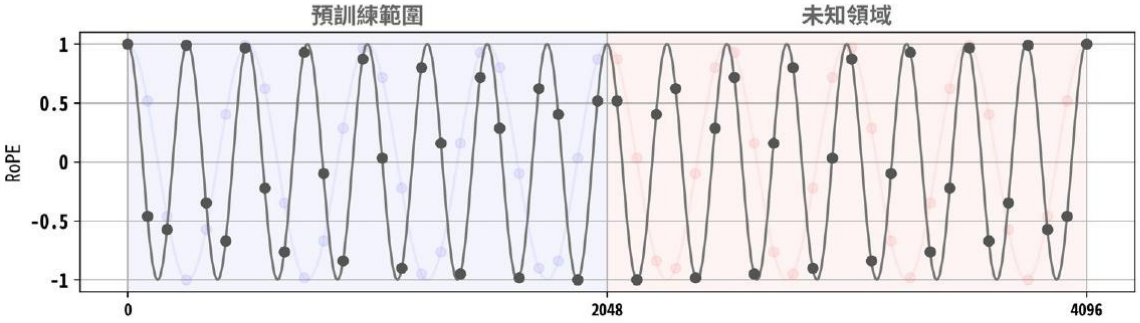


圖 24-6　調整基礎頻率示意圖

那實際上，我們要如何對 RoPE 做這些調整呢？在 HF Transformers 套件裡面，可以透過 `rope_theta` 與 `rope_scaling` 兩個參數來調整：

```
from transformers import LlamaForCausalLM

LlamaForCausalLM.from_pretrained(
    "meta-llama/Meta-Llama-3-8B-Instruct",
```

```
    rope_theta=5e3,
    rope_scaling={"type": "linear", "factor": 2},
)
```

其中 `rope_theta` 就是用來調整基礎頻率的參數，因為會取倒數，所以 `rope_theta` 設定的越小，產生的頻率就會越高。而 `rope_scaling` 則是對位置做縮放，以 `factor=2` 為例，原本在 1024 的旋轉向量，會被放到第 2048 個位置，也就是內插法。

可以使用之前量化章節寫過的困惑度評測程式，來觀察這些位置編碼的縮放對困惑度的影響，例如 TinyLlama 只有訓練 2K 的上下文長度，因此使用 2K 長度的序列測困惑度得到的結果為 8.0234，但是測 4K 序列長度，困惑度居然上升到 139，若是測到 8K 的話更是暴漲到 1291 ！

這時試著將 Scaling Factor 設定為 2 並且重新測試，4K 的困惑度就被壓到 20 左右了！若是把 Scaling Factor 設定為 4 並測試 8K，則困惑度可以壓到 118 左右。雖然說困惑度有所下降，但其實這個困惑度也根本沒辦法用，生成出來的東西依然亂七八糟。但是因為有做擴展設定的模型困惑度比較低，所以 Loss 相對也會比較低，因此若是要用長序列的訓練資料做微調，可以讓訓練更快收斂。

花生狂想曲 💡

但是很顯然的，只調整 RoPE 設定的話，並沒有辦法真正擴展模型的上下文長度。

24.4 LongLoRA

LongLoRA[2] 是一種結合 LoRA 來擴展模型長度的方法，他們提出 **Shift Short Attention（S^2-Attn）**的方法來減少注意力機制的計算量，並結合 LoRA 減少記憶體消耗量，使我們能夠用更少的硬體成本來訓練更長的模型。

原作者實驗發現，如果單純只把模型長度提高而不去做額外的微調，那模型的效果就會很差，但是直接用長序列的資料進行訓練，需要消耗的記憶體又太高了！因此作者提出 S^2-Attn 與 LoRA+ 結合的 LongLoRA 訓練方式，讓我們可以用更少的記憶體來訓練更長的模型。這個 S^2-Attn 的做法大致如下：

1. 將 Attention Heads 切成兩半。
2. 將其中一半的 Attention 位移（Shift）一點點。
3. 將所有 Attention 切成好幾個群組（Group）。
4. 進行 Group-Wise Self-Attention 運算。

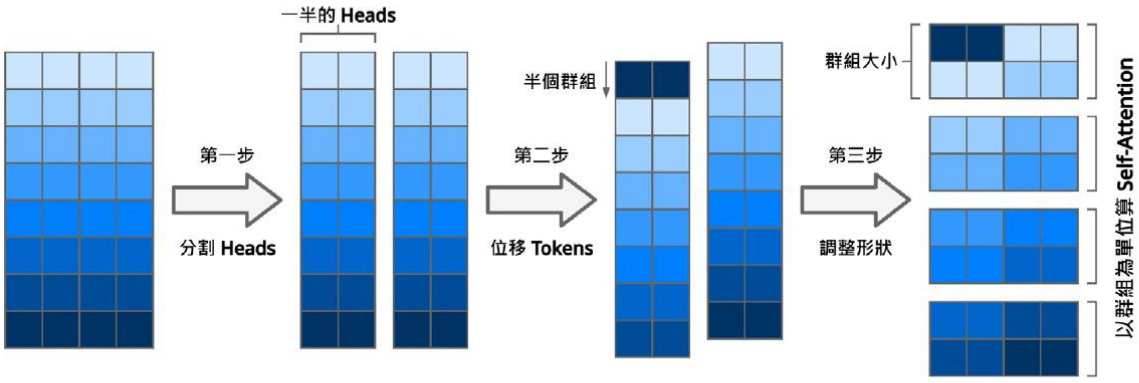


圖 24-7 S^2-Attn 示意圖

位移的目的是要讓群組之間的資訊可以交流，能夠提昇模型訓練的效果。而切成群組做 Group-Wise Self-Attention 則可以降低注意力層的計算量。作者強調

2 https://github.com/dvlab-research/LongLoRA

S^2-Attn 不僅能減少訓練階段的計算量，訓練出來的模型在推論階段也能夠直接使用原本的 Full Attention 進行運算，因此這個方法並不需要修改模型架構。

最後 LoRA+ 其實就是把詞向量與正規化層的權重打開來訓練：

```
model = ModelCls.from_pretrained(...)
model = get_peft_model(model, ...)

for name, param in model.named_parameters():
    if "embed" in name or "norm" in name:
        param.requires_grad_()
        break
```

其中 `embed` 與 `norm` 在不同模型會有不同名稱，需要根據實際情況做調整，這裡是以 Llama 模型的名稱為例。筆者使用 Llama 3 8B 模型測量了一下，如果是使用原版 LoRA 的話，Adapter 的參數量約為 3.4M，但是改用 LoRA+ 的話，Adapter 的參數量會提升到 528M。雖然說參數量大了不少，但這也僅是整個模型 6.5% 的參數量而已，所以需要訓練的參數量依然維持的很少。

24.5 StreamingLLM & Attention Sinks

過去有些開發者可能會採用 Window Attention 的做法來減少長序列輸入的運算量，也就是只計算過去一段固定長度的序列，超過的就通通丟掉不算。但是這樣的做法被證實是不可行的，模型會開始亂輸出。所以又有另外一種做法是結合 Sliding Window 的機制，將整個 Window 滑動一段距離後，重新計算一次所有注意力分數。這樣的做法雖然不會亂吐輸出，但是遇到需要重新計算的地方時，使用者會明顯感受到很長的停頓。

StreamingLLM[3] 的作者在探討 Window Attention 為何會失敗的過程中，觀察到一個相當有趣的現象：在語言模型計算注意力分數的時候，輸入模型的**前幾**

3 https://github.com/mit-han-lab/streaming-llm

個 Token 平均都能獲得相當高的注意力分數。即便把前幾個 Token 換成換行符號這類不具備任何實質語意的 Tokens，依然會獲得相對高的分數。

作者認為 Window Attention 會失敗，就是因為這些分數高的 Tokens 被拋棄了，之所以會有這個現象，作者提出了兩個猜測：

1. 即便下個被預測的 Token **並不需要關注任何上下文**，但是因為注意力層裡面的 Softmax 運算會使注意力分數**無論何時都不會有 Token 是零分**，這個時候前幾個 Tokens 就會來幫忙吸收掉這些多餘的分數。
2. 在自回歸生成的過程中，**後續所有的 Token 都能看到前幾個 Tokens**，因此所有人都認識他們，導致前幾個 Tokens 能夠捕捉一些不重要的注意力。

作者將這前幾個 Tokens 稱為 **Attention Sinks**，Sink 指的是沉陷、沒入的意思。這些 Tokens 相當重要，就像是船錨一樣可以把整艘船穩住而不會亂飄。基於這個觀察，作者提出了 StreamingLLM 的做法，一樣是以 Window Attention 為基礎，但是在截斷注意力時把前幾個 Tokens 的注意力給保留下來。

這樣的做法非常驚人的有效果，不僅能穩住輸出不會亂生成，在推論時也相當有效率，計算量不會隨著長度而增加。因此作者建議以後大家訓練語言模型的時候，一定要多放幾個 Attention Sinks，這樣 StreamingLLM 就可以利用 Attention Sinks 來讓語言模型的處理長度變得**無限長**。這樣的發現，再次重申了 BOS Token 的重要性。

花生狂想曲

StreamingLLM 並不是真的把上下文長度變長，而是一種能讓模型穩定處理長上下文的方法。

可以透過 HF Transformers 套件的 `SinkCache` 類別來使用 Attention Sink 機制進行生成：

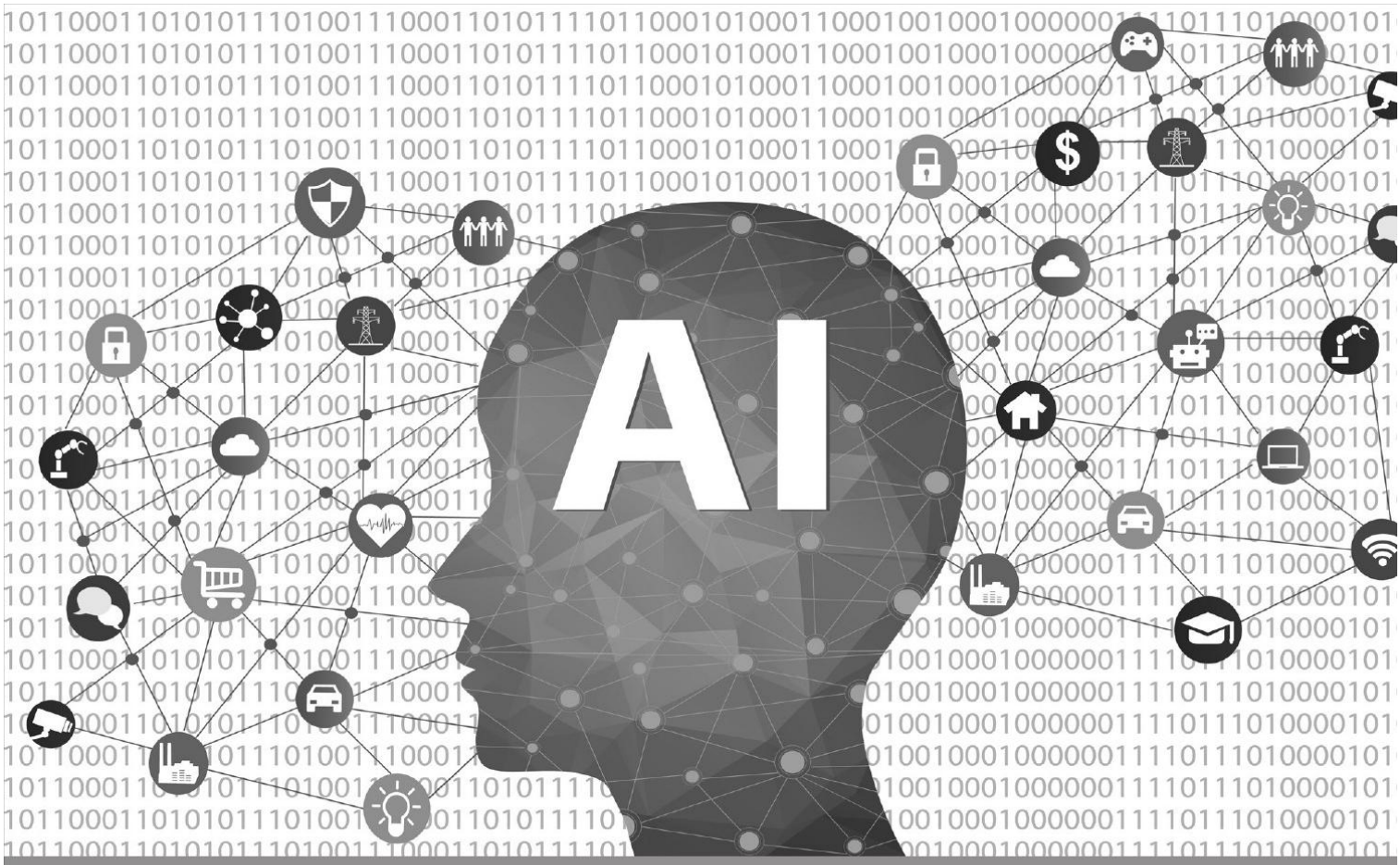
```
from transformers import AutoModelForCausalLM as ModelCls
from transformers import AutoTokenizer as TkCls
from transformers import TextStreamer, SinkCache

m = ModelCls.from_pretrained(...)
tk = TkCls.from_pretrained(...)
ts = TextStreamer(tk)

input_ids = tk(..., return_tensors="pt").to("cuda")
cache = SinkCache(window_length=32, num_sink_tokens=4)
m.generate(input_ids, past_key_values=cache, streamer=ts)
```

可以自己調整看看 `window_length` 與 `num_sink_tokens` 感受一下生成的內容有什麼不同，這兩個數值調的越高，輸出通常越穩定，但能夠節省的記憶體就會少一點。

24.6 結論

本章節介紹了一些與擴展語言模型能處理的上下文長度有關的做法，每個方法都有各自的優點與效果。但是根據筆者的經驗，不管使用哪種方法，只要輸入的文本變長之後，模型產生的回應細緻度就會開始下降，無論是像 Llama 這種小的開源模型，還是像 ChatGPT 或 Claude 這種大的商用模型，都能觀察到類似的現象。

因此面對長文本的任務，筆者還是建議盡可能去做切割，把問題的範圍縮小，或者透過檢索的方法去解決，會是比較兼具效率與準確率的做法。



Chapter

25

資料何其多，硬碟何其小

主管 @tag 我

@penut 在這個 AI 時代，決定實力的不是算力而是資料，去把資料全部下載下來！

我覺得公司的 NAS 會先被塞爆欸 ...

要訓練一個模型，首先要有資料，不僅要有很**大量**的資料，也要有**品質**很好的資料。資料的品質包含：文句是否通順、格式是否合理、內容是否偏頗、資訊是否有害等等。品質越好的資料，越有機會訓練出更好的模型。

花生狂想曲

Data quality determines model ability. 資料品質決定模型能力。

25.1 Hugging Face Datasets

由 Hugging Face 開發的 **Datasets**[1] 套件，除了可以讀寫資料集以外，也能用來下載 HF Hub 上的公開資料集。可以到資料集的**搜尋頁面**[2] 瀏覽有哪些資料集可以用。左邊的過濾器提供各種選項，包含資料集的任務、子任務、資料集規模或語言等等。

如果遇到超大型資料集，但是不想要整份下載下來的話，可以使用套件的串流功能來取樣幾筆資料看看：

1 https://github.com/huggingface/datasets
2 https://huggingface.co/datasets

```
import json
from datasets import load_dataset

ds = load_dataset(
    "togethercomputer/RedPajama-Data-1T",
    "arxiv",  # 選擇 axXiv 子集
    cache_dir="cache",  # 方便清理
    streaming=True,  # 啟用此選項，避免整份資料集被下載到硬碟裡面
)["train"]

# 只取前 10 筆出來看看
subset = [data for _, data in zip(range(10), ds)]

# 內容通常很長，所以寫進檔案裡面慢慢看
with open("subset.json", "wt", encoding="UTF-8") as fp:
    json.dump(subset, fp, ensure_ascii=False, indent=4)
```

每種資料集都有自己的讀取方式，例如 RedPajama 就需要給子資料集的選項，但其實也有滿多資料集是沒有這個選項的，因此讀取前請先參考該資料集的介紹頁面。

在使用 Python 處理資料時，經常會使用 JSON 格式，但 JSON 格式是個沒有對資料進行壓縮的格式，當資料數量很龐大的時候，存放在硬碟上會佔用很巨大的空間。因此有些人會選擇做 Gzip 壓縮，雖然壓縮率很高，但是壓縮與解壓縮的速度非常慢。

Apache Parquet[3] 是個專為機器學習任務所設計的檔案格式，Parquet 具有相當程度的資料壓縮率，但是壓縮速度比 Gzip 快的多，因此是個用來存放大型資料的好幫手。想要使用 Parquet 格式，可以透過 Pandas 套件操作：

```
from datasets import load_dataset
import pandas as pd

ds = load_dataset("wikitext", "wikitext-2-raw-v1", split="train")
```

3 https://parquet.apache.org/

```
text = [{"text": data["text"]} for data in ds]
text = pd.DataFrame(text)

text.to_parquet("wikitext.parquet")  # 儲存資料集
pd.read_parquet("wikitext.parquet")  # 讀取資料集
```

可以來比較一下 JSON、Gzip 與 Parquet 之間讀寫速度的差異，首先寫一個 `Timer` 類別協助我們測量時間：

```
import time

class Timer:
    def __init__(self, title):
        self.title = title

    def __enter__(self):
        self.delta = time.perf_counter()

    def __exit__(self, *_):
        delta = time.perf_counter() - self.delta
        print(f"{self.title}: {delta:.4f}s")
```

接著測量各種格式的讀寫速度：

```
print("=== 寫入速度 ===")
with Timer("JSON"): text.to_json("text.json")
with Timer("Gzip"): text.to_json("text.gz", compression="gzip")
with Timer("Parquet"): text.to_parquet("text.parquet")

print("=== 讀取速度 ===")
with Timer("JSON"): pd.read_json("text.json")
with Timer("Gzip"): pd.read_json("text.gz")
with Timer("Parquet"): pd.read_parquet("text.parquet")
```

得到以下結果：

```
=== 寫入速度 ===
   JSON: 0.0479s
   Gzip: 0.5839s
Parquet: 0.0344s
```

```
=== 讀取速度 ===
   JSON: 0.0288s
   Gzip: 0.0616s
Parquet: 0.0340s
```

可以看到 Gzip 真的慢非常多，而 Parquet 幾乎跟 JSON 差不多快！那這些格式的文本壓縮率如何呢？

```
text.json    11.0 MB
text.gz       3.9 MB
text.parquet  6.0 MB
```

可以看到 Gzip 是壓縮最多的，但 Parquet 可以在幾乎無損的讀寫速度內，達到一定程度的壓縮率，因此當資料規模逐漸增加的時候，Parquet 是個節省硬碟空間又不會損失讀寫速度的好選擇。

然而 Parquet 與其他格式的讀取邏輯相當不同，一般的 JSON 是 Row-Oriented，也就是一筆資料讀完接著下一筆，但 Parquet 是 Column-Oriented，這是什麼意思呢？讓我用一小段程式碼來解釋這個概念：

```
data[i]["text"]  # Row-Oriented 是這樣讀取
data["text"][i]  # Column-Oriented 卻是這樣讀取
```

Row-Oriented 的資料，讀取時需要先指定第幾筆，再指定要讀取該筆資料的哪個欄位。而 Column-Oriented 則是先指定要讀取的欄位，再決定要讀取第幾筆資料。這樣的讀取邏輯其實更適合機器學習的資料集，因為模型不一定會用到每筆資料的所有特徵，所以實驗時只需要讀取會用到的特徵即可。因此，若是要迭代 Parquet 資料集，可能會像是這樣：

```
text = pd.read_parquet("dataset.parquet")
for i in range(len(text)):
    text["id"][i]
    text["title"][i]
    text["article"][i]
```

雖然稍微不直觀一點，但掌握好核心概念，操作起來還是很簡單的喔！

25.2 預訓練資料集

25.2.1 Common Crawl

Common Crawl[4] 是一個大型公開的網路爬蟲專案，它會把網路上的海量資料給爬下來，並提供該網頁的文本內容，許多預訓練資料集都是以 Common Crawl（CC）為基礎建立的。

25.2.2 Project Gutenberg

Project Gutenberg[5] 谷騰堡計畫是從 1971 年開始進行的電子書計畫，目標是提供免費的電子書給大眾閱讀。此計畫的名稱是用來紀念谷騰堡，一位發明活字印刷術並推廣印刷機的人。

25.2.3 The Pile

The Pile[6] 是由 Eleuther AI[7] 提供的資料集，其主成份之一 Pile-CC 就是基於 Common Crawl 的資料集。除此之外，還有很多其他的資料，例如書籍資料 Books3、論文 arXiv、維基百科、GitHub 程式碼等等。另外 The Pile 也有包含 Project Gutenberg 的資料，常見的 PG-19 就是指 1919 年之前的經典文學書籍。

4 https://commoncrawl.org/
5 https://www.gutenberg.org/
6 https://pile.eleuther.ai/
7 https://www.eleuther.ai/

25.2.4 C4 & mC4

C4[8] 全名為 Colossal Clean Crawled Corpus，由 Google 所提出的資料集，同樣是基於 Common Crawl，並做了更進一步的資料清洗，而 mC4[9] 資料集則是後來 Google 提出的多語言（Multilingual）版本。

25.2.5 Stack Exchange

Stack Exchange[10] 是工程師好朋友 Stack Overflow[11] 的系列問答網站，除了程式設計以外還有軟硬體操作、數學、物理、化學、英文等等的領域。而 Stack Exchange 資料集就是基於這些網站爬取下來的網頁文本內容所建立的。

25.2.6 RedPajama & SlimPajama

Pajama 指的是睡衣，而 RedPajama[12] 並不是紅色睡衣，而是一份由 together.ai[13] 所統整的資料集，這份資料集主要整合了 Meta 訓練 LLaMA 時所使用到的資料集，包含 Common Crawl、C4、GitHub、Books、arXiv、Wikipedia 與 StackExchange 等，總共近 1.2T Tokens 的資料集，方便大家重現 LLaMA 的訓練結果。

SlimPajama[14] 則是由 Cerebras[15] 提出的資料集，對 RedPajama 做了更進一步的**去重複（Deduplicate）**處理，得到更精簡的 627B Tokens 的資料集。

---

8 https://huggingface.co/datasets/allenai/c4
9 https://huggingface.co/datasets/mc4
10 https://stackexchange.com/
11 https://stackoverflow.com/
12 https://huggingface.co/datasets/togethercomputer/RedPajama-Data-V2
13 https://together.ai/
14 https://tinyurl.com/llm-slim-pajama
15 https://www.cerebras.net/

補充堅果，頭好壯壯

為什麼**去重複**這麼重要呢？因為重複性的資料可能導致模型產生偏差，試想如果資料集裡面包含了 100 句「土豆是 Potato」，但只有 1 句「土豆在臺灣是 Peanut」，那模型肯定很容易偏頗在前者上。假設這兩種資料各只有一筆的話，模型就可以比較均衡的學習到更具有多樣性的資訊。

25.2.7 The Stack & StarCoderData

The Stack[16] 是由 BigCode 釋出的程式碼相關資料集，包含 358 種程式語言，有將近 6 TB 的資料量。另外還有去重複的版本 **The Stack Dedup**[17] 和 GitHub Issue 版的 **The Stack Issues**[18] 這兩種資料集。

StarCoderData[19] 則是一份基於 The Stack 的程式碼資料集，保留了其中 86 種較常見的程式語言，並加入 GitHub Issue 與 Jupyter Notebooks 等等，總共約 250B Tokens 的文本資料。後來 BigCode 又推出了 **The Stack v2**[20]，包含 600 多種程式語言，資料量更是來到驚人的 67 TB 之多。

一般在訓練 Code LLM 時，並不會真的只拿程式碼的資料去做訓練，而是會加入一些自然語言的資料一起做訓練。在一些文獻中，會以 **PL（Programming Language）**代稱**程式語言**，並使用 **NL（Natural Language）**代稱**自然語言**。

16 https://huggingface.co/datasets/bigcode/the-stack
17 https://huggingface.co/datasets/bigcode/the-stack-dedup
18 https://huggingface.co/datasets/bigcode/the-stack-github-issues
19 https://huggingface.co/datasets/bigcode/starcoderdata
20 https://huggingface.co/datasets/bigcode/the-stack-v2

25.3 對話資料集

25.3.1 Databricks Dolly

databricks-dolly-15k[21] 是由 Databricks[22] 提出的資料集，包含一萬五千筆對話問答的資料，而且這些資料是由 Databricks 上千名員工**人工手動**寫出來的！是當代相當少見的純人工資料集，可惜只有英文。

25.3.2 OASST

OASST[23] 是 Open Assistant[24] 提出的資料集，有十多萬筆資料，涵蓋多種語言，其中就包含中文。但沒有特別說是簡體中文還是繁體中文，所以高機率就是

25.3.3 ShareGPT

ShareGPT[25] 在之前的文章有介紹過，是一個用來分享 ChatGPT 對話內容的瀏覽器外掛。後來 Vicuna 團隊在 ShareGPT 的網頁上進行爬蟲獲得的 ShareGPT 資料集。但現在 ShareGPT 已經關閉爬蟲入口了，只剩別人整理好的 **ShareGPT 資料集**[26] 可以用了。

---

21 https://huggingface.co/datasets/databricks/databricks-dolly-15k

22 https://www.databricks.com/

23 https://huggingface.co/datasets/OpenAssistant/oasst2

24 https://open-assistant.io/

25 https://sharegpt.com/

26 https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered

25.3.4 LMSYS Chat

沒有 ShareGPT 沒關係，由 Vicuna 團隊釋出的 LMSYS Chat[27] 資料集，共有一百萬組對話資料，是從 FastChat Demo[28] 頁面蒐集而來，因此這些資料都是真實的對話資料。

25.4 中文資料來源

25.4.1 政府資料開放平台

政府資料開放平台[29] 提供了很多來自政府提供的資料集，例如前面章節用到的全國路名資料就是來自這裡。對於挖掘臺灣潛在應用而言，是個相當有價值的資料來源。

25.4.2 維基文庫

維基文庫[30] 收集了許多自由文本的內容，例如一些古詩詞和散文經典等等，像是三國演義、西遊記之類的。這些文本大多沒有版權問題，所以拿來訓練是沒問題的，不過這裡面使用文言文的資料居多，缺少白話文資料。

27 https://huggingface.co/datasets/lmsys/lmsys-chat-1m
28 https://chat.lmsys.org/
29 https://data.gov.tw/
30 https://zh.wikisource.org/zh-hant/

25.4.3 開放文學

開放文學[31] 是另外一個收集自由文本的網站，類似維基文庫。雖然文本量可能稍微少一些，但是對爬蟲相對友善，可以直接分析它的 API 呼叫來取得純文本。

25.4.4 漢籍全文資料庫

漢籍全文資料庫[32] 是中研院建立的文本資料庫，以經、史、子、集分類，其中史書的資料相對多一點。

25.4.5 TAIDE

國科會的 TAIDE 專案使用了大量的**繁體中文訓練資料**[33]，且來源多與臺灣相關，例如縣市政府的旅遊網站、各大新聞媒體或教育部的字典等等。雖然不是每份檔案都有結構化的資料可以下載，但也相當具有參考價值。根據 TAIDE 介紹所說，目前這些資料大約是 40B Tokens 左右，可能不算很多，但至少是個不錯的開始！

25.5 評估資料集

完成模型的開發或微調之後，要如何確定模型有沒有進步呢？這時就會需要透過**基準測試（Benchmark）**來評估與比較模型的好壞，因此需要**評估資料集**來做測試，不同的資料集會有不同的評估方法、領域、面向與語言等等。

31 http://open-lit.com/

32 https://hanchi.ihp.sinica.edu.tw/ihp/hanji.htm

33 https://taide.tw/index/training-data

生成式語言模型的評估難度相對高一些，像分類或辨識模型，通常都有標準答案可以計算準確率，但是語言模型通常是生成一段文字，諸如摘要、翻譯或對話等等，似乎都不是有標準答案的任務，那要怎麼評估呢？

最簡單的方法就是把題目變成**選擇題**，給模型數個選項，請它選出正確的選項，這種資料集包含 MMLU、CMMLU 與 TMMLU+ 等都是這樣的設計。

實際上要如何確認模型會選哪個選項呢？可以透過比較 Loss 的方法來進行：

```
choices = [
    "A banana is red.",
    "A banana is yellow.",
    "A banana is blue.",
    "A banana is green.",
]

for text in choices:
    input_ids = tk.encode(text, return_tensors="pt")
    with torch.no_grad():
        outputs = model.forward(input_ids=input_ids, labels=input_ids)
    print(outputs.loss, text)
```

這段程式在比較哪句話的 Loss 比較低，最後輸出結果如下：

```
5.9140 A banana is red.
4.9455 A banana is yellow.
5.8184 A banana is blue.
5.7485 A banana is green.
```

「香蕉是黃色的」Loss 最低，所以可以判定模型傾向於選擇這個選項。但 Loss 低未必代表模型真的會生成這個答案，因此僅能在自動評估上做個參考而已。

除了選擇題以外，也能評估答案與生成之間的**文本相似度**，例如 BLEU 或 ROUGE 分數，通常用在問答、摘要或翻譯任務上，例如 WMT 翻譯就是用這種方法評估。

最近還很流行**請更強的模型來評分**，例如問 ChatGPT 這篇文章的品質如何？或者這兩篇不同模型生成的文章，誰的品質比較好？

在 HF Hub 上可以找到相當多評估資料集來用，但是這麼多的評估資料集，每個資料集的格式不盡相同，評估手法也不一樣，該怎麼辦呢？這時可以透過 EleutherAI 製作的評估套件 `lm-evaluation-harness` 來幫我們，安裝方法如下：

```
git clone https://github.com/EleutherAI/lm-evaluation-harness
cd lm-evaluation-harness
pip install -e .
```

首先查看目前套件支援哪些評估資料集：

```
lm-eval --tasks list
```

以目前繁體中文最常用的評測資料集 TMMLU+ 為例，用法如下：

```
export HF_DATASETS_CACHE=cache

lm-eval --model_args pretrained=meta-llama/Meta-Llama-3-8B \
    --model hf --device cuda:0 --batch_size auto \
    --output_path results --num_fewshot 5 \
    --tasks tmmluplus --limit 0.01
```

透過 `--tasks tmmluplus` 參數指定使用 TMMLU+ 進行評測，設定 `--limit 0.01` 先做少量測試。通常評估會給少量樣本，因此給 `--num_fewshot 5` 來做 5-Shot 評估。將批次大小設定為 `auto`，套件會自行判斷可以設定到多大的批次大小，藉此來提昇評估的速度。

評估完之後，會在 `results` 資料夾底下長出一份 `.json` 檔，裡面存放評估測試的結果，其中 `acc` 就是準確率，而 `acc_norm` 則做了一些正規化，使不同資料集之間的評估結果可以互相比較。不同的評測也會產生不一樣的指標，可以到官方 GitHub[34] 上查看。

34 https://github.com/EleutherAI/lm-evaluation-harness

25.6 模型生成資料

在沒有太多人力資源的情況下，透過模型來蒐集資料是個不錯的方法。像是 GPT-3.5 API 每生成 1000 Tokens 的資料只需要 0.0015 美金，約新台幣 0.045 塊錢。收集一百萬 Tokens 約莫只需要 60 新台幣而已，其實超便宜的！

> 花生狂想曲
> 但是透過模型輸出來蒐集資料，需要注意模型是否存在偏頗的狀況。

像是由 Stanford 提出的 **Alpaca**[35] 專案，裡面就包含了五萬多筆模型生成的資料集。Alpaca 是最常見的一種做法，首先設定一些種子任務：

```
Seed 1: 請把「星爆氣流斬」翻譯成英文
Seed 2: 請問桐人的著名招式是什麼
Seed 3: 請介紹刀劍神域這部動畫
```

然後請 ChatGPT 根據這些種子任務，改寫出更多問句：

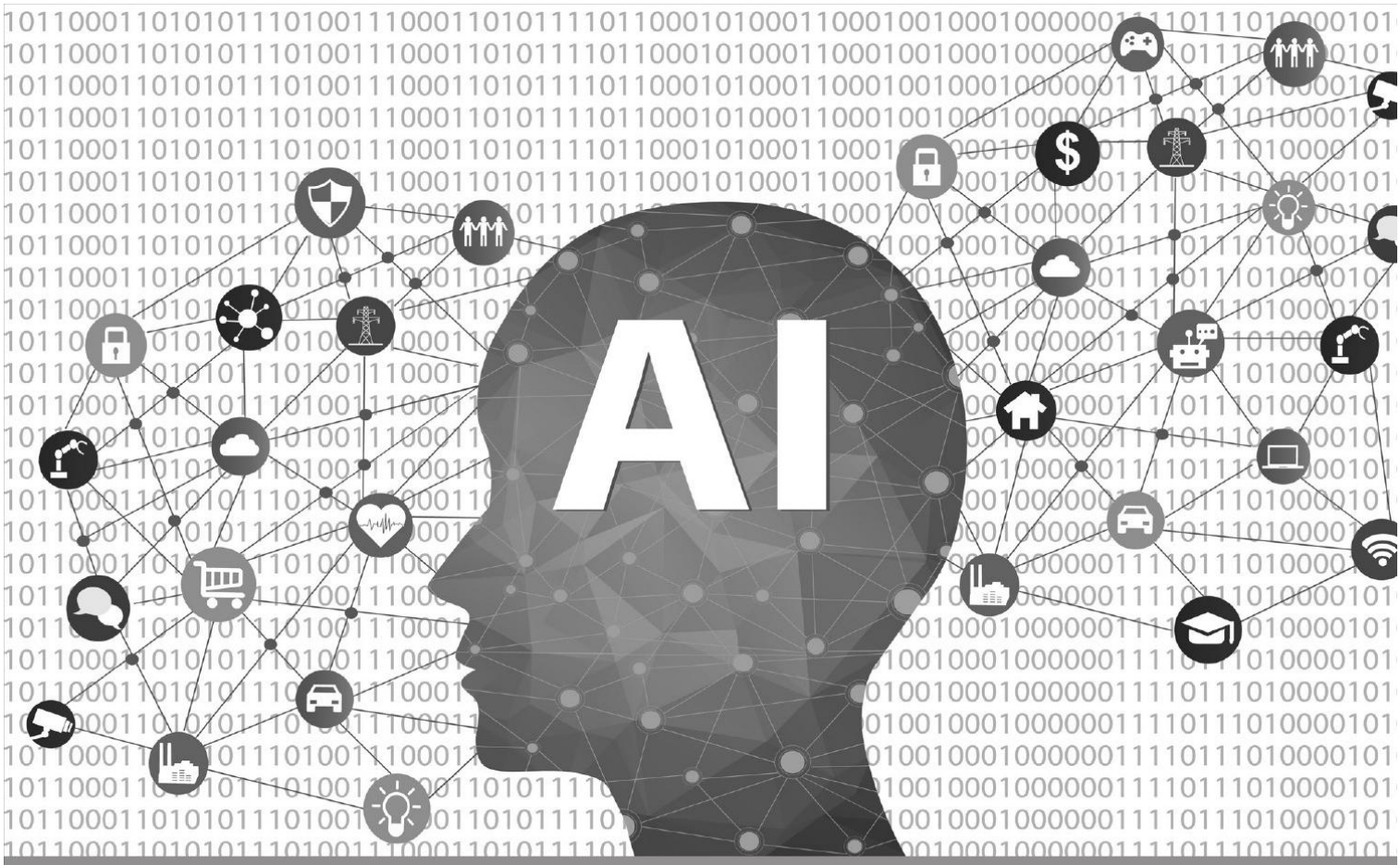
```
Task 1-1: 請把「星爆氣流斬」翻譯成英文
Task 1-2: 請把「大玉螺旋丸」翻譯成英文
Task 1-3: 請把「龜派氣功波」翻譯成英文

Task 2-1: 請問桐人的著名招式是什麼
Task 2-2: 請問鳴人的著名招式是什麼
Task 2-3: 請問悟空的著名招式是什麼

Task 3-1: 請介紹刀劍神域這部動畫
Task 3-2: 請介紹火影忍者這部動畫
Task 3-3: 請介紹七龍珠這部動畫
```

35 https://github.com/tatsu-lab/stanford_alpaca

最後再拿著這些問題直接去問 ChatGPT 來得到答案，這樣就能獲得完整的問答資料集了。除了一般的問答以外，有些時候也會設計一些不能回答或答不出來的問題，藉此訓練模型拒絕回答的能力。

25.7 結論

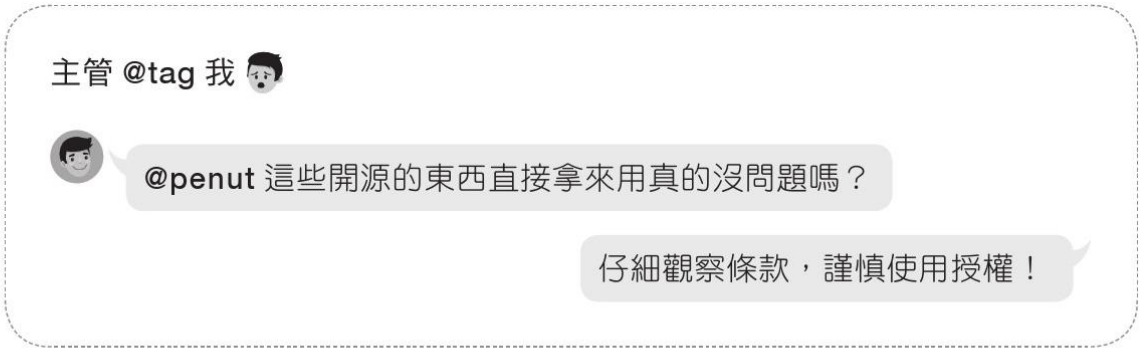
本章節介紹了一些常見的資料集，以及一些中文相關的資料。在繁體中文資料方面，可用於語言模型訓練的資料還是相對稀少一些。作為一個相對資源稀缺的語言使用族群，開發出符合在地語言與文化的資料集需要相當大的努力，希望在這波人工智慧的浪潮下，可以讓大家更加重視這個領域的耕耘。



Chapter

26

授權條款與社會責任



在研究語言模型相關的授權條款時，主要有三個方向：**程式碼**的授權、**資料集**的授權與**模型權重**的授權。有些相對嚴謹的專案，可能會使用不同的條款來分別授權這三樣東西，有些專案甚至會另外寫一份特別的授權條款。

在個人自由研究的階段，可能還不需要太在意授權條款的問題。但是當你的專案準備要公開或商用時，瞭解授權條款來保障彼此的權益，是一個相當重要的課題。本章節帶大家從語言模型的角度出發，探討授權條款的相關議題。

26.1 什麼是「權」？

在談論**授權**（License）之前，可以先來瞭解**著作權**（Copyright）是什麼。著作權是**智慧財產權**（Intellectual Property）的一種，在智慧財產權底下還有商標、專利等等。著作權主要在**保障作者與作品**的權利，其他任何人**未經授權**都不能任意複製、修改或散布等等。

如果想要使用別人的程式碼，那就會需要對方的**授權**。開發者通常會放一個 `LICENSE` 檔案在專案根目錄底下，描述大家在使用此專案程式碼時，應該遵守哪些規定。但即便有了作者的授權，也不代表擁有該作品的著作權。

在 GitHub 上，可以搜尋例如 `license:MIT` 來尋找使用 MIT 授權條款的專案。瀏覽專案時，也可以點進 `LICENSE` 檔案裡面，如果是常見的授權條款，網頁上面就會很貼心的幫你條列這個條款的一些重點：

圖 26-1　在 GitHub 上會列出授權條款的重點

除此之外，在 HF Hub 上面瀏覽模型或資料集時，左邊也有很多授權條款可以篩選，種類真的超級多！

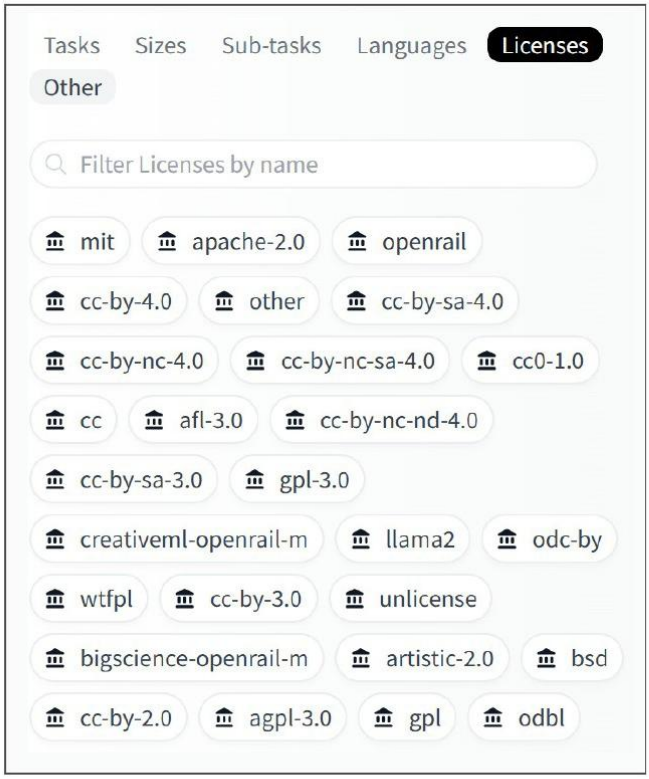


圖 26-2　在 HF Hub 上可以過濾的授權條款種類

26.2 常見授權條款

26.2.1 MIT

MIT 是源自**麻省理工學院（Massachusetts Institute of Technology, MIT）**的一種授權條款，為常見的授權條款裡面規範最寬鬆的一條。任何人都可以免費的複製、修改、再發布，只需要在程式碼的副本處放上一個 MIT License 檔案就好，但你的整份專案還是可以用其他 License 發布或者直接商用等等。

像是 **Ruby on Rails**[1] 框架與 **VS Code**[2] 文字編輯器等專案，皆是使用 MIT 授權條款。

26.2.2 GPL & LGPL

GPL（GNU General Public License）是**自由軟體基金會**[3]**（Free Software Foundation, FSF）**製作的授權條款，這個條款雖然同樣可以修改與商用，但是要求使用者必須使用相同的授權條款，而且必須跟著開源。例如 Meta 釋出的第一版 LLaMA 模型就是使用 GPL 授權條款。

因為必須使用相同的授權條款且強制開源，所以 GPL 專案比較難在閉源的商用專案裡面使用，於是後來出現了 **LGPL（GNU Lesser General Public License）**授權條款，如果只是引用該專案的話，並不需要開源或使用相同授權。但如果有修改該專案原始碼的話，則不適用此規則，必須跟著開源並使用相同授權。

1 https://github.com/rails/rails
2 https://github.com/microsoft/vscode
3 https://www.fsf.org/

像是 **GCC 編譯器**[4] 與 **VLC 影片播放器**[5] 等專案，都是使用 GPL 授權條款，而 **FFmpeg 多媒體工具**[6] 與 **Iris 麥塊光影模組**[7] 就是使用 LGPL 授權條款。

26.2.3 Apache 2.0

Apache 由 **Apache 軟體基金會**[8] 所製作，也是個滿寬鬆的授權條款。Apache 條款要求必須標注有修改的部份，也不能用原專案的商標或名稱來推廣衍生專案。

像是 Hugging Face 的 **Transformers**[9] 與 Google 的 **Tensorflow**[10] 等專案，就是使用 Apache 2.0 授權條款。若基於這些框架去開發出一個自訂版本的框架，是不能用 Transformers 或 Tensorflow 的名號來推廣這個版本的。

26.2.4 BSD

BSD（Berkeley Software Distribution）源自加州大學柏克萊分校（UC Berkeley），也是個可以自由複製、修改與商用的授權條款。BSD 原本有四條規定，被稱為 BSD 4-Clause，但是因為第四條要求衍生專案必須附上貢獻者名單，很多人覺得很麻煩，於是後來出現了把第四條移除的 BSD 3-Clause 或更少的 BSD 2-Clause 等等。

像是 **Numpy**[11] 數學運算套件與 **Django**[12] 後端框架等，也是 BSD 授權條款。

---

4 https://gcc.gnu.org/git/?p=gcc.git;a=blob;f=COPYING3
5 https://github.com/videolan/vlc
6 https://ffmpeg.org/
7 https://github.com/IrisShaders/Iris
8 https://www.apache.org/
9 https://github.com/huggingface/transformers
10 https://github.com/tensorflow/tensorflow
11 https://github.com/numpy/numpy
12 https://github.com/django/django

26.2.5 CC

以上介紹的授權條款通常都是用在程式碼專案上面，而 **CC（Creative Commons）**條款主要針對圖文創作，例如訓練用的資料集就經常使用 CC 條款。在 CC 條款底下還有很多種類，包含：

- BY 是指必須附上原作者的資訊，這個 BY 源自英文 Made By XXX 的 By 用法。
- SA（Share Alike）代表衍生作品必須使用相同的授權。
- NC（Non-Commercial）表示不可商用。
- ND（No Derivative Works）則是禁止修改。

所以像是 CC-BY-SA 就是必須附上作者資訊與使用相同授權條款，CC-BY-NC 則是除了需要附上作者名稱之外，還禁止商用的條款。而 CC0 則是代表作者完全放棄他的著作權，其他人想做複製修改都可以。

26.3 特殊授權條款

26.3.1 OpenRAIL

機器學習的模型權重又跟程式碼和資料集不太一樣了，模型權重不像程式碼是一份可以運作的工具，也不像圖文資料是一種創作，因此前面的條款在某些情況下不太適合模型權重。

在模型上其實更講究**責任歸屬**的問題，因為多數的機器學習模型都是在做**預測**，然而預測沒有 100% 正確的，若實際運作時模型出了問題，責任該如何歸屬？另外，如果有人拿這個模型去做壞事，那模型的作者是否該負起責任？事實上這個議題是相當複雜的，時至今日還是經常產生訴訟糾紛。

於是 OpenRAIL 誕生了，全名為 Open & Responsible AI License，同樣可以讓大家複製、修改和商用等等，同時也要求使用者必須負起責任，不可以拿來做壞事。如果因為不當使用導致你被抓、被罰款，那模型開發者是不需要負責的。

這個授權條款的目的，是為了降低模型開發者會遇到的法律風險。另外也有 BLOOM 的 **BigScience OpenRAIL-m**[13] 或 Stable Diffusion 的 **CreativeML Open RAIL-M**[14] 等不同的版本，也都是類似的聲明。

26.3.2 Llama2 License

第一版的 LLaMA 使用 GPL 授權條款，是比較讓人傷腦筋的授權，許多與 LLaMA 相關的衍生模型，都是使用 Delta Weight 或 LoRA Adapter 的形式釋出，以避免觸犯授權條款的內容。因此其他使用者為了使用這些模型，經常要用各種不同的工具做合併，很麻煩也很耗時。

在 Llama 2 推出之後，使用了全新的 **Llama2 License**[15]，這個授權條款讓大家也能複製、修改、再發布等等。這樣大家就不用再畏畏縮縮的，可以直接把微調過的 Llama2 模型權重整個釋出。

花生狂想曲

在 Llama2 授權條款中有個有趣的附加商業條款，規定如果使用此模型的服務提供廠商，月活躍用戶超過七億的話，必須向 Meta 請求額外的許可。七億欸我的老天鵝！全地球也就七八十億人，到底有哪個服務的月活躍用戶數可以到全球的 10% 啊？大概就只有 Meta 或 Google 這種超級大公司才有吧

13 https://bigscience.huggingface.co/blog/bigscience-openrail-m

14 https://github.com/CompVis/stable-diffusion/blob/main/LICENSE

15 https://ai.meta.com/llama/license/

26.4 相關爭議

AI 領域相對於人類歷史而言，是個相當新穎的領域，也因此誕生了許多爭議。像是 GitHub Copilot 就被提出訴訟[16]，但微軟與 OpenAI 認為 GitHub Copilot 並沒有造成傷害[17]，例如 GitHub Copilot 實際上如何侵害了誰的程式碼等等。已經身中數箭的 OpenAI 更遇到了美國女演員喬韓森認為 OpenAI 未經同意使用她的聲音來訓練語音合成模型[18]。

Getty Images 控告 Stability AI 將他們網站的圖片用於訓練與營利[19]，已經違反他們的版權規定，甚至生成的圖片上面還有 Getty Images 的浮水印[20]。

也有許多作家連署要求，大公司在訓練 AI 時若有用到他們的作品，應該取得同意並給予適當的補償[21]。還有人發現 Books3 資料集裡面包含了許多受版權保護的書籍[22]，而且這是許多模型都有拿來用的資料集，使得這些與訓練資料版權相關的訴訟隨著 AI 的活躍而跟著增長[23]。

前陣子美國著作權局也為了生成式 AI 衍生的版權問題徵求公眾意見[24]，這些訴訟與事件都是相當值得關注的，整個人工智慧從訓練到使用，都涉及了非常多版權問題，這些訴訟的判決結果也將左右未來模型的開發政策與走向。

16 https://www.ithome.com.tw/news/154076
17 https://www.ithome.com.tw/news/155288
18 https://www.ithome.com.tw/news/162990
19 https://www.ithome.com.tw/news/155229
20 https://www.inside.com.tw/article/30654-getty-images-sued-stabilit-ai
21 https://www.ithome.com.tw/news/157853
22 https://www.bnext.com.tw/article/76503/book3-ai-training
23 https://technews.tw/2023/08/28/the-secret-to-chatgpts-intelligence-170000-pirated-books/
24 https://www.ithome.com.tw/news/158526

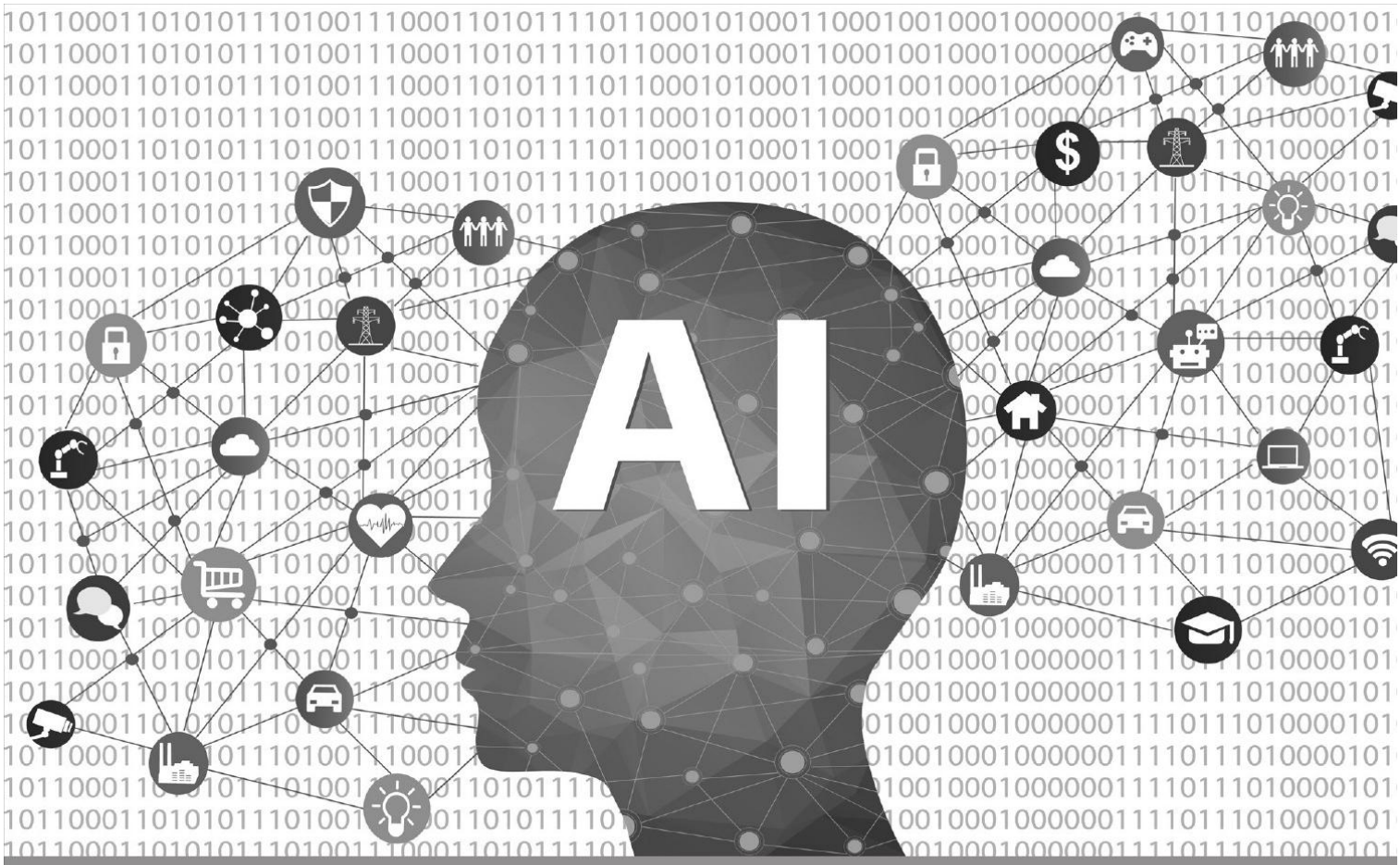
26.5 結論

本章節介紹了一些常見的授權條款以及人工智慧面臨的各種訴訟爭議。身為一個現代的資訊使用者，培養基礎的版權觀念相當重要。在處理海量的訓練資料時，確實很難顧及到每個角落，使每份文本或圖片都是沒有爭議的。

筆者相信，許多開發者都是立意良善，基於熱愛研究與分享的心情在開源自己的成果，這些工具通常是在解決人與機器之間的問題。但是當一份專案走出自己的研究，成為可以被公眾檢視的產品時，就會衍生出許多人與人之間的問題。偏偏社恐如筆者，就是不擅長與人溝通

業界各方也不斷提出許多不同的授權條款，希望能在滿足各種學術或商業用途同時，盡可能保障開發者與使用者的權益。這些條款能夠促使這個生態圈的參與者更無後顧之憂，也因此瞭解這些條款的內容變得相當重要。

最後期許大家，都能當個負責任的 AI 使用者，不要拿 AI 去做壞事，而是用 AI 讓這個世界變得更美好



Chapter

27

學海無涯，學無止境

人工智慧領域的變化相當快速，很有可能筆者今天寫的介紹，明天框架一個大改就不能用了。或者突然出現一個驚人的模型，取代了所有 Transformer 架構，那筆者寫的這本書九成就都沒用了，這也是不無可能的。

因此我們需要不斷的學習，去接觸、去瞭解、去咀嚼新知。但人的時間是有限的，沒有人可以全知全能，要有效率且方向正確的吸收知識，其實是很困難的課題。

最後一個章節，就來跟大家分享一些吸收新知的管道吧！

27.1 論文

看論文是個非常好的管道，可以到各大 AI 龍頭的 Publication 頁面尋找他們發表的論文，也可以用 **Google Scholar**[1] 尋找相關論文，或者使用 **Scholar GPT**[2] 幫你找論文，而筆者自己比較常關注 arXiv 上 **cs.CL**[3]（Computer Science, Computation and Language）領域的最新論文。

arXiv 上的論文有個好處，就是很多論文都會附上論文原始碼，通常是 Latex 格式。除了純文字格式以外，往往也能看到一些作者的註解，有些註解甚至是用中文寫的，這些是從 PDF 上看不到的部份。有時候這些註解反而能讓人豁然開朗，~~原來實驗數據這樣呈現是為了隱藏系統的缺陷啊~~。

雖然「論文」一詞給人很學術的感覺，光聽就想睡，但是在資訊領域裡面，其實就很像我們在鐵人賽裡面看到的技術文章，只是他是用英文寫的而已。推薦可以看看那些有公開原始碼專案的論文，將程式碼與論文搭配著看，可以更容易吸收裡面的內容。

1 https://scholar.google.com.tw/
2 https://chatgpt.com/g/g-kZ0eYXlJe-scholar-gpt
3 https://arxiv.org/list/cs.CL/recent

27.2 GitHub 趨勢

在 GitHub 趨勢[4] 上可以看到最近的熱門專案，雖然不是每個都跟語言模型有關，但這股熱潮應該還會再持續一陣子，所以經常能看到與語言模型相關的專案。也可以下載手機版的 GitHub App，有事沒事就打開來看看有什麼有趣的專案，喜歡的話就幫他點顆星，這樣 GitHub 也會推薦一些相關的專案給你參考。

27.3 Hugging Face

27.3.1 Blog

HF Blog[5] 常會分享一些技術觀念文章，因為 Hugging Face 的開發團隊經常在整合最新潮的技術，完成整合後也會在第一時間發表到部落格上，有時套件本身甚至都還沒進版。因此關注 HF Blog 也能觀察到現在流行什麼樣的技術，大家都在關注哪方面的技術等等。

27.3.2 Trending

從 HF Hub[6] 首頁可以看到最近的趨勢，大家都在下載哪些模型、資料集，或者在玩哪些 Space 的 Demo 系統。可以點進這些熱門的模型看看，去瞭解他們又改進了哪些模型架構或訓練手法，在留言區也能看到大家為什麼喜歡這個模型，或者這個模型還有哪些缺陷需要改進。

4 https://github.com/trending
5 https://huggingface.co/blog
6 https://huggingface.co/

27.3.3 Daily Paper

假如你不像筆者每天都在當薪水小偷，有很多時間可以隨意閱覽論文，那可以考慮看看 HF Daily Paper[7] 每天推薦的論文。這裡會列出的論文可能不僅限於文字領域，也經常看到圖像或語音相關的論文，大家也經常在這裡討論那些論文的內容，有時還會看到該論文的原作者在底下熱心回覆喔！

如果你有在 arXiv 上發現喜歡的論文，卻沒有在 HF Daily Paper 上刊登，也可以幫忙建立頁面，只要在搜尋欄輸入該論文的編號，進去之後點 `"Index … on the Hub"` 就可以囉！可以幫喜歡的論文按個讚，或者在留言處分享你的心得。

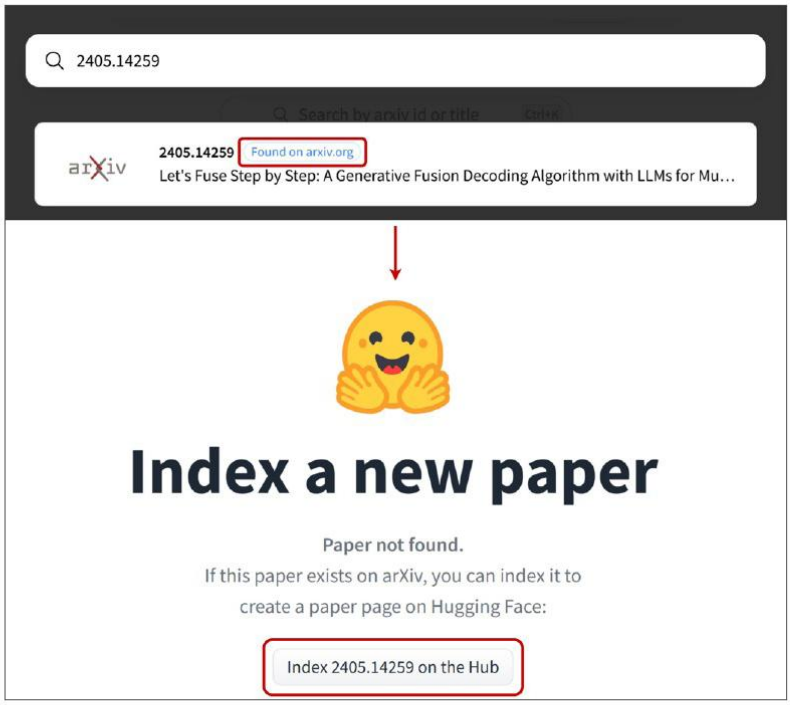


圖 27-1　在 HF Daily Paper 上建立 arXiv 論文頁面

如果想找更多相關的論文，可以在留言處輸入 `@librarian-bot recommend`，就能召喚一個機器人幫你推薦更多論文喔！

7　https://huggingface.co/papers

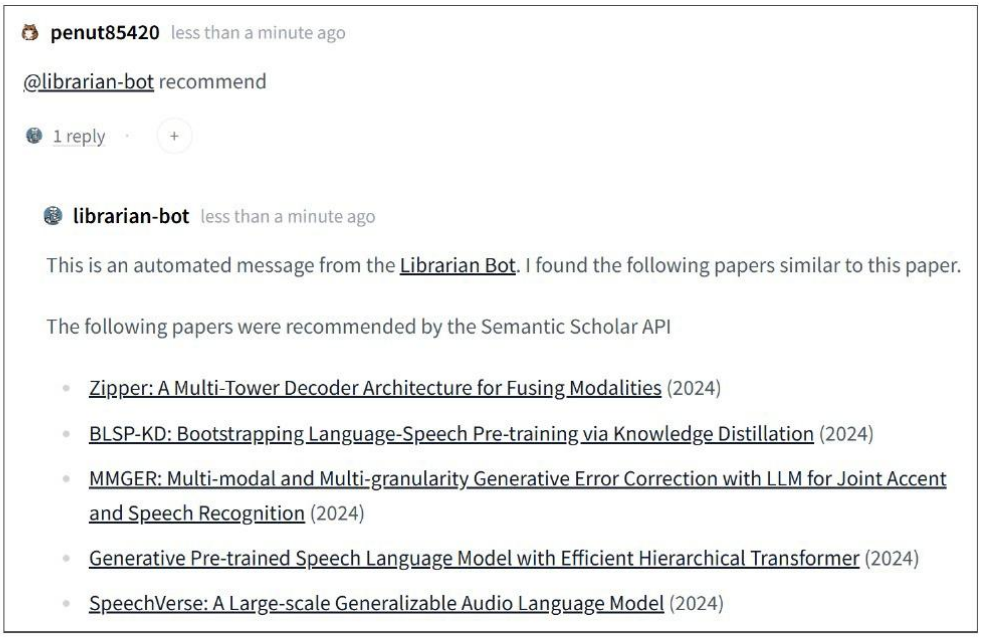


圖 27-2　請機器人推薦更多相關論文

27.4　iThome

iThome[8] 分享的技術新聞也是相當優質！經常報導與最新的時事、技術相關的新聞，而且記者通常都會附上來源，對於有疑惑的部份可以點進資料來源研究原文，也能借助 ChatGPT 的力量來快速閱讀與核實這些新聞。

27.5　與人互動

最後別忘了，與人互動也是個很好的學習方法。如果在讀論文時遇到問題，不妨試試看寄封電子郵件詢問原作者，筆者就曾經這樣做並且得到了良好的回應，解決了當下的煩惱，令人相當感激。

8　https://www.ithome.com.tw/

如果是在 GitHub 上遇到問題，可以發 Issue 尋求協助，但請記得遵循該專案的相關規範。保持謙虛有禮的態度，也更容易讓大家願意協助你解決問題。

27.6 未來發展

在筆者打鐵人賽的期間，OpenAI 推出的 GPT-4V 突然就會理解圖片的功能了，而在筆者準備出版期間，還推出了 GPT-4o 能夠與人流暢的語音互動，使用起來的效果相當驚人。筆者相信未來趨勢會是**多模態（Multimodality）**，不僅只是輸入文字、圖片，甚至能直接產生聲音、影片等等。

在程式語言方面，像是 Rust[9] 與 Mojo[10] 也是值得關注的程式語言，例如 HF Tokenizer[11] 套件底層就是以 Rust 實做的。Mojo 一直到最近才開放給大眾使用，但已經出現 llama2.mojo[12] 這種專案，成果也是相當驚人。

有人在強化底層實做，也有人在豐富上層框架。從 vLLM 這種高速推論的框架，到 LangChain 這種豐富簡潔的應用開發框架，都在不斷改進我們使用語言模型的體驗。

除了文字模型以外，還有許多其他的 AI 模型，例如 Whisper 語音辨識模型、Stable Diffusion 圖像生成模型、Suno 音樂生成模型，甚至還有 Kosmos-2.5 這種排版文件生成的模型，也推薦給各位讀者參考。

願人工智慧的技術發展能讓我們的生活越來越美好！

---

9 https://www.rust-lang.org/
10 https://www.modular.com/mojo
11 https://github.com/huggingface/tokenizers
12 https://github.com/tairov/llama2.mojo